rai-bvar: Composable Bayesian VARs with Stochastic Volatility, Validated Samplers and a Fair Out-of-Sample Verdict Engine in Python

Yigit Okar[1,2*]
[1]Rational AI, Inc., New York, NY, USA
[2]Care Artificial Intelligence Research and Consulting LLC, Brooklyn, NY, USA

October 2026

Abstract

rai-bvar, an Apache-2.0 Python package, is built on the position that a density-forecast gain credited to a richer mean or prior in a Bayesian VAR (BVAR) is interpretable only with the volatility specification held fixed. A BVAR composes mean, volatility and prior components, which select the engine. Conjugate Minnesota models, with dummy observations and the Giannone–Lenza–Primiceri hierarchy, go to an analytic engine. Stochastic volatility, outliers and Student-t errors run on hand-written JAX Gibbs samplers, in recursive structural or corrected reduced form. Both engines return one contract, path draws with per-draw conditional Gaussian moments, so log scores are Rao–Blackwellised (RB). At $h > 1$ an opt-in MCMC estimator integrates intermediate outcomes out. A verdict engine calls a model better only if it differs from its reference along one axis, gains over three Monte Carlo standard errors in log score, is separated from it by a rolling-window test or the model confidence set (unproven under expanding windows), and both converged at 95% or more of the origins. Samplers are checked by identities, block-level tests, Geweke tests, simulation-based calibration (SBC), analytic twins and parity tests; full-size Geweke and SBC tests pass on the release commit. Eighteen curated bugs form a release gate for the identity, block-level, Geweke and SBC tiers: in all 37 mutant-by-tier checks the bug is caught and the unmodified sampler passes. An illustrative replication of Clark and Ravazzolo (2015a) on FRED-QD missed 2 of 6 criteria in a first full run; after remedies registered before a second run (not blind to the first), the acceptance run met 5 of 6 as coded. The unmet one asks the unemployment gain at $h = 8$ to be below that at $h = 1$: on the path-integrated RB score it is larger, mostly through three crisis origins; on their Gaussian score it shrinks, as in the original.

Keywords: Bayesian vector autoregression; stochastic volatility; Gibbs sampling; density forecasts; sampler validation; reproducibility; Python; JAX.

1 Introduction

rai-bvar is a Python package for Bayesian vector autoregressions (BVARs) built around one question, stated in its specification as

*Written with AI coding agents (Claude, Anthropic, used through Claude Code); the author is responsible for its content. Appendix J states what the agents did and what the record shows of the author's part.

> "Does added model complexity improve out-of-sample density forecasts once time-varying volatility is accounted for?"

The qualification matters. The volatility of U.S. macroeconomic shocks has changed over the post-war period (Cogley and Sargent, 2005; Primiceri, 2005; Sims and Zha, 2006), and allowing for time-varying volatility has been found to improve BVAR density forecasts (Clark, 2011; Clark and Ravazzolo, 2015a; Carriero et al., 2016); Chan (2024) surveys BVARs with stochastic volatility (SV), and Carriero et al. (2015) study how other specification choices affect forecast accuracy. When a richer mean or prior is compared with a baseline that also differs in volatility, the gain combines the two changes. The package therefore adopts a design position, a rule of the software and not an empirical finding: a mean or prior effect is measured with the volatility specification held fixed, against the linear Minnesota model under the same SV and never against constant volatility.

Our thesis is that such a comparison is only as credible as its weakest link. We name six links as requirements on the software, not claims about existing tools, and say where each is addressed.

1. *The sampler may target the wrong posterior.* A published equation-by-equation sampler for large BVARs with SV (Carriero et al., 2019) did so, as Bognanni (2022) showed; Carriero et al. (2022) corrected it. Samplers need checks that are shown to catch known errors (Section 8).

2. *Every forecast origin is a separate MCMC fit, and each must converge*; a gain computed from unconverged fits should not be certified (Sections 6.6, 8.9 and 11.4).

3. *The log score is itself a Monte Carlo estimate.* At horizons beyond one step its error can be large in the tails while the reported standard error stays small (Section 6.4).

4. *Triangular SV models depend on the variable ordering* (Arias et al., 2023; Chan et al., 2024), so a gain should be reported with its spread across orderings (Sections 6.7 and 11.5).

5. *Tests must be valid for the design.* For nested models under an expanding window, the tests of Diebold and Mariano (1995) lack their standard limit (Clark and McCracken, 2001) and the framework of Giacomini and White (2006) needs a rolling or fixed window; the package then labels those p-values descriptive and decides through the model confidence set (Hansen et al., 2011), whose validity in that design is itself open, on proper scores (Gneiting and Raftery, 2007) (Sections 6.5, 6.6 and 12).

6. *Analysis choices can be made after the scores are seen*, unless the settings are fixed in code before the scores exist (the pre-registration in Section 11.4).

Two infrastructure requirements follow: one estimation interface and output format for conjugate and non-conjugate models (Sections 4.2 and 6.2), and no look-ahead, so that data-dependent prior settings such as Minnesota scales are recomputed inside every estimation window (Section 6.5).

The package metadata describe rai-bvar as "Ablation-ready Bayesian VARs: composable mean, volatility and prior components with a fair out-of-sample verdict engine." Five design commitments shape it: (i) the composition, never the user, chooses the engine, sending conjugate Minnesota models to an analytic engine and compositions with SV, outliers or Student-t errors to hand-written JAX Gibbs samplers; (ii) both engines return one predictive contract; (iii) one backtest and one verdict engine score and compare every model; (iv) validation ships as library code, not only as tests; and (v) the implementation is licensed under Apache-2.0, with GPL-licensed packages run only as numerical oracles (qualified in Section 2). "Ablation-ready" describes the architecture, in which a new mean, volatility or prior component is evaluated against every existing one through one contract and one verdict rule. The release described here (Phases 0–2 of the roadmap) populates the volatility

axis; the mean axis is linear and the prior axis is the Minnesota family (Section 3). It is release 0.2.0 (tag `v0.2.0`), classified as pre-alpha: commit 4ac7969 of `https://github.com/yigitokar/rai-bvar` (Appendix G), with the code of commit 7f880e1, on which the release validation record was made; "the release commit" below refers to that code.

rai-bvar makes each link checkable, and this paper shows the evidence for each. The econometrics is not new; the contribution is the integration, the contracts and the evidence of correctness:

- **C1, a factorial composition grammar** (Section 4). `BVAR(mean=...,vol=...,lags=...)` resolves to an engine by fixed rules and rejects invalid compositions at construction; `spec()`, `axes` and `spec_hash(model)` make "differs along exactly one axis" checkable.

- **C2, a populated volatility axis** (Section 3). Constant volatility under inverse-gamma, inverse-Wishart or IW-equivalent (Chan, 2022) priors, and SV with AR(1) or random-walk log-variance, outliers (Carriero et al., 2024), Student-t errors (Jacquier et al., 2004) or both, in two recursive structural forms and the corrected reduced form (Carriero et al., 2022), though not every combination in every form (Section 3); dummy-observation priors (Doan et al., 1984; Sims and Zha, 1998; Giannone et al., 2015) are available on both engines.

- **C3, one predictive contract** (Sections 6.2 to 6.4). Path draws with per-draw conditional means and Cholesky factors give a Rao–Blackwellised log score and an effective-sample-size-adjusted Monte Carlo standard error (MCSE) (Geweke and Amisano, 2010; Krüger et al., 2021). We diagnose its tail failure at $h > 1$ and implement, for MCMC forecasts, an opt-in estimator that integrates the intermediate outcomes out given each draw's volatility path, conditioning as Hiraki et al. (2026) do for predictive moments.

- **C4, a backtest and verdict engine that encode "volatility first"** (Sections 6.5 to 6.6). Look-ahead-free refits, one-axis pairing, grid harmonisation, a three-MCSE gate, a validity policy for Diebold–Mariano tests with the Harvey–Leybourne–Newbold correction (Harvey et al., 1997), model-confidence-set separation and a convergence gate (Vehtari et al., 2021) form one automatic rule; the combination is the claim, and its size and power are unstudied (Section 12).

- **C5, validation as library code** (Section 8). `rai_bvar.validation` implements the statistics and independent reference models of identity checks, block-level checks, Geweke tests and simulation-based calibration (SBC), which with analytic twins, parity tests and a mutation suite form seven tiers (Geweke, 2004; Cook et al., 2006; Gelman, 2017; Talts et al., 2018; Modrák et al., 2025; Grosse and Duvenaud, 2014; Gandy and Scott, 2021; Säilynoja et al., 2022). Geweke and SBC tests are registered for 23 composition rows, among them that of the replication's primary SV arm, and Geweke tests for 7 block configurations. Mutation testing (DeMillo et al., 1978; Jia and Harman, 2011) gives the identity, block-level, Geweke and SBC tiers a release gate: 18 curated sampler bugs, each with a confirmed catching tier, give 37 mutant-by-tier checks, all caught with passing controls; this is a regression gate, not an estimate of power.

- **C6, observations on validation methods** (Sections 8 and 9.3). Exact-mode SV is exact for any mixture table, so a coarse table gives Geweke tests power against exact-correction errors; the log-likelihood as a test quantity catches the triangular-step error of Carriero et al. (2019), which parameter ranks missed in recorded runs, consistent with Modrák et al. (2025); per-replication SBC effective sample sizes cannot be certified from 99 kept draws; and dummy-observation rows admit a closed-form variance marginal and an exact rejection envelope.

- **C7, two float64 failures fixed in the kernel without jitter** (Section 9). A Householder-QR draw of the contemporaneous coefficients replaces a Cholesky draw whose formed precision lost its small eigenvalue, and a cancellation-free excess-pivot recursion factorises the random-walk SV path system at $\sigma \approx 5 \times 10^{-9}$; we claim this application of subtraction-free elimination (Grassmann et al., 1985; Alfa et al., 2002; Ye, 2008; Demmel et al., 2008), not the algorithm.
- **C8, reproducible JAX MCMC with stated limits** (Section 4.6). Counter-based random keys (Salmon et al., 2011) are derived per chain, sweep, block and equation; invariances are tested bitwise on toy kernels and on production chain prefixes under `chains="map"`, and to a relative 10^{-9} for production `vmap` and `shard_map` runs; nothing is guaranteed across platforms or JAX versions.
- **C9, an empirical illustration** (Section 11). A pre-registered qualitative replication of Clark and Ravazzolo (2015a) on FRED-QD with all 24 variable orderings illustrates the workflow; it does not test the econometric claim. In a first full run criteria 1, 2, 3 and 5 of six were met and criteria 4 and 6 were not, which we traced to marginal $\hat{R}$ misses at the convergence gate and to the tail failure of the path-conditioned score at a few crisis origins. The acceptance run's settings respond to that diagnosis, so its pre-registration is not blind, but they were committed before the run was launched. As coded, its outcome is: criteria 1, 2, 3, 5 and 6 met; 4 not met. With every reduced-form fit converged on the gate's primary quantities, stochastic volatility is "better" at $h = 1$ in all 24 orderings. Criterion 4 fails because the path-integrated RB unemployment gain is larger at $h = 8$ (+0.273) than at $h = 1$ (+0.160); about 72% of the $h = 8$ sum comes from three crisis origins, and on Clark and Ravazzolo's Gaussian-approximation score the gain shrinks, as in their results, although the criterion then fails on pre-crisis GDP instead (Section 11.5). We report this difference without tuning and leave its causes as untested hypotheses.

The priors, SV samplers, mixture approximations, ancillarity–sufficiency interweaving, the corrected triangular algorithm, precision samplers, predictive-ability tests, scoring rules, Geweke and SBC tests, mutation testing, subtraction-free elimination and conditional (Rao–Blackwellised) predictive densities are existing methods, credited where used.

Section 2 covers related software and licensing, Sections 3 to 6 the models, architecture, samplers, forecasts and verdicts, Section 7 two examples, Sections 8 to 10 validation, numerical robustness and performance, and Sections 11 to 13 the replication, limitations, and a checklist and roadmap.[1]

2 Related software, numerical oracles and licensing

Existing packages had the following roles, with the versions used (parity results are in Section 8.6). The R package BVAR (Kuschnig and Vashold, 2021), version 1.0.5 (Kuschnig and Vashold, 2024), was run as the oracle for the hierarchical prior of Giannone et al. (2015), for the sum-of-coefficients and single-unit-root dummy observations, and for Metropolis–Hastings sampling of the hyperparameters, and the Python package bvar of the Bank of England (Labonne et al., 2026), version 0.3.5 (MIT licence), as a second oracle for the analytic engine. stochvol (Kastner, 2016; Hosszejni and Kastner, 2021), version 3.2.1, was run as the oracle for univariate SV (Appendix D.7). bayesianVARs (Gruber et al., 2026), version 0.1.8, was planned as the oracle for the corrected reduced-form sampler but could not be built without a Fortran compiler, so we state no finding about it; Section 8.6

[1]The use of AI coding agents, and what "independent" means for their reviews, is stated in Appendix J; the licensing statement and the notes in which GPL source was read are in Appendix H.

describes its substitute. bvartools (Mohr, 2026) was an architectural reference, never run, and none of the later-phase oracles, factorstochvol (Kastner, 2026), bvars (Liu et al., 2026), bsvars (Woźniak, 2026), bsvarSIGNs (Wang and Woźniak, 2026) and bvarsv (Krueger, 2015), is run yet.

Other BVAR software is publicly available in Python (bvar-toolkit-python, Chan, 2026b; impulso, Pinder, 2026; srvar-toolkit, Shaw, 2026; and the `BayesianVARMAX` model of pymc-extras, PyMC Developers, 2026), in R (bvhar, Kim and Baek, 2026, whose Python version is in development; BGVAR, Boeck et al., 2022) and in MATLAB (the BEAR toolbox, Dieppe et al., 2016; bvar-toolkit, Chan, 2026a), and frequentist VAR estimation is available in statsmodels (Seabold and Perktold, 2010); we do not compare features.

The MCMC engine is written in JAX (Bradbury et al., 2018; Frostig et al., 2018), the analytic engine in NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020), forecasts and MCMC posteriors are returned as xarray objects (Hoyer and Hamman, 2017), and the continuous ranked probability score (CRPS), its threshold-weighted version and the energy and variogram scores come from scoringrules (Zanetta and Allen, 2024) (in R, scoringRules; Jordan et al., 2019). The samplers are hand-written Gibbs kernels by design, not models in a probabilistic programming language such as PyMC (Abril-Pla et al., 2023) or NumPyro (Phan et al., 2019); each exposes its exact conditional or Metropolis–Hastings acceptance ratio, on which the identity tests (Section 8.2) and the deterministic replay of failures (Section 4) rely; we make no claim about relative mixing.

rai-bvar is released under the Apache License, version 2.0. The R oracles BVAR (GPL-3) and stochvol (GPL $\geq$ 2) are GPL-licensed, so the project's specification requires algorithms to be implemented from the published papers, not translated from GPL source. The development practice was:

> No code was translated from GPL sources. Algorithms were implemented from the published papers. GPL packages were run as numerical oracles, and in a few documented cases (named in the repository notes) their source was read to pin down an oracle's behaviour, defaults or constants, never to write implementation code.

This is weaker than a clean-room process in the strict sense, in which implementers never see the source, and we do not claim that stronger property; Table H.1 in Appendix H lists the five repository notes that record a reading of GPL source, with the version read and the purpose. The FRED-MD and FRED-QD databases (McCracken and Ng, 2016, 2021) are downloaded on demand and checked against SHA-256 checksums, never bundled, because some series carry third-party copyrights.

3 The model framework

No model class of rai-bvar is new; each is credited where it is defined, and the package's own choices within a class (the Mode I scaling, the default SOC scope) are marked as such. Table 1 lists the components by axis, the validation rows in which each appears, and the roadmap. The data are n variables y_t, observed at $t = 1, \ldots, T$ after p initial observations, with regressors $x_t = (d_t', y_{t-1}', \ldots, y_{t-p}')' \in \mathbb{R}^k$, $k = n_d + np$, where d_t holds $n_d \in \{0, 1, 2\}$ deterministic terms (none, a constant, or a constant and a linear trend). Y (columns y_i) and X stack the observations, $Y_{<i}$ holds the first $i - 1$ columns of Y, and the column order of the data is the recursive ordering. κ is the Minnesota overall tightness (code `lambda_`), ι (iota, not the index i) the position of equation i where it enters a prior formula as a number, $\lambda_{i,t}$ a volatility, and $\mathrm{N}(m, V)$ has variance V; Table A.1 in Appendix A lists the rest of the notation, with the code names.

3.1 The recursive structural form

Every model estimated by MCMC is written as

$$Ay_t = \Gamma x_t + \Lambda_t^{1/2}\varepsilon_t, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathrm{N}(0, I_n), \tag{1}$$

with A unit lower triangular (entries a_{ij}, $j < i$), Γ with rows γ_i', and $\Lambda_t = \mathrm{diag}(\lambda_{1,t}, \ldots, \lambda_{n,t})$. The reduced form is $y_t = B'x_t + u_t$, $u_t \sim \mathrm{N}(0, \Sigma_t)$, with $B' = A^{-1}\Gamma$ and $\Sigma_t = A^{-1}\Lambda_t A^{-1\prime}$. As $|A| = 1$, the log likelihood given the first p observations is the reduced-form Gaussian log density with no Jacobian term,

$$\log p(Y \mid A, \Gamma, \lambda) = \sum_{i=1}^{n}\sum_{t=1}^{T}\Big[-\tfrac{1}{2}\log(2\pi\lambda_{i,t}) - \frac{e_{i,t}^2}{2\lambda_{i,t}}\Big], \qquad e_{i,t} = y_{i,t} + \textstyle\sum_{j<i} a_{ij}y_{j,t} - x_t'\gamma_i. \tag{2}$$

With priors independent across equations, equation i is, given the volatilities, a heteroskedastic regression with $k + i - 1$ regressors, so a sweep costs $O(nk^3) = O(n^4p^3)$ rather than the $O(n^6p^3)$ of a joint draw. This is exact in the structural form; a prior on the reduced-form B couples the equations, and there the package uses the corrected algorithm of Carriero et al. (2022) (Section 5).

3.2 Priors

All coefficient priors are of the Minnesota type (Litterman, 1986), in a conjugate form (analytic engine) or a structural or reduced form (MCMC engine; Section 4 resolves `form="auto"`); Appendix A.1 collects the options left out below.

Conjugate form. The analytic engine uses the normal–inverse-Wishart (NIW) prior (Kadiyala and Karlsson, 1997; Bańbura et al., 2010)

$$\Sigma \sim \mathrm{IW}(\mathrm{diag}\,\psi, d), \qquad \mathrm{vec}\,B \mid \Sigma \sim \mathrm{N}(\mathrm{vec}\,b, \Sigma \otimes \Omega), \qquad \Omega_{(l,j)} = \frac{\kappa^2(d-n-1)}{l^{\alpha}\psi_j}, \qquad d = n+2, \tag{3}$$

with Ω diagonal, (l, j) the coefficient on lag l of variable j, α the lag decay and $\Omega_{(d)} = \kappa_d$ for deterministic terms. The scales ψ_j are residual variances of univariate AR(p) regressions on the estimation window, and b is zero except on the own first lag, where it is 1 for series in levels or log levels (transformation codes 1 and 4). The Kronecker structure ties own- and cross-lag tightness (Kadiyala and Karlsson, 1997) and the inverse Wishart fixes the contemporaneous prior (Appendix B.1), so the structural form's scales κ_c (`cross`, on the prior standard deviation of other variables' lags) and κ_a (`contemp`, on the prior variance of a_{ij}) must be 1 here. The prior stays conjugate with sum-of-coefficients (SOC) and single-unit-root (SUR) dummy rows (Doan et al., 1984; Sims and Zha, 1998), built as in Giannone et al. (2015) except that by default (`soc_scope="persistent"`, our convention) SOC rows exist only for variables with a non-zero own-lag prior mean; with the hierarchy of Giannone et al. (2015) for the hyperparameters ϑ (`hierarchy="fixed"`, `"eb"`, the default, or `"mh"`); and with the COVID-19 scaling of Lenza and Primiceri (2022), which divides the rows of Y and X by a common scale s_t, equal to 1 before a start period t^* (by default the period containing March 2020).

Structural form. This default of the MCMC engine centres the coefficients conditionally on the contemporaneous row,

$$\mathrm{E}[\gamma_i \mid a_i] = M\tilde{a}_i, \qquad \tilde{a}_i = (a_{i1}, \ldots, a_{i,i-1}, 1, 0, \ldots, 0)', \tag{4}$$

with M the reduced-form prior mean b, so that $\mathrm{E}[B' \mid A] = M'$ for every A (D3 in the design documents; Appendix B.1). With M the random-walk mean (identity on the first lag, zeros elsewhere), this is the conditional centring of the structural prior of Sims and Zha (1998), who normalise the structural shocks to unit variance and leave the diagonal of A free; a linear centring is unchanged when a_i and γ_i are rescaled together. Sims and Zha (2006) write it, in their notation, as $A_+ = D + SA_0$ with a zero-mean prior on D (in ours, column i of D is $\gamma_i - M\tilde{a}_i$ up to the scale of the row). In the coordinates $\delta_i = \gamma_i - M_{<i}a_i$ ($M_{<i}$ the first $i-1$ columns of M; unit Jacobian), a_i and δ_i are independent a priori with diagonal precisions (Algorithm C.2).

Three prior modes share this form (Appendices B.1 and B.2; tests in Appendix B.3). *Mode I*, the package's own design and its default, used by `Constant("ig")` and every SV arm, puts the fixed scale $\bar{\sigma}_i^2 = \psi_i/\iota$ in place of λ_i in the coefficient prior variances. At $\kappa_c = \kappa_a = 1$ it matches exactly the reduced-form mean (for every A), the marginal coefficient variances and $\mathrm{E}\,\Sigma = \operatorname{diag}\psi$ of the NIW prior, but not its distribution, and avoids the growth $\mathrm{E}\,\Sigma_{ii} = 2^{i-1}\psi_i$ along the ordering (Chan et al., 2024) of the unscaled choice. *Mode II* (`Constant("iw")` with a structural mean) scales both blocks by λ_i instead; at $\kappa_c = \kappa_a = 1$ it is exactly the structural form of the NIW prior (Chan, 2022, Corollary 1), so its posterior is the NIW posterior, and with $\kappa_c \neq 1$ it is the asymmetric conjugate prior of Chan (2022) with the centring (4). `Constant("iw_equivalent")` scales only a_i by λ_i; at $\kappa_a = 1$ the implied Σ is exactly $\mathrm{IW}(\operatorname{diag}\psi, n+2)$ (Chan, 2022, Proposition 1), and in reduced form the model is invariant to the ordering of the variables.

Reduced form. The columns π_j of B get independent priors $\mathrm{N}(m_j, \psi_j\Omega^{(j)})$, with m_j column j of M and $\Omega^{(j)}$ the Ω of (3) with the entries of other variables' lags multiplied by κ_c^2, which match the NIW marginal variances at $\kappa_c = 1$; the prior on (A, λ) is the structural one.

Hyperparameters on the MCMC engine. These are fixed, by default at the empirical-Bayes mode of the analytic conjugate twin fitted at $\kappa_c = \kappa_a = 1$ on the same window, so that every volatility arm in a comparison shares the mean prior; except in Mode II this is a documented plug-in, not the estimated model's own mode (Appendix A.1).

3.3 Volatility arms

Constant. Under `Constant`, $\lambda_{i,t} = \lambda_i \sim \mathrm{IG}(1 + \iota/2, \psi_i/2)$, with mean $\bar{\sigma}_i^2 = \psi_i/\iota$, in all three modes.

Stochastic volatility. Under `SV` the log-variance $h_{i,t} = \log\lambda_{i,t}$ of each structural equation follows

$$h_{i,t} = \mu_i + \phi_i(h_{i,t-1} - \mu_i) + \sigma_i\eta_{i,t}, \qquad \eta_{i,t} \overset{\text{iid}}{\sim} \mathrm{N}(0,1), \tag{5}$$

or, with `rw=True`, the random walk $h_{i,t} = h_{i,t-1} + \sigma_i\eta_{i,t}$. The package's default priors, in the parametrisation of stochvol (Kastner, 2016; Hosszejni and Kastner, 2021), are $\mu_i \sim \mathrm{N}(\log(\psi_i/\iota), 10)$, centred in each window on the log of the Mode I scale; $(\phi_i + 1)/2 \sim \mathrm{Beta}(20, 1.5)$; $\sigma_i^2 \sim B_\sigma\chi_1^2$ with $B_\sigma = 1$ (0.1 for the random walk), or an inverse gamma; $h_{i,0}$ from the stationary law, or $\mathrm{N}(\log(\psi_i/\iota), 4)$ for the random walk. The replication's primary arm (Section 11), reduced form × `SV(rw=True)`, is validation row 18; its Student-t variant has no row of its own (Section 8.7). The factorisation $\Sigma_t = A^{-1}\Lambda_t A^{-1\prime}$ with a constant unit lower-triangular A and independent univariate log-variances is the volatility model of Cogley and Sargent (2005), who combine it with drifting coefficients and random-walk log-variances; Primiceri (2005) uses the same triangular factorisation but lets A drift and leaves the covariance of the log-volatility innovations unrestricted. With

Table 1: Components by axis, the engine (and form) estimating each, and its validation rows (Section 8; rows 1–18 and 201–205 of `joint.py` at 7f410dc: 15–16 vary the SV priors, 17 is zero-mean univariate SV, 201–205 add SOC/SUR rows). "Later": `SPEC.md` §4.4, §13.

Axis	Component	Engine (form)	Validation rows
Mean	`Linear`, $x_t'\gamma_i$	both engines	all except 17
	Later: BART (Phase 4); TVP, threshold and Markov-switching means (Phase 5).		
Prior	`Minnesota`, conjugate NIW; hierarchy fixed, EB or MH	analytic	none (analytic tests, parity)
	structural, Mode I	MCMC	1–4, 12, 14–16, 201, 204
	structural, Mode II	MCMC	5, 202
	structural, `iw_equivalent`	MCMC	6, 205
	reduced form	MCMC	7–11, 13, 18, 203
	SOC/SUR dummy rows	both engines	201–205
	Later (Phase 3): horseshoe, normal-gamma, Dirichlet–Laplace, R2D2 and SSVS priors; the asymmetric conjugate prior as a prior class.		
Volatility	`Constant("iw")`	analytic; MCMC (Mode II)	5, 202
	`Constant("ig")`	MCMC (both forms)	1, 7, 201, 203
	`Constant("iw_equivalent")`	MCMC (both forms)	6, 11, 205
	`SV()`, AR(1) log-variance	MCMC (both forms)	2, 8, 15–17, 204
	`SV(rw=True)`	MCMC (both forms)	14, 18
	`SV(outliers=True)`	MCMC (both forms)	3, 9
	`SV(tails="t")`	MCMC (both forms)	4, 10
	`SV(outliers=True, tails="t")`	MCMC (both forms)	12, 13
	Later: common SV (Carriero et al., 2016) (Phase 3); factor SV (Kastner, 2019) and order-invariant SV (Chan et al., 2024) (Phase 5).		
COVID-19	scaling of Lenza and Primiceri (2022)	analytic only	none (analytic tests)
	`SV(outliers=True)`	MCMC	3, 9, 12, 13
	Later: the scaling on the MCMC engine (Phase 3).		
Benchmarks	`RandomWalk`, `OLSVAR`	own simulator	none

constant coefficients and `rw=True`, the volatility specification is that of the BVAR with stochastic volatility of Clark (2011) (constant A, independent random-walk log-variances), who takes it from Cogley and Sargent (2005) and writes the VAR in mean-adjusted form with a steady-state prior.

Robust SV. Outliers and Student-t errors are transitory scales of the structural residual,

$$e_{i,t} = o_{i,t}\,\tau_{i,t}^{1/2}\,e^{h_{i,t}/2}\,\varepsilon_{i,t}, \qquad \lambda_{i,t} = e^{h_{i,t}}\,o_{i,t}^2\,\tau_{i,t}, \tag{6}$$

and $\lambda_{i,t}$ in (2) is then this effective variance. With `outliers=True`, as in Carriero et al. (2024), $o_{i,t} \in \{1,\dots,20\}$ is 1 with probability $1-p_i$ and otherwise uniform on $\{2,\dots,20\}$, with a Beta prior on p_i. The two-part outlier distribution and the Beta prior on p_i follow the unobserved-components model of Stock and Watson (2016), whose outlier scale is uniform on $[2,10]$; the discrete grid follows Carriero et al. (2024), who describe it as a tractable approximation. With `tails="t"`, $\tau_{i,t} \sim \mathrm{IG}(\nu_i/2,(\nu_i-2)/2)$, a standardised t ($\mathrm{E}\,\tau_{i,t}=1$; unit scale in Appendix A.1), and ν_i is uniform on $\{3,\dots,40\}$; the two components can be combined.

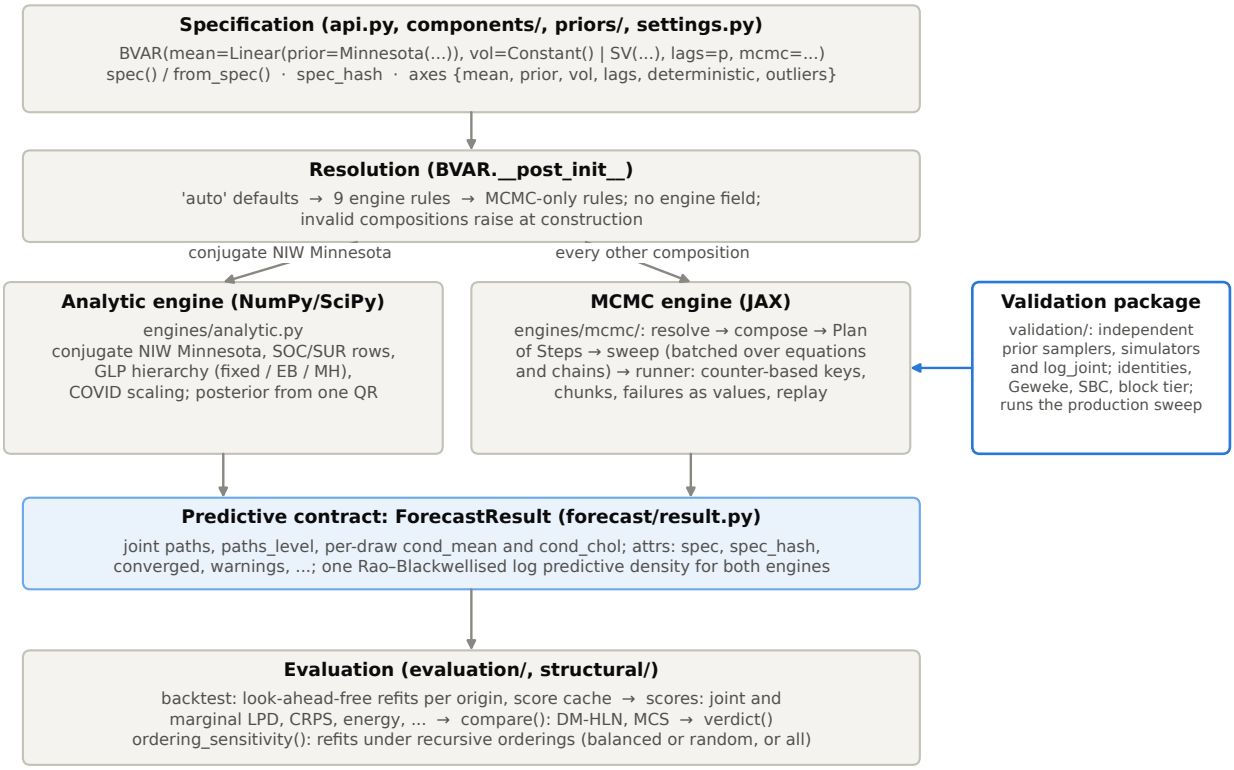


Figure 1: Architecture of rai-bvar. Specifications and engine resolution select the analytic (NumPy/SciPy) or the MCMC (JAX) engine. Both return the same `ForecastResult` contract, which feeds the backtest, the scores, `compare()` and `verdict()`. The validation package wraps the production sweep.

4 Software architecture

4.1 Design rules

Nine design rules fix the shape of the package. Five shape the interface: composition over inheritance (R1: a model is `BVAR(mean, vol, ...)`, with no model-specific classes), volatility first (R2), one predictive contract for both engines (R3), the variable ordering surfaced rather than hidden (R8), and no look-ahead (R9; Section 6.5). Three govern the internals: float64 throughout, no explicit inverses, precision-based samplers and log-space densities (R4); no sampler merged without passing Geweke and simulation-based calibration tests (R6; Section 8); and one MCMC backend, hand-written Gibbs kernels in JAX (Bradbury et al., 2018; Frostig et al., 2018) with no probabilistic programming language in the core (R7). R5 concerns licensing (Section 2). Figure 1 shows the resulting layers.

4.2 Composition and engine resolution

A model is a frozen dataclass `BVAR(mean, vol, lags, deterministic, outliers, mcmc)` with no engine field. Its constructor resolves the `"auto"` defaults, applies nine rules that map a resolved (prior form, volatility prior) to an engine and mode, and, for MCMC compositions, rejects what the engine does not support (Table C.1 in Appendix C.1): a conjugate prior runs on the analytic NIW engine with `Constant("iw")` and is an error with any other volatility; a structural prior runs on the MCMC engine, in Mode II (the NIW twin) with `Constant("iw")`, with the (a, λ) prior of Chan

(2022) under `Constant("iw_equivalent")` and in Mode I otherwise; and a reduced-form prior runs the corrected triangular step on the MCMC engine, except with `Constant("iw")`, which is an error. `BVAR()`, `BVAR(vol=Constant("ig"))` and `BVAR(vol=SV())` reach the first, the structural Mode I `Constant` and the structural Mode I SV rules. Every error is raised at construction, and tests check every rule of the table (the seven non-error rules case by case) and every error.

`spec()` returns plain JSON and `from_spec(spec())` round-trips (a property-based test). Because the resolved components are stored, `BVAR()` still resolves to the Phase 1 composition, and every Phase 1 (conjugate) model keeps the specification hash, name and axes recorded with the Phase 1 code (golden tests). The property `axes` splits a model into mean, prior, volatility, lags, deterministic terms and outliers, and each axis label encodes every non-default setting, so "differs along exactly one axis" is a property that `compare()` checks (Section 6.6); the Mode II prior label `scaling=conjugate` keeps a Mode II model from being paired with a Mode I model along the volatility axis. `import rai_bvar` imports no JAX (a subprocess test checks this); the MCMC engine is loaded at the first MCMC fit.

4.3 The analytic engine

The conjugate engine uses NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020). One QR decomposition of the stacked Minnesota dummy rows, SOC/SUR rows and data gives the posterior; $(R^\top R)^{-1}$ is never formed, and draws use the Bartlett decomposition. Empirical Bayes maximises the log marginal likelihood plus the log hyperprior by L-BFGS-B, and the hierarchical option is the random-walk Metropolis–Hastings scheme of Giannone et al. (2015). The engine also computes recursive impulse responses and the forecast-error variance decomposition.

4.4 The MCMC engine in six ideas

1. *The static/traced split is the compilation cache key.* Choices that fix a shape or a branch live in a frozen, hashable `StaticConfig`; data, priors, hyperparameters, seeds and the sample length are traced. Time is padded, by default to $T_{\text{pad}} = 32\lceil T/32 \rceil$, with zero-weight rows, so backtest windows in one bucket share compiled programs; a test of two such windows sees no recompilation.

2. *Equations and chains are* ***`vmap`*** *axes.* Given the volatilities the structural equations are conditionally independent, so kernels are written for one equation and batched, and the batched phase is an exact Gibbs block. Chains run under `vmap` (default), `lax.map` or `shard_map`.

3. *The Gibbs order is data.* A `Plan` of `Step`s with declared read and write sets is printed by `fit.plan.describe()` (Figure C.1). Invariants are checked for every composition, including a tree diff showing that each step writes only what it declares. The mixture indicators are local to the volatility step, so nothing is drawn between them and the log volatilities (Del Negro and Primiceri, 2015).

4. *Kernels are pure and take injected noise:* `sample_core(draw_noise(key, ...), ...)`. Every exact draw exposes its conditional and every Metropolis–Hastings step its log acceptance ratio; the identity tests of Section 8 use both, and the failure replay evaluates the conditionals.

5. *Randomness is counter-based* (Salmon et al., 2011). Keys are `fold_in` chains over stream, data salt, chain, absolute sweep, block, equation and sub-draw, with unsigned 32-bit counters and no key in any loop carry, so a draw depends on its position in the chain, not on how sweeps are grouped into compiled calls.

6. *Failures are values.* Nothing raises inside compiled code and no jitter is added. Each chain keeps its first failure as (code, *it*) (Algorithm 1); the host replays the failing chunk deterministically, evaluates the failing block's exposed conditional, and raises `MCMCNumericalError` with the block, equation, iteration and diagnostics. No chain is dropped. Section 9 describes such failures.

4.5 Settings and runtime

`MCMCSettings` is part of the specification: by default 4 chains after a burn-in of 1000, with 10,000 kept draws per chain for a composition with stochastic volatility and 2000 otherwise, and convergence thresholds $\hat{R} < 1.01$ and bulk ESS > 400 (Section 6.6); an SV fit with fewer kept draws warns (Section 12). `Runtime` is not: it holds the device count (default 1), the kept draws per compiled call, a memory budget checked only before sampling, a memory-mapped store and the replay option. The backtest never passes a `Runtime`, so cached scores do not depend on the machine's device count. An `MCMCFit` holds the draws as an xarray `DataTree` (Hoyer and Hamman, 2017), diagnostics, a `summary()` (prior mode, hyperparameters, implied reduced-form prior moments, ordering), the `converged` flag, `forecast()`, `irf()` and the final state for warm starts.

4.6 Reproducibility, stated with its limits

Table G.1 in Appendix G gives each guarantee with its scope and test. Bit-for-bit invariance to chunk and burn-in piece size, aligned thinning, plan fusion, `vmap` over chains versus a loop, and the storage backend is tested with the runner's test doubles ("toys"), and with the production kernels for chain-count prefixes under `chains="map"`. With the production kernels, chain-count prefixes under `vmap` (SV and CCCM kernels) and `vmap` versus `shard_map` over four devices (structural SV) are equal only to rounding (relative tolerance 10^{-9} over a few burn-in sweeps), because XLA vectorises reductions differently for different batch sizes; in-loop and post-hoc forecasts agree to 10^{-12}. Chunk invariance with the production kernels follows from the key layout but is not tested, and nothing is guaranteed across platforms, JAX versions or PRNG implementations.

4.7 Extension protocol

New components implement `MeanBlock`, `VolBlock`, `CoefPrior` or `GlobalStep` and register an independent reference implementation, from which the validation package builds their per-block and per-composition Geweke configurations. That later components will plug in without editing the sweep, runner or forecast core is a design intention, not yet tested.

5 The MCMC samplers

We claim no new sampler: each block follows published work, credited where it is used, and what was derived in our design notes is marked as such. The full pseudocode is in Appendix C. We write $\lambda_{i,t}^{\text{eff}} = e^{h_{i,t}} o_{i,t}^2 \tau_{i,t}$ for the effective variance of structural equation i ($\lambda_i o_{i,t}^2 \tau_{i,t}$ under constant volatility); sums over t run over observed rows (padded rows have weight zero).

5.1 One sweep

In structural form each equation, under `vmap`, draws (δ_i, a_i) jointly from one Gaussian conditional, recomputes its residual and runs its volatility block atomically (Algorithm 1). In that block the transitory steps ($o \to p \to \nu \to \tau$) run before the persistent core, in which the SV indicators are drawn just before h. In reduced form a pre-phase step draws $\Pi \mid A, \lambda$ (Section 5.3), then each

equation draws $a_j \mid \Pi, \lambda_j$. The coefficient steps condition on the exact Gaussian likelihood given λ^{eff} and are marginal of the indicators, as Del Negro and Primiceri (2015) require; the transitory steps with τ integrated out are partially collapsed Gibbs blocks (van Dyk and Park, 2008).

Algorithm 1 One Gibbs sweep of one chain (compact form).

Input: chain key, absolute sweep index it (unsigned 32-bit), state
1: **if** reduced form **then** $\Pi \leftarrow$ CCCM scan over $\pi_j \mid \pi_{-j}, A, \lambda^{\text{eff}}$ ▷ Section 5.3
2: **for all** equations i **in parallel** (`vmap`); keys `fold_in`(it, block, i, sub-draw) **do**
3: structural: $(\delta_i, a_i) \sim N(K_i^{-1} b_i, K_i^{-1})$; reduced: $a_i \mid \Pi, \lambda_{i,\cdot}^{\text{eff}}$ by QR ▷ Section 5.2
4: $e_{i,t} \leftarrow$ structural residual; $o_i \to p_i \to \nu_i \to \tau_i$ given e_i and the persistent variance ▷ Section 5.6
5: on $\log \tilde{e}_{i,t}^2 = \log e_{i,t}^2 - 2 \log o_{i,t} - \log \tau_{i,t}$: Constant $\lambda_i \sim$ IG (11); SV $s_i \to h_i \to \theta_i^{\text{SV}}$ (Algorithm C.7)
6: **end for**
7: keep the first failure (code, it), code $= 1 + 1000 \cdot \text{block} + \text{equation}$

5.2 The structural coefficient draw

In the δ-coordinates of Section 3, every equation regresses on one shared design $F = [X \mid XM_{<n} - Y_{<n}]$; the inactive slots a_{ij}, $j \geq i$, are padded so that the active draw stays exact (Algorithm C.2). With the weighted Gram $F^\top \Lambda_i^{-1} F$, $\Lambda_i = \text{diag}(\lambda_{i,\cdot}^{\text{eff}})$ in the precision K (Appendix C.3), the draw is a Jacobi-scaled Cholesky precision draw,

$$\theta_i = s \odot L^{-\top}\big(L^{-1}(s \odot b) + z\big), \qquad LL^\top = \text{diag}(s)\, K\, \text{diag}(s), \qquad s = \text{diag}(K)^{-1/2}, \tag{7}$$

with $z \sim N(0, I)$, so $\theta_i \sim N(K^{-1}b, K^{-1})$ exactly. Under constant volatility without a transitory scale a deterministic rule picks one of three exact fast paths, `eigen`, `fixed_factor` or `cholesky_const` (Appendix C.3). The paths share the target; their tests need a conditioning-aware tolerance (Section 8). Where a_i is drawn on its own, as in the reduced form, a Householder QR of the whitened design gives the same draw as (7) for the same z without forming the precision (Algorithm C.4); Section 9 shows why.

5.3 The corrected reduced-form step

The columns π_j of Π are drawn from their exact full conditionals (Carriero et al., 2022):

$$K_j = V_j^{-1} + X^\top \text{diag}(W_{\cdot j})\, X, \qquad b_j = V_j^{-1} m_j + X^\top\big(g_{\cdot j} + W_{\cdot j} \odot X\pi_j\big), \tag{8}$$

where $W_{tj} = (\Sigma_t^{-1})_{jj} = \sum_{i \geq j} A_{ij}^2 / \lambda_{i,t}^{\text{eff}}$ (with $A_{jj} = 1$) and $g_{tj} = (\Sigma_t^{-1} u_t)_j$, with u_t the current reduced-form residual, so A is never inverted. The scan is block Gauss–Seidel on the joint precision of $\text{vec}\,\Pi$ (Fox and Parker, 2017). The earlier triangular step of Carriero et al. (2019) targets the wrong posterior (Bognanni, 2022; Appendix B.8); it survives as mutant M1 (Section 8.8).

5.4 The stochastic-volatility core

The SV core (Algorithm C.7) works on $y_t^* = \log(\tilde{e}_t^2 + c)$ with, by default, the relative offset $c = 10^{-12}\psi$ (an absolute offset can be set). The $\log \chi_1^2$ density is approximated by the ten-component mixture of Omori et al. (2007), with component means m_j and variances v_j (the seven-component mixture of Kim et al. (1998) is an option), and the indicators are drawn by inverse CDF. The path $(h_0, \ldots, h_T)$ is drawn in one block from its tridiagonal Gaussian conditional (Rue, 2001; Chan and Jeliazkov, 2009; McCausland et al., 2011), the "all without a loop" (AWOL) sampler in the

terminology of Kastner and Frühwirth-Schnatter (2014), in non-centred scaled form $\tilde{Q} = \sigma^2 Q$. By default (`exact=True`) an exact Metropolis–Hastings correction removes the mixture approximation: with g the mixture density and

$$w(h) = \sum_t \big[\log N(\tilde{e}_t; 0, e^{h_t}) - \log g(y_t^* - h_t)\big], \tag{9}$$

the path proposal is accepted with probability $\min\{1, e^{w(h')-w(h)}\}$ (Del Negro and Primiceri, 2015, Algorithm 3); this acceptance probability has the form of eq. (11) of Stroud et al. (2003). The chain is then exact for any mixture table and offset $c \geq 0$, which the validation exploits with a deliberately coarse mixture (Section 8). The centred steps need no correction (Appendix B.4). The non-centred step (on by default, `asis=True`) interweaves (Yu and Meng, 2011; Kastner and Frühwirth-Schnatter, 2014); in exact mode it is accepted or rejected once, with a ratio combining the auxiliary ratio and the change in w, which is the Metropolis–Hastings ratio under the exact target for the auxiliary step's proposal (Appendix B.4); Hosszejni and Kastner (2019) instead correct the h step and update the parameters without the mixture approximation, in separate steps, and we do not claim the combination as new. A negative σ' is folded together with $\tilde{h}$, which preserves the non-centred posterior by the sign symmetry used by Frühwirth-Schnatter and Wagner (2010); the cold start is described after Algorithm C.7.

5.5 The random-walk path as a path Laplacian

For $h_t = h_{t-1} + \sigma\eta_t$, $h_0 \sim N(m_0, V_0)$, the scaled precision is $\tilde{Q} = \sigma^2 Q = \mathrm{Lap}_w + \mathrm{diag}(\delta)$: a path-graph Laplacian with unit edge weights w_t (zero after the last live state) plus $\delta_0 = \sigma^2/V_0$ and $\delta_t = \sigma^2/v_{s_t}$. The sampler (Algorithm C.8) carries each pivot as its excess q_t over the next edge:

$$q_0 = \delta_0, \quad q_t = \delta_t + \frac{w_{t-1}\, q_{t-1}}{w_{t-1} + q_{t-1}}, \quad p_t = q_t + w_t, \quad \log|\tilde{Q}| = \sum_t \big[\log w_t + \mathrm{log1p}(q_t/w_t)\big], \tag{10}$$

with $\log q_t$ where no next edge exists. The p_t are the Cholesky pivots of $\tilde{Q}$ (Appendix B.9), but every term is non-negative, so nothing cancels; assembling $\tilde{Q}$ first rounds away each σ^2/v_{s_t} below the spacing of its diagonal, and the level-direction pivot becomes rounding noise (Section 9). The recursion is the subtraction-free (GTH-type) elimination for diagonally dominant M-matrices parametrised by off-diagonal entries and row sums (Grassmann et al., 1985; O'Cinneide, 1996; Alfa et al., 2002; Ye, 2008), as described by Demmel et al. (2008, Section 2.3.4), applied to a tridiagonal matrix whose row sums are the δ_t; we claim its application to SV path sampling, not the algorithm.

5.6 Transitory components

The outlier scale $o_t \in \{1, \ldots, K\}$ ($K = 20$ by default) has logits $\log \varpi_k - \log k - z_t^2/(2k^2)$, with $z_t^2 = e_t^2/\lambda_t^{\mathrm{pers}}$, $\varpi_1 = 1-p$ and $\varpi_{k>1} = p/(K-1)$ (Carriero et al., 2024; after the continuous mixture of Stock and Watson, 2016), and $p \mid o$ is beta (Table C.2). For Student-t errors (Jacquier et al., 2004), ν is drawn on its grid ($\{3, \ldots, 40\}$ by default) with τ integrated out, then τ_t (Table C.2 gives all conditionals, including the collapsed outlier step used with both components). The log-gamma variates come from a vectorised Marsaglia and Tsang (2000) sampler, not from `jax.random.loggamma`, because under `vmap` a rejection loop runs until the last lane accepts; the sampler stays exact (Appendix C).

5.7 The constant-volatility core

Under constant volatility

$$\lambda_i \mid \cdot \sim \mathrm{IG}\Big(c_i + \tfrac{T}{2} + \tfrac{n_{\text{active}}}{2},\; C_i + \tfrac{1}{2}\textstyle\sum_t \tilde{e}_{i,t}^2 + \text{quad}\Big), \tag{11}$$

where $(n_{\text{active}}, \text{quad})$, the number of coefficients whose prior scales with λ_i and half their prior quadratic form, is passed from the coefficient step of the same equation: zero in Mode I, all $k+\iota-1$ coefficients in Mode II, and the $\iota-1$ contemporaneous ones under `iw_equivalent` (ι the one-based equation index). This is the only interaction between the prior and the volatility block that the plan allows (for the SOC/SUR dummy rows in Mode II see Section 9.3).

6 Forecasts, the predictive contract and the verdict engine

Both engines return a `ForecastResult`, and everything downstream of a fit (scores, backtests, tests and verdicts) reads only that object. None of the scores, tests or estimators below is new; the package adds one contract that makes them available, with their Monte Carlo error, to every composition of Section 3.

6.1 Joint path simulation

For each selected posterior draw $s = 1, \ldots, S$ and forecast origin T, the MCMC engine first simulates the future log variances, including future outlier and Student-t scales, which give the effective variances $\lambda^{\text{eff}}_{s,T+h}$; it then runs over $h = 1, \ldots, H$, building $x_{s,T+h}$ from the simulated path:

$$m_{s,h} = B_s' \, x_{s,T+h}, \qquad L_{s,h} = A_s^{-1} \operatorname{diag}\big(\lambda^{\text{eff}}_{s,T+h}\big)^{1/2}, \qquad y_{s,T+h} = m_{s,h} + L_{s,h}\, \varepsilon_{s,h}, \tag{12}$$

with $\varepsilon_{s,h} \sim N(0, I_n)$ and $B_s' = A_s^{-1}\Gamma_s$ in structural form ($B_s = \Pi_s$ in reduced form). Because A_s is unit lower triangular, $L_{s,h}$ is the Cholesky factor of the conditional covariance at no extra cost, and $y_{T+h} \mid \theta_s, y_{s,T+1:T+h-1}, \lambda_{s,T+h} \sim N(m_{s,h}, L_{s,h}L_{s,h}')$ holds exactly. The analytic engine uses $L = \text{chol}(\Sigma)$ for its NIW posterior draws (Appendix C.7, which also covers impulse responses and variance decompositions).

6.2 The predictive contract

A `ForecastResult` wraps an `xarray` dataset (Hoyer and Hamman, 2017) with the fields `paths` (joint paths on the transformed scale), `paths_level` (their pathwise inverse, never applied to quantiles), `cond_mean` ($m_{s,h}$) and `cond_chol` ($L_{s,h}$), and attributes (Table C.3). Its invariants, that $L_{s,h}^{-1}$(`paths` − `cond_mean`) is iid $N(0, I)$, that each draw is one joint path across horizons and variables and that stacking is chain-major, are tested for every registered benchmark and analytic-engine model and for 13 MCMC compositions, eight structural and five reduced-form, among them reduced form × SV(rw), the primary arm of the replication in Section 11. Reduced form × SV(t) and SV(outliers, t), and the replication's variant SV(rw, t), are not among them (Section 8.7).

6.3 The Rao–Blackwellised log predictive density

The default estimator (SPEC §7) is the mixture of the conditional Gaussian densities,

$$\log \hat{p}_h(y) = \text{logsumexp}_s \log N\big(y;\, m_{s,h},\, L_{s,h}L_{s,h}'\big) - \log S, \tag{13}$$

a standard mixture log score (Geweke and Amisano, 2010; Krüger et al., 2021). A leading block $\{1, \ldots, j\}$ of the ordering uses $L_{s,h}[1{:}j, 1{:}j]$, and any other subset factors the selected rows of $L_{s,h}$ by QR, so marginal scores are exact. With weights $w_s = \exp(\log p_s - \max_r \log p_r)$, the reported Monte Carlo standard error is the delta-method value

$$\widehat{\mathrm{se}}\big(\log \hat{p}_h\big) = \frac{\mathrm{sd}(w)}{\sqrt{S}\,\bar{w}} \times \sqrt{S/\widehat{\mathrm{ESS}}_{\mathrm{ac}}(w)}, \tag{14}$$

with the sample standard deviation, where $\mathrm{ESS}_{\mathrm{ac}}$ is the autocorrelation effective sample size of the weights over chains and draws (MCMC forecasts only; the factor is 1 when it is undefined). Distinct from it is the Kish effective number of weights $\mathrm{ESS}_{\mathrm{K}} = (\sum_s w_s)^2 / \sum_s w_s^2$ (Kish, 1965), which measures how many draws carry the estimate.

6.4 The $h > 1$ tail problem and the path-integrated estimator

Failure. For $h > 1$ the moments in (13) are conditioned on the simulated path $y_{s,T+1:T+h-1}$ of draw s. For an outcome far in the tail, only draws whose paths wandered towards it receive weight, in the limit one draw ($\mathrm{ESS}_{\mathrm{K}} \approx 1$). The mixture $\hat{p}_h$ is unbiased for p_h, so by Jensen's inequality its logarithm is biased low, and the bias is large exactly when the weights are heavy-tailed; (14) estimates the spread, not this bias. In the replication of Section 11 (Constant arm, unemployment, $h = 8$, origin 2008Q2), an agent review of the quick run, recorded in the repository notes, found -93.6 with 1000 forecast draws, -46 to -63 with 8000 and -36 with 64,000, against -19.1 for the path-integrated estimator below on the same 1000 draws (itself a Monte Carlo estimate), while the MCSE stayed near 1 nat, the ceiling of the formula: before the MCMC adjustment the delta-method MCSE cannot exceed one nat, and it reaches one when the weight sits on a single draw (Appendix B.5 in Appendix B.5).

Fix (`lpd_method="integrated"`). Given θ_s and the drawn scale path $\lambda_{s,T+1:T+h}$, the intermediate outcomes can be integrated out. With the simulated shocks $u_{s,k} = y_{s,T+k} - m_{s,k}$ and the moving-average coefficients $\Phi_{s,j}$ of draw s ($\Phi_{s,0} = I$; Appendix B.5),

$$\begin{gathered} y_{T+h} \mid \theta_s, \lambda_{s,T+1:T+h} \sim N\big(\tilde{m}_{s,h},\, M_{s,h} M_{s,h}'\big), \qquad \tilde{m}_{s,h} = m_{s,h} - \sum_{k=1}^{h-1} \Phi_{s,h-k}\, u_{s,k}, \\ M_{s,h} = \big[\Phi_{s,h-1} L_{s,1} \mid \Phi_{s,h-2} L_{s,2} \mid \cdots \mid \Phi_{s,0} L_{s,h}\big] \quad (n \times hn). \end{gathered} \tag{15}$$

The estimator uses these moments in (13) and (14) without forming $M_{s,h}M_{s,h}'$ or drawing new random numbers, so the weights no longer depend on where a draw's path went; at $h = 1$ the conditioned code runs, bitwise. With one parameter draw and constant volatility it is exact, matching a companion-form oracle at $h = 2, \ldots, 8$ to 10^{-10} (Figure 2 and Appendix B.5). The integrated estimator is the mixture-of-parameters, or Rao–Blackwellised, estimator (Krüger et al., 2021, Section 2.1) with each draw's simulated volatility path in the conditioning set, as Krüger et al. (2021, Section 5.2) condition on a draw's latent states. With constant volatility it is the Monte Carlo average over posterior draws of a linear Gaussian model's exact h-step predictive likelihood (Warne et al., 2017), and Hiraki et al. (2026, Appendix D) compute multi-step predictive moments in the same way, from simulated log-volatility paths and a companion form, for a factor SV-in-mean VAR; we claim the implementation, not the estimator.

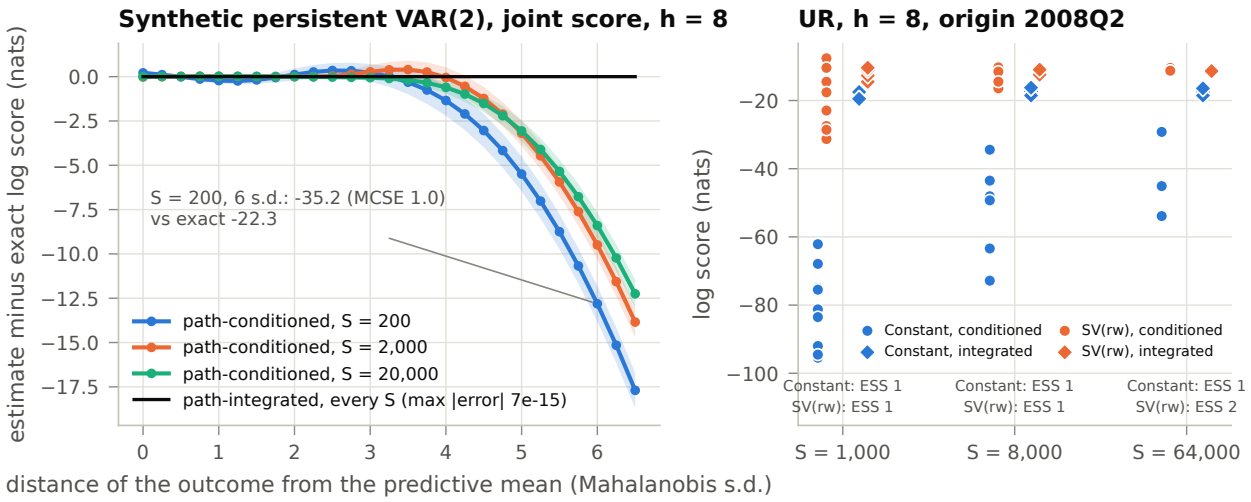


Figure 2: Path-conditioned (the specification's default) against path-integrated log score at $h > 1$. Left: synthetic persistent VAR(2), $h = 8$, one parameter draw; error against the outcome's Mahalanobis distance. Right: Constant (and SV(rw)), unemployment, $h = 8$, 2008Q2, at the acceptance MCMC budget, with the spread across seeds and $\mathrm{ESS_K}$.

Residual error. The integrated estimator is still an average over draws whose weights concentrate in the tails: at the same cell, an agent review recorded in the repository notes found $\mathrm{ESS_K}$ of 2–6 for the Constant arm with 1000 forecast draws, so the pre-registered acceptance run uses 8000 (Appendix F.2). The estimator is opt-in and covers MCMC forecasts of linear or zero means (scope in Appendix B.5).

6.5 Scores, tests and backtests

Scores are computed per origin, horizon and model on the transformed scale, with the strictly proper (Gneiting and Raftery, 2007) joint or per-variable log score as the primary score (the others are listed in Appendix C.8). Loss differences are tested with the Diebold–Mariano statistic (Diebold and Mariano, 1995), corrected as in Harvey et al. (1997), against Student-t_{P-1} quantiles (Equation (C.2)); the long-run variance uses a rectangular kernel with $h-1$ autocovariances, with Bartlett weights only if that estimate is not positive. The correction was derived for squared-error-type losses, and we apply it to log-score losses without a separate argument. Following Giacomini and White (2006), its p-values count as valid only with rolling or fixed windows and are labelled descriptive otherwise, since with expanding windows and nested models the statistic is not asymptotically standard (Clark and McCracken, 2001). The second route is the model confidence set (Hansen et al., 2011) on the joint log-score losses. Under expanding windows a "better" verdict can therefore only come through the MCS route (Section 6.6), which results should report. `backtest()` runs the origins from a start to an end date with an `Expanding` window (tests descriptive) or a `Rolling` one (tests valid), and at each origin refits the model on `window.select(data.upto(origin))`, so every data-dependent prior quantity is recomputed from the data available then (R9; seeding and caching in Appendix G.1).

6.6 `compare()` and the verdict

`compare()` backtests every model on identical origins, windows and outcomes and evaluates every statistic on the origins where all models have a finite joint log score. For a model M and a reference

B at horizon h, let ΔLPD be the summed difference of their log scores over those origins, $\mathcal{D}(M, B)$ the set of declared axes (mean, prior, volatility, …) on which their axis labels differ, p_{DM} the one-sided DM–HLN p-value and $\mathcal{M}_{25\%}$ the 75% model confidence set ($\alpha = 0.25$), the models whose MCS p-value is at least 0.25. The default rule is

$$\boxed{\begin{aligned} \text{better} \iff & \big|\mathcal{D}(M,B)\big| = 1 \ \wedge \ \Delta\text{LPD} > 0 \ \wedge \ |\Delta\text{LPD}| > 3\,\text{MCSE} \\ & \wedge \ \big[(\text{rolling} \wedge p_{\mathrm{DM}} < 0.10) \ \vee \ (M \in \mathcal{M}_{25\%} \wedge B \notin \mathcal{M}_{25\%})\big], \\ & \text{with MCSE} = \Big(\textstyle\sum_t \big(\widehat{\mathrm{se}}^2_{M,t} + \widehat{\mathrm{se}}^2_{B,t}\big)\Big)^{1/2}, \end{aligned}} \tag{16}$$

"worse" is its mirror image, and anything else is "inconclusive" (Figure C.2 in Appendix C.8 draws the rule as a decision flow). With two models the $T_{\max}$ MCS reduces to a two-sided block-bootstrap test of equal mean loss at level 0.25, about 12.5% one-sided once $\Delta\text{LPD} > 0$, looser than the 10% DM route, and its circular block length round($P^{1/3}$) (5 at $P = 106$) does not grow with h. The set is formed over every model passed to `compare()`, so a pair's verdict depends on the comparison set (Section 11.5). Under expanding windows an MCS-route verdict is no better established than the DM p-value it replaces; both are best read as descriptive. A convergence gate then forces "inconclusive" if either model converged at fewer than 95% of the origins. A fit is converged when no chain raised an error, at least two chains ran, any empirical-Bayes step converged, and every primary quantity (log-likelihood, own-first-lag coefficients and volatility parameters; Appendix D.9) has a rank-normalised split $\hat{R}$ below 1.01 and a bulk ESS above 400 (Vehtari et al., 2021). Pairs that differ along more than one axis, or whose specs differ without a declared axis, are "not comparable". `verdict(axis="vol")` and `grid()` build such one-axis pairs for a volatility grid (Appendix C.8). Axis labels are declared, not derived: against `Constant("iw_equivalent")` an SV arm also changes the prior of the contemporaneous coefficients a_{ij} (the Mode I prior instead of the λ_i-scaled one of Chan, 2022), which against `Constant("ig")` it would not. Each ingredient is standard and their combination is the package's; the rule's size and power have not been studied, and the quadrature MCSE treats the two models' errors as independent although both use the same per-origin forecast seed (Section 12).

6.7 Ordering sensitivity

Because a composition with a triangular factorisation generally depends on the variable ordering (Arias et al., 2023; Chan et al., 2024), `rb.ordering_sensitivity()` re-evaluates a model under all, Latin-square-balanced or random permutations and flags a metric, such as the range of log scores over orderings, only above its threshold and with a signal-to-noise ratio above 3 against the noise floor $\sigma_{\mathrm{MC}} = \max(\text{sd across seeds}, \text{MCSE})$ (Appendix C.8). The only order-invariant MCMC composition in Phase 2, reduced form × `Constant("iw_equivalent")` (Chan, 2022), is the replication's Constant arm and so a negative control for the check (Section 11; Section 12 for the first full run).

7 Using rai-bvar: a walk-through

Two short scripts show the user interface on simulated data, a stationary three-variable quarterly VAR(1) for output growth, inflation and an interest rate, stored as raw levels with FRED-style transformation codes, so they run without network access. The listings are verbatim excerpts with the scripts' own line numbers; Appendix G.2 gives the scripts and their output in full (the MCMC output is bitwise reproducible only on the same platform and package versions; Section 4).

7.1 Data

`VARData.from_frame(raw, tcodes={...})` takes raw levels indexed by dates or periods, rejects malformed input and uses the column order as the recursive ordering of every triangular model; `upto`, `tail`, `select` and `realized` give the data available at an origin, rolling windows, reorderings and future outcomes (Appendix A.2). The transformation codes 1–7 are those of FRED-MD and FRED-QD (McCracken and Ng, 2016, 2021), each with a pathwise inverse, and they set the own-first-lag prior mean (1 for levels and log levels, 0 for differences). `rb.datasets.load("fred-qd", vintage="2026-09")` downloads a pinned vintage of FRED-QD (McCracken and Ng, 2021), checks its SHA-256 checksum and caches it; the files are not bundled. The named FRED-QD variable sets load with the package's own codes by default (`tcodes="rai"`), which differ from the benchmark codes (McCracken and Ng, 2021, Table A1) for price indices (5, log first differences, instead of 6), for interest rates and the unemployment rate (1, levels, instead of 2) and for housing starts (4 instead of 5); `tcodes="official"` uses the codes in the file.

7.2 An analytic BVAR, a forecast and a comparison

Listing 1: `ex_a.py` (excerpt): an analytic BVAR, a forecast and a rolling-window comparison.

```
29  data = rb.VARData.from_frame(raw, tcodes={"gdp": 5, "prices": 5, "rate": 1})
31  print("prior means of own first lag:", data.prior_means())
34  bvar = rb.BVAR(mean=rb.Linear(prior=rb.Minnesota(soc=1.0, sur=1.0)), lags=2)
36  fit = bvar.fit(data)
38  fc = fit.forecast(horizon=4, draws=2000, seed=0)
43  # Pseudo out-of-sample comparison on rolling windows (valid DM/AG p-values).
44  res = rb.compare(
45      {"RW": rb.RandomWalk(), "OLS-VAR(2)": rb.OLSVAR(lags=2), "BVAR(2)": bvar},
46      data,
47      origins=rb.Origins(start="2010Q1", end="2019Q3"),
48      window=rb.Rolling(size=120),
49      horizons=[1, 4],
50      baseline="RW",
51  )
57  v = res.verdict()
```

Output of Listing 1, lines 28–29 (verbatim; full output: Listing G.2)

```
OLS-VAR(2) vs RW h=1: ΔLPD=+14.45 (MC s.e. 0.10); DM p=0.014 (valid: rolling window); MCS25: model in,
    ↪ reference out
BVAR(2) vs RW h=1: differs along ['deterministic', 'lags', 'mean', 'model', 'outliers', 'prior', 'vol']; a
    ↪ verdict needs exactly one axis (SPEC R2)
```

The log difference (code 5) consumes the first period, and the own-first-lag prior means follow the codes: 0, 0 and 1. The BVAR of line 34 adds SOC and SUR dummy observations to the Minnesota prior and keeps the default volatility, so it resolves to the conjugate NIW model on the analytic engine, with empirical Bayes in the hierarchy of Giannone et al. (2015) for the tightness and the dummy weights. Over 39 rolling-window origins at $h = 1$, the OLS VAR(2) is "better" than the random walk on the `model` axis: ΔLPD +14.45 (MCSE 0.10), a one-sided DM–HLN p-value of 0.014 (the table prints the two-sided 0.029; verdicts use one-sided values) and MCS separation. The comment in line 43 says "DM/AG"; the test implemented is the Diebold–Mariano test with the Harvey–Leybourne–Newbold correction applied to log-score losses (Section 6.5). The BVAR has the larger ΔLPD (+15.37), yet it is "not comparable" with the random walk: the two differ along seven axes, and a verdict needs exactly one (R2).

7.3 The same model family on the MCMC engine

Listing 2: `ex_b.py` (excerpt, same data): SV on the MCMC engine and a volatility-axis comparison.

```
40 mc = rb.MCMCSettings(chains=4, draws=1000, burn=1000)
41 sv = rb.BVAR(vol=rb.SV(), lags=2, mcmc=mc)
43 fit = sv.fit(data.upto("2019Q3"))
44 s = fit.summary()
47 fc = fit.forecast(horizon=4, draws=2000, seed=0)
50 print("joint LPD h=1 and MCSE:", fc.log_pred_density(y_obs, horizon=1, return_mcse=True))
57 models = {
58     "Linear x Constant": rb.BVAR(vol=rb.Constant("ig"), lags=2, mcmc=mc),
59     "Linear x SV": sv,
60 }
63 res = rb.compare(
64     models, data,
65     origins=rb.Origins(start="2017Q1", end="2019Q3"),
66     window=rb.Rolling(size=120),
67     horizons=[1],
68     draws=1000,
69     baseline="Linear x Constant",
70 )
72 v = res.verdict(axis="vol")
```

Output of Listing 2, line 39 (verbatim; full output: Listing G.4)

```
Δ
LPD=-0.11 (MC s.e. 0.10, not precise enough); DM p=0.421 (valid: rolling window); MCS25: model in, reference
    ↪ in; converged at only 0% of origins
```

With `vol=SV()`, the default mean `Linear(prior=Minnesota())` moves to the MCMC engine, in structural form with the Mode I prior; there is no engine argument to set. Its Minnesota hyperparameters are an empirical-Bayes plug-in from the conjugate twin on the same window (for SV not a type-II maximum-likelihood estimate), so every volatility arm on that window receives the same mean prior. The listing sets a deliberately small budget of 4×1000 kept draws, a tenth of the SV default, for which the fit warns; on this homoskedastic DGP the SV parameters ϕ_i and σ_i mix slowly, and the fit is flagged as not converged. The forecast container and its log-density interface are those of the analytic case (R3). In the final comparison, whose arms are structural Mode I models that differ only in volatility, SV has a ΔLPD of -0.11 over 11 rolling-window origins with an MCSE of 0.10, too imprecise for any verdict (one-sided DM p-value 0.421 for "worse"; the reasons line prints the smaller of the two one-sided values), and it converged at none of them, so the convergence gate forces "inconclusive" whatever the other evidence says. This is intended: the gate refuses to certify a comparison with an unconverged fit (Section 11 shows how it decided a replication criterion and how a fit-level diagnostic set the MCMC budget).

7.4 Inspecting fits and failures

An MCMC fit exposes its Gibbs plan (`fit.plan.describe()`; Figure C.1), diagnostics, `converged` flag, `warnings` and stage timings. A numerical failure inside a kernel raises `MCMCNumericalError`, which names the block, equation, chain and sweep and carries the diagnostics of a deterministic replay (Sections 4.4 and 9).

8 Validating the samplers

The specification of rai-bvar lets no sampler into the code base until it passes Geweke and simulation-based calibration (SBC) tests. The rule answers the first threat of Section 1, a published sampler

that targeted the wrong posterior (Carriero et al., 2019; Bognanni, 2022; Carriero et al., 2022). A passing suite is informative only if one knows what it would have caught, so we inject known errors into the production samplers and check that each is caught (Section 8.8).

Seven tiers carry the evidence: identities (U), block-level tests (B), Geweke tests (G), SBC (S), analytic twins (A), parity with independent implementations (P) and mutation testing (M) (rules, cadence, test counts and costs in Table D.1 and Appendix D.10). Tier U, the per-PR Geweke size G-small and kernel-level Tier B run on every pull request (PR); the continuous-integration job named "nightly" runs weekly and carries the rest.

8.1 Harness principles

Two implementations. Everything a test treats as the truth (prior samplers, simulators of observables and the log joint density) comes from a work package whose owner wrote no kernel code, except the SOC/SUR dummy-row reference, which the kernels' owners added later from the model. The owners were separate AI agent instances, not people: the rule secures separate code paths, not independent judgement (Appendix J). No reference imports a kernel module; a subprocess test enforces this. The algebra differs too: references are joint densities on the natural parameters and kernels conditional formulas, and for the dummy rows the reference works in covariance form ($m \times m$ systems, Matheron's rule; Journel and Huijbregts, 1978; Chilès and Delfiner, 2012; Wilson et al., 2020) and the kernels in precision form ($q \times q$ Gram updates). Mixture constants are transcribed twice and must agree on every printed digit. An oracle shares any convention we copied into it, whereas Geweke and SBC tests share only our prior and simulator, so the two are complementary (the rule's origin: a design-review erratum, Appendix I).

Production code; failures fail. A test chain's step is the production `prepare(data, prior)` followed by one production sweep, never a copy, under proper, tight test priors in which every term matters (Appendix D.4). A non-finite test function gives $p = 0$ and is counted, and a failed SBC replication fails the test; the only filter acts on the simulated data alone ($\tilde{y}$ finite, $\max|\tilde{y}| < 10^3$) and leaves ranks uniform, because uniformity holds conditionally on the data (Modrák et al., 2025, Sections 3.1–3.2, Theorems 4 and 5).

Fixed seeds, sequential decisions. Keys derive from (seed, tier, CRC-32 of the composition name, chain), so results do not depend on how chains are vectorised. A false alarm is then deterministic, so a marginal Bonferroni p-value is settled by the sequential procedure of Gandy and Scott (2021, Algorithm 3), which reruns a fresh, four times larger experiment at family level α (Appendix D.3); per PR ($\alpha = 10^{-3}$, three stages) this costs about 30% extra effort under the null. Holm's procedure (Holm, 1979) only names the failing functions.

8.2 Tier U: deterministic identities

For a block b with implemented conditional $q_b(\theta_b \mid \theta_{-b}, y)$ and two states x_0, x_1 that differ only in θ_b, the identity (Grosse and Duvenaud, 2014) $\log q_b(x_1) - \log q_b(x_0) = \log p(x_1, y) - \log p(x_0, y)$ is checked against the independent log joint density; for a Metropolis–Hastings step, the computed log acceptance ratio before truncation must equal $\log \pi(x') + \log q(x \mid x') - \log \pi(x) - \log q(x' \mid x)$, including the mixture-correction weights (9), the combined accept/reject of the σ^2 step and the ASIS Jacobians. Both hold to $|\text{LHS} - \text{RHS}|/(1 + |\text{RHS}|) \leq 10^{-8}$. Dense oracles with injected noise check the linear algebra at 10^{-10} to 10^{-13}; as two exact samplers of one Gaussian agree

only to $O(\epsilon_{\text{mach}}\,\text{cond}(K))$, path-against-path tests use envelopes that grow with $\text{cond}(K)$, and infrastructure checks complete the tier (Appendix D.2).

8.3 Tier G: Geweke tests

Following Geweke (2004), the marginal-conditional simulator draws M_1 independent pairs $\theta \sim p(\theta)$, $y \sim p(y \mid \theta)$, and the successive-conditional simulator runs C independent chains of M_c steps ($y \sim p(y \mid \theta)$, then one production sweep), each started at a prior draw with no burn-in. With $\bar{g}_{\text{MC}}$ and $\hat{\sigma}_g^2$ the marginal-conditional mean and variance of a test function g, and $\bar{g}_{\text{SC}}$ and s_C^2 the mean and variance of the C chain means,

$$z_g = \frac{\bar{g}_{\text{MC}} - \bar{g}_{\text{SC}}}{\sqrt{\hat{\sigma}_g^2/M_1 + s_C^2/C}}, \qquad p_g = 2\Pr\big(t_{C-1} > |z_g|\big), \tag{17}$$

and the test fails iff $\min_g p_g \le \alpha/G$ over the G functions. A prior-started chain is stationary from its first sweep under the null, so the size needs only that the sweep leave the posterior invariant; ergodicity affects power. Talts et al. (2018, Section 3) note that the z-scores of Geweke (2004), computed from a single successive-conditional chain, are meaningful only once that chain has converged; here, with C independent prior-started chains and the variance of their means in (17), slow mixing lowers power. Test priors follow the rule $r = \text{Var}_{\text{prior}}/\text{Var}_{\text{post}} \le 5$, derived for the scalar conjugate model only (Appendix B and Section 12); test functions must have finite variance, which excludes raw data moments under the SV test prior (Appendix D.4), and include the exact log-likelihood (2). G-small runs per PR and the larger G-full weekly, at $n \le 3$ (Appendices D.3 and D.5); rows with the approximate (`exact=False`) SV sampler are recorded but do not gate. The 15 PR compositions registered before row 18 pass G-small (Table D.2; commit and machine in Appendix D.3), largest final-stage $|z|$ 3.03 against critical values of 4.10–4.50, row 201 at the fresh second stage; row 18, added later as the sixteenth PR row, is reported in Section 8.7. On the release commit all 16 pass G-small on Linux, and all 23 compositions pass G-full, row 2 at the third stage, with no non-finite test function in the twelve `exact=False` rows (Table D.3). The weekly run of 22 October had failed G-full for `reduced_sv` on one chain, the float64 failure fixed in the kernel before both full replication runs that the paper reports (Section 9.1); a committed test replays that chain, and on the Apple M5 Max the production step now succeeds. The Geweke simulator also reaches states far outside those that ordinary data produce, which is how a float64 weakness of a coefficient draw was found (Section 9.1).

8.4 Tiers S and B: simulation-based calibration and block-level tests

SBC (Cook et al., 2006; Gelman, 2017; Talts et al., 2018; Modrák et al., 2025) checks that the rank of a prior draw $\tilde{\theta}$ among posterior draws given data $\tilde{y}$ simulated from it is uniform (a variant conditional on observed data, Säilynoja et al., 2025, is related work). Per replication, a pilot chain started at an independent prior draw gives IAT_i, the largest Geyer initial-monotone estimate (Geyer, 1992) over the quantities and their tail indicators, and a fresh chain keeps L draws every $t_i = \min(\lceil 2\,\text{IAT}_i \rceil, t_{\max})$ sweeps; as t_i depends only on $\tilde{y}$ and pilot noise, ranks stay uniform. Uniformity is tested with the γ statistic of Säilynoja et al. (2022), with exact far-tail p-values from a forward recursion (Appendix D.5). $L = 99$ kept draws cannot certify a per-replication effective sample size (ESS) of $0.8L$, so that ESS is predicted from the pilot IAT, and a second gate on the kept-draw ESS pooled over replications (Kish, 1965) catches pilots that underestimate the IAT. The thinning cap is $t_{\max} = 200$ (400 for row 18), raised from the design's 50 by the ESS rule's own remedy (Appendix D.5). Besides the parameters, the ranked quantities include h at three dates,

log-likelihoods, $\log p(\theta)$, and h_{T+1} and y_{T+1} drawn through the production forecast core; such data-dependent quantities are needed, because a "posterior" equal to the prior passes SBC on every data-free quantity (Modrák et al., 2025, Section 3.4, Theorem 7). The configuration is $N = 1000$, $L = 99$, $\alpha = 10^{-2}$ for 23 compositions (rows 1–18 and 201–205) at $n \leq 3$. On the release commit all 23 pass, row 15 at the second stage (Table D.3). In Tier B, blocks that draw exactly from their conditional get block-conditional SBC, and Metropolis–Hastings, ASIS and Gibbs-scan blocks one-block invariance tests, both at Bonferroni $\alpha = 10^{-3}$ inside the sequential wrapper (Appendix D.5).

8.5 Tier A: analytic twins and exact oracles

Analytic twins and exact oracles compare the MCMC engine with closed forms, quadrature or exact draws (Appendix D.6): A1 equates the structural NIG log marginal likelihood of Chan (2022) with the analytic engine's to 10^{-10}, with and without SOC/SUR rows; A2 and A3 compare MCMC structural Mode II × `Constant("iw")` with the analytic NIW posterior at $n = 3$ and at $n = 20$, $p = 4$, $T = 300$, a size too expensive for Geweke and SBC tests; A4 checks Chan's asymmetric conjugate prior against its closed form, A5 Mode I × Constant against quadrature, A6 the exact-h, ASIS and step-order kernels on toy posteriors, and A7 the reduced-form Constant sampler against an exact system-wide coefficient draw. Negative controls give the comparisons measured power: A2 rejects one extra degree of freedom, A5 an inverse-gamma shape off by one half, and A6 and A7 deliberately wrong kernels (the uncorrected non-centred move, the step order of Primiceri, 2005, the CCM-2019 step). Apart from the A6 toys, the twins cover constant volatility only.

8.6 Tier P: parity with independent implementations

Parity compares the 5, 25, 50, 75 and 95% quantiles of m quantities by a z-statistic built from both implementations' Monte Carlo standard errors, passing iff $|z| \leq \max(4, z_{1-0.01/(2m)})$ (Appendix D.7). The analytic engine matches R BVAR 1.0.5 (Kuschnig and Vashold, 2021) and the Bank of England's Python bvar 0.3.5 (Labonne et al., 2026) in log marginal likelihood to a tested relative 10^{-8}, with recorded agreement below 10^{-11} (BVAR, exact cases) and at most 1.5×10^{-11} (bvar). On the MCMC side, stochvol 3.2.1 (Kastner, 2016; Hosszejni and Kastner, 2021) against `ZeroMean` × `SV` ($n = 1$; main and default-prior settings, four datasets) gives largest $|z|$ 1.63–2.59 in the PR tier and 1.64–3.57 in long runs, against a bound of 4.00. Our standardised-t SV sampler is checked against an exact importance-sampling posterior on a four-observation problem (largest $|z|$ 1.83), an oracle that needs no second sampler. The planned reduced-form reference, bayesianVARs (Gruber et al., 2026), could not be built in our test environment and was not run; against a substitute R sampler that draws the reduced-form coefficients jointly from their exact system-wide conditional, the largest $|z|$ are 2.76, 3.45 and 3.56 on three fixtures (bounds 4.05–4.51), so the reduced-form criterion is met against this substitute, not against bayesianVARs. There is no external VAR-level oracle for structural Mode I × SV, the default MCMC composition, nor for exact-mode SV beyond reweighted single-series runs; these rest on Tiers U, B, G and S (Table D.5).

8.7 Coverage of the compositions the replication scores

The primary arm of the replication in Section 11, reduced-form Minnesota × SV(rw), is validation row 18, `reduced_sv_rw`, registered after the acceptance run's commit in a commit that changes no sampler code (Appendix D.5), and the row's configuration equals the arm's in every component field except the shapes, the test priors and an inert ν grid. Like rows 7–10 it runs G-small per PR and G-full and SBC weekly, with its `exact=False` variant recorded, and the arm is among the 13 MCMC compositions of the contract tests and among those fitted through the public `BVAR.fit`

path. The row's largest $|z|$ is 1.98 in G-small (critical value 4.42), in a run recorded with that commit, and 1.90 in G-full (3.94) on the release commit, where the arm also passes all ten contract tests (Appendix D.5). In one-off runs outside the mutation suite its G-small rule catches three of the suite's mutants and two further errors, and a variant G-small test at $h_0 \sim \mathrm{N}(-1, 4)$, run on every PR, catches a prior variance used as a precision, which the default test prior hides (Appendix D.5). At the shared thinning cap of 200 the γ test and the pooled-ESS gate were clean, but the per-replication ESS rule held for only 987 of 997 replications, short of 99%; after that miss the row's cap was raised to 400, at which it passes. With a 4000-sweep pilot instead of the design's 1000, the per-replication rule fails at 400 too (981 of 997), while the γ test and the pooled-ESS gate stay clean. The slow quantities are the reduced-form coefficients Π, mostly intercepts, on about 1–2% of prior-drawn $T = 50$ datasets: a mixing tail, not a calibration error (Appendix D.5 and Section 12). The baseline, reduced form × `Constant("iw_equivalent")`, is row 11, and the AR-SV(0.9) arm runs the code path of row 8 with a different prior on ϕ. The Student-t variant, SV(rw, t), has no row of its own; it rests on row 18, on the Student-t component of rows 4 and 10 (standardised scale, ν on a grid) and, for its fixed $\nu = 10$ and unit scale, on Tier U checks of the transitory block.

8.8 Tier M: the mutation suite

The 18 mutants (Table 2) are injected only through production seams (Appendix D.8); a tier catches a mutant only if it fails on the mutant and passes on the unmutated control with common seeds, a per-PR test requires a confirmed catching tier for every mutant, and an uncaught mutant blocks the release. Tier U catches 14 mutants, G all 18 (M7 with a coarse mixture table, M11 and M12 under the auxiliary-mixture sampler), B the four with a B check and S the one with an S check: 37 checks. All 37 catch their mutant with a passing control, in about three minutes with a cold compilation cache (Appendix D.10), and recorded runs at earlier commits on x86-64 Linux also caught every mutant (Appendix D.8; Figure D.1 shows each statistic against its threshold). Mutation testing is established (DeMillo et al., 1978; Jia and Harman, 2011); it has been applied to scientific software with tolerance-based oracles (Hook and Kelly, 2009) and, with generic operators, to probabilistic models (Jarma et al., 2026), and probabilistic programming systems have been tested with generated programs and differential oracles (Dutta et al., 2018). Deliberately seeded errors are how Geweke (2004), Cook et al. (2006, Section 3) and Grosse and Duvenaud (2014, Section 3.1) demonstrate their checks, and Gandy and Scott (2021, Section 4) compare tests on erroneous samplers; Geweke also found errors in two published posterior simulators. The suite uses curated algorithm-level mutants, some taken from published errors, as a release gate. Because each mutant's catching tiers were confirmed when it was registered, the 37 catches show that the suite still catches these errors, not how often the tiers would catch an unknown bug.

The suite taught five lessons. *(1) Identity tests and joint tests are both needed.* The four mutants without a U check are the non-local errors: a step order (M2, the indicator placement of Primiceri (2005), corrected by Del Negro and Primiceri (2015)), a simulator that draws fresh latents (M3), a sampler–model prior mismatch (M5) and a model approximation (M16); every conditional-formula error is caught by an identity or closed form at one joint draw. *(2) Exactness changes what counts as a bug.* Exact-mode SV is exact for any mixture table and indicator law, provided the correction (9) is computed from the same logits and table as the indicator draw, an implementation invariant that no mutant breaks on one side only; so M11 and M12 are bugs only of the auxiliary-mixture sampler, and the uncorrected move M7, invisible with the 10-component table at our test sizes, is caught once a one-component table is injected in mutant and control alike: degrade an approximation on purpose to test its exact correction. *(3) The log-likelihood is a test quantity.* Block SBC catches M1 through the rank of the exact log-likelihood; one-block invariance of the whole CCCM scan

Table 2: Mutation suite: injected error and, per catching tier, the mutant / control statistic (seams, compositions, U tolerances: Appendix D.8); all 37 checks catch their mutant with a passing control. U: identity or closed-form error at one joint draw; B, S: Bonferroni-adjusted q or minimum p; G: largest $|z|$ at G-small at the mutation compositions (critical values 4.10–4.40); M12's U check is a closed-form check with tolerance 10^{-4}. From the release validation record (Appendix D.8); the committed evidence differs only for M2 (702 / 2.48).

ID	Injected error	Catching tier: mutant / control
M1	CCM-2019 triangular step, not the corrected one	U 1.70 / 1.6e−14; B p 2.8e−11 / 0.10; G 20.8 / 3.03
M2	stale indicators: carried over, drawn after h	G 705 / 2.48
M3	simulator draws fresh latents	G 23.5 / 2.59
M4	τ conditional with shape $\nu/2$, not $(\nu+1)/2$	U 0.163 / 5e−15; G 281 / 2.12
M5	$B_\sigma \times 2$ in the sampler only	G 50.3 / 2.59
M6	non-centred $\lvert\sigma\rvert$ without flipping $\tilde{h}$	U 1.94 / 0; G 7.4 / 2.34
M7	exact-mode non-centred move left uncorrected	U 2.87 / 0; G 77.6 / 2.28
M8	centring (4) dropped ($\gamma_i = \delta_i$)	U 0.585 / 0; G 101 / 2.77; S q 1.6e−13 / pass[a]
M9	outlier pmf without $1/k$	U 0.699 / 1e−15; G 1474 / 2.51
M10	$b_p + T$ instead of $b_p + T - N_1$ in the p update	U 0.120 / 1e−14; G 30.1 / 2.51
M11	indicator pmf without σ_j^{-1}	U 0.112 / 6e−16; B q 4.9e−19 / 0.93; G 86.9 / 2.18
M12	mixture means shifted by -1.2704 twice	U 1.2704 / 8.3e−5; B q 2.5e−162 / 0.93; G 444 / 2.18
M13	IG shape with T_{pad} instead of T_{eff}	U 1.64 / 3e−15; G 429 / 2.77
M14	a-row regression on $+u_{<j}$ instead of $-u_{<j}$	U 14.9 / 9e−15; G 325 / 3.03
M15	ASIS map without the $\tilde{h}$ rescale	U 0.457 / 6e−17; B q 1.8e−14 / 1.0; G 50.5 / 2.34
M16	offset $c = 0.1$ at $\lambda \approx 0.01$, approximate sampler	G 264 / 1.99
M17	Mode I dummy-row weight $1/\lambda_i$, not $1/\bar{\sigma}_i^2$	U 0.060 / 2.4e−15; G 23.1 / 1.94[b]
M18	SOC/SUR rows dropped from the CCCM Π step	U 0.405 / 2.9e−14; G 99.7 / 2.61

[a]The control passes at stage 3 ($q = 0.0069, 0.065, 0.95$). [b]Control on the dense eigen path, passing at stage 2 after a stage-1 q of 0.0012; 2.44 on the `cholesky_const` path.

did not, in a recorded run not repeated here ($q = 0.29$), consistent with Modrák et al. (2025) on how test quantities shape sensitivity. *(4) Moves must preserve the measure.* Only the joint fold $(\sigma, \tilde{h}) \mapsto (-\sigma, -\tilde{h})$ leaves the non-centred posterior invariant (Frühwirth-Schnatter and Wagner, 2010); taking $|\sigma|$ alone (M6) is caught by U and G. *(5) Wrong dummy-row weights are loud* (M17 and M18 in Table 2).

8.9 Runtime convergence diagnostics and the gate

The convergence flag that the gate of Section 6.6 reads is computed for every fit from rank-normalised split $\hat{R}$, the larger of its bulk and folded versions, and the bulk ESS (Geyer, 1992; Vehtari et al., 2021) of a primary set of quantities. The secondary sets, which include the reduced-form intercepts, are reported but never gate (set sizes in Appendix D.9). The gate is strict for SV: a pilot study predicted that SV fits at the default budget would routinely fail through ϕ and σ, the production benchmark ($4 \times 10{,}000$ draws) has `converged = False` in all six runs, and in the first full Clark–Ravazzolo run (four chains of 5000 draws) the SV arms converged at 37–68% of the main-stage origins and SV(rw) at 57–82% across orderings (acceptance run: Section 11).

9 Numerical robustness: what the validation and replication runs found

Three failures surfaced in the Phase 2 validation and replication runs, none from a user report: two float64 failures at states that are legitimate under the model, F1 in a Geweke test with a

heavy-tailed test prior and F2 in a replication of 24 orderings at 106 origins, and F3, a bug in the failure-reporting path (Table E.1). In F1 and F2 the information was lost when the matrix was assembled, not when it was factored, so both fixes factor the data instead; because failures are values and are never jittered (Section 4.4), both were reported rather than masked.

9.1 F1: a Geweke chain in the tail of the test prior

The weekly G-full Geweke test (Geweke, 2004) of `reduced_sv` failed on chain 3214 of 4000 at sweep 194 with status 1003 (block `EQ_CONTEMP`, equation 2), with equation 0 at log-variances $h \approx 31$–37 and $|y| \approx 2 \times 10^8$. The precision of equation 2's contemporaneous coefficients, $K_a = V_a^{-1} + Z^\top W Z$ (regressors Z, inverse variances W), had exact eigenvalues 14.8 and 1.9×10^{17}; forming $Z^\top W Z$ lost the small one and the Cholesky pivot was zero, although the conditional is benign, with standard deviations 0.20 and 0.16. Rather than narrow the test prior, which would have hidden a real float64 weakness, the fix draws a from a Householder QR factorisation of the whitened stacked design (Section 5.2 and Appendix B.9): its R is the upper Cholesky factor of K_a, so the draw is the same function of the noise, but K_a is never formed and the condition number is $\mathrm{cond}(K_a)^{1/2}$, not its square, the textbook case for QR over the normal equations (Higham, 2002; Björck, 1996). Tests check the QR draw against the Cholesky draw and a 60-digit exact conditional and replay chain 3214, where the production step now succeeds (on the Apple M5 Max; Appendices D and E.1). On x86-64 Linux the replay test failed in the one weekly run there, before the release, and the cause is not known; the other F1 tests pass there in continuous integration.

9.2 F2: the random-walk SV draw at $\sigma \approx 5 \times 10^{-9}$

The first full Clark–Ravazzolo attempt (Table F.2) aborted with status 2001 (block `EQ_VOL`, equation 0) in an SV(rw) reduced-form fit at origin 1987Q2, burn-in sweep 318, with GDP's volatility scale at $\sigma = 4.98 \times 10^{-9}$. The state is valid: the non-centred prior $\sigma \sim N(0, B_\sigma)$, equivalently $\sigma^2 \sim B_\sigma \chi^2_1$, has positive density at $\sigma = 0$, and the same sweep's non-centred move would have returned σ to 0.031. In the scaled system $\tilde{Q} = \mathrm{Lap}_w + \mathrm{diag}(\delta)$ of Section 5.5, the terms $\delta_t = \sigma^2 / v_{s_t} \approx 10^{-17}$ fell below the floating-point spacing at the Laplacian's diagonal entries (2), so the assembled matrix was the singular path Laplacian and the level pivot, $\approx \sigma^2 (1/V_0 + \sum_t 1/v_{s_t}) \approx 10^{-15}$, came out zero. The excess-pivot recursion of Section 5.5, which we read as an application of known subtraction-free elimination, not a new algorithm, factors the same matrix without assembling it, with non-negative terms only, so nothing cancels. At a synthetic test state ($\sigma = 5 \times 10^{-9}$, 88 periods) it matches the smallest pivot, $\log|\tilde{Q}|$ and h of a 60-digit reference to a relative 10^{-12}, on the Apple M5 Max and on Linux in continuous integration. Before the fix, the old path could also return a finite but wrong answer without reporting a failure, because the step accepts any finite positive pivot: at the same synthetic test state, not the production fit, its level pivot is a rounding residue (2.22×10^{-16} against 5.308×10^{-15}) and h_0 is off by about 70, and over a grid of σ it fails loudly only at $\sigma \leq 2.2 \times 10^{-9}$ (Figure E.1, Appendix E.1). Of the replication's runs, only the quick run predates this fix, so its random-walk SV numbers may have been affected and serve only as context (Appendix F.5). The random-walk integrated likelihood had the same flaw and now shares the factorisation (Appendix E.1). The fix removes the crash, not the slow mixing near $\sigma = 0$ (Sections 11.4 and 12).

F3: the replay of F2 crashed. The deterministic replay of F2 ran the production CCCM step on host arrays and itself crashed with a `TracerArrayConversionError`, so F2's diagnostics were

lost. A later commit fixes this and tests the replay with the production program instead of toy kernels.

9.3 SOC/SUR dummy rows on the MCMC engine

In Mode II the dummy rows are weighted by $1/\lambda_i$, so the λ_i step must count them; a hand-injected omission, recorded in the repository notes but not committed as a test, was caught by the analytic twin (Section 8.5) but not by the dense oracles of the coefficient step. For exact Geweke and SBC references, $\lambda_i \mid d$ is inverse gamma under Mode II, and under `iw_equivalent`, where it is not, the base prior $\mathrm{IG}(c, C)$ is an exact rejection envelope (Appendix B.6). The λ draws pass a Kolmogorov–Smirnov test against quadrature, and row 205 passes Geweke and SBC (Appendix B.6). The $O(q^2)$ eigen fast path survives the dense prior precision that the rows induce.

9.4 Notes for other implementations

These generic checks name no package. Implementations that (i) form a weighted Gram for a Gaussian conditional draw should test regressors whose scales differ by many orders of magnitude (F1); (ii) solve the random-walk SV system in scaled form should test $\sigma \to 0$ (F2); (iii) fold a non-centred sign should flip $(\sigma, \tilde{h})$ jointly, the map that leaves the posterior invariant (Frühwirth-Schnatter and Wagner, 2010), not σ alone (mutant M6, Section 8.8); and (iv) score multi-step forecasts conditionally on simulated paths should inspect the Kish effective number of weights ESS_K for tail outcomes (Section 6.4).

9.5 Platform caveats

On the macOS development machine (NumPy and SciPy on Accelerate), `slogdet` and `det` warned of division by zero on well-conditioned tridiagonals, and a Phase 1 empirical-Bayes test of the analytic engine has repeatedly crashed the interpreter, for unknown reasons (Appendix E.3); we attribute neither to a particular library, and Linux CI is the reference platform. No run of that test, which is marked slow, is on record on Linux: the one weekly x86-64 run in continuous integration stopped at its time limit before reaching it.

10 Performance

The performance target, BENCH-P2, is one realistic SV workload: structural Linear × SV ($n = 20$, $p = 4$, $T = 300$) fitted to a simulated stationary VAR(4) with SV, with four chains of 1000 burn-in and 10,000 kept sweeps, fixed hyperparameters and one CPU device, within 120 s. The measured quantity is the end-to-end wall time of `BVAR.fit()`, including compilation of the JAX program (Bradbury et al., 2018), storage and diagnostics; each fit runs in a fresh process, three times with an empty (cold) and three times with a populated (warm) persistent compilation cache.

The reference measurement of the production engine is the committed baseline, taken on an Apple M5 Max with desktop applications open (one-minute load average 2.4–6.1 before the six runs), whereas the protocol asks for an idle machine with at least eight performance cores; the record lists six. The median wall time is 52.3 s cold and 49.6 s warm, of which compilation takes 3.46 s cold and 1.00 s when the executables are loaded from the cache (medians over the three cold and the three warm runs). One sweep of all four chains takes 3.90 ms and peak resident memory is 3,043 MiB (medians over all six runs; Appendix E.2). These times are within the target, but on a measurement that does not follow the protocol, so the protocol's outcome is open; the numbers

belong to this machine, load and commit and are not portable. No run on an idle machine exists, and none on the release commit: later commits, D-35 among them, change engine code, including the resolution and runner modules that `fit()` calls, so the timings are those of commit deb3929.

Mixing, not speed, is the binding constraint. The slowest quantities, the SV scales σ_i and persistences ϕ_i, reach a minimum bulk ESS per second of 10.2 and 11.2 over the warm-run wall time, against 403 for the own first-lag coefficients. Every run reports `converged = False`; the runs share a seed and give identical draws, so this is one outcome seen six times: according to the commit message, 4 of the 101 primary quantities miss the R-hat threshold of 1.01 (worst 1.016, a σ_i), while every group's minimum bulk ESS exceeds 400. Even at the SV default budget of release 0.2.0 ($4 \times 10{,}000$ kept draws), this workload thus fails the convergence rule of Section 6.6; Section 12 discusses the budget.

On a shared 2-vCPU x86-64 runner of the continuous-integration bench job, at commit 779957f (before D-35), the same fit took 228.9 s cold and 198.4 s warm; the design reports that run without gating it (Appendix E.2). The four-device stretch target of 60 s, the reduced-form CCCM sampler, SV with Student-t errors or with outliers plus t, and an idle x86 machine have no production measurement, and timings of design stand-ins and pilot prototypes are not quoted (Appendix E.2); Table E.2 gives the full benchmark record and Table E.3 the kernel microbenchmarks.

11 Illustration: a pre-registered replication of Clark and Ravazzolo (2015a)

11.1 Role and target

The replication is one of the project's Phase 2 exit criteria, and it is qualitative: the target is the ranking reported by Clark and Ravazzolo (2015a), that VARs with stochastic volatility beat homoskedastic VARs in density forecasts, not their numbers cell by cell. It exercises the whole stack on real data and illustrates the workflow; it does not test the econometric claim beyond these data and models. We use the working paper's numbers (Clark and Ravazzolo, 2012, Tables 1–3, VAR block); the article's tables were not accessed (Appendix F.6). The working paper writes the errors of its time-varying-volatility VARs with a unit lower-triangular matrix A (Clark and Ravazzolo, 2012, Section 3.2, eqs. 7–11) but does not state the order of the variables, so we run all 24 orderings, with the order of its Section 2 and its tables, GDP, INFL, UR, TBILL, as the identity ordering.

11.2 Data and models

The data are FRED-QD and FRED-MD, vintage 2026-09 (McCracken and Ng, 2021, 2016), checked against their SHA-256 checksums; estimation starts in 1965Q1, as in Clark (2011), not in Clark and Ravazzolo's 1955Q1, which FRED-QD (from 1959) cannot provide (Appendix F.4). The forecasts are pseudo real time on the final vintage, not on real-time vintages (Croushore and Stark, 2001). Table F.3 lists the series and transformations, and Appendix F every deviation from the original design. Run-time assertions (Table F.3) check the quarterly averaging of the monthly series and, for the primary arm, the absence of look-ahead: at 1985Q1, 2007Q4 and 2011Q2 the resolved prior, its hyperparameters and the padded data are bitwise identical when the vintage itself is cut at the origin (`true`).

Every arm shares the mean block, Clark and Ravazzolo's Minnesota prior on the reduced form with $p = 4$ (Table F.4), sampled with the corrected triangular algorithm (Carriero et al., 2022). The volatility arms (Table F.4) are: the order-invariant baseline `Constant('iw_equivalent')` (Chan,

2022) in place of the normal-diffuse VAR; the primary arm SV(rw), Clark and Ravazzolo's VAR-SV; the library's AR(1) default SV; their variants SV(rw,t10) and AR-SV(0.9); and a structural Mode I pair. Apart from the variants' priors on ν and ϕ, the SV priors are the library defaults (Section 3.3), not Clark and Ravazzolo's, and no prior was tuned. Against this baseline the SV arms also differ in the prior of the contemporaneous coefficients (Section 6.6).

11.3 Evaluation design

Forecasts come from an expanding window at 106 origins 1985Q1–2011Q2, of which the 92 up to 2007Q4 form the pre-crisis sample, at horizons of $1, 2, 4, 8$ quarters. Both samples are defined by origin, so every horizon uses all of their origins: the pre-crisis targets at $h = 8$ run to 2009Q4, and the full-sample ones to 2013Q2. In the notes to their Tables 1–3, Clark and Ravazzolo's forecasts at origin t use data through $t - 1$ and are for periods t to $t + 7$ (Clark and Ravazzolo, 2012), so for the same origin their h-step target, $t + h - 1$, is one quarter earlier than ours, although both lie h quarters past the last observation. Since their last vintage, 2011Q4, ends in 2011Q3, their samples at $h = 4$ and $h = 8$ cannot contain every origin to 2011Q2 (nor at $h = 2$ with second-release actuals for growth and inflation), and the working paper does not say whether the 2007Q4 end of the pre-crisis sample applies to origins or to targets. Each origin is a separate MCMC fit with its own seed. Under the expanding window the Diebold–Mariano (DM/HLN) p-values are descriptive, so a "better" verdict must come through the model confidence set (MCS; Section 6.5), itself a descriptive route here (Section 6.6).

The primary score is the Rao–Blackwellised (RB) log predictive density of Section 6.3: path-conditioned at $h > 1$ in the first full run, path-integrated in the acceptance run, which also reports the path-conditioned score on the same draws as a secondary number that never enters a rule; the two coincide at $h = 1$. For parity we report Clark and Ravazzolo's Gaussian-approximation score (Adolfson et al., 2007) from the same draws, proper but not strictly proper (Appendix F.4), and flag the cells in which its sign differs from the RB score's; CRPS and RMSE ratios complete the set. Verdicts are reported with the convergence gate on, as `compare()` returns them, and off.

An origin is *RB-degenerate* at horizon h when the delta-method MCSE of the joint RB score of either arm is at least 0.9: the weights then sit on about one draw, and the MCSE saturates near one nat however far off the estimate is. Before the MCMC adjustment the MCSE is a function of the Kish effective number of weights alone and at most one nat, so that, before the adjustment, the threshold corresponds to $\mathrm{ESS_K} \approx 1.2$ (Appendix B.5). The flag is descriptive: flagged origins are kept, because dropping exactly the tail origins would bias the comparison. The ordering stage runs `ordering_sensitivity(mode='backtest', design='all')` with one noise replicate for SV(rw) and `Constant`, then a `compare()` verdict per ordering, each ordering with its own backtest seed. `Constant('iw_equivalent')` is order invariant, so its spread across orderings is Monte Carlo noise, which makes it a negative control.

Criteria. The replication plan states six criteria in words (Table F.1), and the script turns them into rules of the project's own making, quoted and discussed in Appendix F.3; criterion 4's rule, for example, compares $h = 8$ with $h = 1$ only. No rule was changed after the first run. Only criterion 6 uses the convergence gate, and only $h = 1$ scores, which do not depend on the estimator.

11.4 The first full run, its diagnosis and the pre-registration

The numbers of the first full run are history, not results. It used 4×5000 kept draws (burn-in 1000), 1000 forecast draws and the path-conditioned LPD, after an attempt that aborted on the

failure of Section 9.2 (Table F.2). It met criteria 1, 2, 3 and 5 and missed 4 and 6.

Criterion 4 failed on one cell, full-sample unemployment at $h = 8$ (+1.954 against +0.159 at $h = 1$), driven by three 2007–08 origins with differences of +40.4 to +41.6 nats, where the path-conditioned estimator had collapsed (Section 6.4). Criterion 6 failed on the convergence gate alone: SV(rw) was "better" at $h = 1$ in 0 of 24 orderings with the gate on and in 24 with it off, having converged at 57–82% of origins (`Constant`: 100%; Appendix F.9). Refits of SV(rw) at the main-stage seeds, with fit-level diagnostics and no forecasts or scores (Table F.9), showed every base-budget failure to be narrow, on a σ quantity with $\hat{R} < 1.03$, and all 106 origins converging at $4 \times 20{,}000$ in the identity ordering (72 at 4×5000) and in the two worst orderings (71 and 63). Near the threshold the count moves with the seed (72 against 69 origins under the first run's two seeds; Appendix F.8), so budgets are best sized with such diagnostics before any score exists.

What was registered. The pre-registration (Appendix F.2) changes exactly three settings: $4 \times 20{,}000$ kept draws (burn-in 1000) for every MCMC arm in every stage, the path-integrated LPD as the primary score, and 8000 forecast draws per origin, which cut the thinning error of `Constant`'s estimate at a crisis cell from 0.75–1.06 to 0.09–0.19 nats (standard deviation); everything else, the coded criteria and the 95% gate included, is unchanged. It was committed and pushed to the hosted repository before the run was launched from a clean checkout of that commit: GitHub's repository activity log records the creation of the branch at that commit at 16:10:46 UTC, ten seconds before the run's first log line and about a minute and a half before its first score (Appendix F.1). It says that the run is not blind to the first, that the criteria are evaluated as coded whatever the outcome, that criterion 4 may still fail on unemployment, and that no further run with other settings is made after the scores are seen, except to fix a bug.

11.5 Results of the acceptance run

Reading rules. The paper's reading rules, which are not part of the pre-registration (Appendix F.7), let the RB scores at $h \geq 2$ move with the estimator, the $h = 1$, CRPS, RMSE and Gaussian numbers by Monte Carlo noise only, and the converged shares and gate-on verdicts with the budget. The $h = 1$ log-score gains of every SV arm differ from the first run's by less than the rules' Monte Carlo yardstick (SV(rw)'s joint gain: +0.819 against +0.809), a descriptive comparison: the yardstick is the largest move over all arms between one earlier pair of runs, one of which predates a sampler fix (Appendix F.7).

Outcome and headline. As coded, the outcome is: criteria 1, 2, 3, 5 and 6 met; 4 not met (Table F.1). Every reduced-form arm converged at 100% of origins in both samples, on the gate's primary quantities, which leave the intercepts out (Section 12; the report has no share for the structural pair), and no reduced-form origin is RB-degenerate (0 at every horizon), a flag that detects only a near-total collapse of the weights onto one draw. SV(rw)'s gain over `Constant` is positive for every variable at $h = 1$ and 2 in both samples, as criterion 1 requires, its CRPS and RMSE ratios at $h \leq 2$ are below one, and the joint $h = 1$ gain is +0.819 nats per quarter (descriptive one-sided DM $p < 0.001$; Table 3 and Figure F.3).

Criterion 4. Criterion 4 fails on one of its four parts, as the pre-registration allowed. Its rule asks the full-sample gain at $h = 8$ to be below that at $h = 1$ for unemployment and the bill, and the pre-crisis gain at $h = 8$ to be positive for GDP and inflation. The bill's gain falls from +0.377 to −0.195, and the pre-crisis gains are +0.105 for GDP and +0.307 for inflation (Table F.6); the unemployment gain rises, from +0.160 at $h = 1$ to +0.273 at $h = 8$, path-integrated. The

Table 3: SV(rw) against `Constant`, full sample (106 origins), acceptance run. ΔLPS: mean per-origin log-score difference in nats per quarter, path-integrated RB estimator (primary), path-conditioned RB estimator on the same draws (secondary, never in a rule) and Gaussian approximation (parity metric). "Sign": RB and Gaussian ΔLPS differ in sign. "Degen.": RB-degenerate origins (joint MCSE ≥ 0.9 for either arm; the count is per horizon and the same for all variables). DM p: one-sided DM/HLN p-value for "SV(rw) is better", descriptive under the expanding window. CR: Clark and Ravazzolo's VAR-SV against VAR, working-paper Tables 1–3, Gaussian-approximation log score.

		ΔLPS						Ratio		CR		
Var.	h	Integr.	Cond.	Gauss.	Sign	Degen.	DM p	CRPS	RMSE	ΔLPS	CRPS	RMSE
GDP	1	+0.126	+0.126	+0.103	no	0	0.002	0.897	0.908	+0.193	0.864	0.896
	2	+0.124	+0.124	+0.065	no	0	0.002	0.894	0.901	+0.133	0.856	0.878
	4	+0.089	+0.089	−0.061	yes	0	0.063	0.927	0.944	+0.090	0.890	0.932
	8	+0.108	+0.108	+0.010	no	0	0.008	0.940	0.977	+0.060	0.927	1.007
INFL	1	+0.233	+0.233	+0.210	no	0	< 0.001	0.845	0.899	+0.075	0.954	0.966
	2	+0.269	+0.268	+0.270	no	0	< 0.001	0.807	0.848	+0.083	0.944	0.949
	4	+0.255	+0.254	+0.257	no	0	< 0.001	0.767	0.788	+0.155	0.895	0.915
	8	+0.293	+0.293	+0.326	no	0	< 0.001	0.720	0.725	+0.138	0.887	0.883
UR	1	+0.160	+0.160	+0.122	no	0	0.008	0.936	0.969	+0.162	0.922	0.967
	2	+0.206	+0.195	+0.048	no	0	0.082	0.924	0.946	+0.098	0.897	0.930
	4	+0.251	+0.578	−0.248	yes	0	0.172	0.929	0.937	−0.182	0.906	0.914
	8	+0.273	+1.309	−0.526	yes	0	0.282	0.981	0.965	−0.707	1.001	0.971
TBILL	1	+0.377	+0.377	+0.315	no	0	< 0.001	0.798	0.935	+0.284	0.822	0.933
	2	+0.153	+0.156	+0.095	no	0	0.077	0.895	0.945	+0.043	0.915	0.944
	4	−0.066	−0.066	−0.142	no	0	0.681	0.983	0.968	−0.304	1.010	0.974
	8	−0.195	−0.193	−0.230	no	0	0.857	1.019	0.986	−0.403	1.059	0.990
joint	1	+0.819	+0.819	+0.654	no	0	< 0.001					
	2	+0.743	+0.794	+0.463	no	0	0.003					
	4	+0.673	+1.997	+0.002	no	0	0.056					
	8	+0.604	+3.304	−0.350	yes	0	0.119					

following numbers are secondary and descriptive: reported next to the rule, they never enter it (Table F.5). The unemployment gain at $h = 8$ is concentrated: its three largest per-origin values, +7.595, +6.626 and +6.515 nats at 2007Q4–2008Q2 (Figure F.1; first run: +40.4 to +41.6), make up 72% of the 106-origin sum, and without the three values largest in absolute value the mean is +0.079 at $h = 8$ against +0.151 at $h = 1$; its median and the pre-crisis mean also fall with the horizon (Table F.5). Each of these origins carries Monte Carlo error of about 1.3 nats per arm, which moves a 106-origin mean by hundredths of a nat (Appendix F.2). On Clark and Ravazzolo's Gaussian-approximation score, a reported secondary result, the unemployment gain shrinks as in their results, from +0.122 to −0.526, but criterion 4 would still fail on that score, on the pre-crisis GDP gain at $h = 8$ (−0.001). The path-conditioned estimator puts the $h = 8$ unemployment gain at +1.309 (first run: +1.954); the pre-crisis triple is in Table F.6.

Verdicts and their routes. With every reduced-form arm converged, the gate changes no verdict (Table F.7), and every "better" verdict, in either sample, comes through the MCS route (the SV arm in the 75% MCS, `Constant` out), as it must under the expanding window. Verdicts depend on the comparison set. The main stage passes all five reduced-form arms (`Constant`, SV(rw), SV, SV(rw,t10), AR-SV(0.9)) to one `compare()`, so its MCS is formed over all five: SV(rw)'s full-sample verdict is "better" at $h = 1$ and "inconclusive" at $h = 2$, 4 and 8, where that MCS retains neither

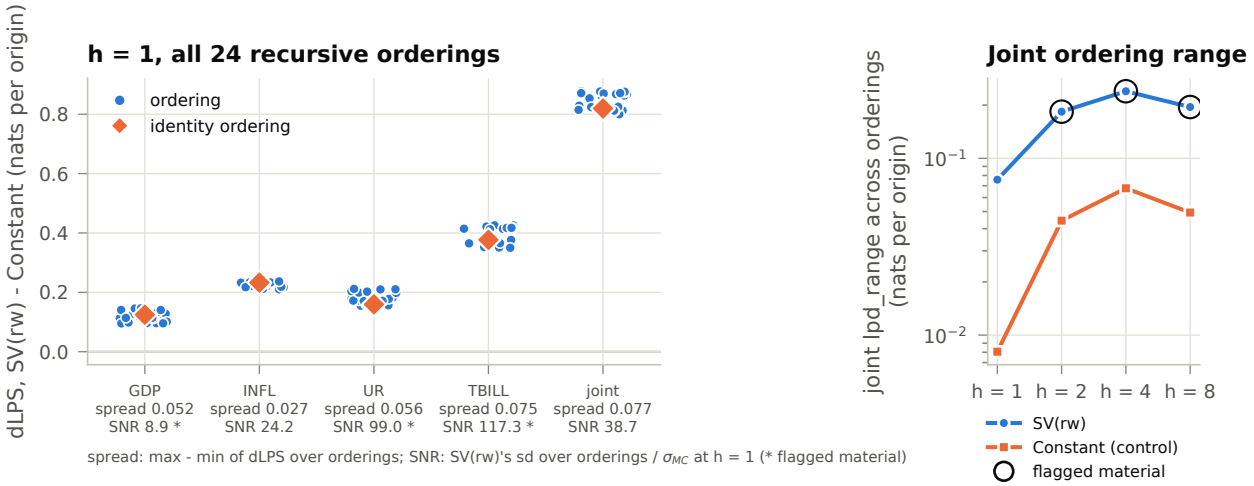


Figure 3: Ordering robustness, acceptance run. Left: ΔLPS (SV(rw) − `Constant`) at $h = 1$ per variable and joint, one point per ordering (24), with the identity ordering marked; each column gives the spread across orderings and SV(rw)'s signal-to-noise ratio, the quantity the materiality flag compares with 3. Right: joint ordering range by horizon for SV(rw) and for the order-invariant `Constant` arm (negative control), with flagged cells marked. At $h \geq 2$ the scores are path-integrated. The flags' noise floor rests on one replicate per arm, so a flag, or its absence, is one realisation.

SV(rw) nor `Constant`, although at $h = 2$ the total gain is +78.713 nats with an MCSE of 0.638. The ordering stage compares SV(rw) with `Constant` alone, and there its verdict is "better" at every horizon in all 24 orderings, at $h = 8$ with descriptive one-sided DM p-values from 0.086 to 0.154, mostly above the DM route's 0.10: the two-model MCS is the looser test (Section 6.6). Criterion 5 is met: neither variant is better than both plain SV arms (DM $p < 0.10$) at any joint horizon or marginal cell.

Ordering robustness. Across the 24 orderings (Figure 3) SV(rw)'s joint $h = 1$ score ranges over 0.076 nats, against a gain of +0.819; for GDP, the smallest gain, over 0.052 against +0.126. SV(rw) converged at 100% of origins in every ordering (first run: 57–82%; Figure F.2), and its verdict is "better" in 24 of 24 orderings with the gate on and 24 with it off; so criterion 6, which the first run missed on the gate alone, is met. The orderings share the data, so these are 24 dependent comparisons, not 24 confirmations. For scale, the order-invariant `Constant` arm, whose spread across orderings is the Monte Carlo noise of another sampler and so at best a lower bound on SV(rw)'s, has ranges of 0.001–0.068 nats over its 20 variable–horizon cells (0.008–0.068 for the joint score), against 0.027–0.244 (0.076–0.240) for SV(rw). The materiality flags rest on a noise floor estimated from one replicate (Section 12): `Constant`, flagged at two cells in the first run (Appendix F.9), is not flagged here (`material = false`), which is one realisation, and SV(rw) is flagged at 14 of its 20 cells. Part of SV(rw)'s ordering dependence may come from its priors, whose centring depends on an equation's position (μ_i and $h_{i,0}$ at $\log(\psi_i/\iota)$; Section 3.3), not only from the triangular factorisation.

Interpretation. On these data and models the run reproduces the ranking that the replication targets (criteria 1–3 and 6; criterion 5, on the variants, also holds). On the primary score it does not reproduce the horizon pattern for unemployment, whose gain falls from +0.162 at $h = 1$ to −0.707 at $h = 8$ in Clark and Ravazzolo's working paper (Table 3) and rises here, mostly through

three crisis origins (on their score it falls; above). As registered, we report this as a difference from their results and tuned nothing in response (Appendix F.2). The pre-registration names three candidate causes: the final vintage instead of real-time data, Chan's `iw_equivalent` baseline instead of the normal-diffuse VAR, and the library's default SV priors instead of theirs. These are hypotheses, which this run does not test, although Clark and Ravazzolo's data are in the journal's archive (Appendix F.4); the pre-registration rules out further runs with other settings. No run-level warning reports a convergence failure, as the first run's did.

11.6 Caveats

The illustration is limited by final-vintage instead of real-time data; the working paper's numbers instead of the article's; Chan's `iw_equivalent` baseline instead of the normal-diffuse VAR, and the library's SV priors instead of Clark and Ravazzolo's (further differences in Appendix F.4); descriptive significance, with verdicts that come through the MCS, whose validity in this setting is open; operational thresholds of our own choosing; a pre-registration that is not blind; one backtest seed in the main stage; and a change of the primary estimator between the runs at $h > 1$, which is why every LPD number above names its estimator. The DM components of criteria 1, 5 and 6 are descriptive under the expanding window, so these criteria are coded descriptive checks, not tests. Criterion 4 compares gains on the RB score with a pattern that Clark and Ravazzolo measured on the Gaussian approximation, a proper but not strictly proper score (Appendix F.4); that is a limitation of the criterion's design, which compares point estimates of the gains with no allowance for their uncertainty. The primary arm, reduced form $\times$ SV(rw), was not a registered validation composition at the run's commit; since a later commit that changes no sampler code, it is row 18. On that row the Geweke and contract tests pass and five injected bugs are caught; its SBC γ test and pooled-ESS gate are clean, while the per-replication ESS rule missed at the shared thinning cap of 200, passes at the cap of 400 set for the row after that miss, and fails with a 4000-sweep pilot (Section 8.7). Its slow-mixing coefficients are a limitation (Section 12); its Student-t variant has no row and still rests on its validated parts.

12 Limitations

Scope. The package implements Phases 0–2 of its roadmap. The mean axis is linear and the prior axis is the Minnesota family (conjugate, structural and reduced form, with dummy-observation priors and the GLP hierarchy); the volatility axis is the one that is populated. Global-local priors, common stochastic volatility and conditional forecasts (Phase 3), BART means, generalised impulse responses and approximate leave-future-out cross-validation (Phase 4; Bürkner et al., 2020), time-varying parameters, threshold and Markov switching, factor and order-invariant SV (Phase 5), and identification beyond the recursive scheme (Phase 6) are roadmap items, not features; for the methods see, for example, Huber and Rossini (2022), Clark et al. (2023), Gruber and Kastner (2025), Kastner (2019), Chan et al. (2024) and Bertsche and Braun (2022). That later components plug in without editing the sweep, runner or forecast modules is a design intention that no component has yet tested. The data layer rejects missing values inside the sample, has no mixed-frequency support, and ships no real-time vintages; it only provides the hook for them.

The verdict engine. The verdict rule is implemented and unit-tested, but its statistical properties are not established. The size and power study on simulated data-generating processes that the specification calls for has not been implemented, so we claim no false-positive rate for "better".

What exists are tests of the Diebold–Mariano size and power, of MCS behaviour and of the verdict logic. The MCSE of a total ΔLPD adds the two models' per-origin MCSEs in quadrature, although both models' forecasts at an origin draw on random streams derived from the same backtest seed and origin; whether the factor of three is conservative is unknown. Under expanding windows a "better" verdict can only come through the MCS, whose validity in that setting is open (cf. Hansen et al., 2011); with two models it is a two-sided test at 25%, its block length does not grow with h, and a pair's verdict depends on the other models passed to `compare()` (Sections 6.6 and 11.5). Verdicts are per horizon, with no rule that combines horizons, and the DM long-run variance (Section 6.5) is not the Newey–West variance (Newey and West, 1987) that the specification states.

The convergence gate. SV fits routinely miss the $\hat{R} < 1.01$ and bulk ESS > 400 targets. The prototype pilot predicted this, through ϕ and σ, at the earlier default budget of 4×2000 draws, at which no SV arm of the quick Clark–Ravazzolo run converged at any origin. It persists at larger budgets: all six runs of the performance benchmark ($4 \times 10{,}000$ draws) report `converged = False`, and so did many SV(rw) fits of the first full Clark–Ravazzolo run (4×5000; Section 11.4). Part of this is noise in the rule itself: $\hat{R}$ is an estimate, and at a bulk ESS of a few hundred the rule $\hat{R} < 1.01$ also fails by chance, often (cf. Vehtari et al., 2021), so the gate acts much like a floor on the ESS, and near the threshold the converged count moves with the seed by several origins (Section 11.4). The gate is strict in that sense, and the price is that users need large budgets: the acceptance run used $4 \times 20{,}000$ draws, at which every reduced-form arm converged at 100% of origins and SV(rw) did so in every ordering, and it took 12,905 s (3 h 35 min) with 10 worker processes (Section 11.5). Release 0.2.0 therefore raises the default for SV compositions to $4 \times 10{,}000$ kept draws and warns below it; the rule itself is unchanged, and the new default raises the floor without guaranteeing convergence: the performance benchmark still fails the rule at it (Section 10), and in the comparison example of the package's README the SV arm converges at 10 of its 11 origins. The gate also leaves the reduced-form intercepts out: it gates on the log-likelihood, each equation's own first-lag coefficient and the volatility parameters, only reports the intercepts and the contemporaneous coefficients, and does not monitor the other lag coefficients, so "converged" certifies these primary quantities only. In the SBC of row 18, the replication's primary arm, about 1–2% of prior-drawn $T = 50$ datasets mix very slowly in the reduced-form coefficients, most often an intercept (Section 8.7). Whether the replication's fits mix as slowly is unknown; their secondary diagnostics are in the run's cache, not in its report.

Log scores. The path-conditioned estimator remains the default at $h > 1$. The path-integrated estimator is opt-in and covers MCMC forecasts of linear or zero means only; analytic, benchmark and pooled forecasts raise at $h > 1$. It removes the path-conditioning error but not the Monte Carlo error: it is still an average over the forecast's parameter and volatility-path draws, whose weights concentrate in the tails. At a crisis origin, SV(rw)'s 1000-draw estimate varied across forecast seeds by more than its delta-method MCSE, and with 80,000 forecast draws the values still differed by about 1.3 nats per arm across fit seeds, an MCMC error that more forecast draws do not remove (Appendix F.2). In the acceptance run the RB-degenerate flag fired only at 1 origin, 2008Q2 at $h = 4$ in the structural pair, whose convergence the report does not record; the flag detects only a near-total collapse of the weights and bounds the delta-method MCSE, not this error. All scores are on the transformed (model) scale. The Monte Carlo noise of the subsampled energy and variogram scores is not reported, and there is no SBC of y_{T+h} for $h > 1$.

The ordering diagnostic. The noise floor of `ordering_sensitivity` in the replication rests on one replicate (`noise_reps=1`), and in backtest mode it has no MCSE term. In the first full run it flagged the order-invariant `Constant` arm as material, at two $h > 1$ cells. In the acceptance run, at the larger budget, it does not (`material = false`); one run without a false flag does not establish the diagnostic's false-positive rate, and the flags it raises on SV(rw) rest on the same one-replicate floor: the spread of two runs, under which a signal-to-noise ratio above 3 is common even under order invariance, so the flags rest mainly on the fixed thresholds.

Validation evidence. Geweke tests and SBC run at $n \leq 3$; the realistic size ($n = 20$, $p = 4$, $T = 300$) is covered only by the analytic twins, and only for constant-volatility compositions. Our parity tier has no external VAR-level oracle for structural Mode I $\times$ SV. The replication's primary arm, reduced form $\times$ SV(rw), has been a validation row only since after the acceptance run (row 18; Section 8.7); its Student-t variant has no row and no contract fit. Test priors can hide bugs: row 18's default test prior hides a prior variance used as a precision, which a variant G-small test catches (Section 8.7; 13.8 against 2.33). The release validation record of the weekly tier is one run on one machine (an Apple M5 Max), the per-PR tier on the release commit has pass or fail outcomes only (continuous integration on Linux), so the G-small statistics of Table D.2, row 18's among them, come from earlier commits; the x86-64 mutation runs predate the release, and the one x86-64 run of the weekly tier, also earlier, stopped half-way at its time limit with the replay test of F1 failing (Appendix D). The test-prior design rule $r \leq 5$ is a derivation for the scalar conjugate model, unverified for ϕ, σ, p and ν, and low relative numerical efficiency of some Geweke functions lowers power. The suite is checked only against 18 curated mutants, which says little about its power against unknown bugs. Whether the 10-component mixture approximation is visible to G-full with unknown SV parameters is undetermined. The reduced-form parity check against bayesianVARs was replaced by an in-house oracle because the package could not be built in the test environment, and all third-party parity holds for pinned versions (stochvol 3.2.1, BVAR 1.0.5, BoE bvar 0.3.5).

Numerics, performance and platforms. The formed-Gram draws that were not rewritten, among them the structural coefficient step and the CCCM Π step, still accept any positive pivot, with no relative floor, although on the old random-walk path a tiny positive pivot could give a finite but wrong draw with no failure reported (Section 9.2 and Figure E.1); whether reachable states expose them is unknown. The excess-pivot recursion removed the crash and the wrong draws of the random-walk volatility step near $\sigma = 0$, not the slow mixing there. The performance evidence comes from one Apple M5 Max that was not idle and one 2-vCPU x86-64 continuous-integration runner; there is no idle x86 measurement and no multi-device timing of the production engine. Bitwise reproducibility holds only within one platform and version set: invariance to chunking is tested on runner toys, and the toy chain-prefix test is bitwise on macOS and agrees only to rounding on x86 Linux. The macOS caveats of Section 9.5 apply.

Release state. The package is at version 0.2.0, classified as pre-alpha; it is not on PyPI and installs from the repository (Appendix G).

13 Conclusion

A model comparison is only as credible as its weakest link: the sampler, convergence at the origins, the Monte Carlo error of the score, the variable ordering, the validity of the test, and choices made after the scores were seen. rai-bvar makes each of these links checkable. The econometrics

is not new; the contribution is the integration, the contracts between the components, and the evidence, with its stated gaps, that the samplers target the intended posteriors. The validation code behind that evidence ships with the library. In the pre-registered replication of Clark and Ravazzolo (2015a), the acceptance run's outcome as coded is: criteria 1, 2, 3, 5 and 6 met; 4 not met; the path-integrated RB unemployment gain is larger at $h = 8$ than at $h = 1$, mostly through three crisis origins, while on the original's Gaussian score it shrinks, as there (Section 11.5).

A checklist. Lessons for forecasters and for implementers of the same algorithms:

1. Write every kernel as a pure function of injected noise, exposing each exact draw's conditional and each Metropolis–Hastings log acceptance ratio (Section 4.4).
2. Write the reference samplers, simulators and joint densities independently of the kernels, and enforce the separation with an import test (Section 8.1).
3. Check an identity at a joint draw for each conditional and Metropolis–Hastings step; run prior-started Geweke chains and SBC on data-dependent quantities such as the log-likelihood (Section 8).
4. Measure the suite's power with injected bugs, among them errors of published samplers, and degrade an approximation on purpose to test its exact correction (Section 8.8).
5. Treat numerical failures as values: never add jitter; fix the kernel, not the test prior (Section 9).
6. Gate verdicts on the share of origins (95% by default) at which a fit's primary quantities converged, say which quantities those are, and size the MCMC budget with fit-only diagnostics before any score exists (Sections 6.6 and 11.4).
7. Do not read the delta-method MCSE of an RB log score as a bound on its tail error: inspect the Kish effective number of weights $\mathrm{ESS}_{\mathrm{K}}$ (Kish, 1965); at $h > 1$, integrate the path out and use enough forecast draws (Section 6.4).
8. Report RB and Gaussian-approximation scores side by side, the ordering spread next to the gain, how concentrated a gain is across origins, and an order-invariant negative control (Sections 11.3 and 11.5); state a replication's criteria on the target's own scoring rule, or on both, and with an allowance for the uncertainty of the estimates they compare.
9. Treat Diebold–Mariano p-values under expanding windows as descriptive, and state which route produced each verdict and which models formed the comparison set (Sections 6.5 and 6.6).
10. Pre-register settings as code, deposit or push the pre-registration before launch, say how blind it is, and check the absence of look-ahead at run time (Sections 11.2 and 11.4).

Roadmap and availability. The roadmap continues by phase (Section 12); we make no promise about results. The next priority is the size and power study of the verdict engine on simulated data, followed by the Phase 3 prior axis, so that a two-axis ablation can be shown. rai-bvar is licensed under Apache-2.0; release 0.2.0 is commit 4ac7969 of `https://github.com/yigitokar/rai-bvar`, and Appendix G lists its reports, pre-registration and validation records.

A Notation and conventions

The paper uses mathematical symbols where the code uses field names, and in two places the specification and the code use different symbols for the same object. Table A.1 fixes the convention used throughout and gives the code name of each symbol. Two choices matter for reading the code. First, the Minnesota overall tightness is written κ, although its field is `Minnesota.lambda_`, because $\lambda_{i,t}$ denotes the structural variance of equation i at time t. Second, the text numbers equations $i = 1, \ldots, n$ in the recursive order, while the code indexes them from 0; where the position enters a prior formula as a number it is written ι ($\iota = i$ in the text's numbering, the code index plus one), so that, for example, the Constant prior of equation i is $\lambda_i \sim \mathrm{IG}(1 + \iota/2, \psi_i/2)$; the derivations of Appendix B mostly index equations by ι. The letter h is overloaded in the usual way: with subscripts, $h_{i,t} = \log \lambda_{i,t}^{\mathrm{pers}}$ is a log-volatility; without, $h \in \{1, 2, 4, 8\}$ is a forecast horizon.

All floating-point work is in float64, densities are evaluated in log space, and log scores are in nats. Data are stored as raw levels with FRED transformation codes (McCracken and Ng, 2016); each code has a forward map and an inverse that is applied to whole simulated paths, never to quantiles or means, because nonlinear transforms do not commute with expectations. Table A.2 lists them.

A.1 Prior options and defaults

This subsection records the options and defaults of the priors of Sections 3.2 and 3.3 that the main text leaves out.

Scales under the COVID-19 scaling. With the scaling of Lenza and Primiceri (2022) (below), the default scales ψ_j (residual variances of univariate AR(p) regressions with a constant on the estimation window) are computed on the periods before t^* when enough of them exist.

Dummy rows. The default `soc_scope="persistent"` keeps differenced variables from being pulled towards a unit root. `soc_scope="all"` restores the SOC rows of Giannone et al. (2015) for every variable, including those with a zero own-lag prior mean.

The COVID-19 scaling. The scale s_t of Lenza and Primiceri (2022) is 1 before the start period t^*, free in t^* and the next two periods, and decays geometrically towards 1 afterwards; the log marginal likelihood gains the Jacobian $-n \sum_t \log s_t$.

The hierarchy. The hierarchy of Giannone et al. (2015) is used with their default hyperpriors, plus a prior on α, which Giannone et al. (2015) fix at 2, used only when α is estimated. `hierarchy="eb"` maximises $\log p(Y \mid \vartheta) + \log p(\vartheta)$, and `"mh"` samples ϑ by random-walk Metropolis with (B, Σ) drawn exactly given each ϑ. With dummy rows, $p(Y \mid \vartheta)$ is the marginal likelihood of dummies and data divided by that of the dummies alone.

Empirical Bayes on the MCMC engine when $\kappa_c \neq 1$. Hyperparameters are fixed during MCMC: at given values (`hierarchy="fixed"`) or, by default (`"eb"`), at the empirical-Bayes mode of the analytic conjugate twin, fitted at $\kappa_c = \kappa_a = 1$ on the same window. Two cases differ when $\kappa_c \neq 1$: in structural form the mode of the per-equation closed-form marginal likelihood of Chan (2022), plus the same log hyperprior, is used instead, with κ_c and κ_a held fixed and the twin's mode as the starting point; in reduced form, where no closed form exists, the twin's mode is used and a note is recorded with the fit.

Options the MCMC engine rejects. `hierarchy="mh"` and the COVID-19 scaling are rejected on MCMC compositions, where SV with outliers treats the 2020 observations; Appendix C.1 lists every construction-time rejection.

The unit-scale Student-t. With `tails="t"` and `t_scale="unit"`, $\tau_{i,t} \sim \mathrm{IG}(\nu_i/2, \nu_i/2)$, the scale mixture of Jacquier et al. (2004), in place of the standardised default $\mathrm{IG}(\nu_i/2, (\nu_i - 2)/2)$ with

Table A.1: Notation. "Code" gives the field or variable name in rai-bvar; defaults are those of the committed specification classes.

Symbol	Meaning	Code (default)
n, p, n_d	variables, lags, deterministic terms ($n_d \in \{0,1,2\}$)	`lags`, `deterministic` (`"n"`, `"c"`, `"ct"`)
$k = n_d + np$	regressors $x_t = (d_t', y_{t-1}', \dots, y_{t-p}')'$	
T (T_{eff}), T_{pad}	effective observations; padded length (bucket of 32 rows)	`T_eff`, `T_pad`
$q = k + n - 1$	coefficient slots of a structural equation ($k+i$ active in equation i)	
A, a_{ij} $(j < i)$	unit lower-triangular contemporaneous matrix	`eq.struct.a`
Γ, γ_i; $B' = A^{-1}\Gamma$; Π	structural, reduced-form ($k \times n$) coefficients	`eq.mean`, `glob.mean.Pi`
M, m_i, $\tilde{a}_i$	reduced-form prior mean, its column i, $\tilde{a}_i = (a_{i1}, \dots, a_{i,i-1}, 1, 0, \dots, 0)'$	`mean` (from tcodes)
$\delta_i = \gamma_i - M_{<i} a_i$	δ-coordinates (unit Jacobian)	`eq.mean.delta`
κ, α	overall tightness, lag decay	`lambda_` (0.2), `alpha` (2.0)
κ_c, κ_a, κ_d	cross-lag tightness (scales the s.d.), contemporaneous scale, deterministic-term variance	`cross` (1.0), `contemp` (1.0), `var_const` (10^7)
ψ_j	AR(p) residual variance of variable j in the window	`psi="ar"`
ϑ	Minnesota hyperparameters that the hierarchy fixes, estimates by EB or samples (κ, α, ψ, SOC/-SUR weights)	`hierarchy` (`"eb"`)
$\Sigma \sim \text{IW}(\Psi, d)$	$\Psi = \text{diag}\,\psi$, $d = n + 2$, $\text{E}[\Sigma] = \Psi/(d - n - 1)$	`ConjugateProblem.d`
$\text{IG}(c, C)$	inverse gamma, shape c, scale C; density $\propto \lambda^{-c-1} e^{-C/\lambda}$	
$\lambda_{i,t}$, $\lambda_{i,t}^{\text{eff}}$	structural variance; effective variance $e^{h_{i,t}} o_{i,t}^2 \tau_{i,t}$	`log_lam`, `h`
μ_i, ϕ_i, σ_i, B_σ	SV level, persistence, volatility of volatility, $\sigma_i^2 \sim B_\sigma \chi_1^2$	`SV(...)` priors
s_t, (p_j, m_j, v_j), c_i	mixture indicator, mixture table, offset $c_i = 10^{-12}\psi_i$	`mixture` (OCSN-10)
$o_{i,t} \in \{1, \dots, K\}$, p_i	outlier scale ($K = 20$) and outlier probability	`outlier_max`
$\tau_{i,t}$, ν_i	Student-t mixing scale, degrees of freedom $\nu \in \{3, \dots, 40\}$	`tails="t"`, `nu_prior`
S, H, $L_{s,h}$, $m_{s,h}$	forecast draws, horizon, conditional Cholesky factor and mean	`cond_chol`, `cond_mean`
$\tilde{m}_{s,h}$, $M_{s,h}$	path-integrated mean and factor of draw s at horizon h (D-34)	from `lag_coefs`
C, M_c, M_1, G	Geweke: chains, sweeps per chain, iid draws, test functions	`GewekeConfig`
N, L	SBC replications and kept draws per replication	`SBCConfig`

$\text{E}\,\tau_{i,t} = 1$.

Outliers. The grid $o_{i,t} \in \{1, \dots, 20\}$ is the discretisation by Carriero et al. (2024) of a uniform outlier scale on $[2, 20]$. The Beta prior on the outlier probability p_i of `SV(outliers=True)` encodes, by default, one outlier every 4 years over 10 prior years (every 10 years with t errors).

A.2 Data input and the FRED loader

`VARData.from_frame(raw, tcodes={...})` raises an error on gaps in the index, duplicate columns, infinite values, logs of non-positive values and missing values inside the sample; leading and trailing

Table A.2: Transformation codes, their pathwise inverses and the own-first-lag prior mean they imply. x is the raw series, y the transformed one, T the last observed period; sums and products run over $j = 1, \dots, h$.

tcode	y_t	Pathwise inverse for x_{T+h}	History	Own-lag mean
1	x_t	$x_{T+h} = y_{T+h}$	0	1
2	Δx_t	$x_{T+h} = x_T + \sum_j y_{T+j}$	1	0
3	$\Delta^2 x_t$	$\Delta x_{T+h} = \Delta x_T + \sum_j y_{T+j}$, then $x_{T+h} = x_T + \sum_j \Delta x_{T+j}$	2	0
4	$\log x_t$	$x_{T+h} = \exp(y_{T+h})$	0	1
5	$\Delta \log x_t$	$x_{T+h} = x_T \exp(\sum_j y_{T+j})$	1	0
6	$\Delta^2 \log x_t$	as code 3 on $\log x$, then exp	2	0
7	$\Delta(x_t/x_{t-1} - 1)$	$r_{T+h} = r_T + \sum_j y_{T+j}$ with $r_t = x_t/x_{t-1} - 1$, then $x_{T+h} = x_T \prod_j (1 + r_{T+j})$	2	0

incomplete periods are trimmed, with a warning except for the leading periods that the transformations themselves consume. `realized(origin, h)` returns the future outcomes on the transformed scale. The `variables` argument of `rb.datasets.load` takes a named set or a list of series (the order given is the ordering), and `tcodes` takes `"rai"`, `"official"` or a mapping. The named variable sets (`"small"`, `"medium"`, `"large"`) contain only series published by U.S. federal agencies, and interior publication gaps are interpolated (linearly, in logs for logged series) with a warning that lists every filled period (`fill_gaps=True`).

B Derivations

Each derivation below states the committed test that checks it, or says that none exists. None of the results is claimed as new except where a subsection says otherwise; the sources are credited in each.

B.1 The structural form of the normal–inverse-Wishart prior

Let $\Sigma \sim \mathrm{IW}(\Psi, d)$ with $\Psi = \mathrm{diag}(\psi)$, and $\mathrm{vec}(B) \mid \Sigma \sim \mathrm{N}(\mathrm{vec}\, M, \Sigma \otimes \Omega)$, the conjugate Minnesota prior of the analytic engine. Write $\Sigma^{-1} = A' D^{-1} A$ with A unit lower triangular and $D = \mathrm{diag}(\sigma_1^2, \dots, \sigma_n^2)$, and set $\Gamma = AB'$, so that $A\Sigma A' = D$.

Proposition B.1. *Independently over $\iota = 1, \dots, n$,*

$$\sigma_\iota^2 \sim \mathrm{IG}\Big(\frac{d - n + \iota}{2}, \frac{\psi_\iota}{2}\Big), \qquad a_\iota \mid \sigma_\iota^2 \sim \mathrm{N}\big(0, \sigma_\iota^2\, \mathrm{diag}(\psi_1^{-1}, \dots, \psi_{\iota-1}^{-1})\big), \qquad \gamma_\iota \mid a_\iota, \sigma_\iota^2 \sim \mathrm{N}(M\tilde{a}_\iota, \sigma_\iota^2 \Omega). \tag{B.1}$$

Proof sketch. The inverse-Wishart partition theorem (Gupta and Nagar, 2000) gives the first two statements when applied twice. The leading $\iota \times \iota$ block is $\Sigma_{1:\iota} \sim \mathrm{IW}(\Psi_{1:\iota}, d - n + \iota)$. Partition that block as $(\iota - 1, 1)$: the Schur complement $\sigma_\iota^2 = \Sigma_{\iota\iota \cdot <\iota}$ is $\mathrm{IW}_1(\psi_\iota, d - n + \iota)$, that is, the inverse gamma above, and the regression coefficient $a_\iota = -\Sigma_{<\iota}^{-1} \Sigma_{<\iota,\iota}$ is conditionally $\mathrm{N}(0, \sigma_\iota^2 \Psi_{<\iota}^{-1})$; the sign is irrelevant for a zero-mean law. The theorem also makes $(a_\iota, \sigma_\iota^2)$ independent of $\Sigma_{1:\iota-1}$, of which every earlier (a_j, σ_j^2) is a function, which gives independence across equations. For the third statement, $\gamma_\iota = B\tilde{a}_\iota$, so $\mathrm{E}[\gamma_\iota \mid A, D] = M\tilde{a}_\iota$ and $\mathrm{Cov}(B\tilde{a}_\iota, B\tilde{a}_j \mid \Sigma) = (\tilde{a}_\iota' \Sigma \tilde{a}_j)\, \Omega = \delta_{\iota j} \sigma_\iota^2 \Omega$ because $A\Sigma A' = D$. Finally, $(B, \Sigma) \leftrightarrow (A, D, \Gamma)$ is a bijection and the likelihoods coincide because $|A| = 1$, so the posteriors coincide as well. ∎

The direction "inverse-Wishart implies the structural normal–inverse-gamma form" is Corollary 1 of Chan (2022), and his Proposition 1 is the converse; the conditional centring $\mathrm{E}[\gamma_\iota \mid a_\iota] = M\tilde{a}_\iota$ with the random-walk mean (identity on the first lag, zeros elsewhere) is the centring of the structural prior of Sims and Zha (1998), who normalise the structural shocks to unit variance instead of fixing the diagonal of A at one; rescaling a_ι and γ_ι together leaves a linear centring unchanged. Sims and Zha (2006) restate it, in their notation, as $A_+ = D + SA_0$ with a zero-mean prior on D; their D is not the diagonal D above, and its column ι is $\gamma_\iota - M\tilde{a}_\iota$ up to the scale of the row. Three consequences are used in the paper. (i) With $d = n+2$, $\sigma_\iota^2 \sim \mathrm{IG}(1+\iota/2, \psi_\iota/2)$ and $\mathrm{E}[\sigma_\iota^2] = \psi_\iota/\iota$; this is the Constant prior of every Mode. (ii) Mode II at $\kappa_c = \kappa_a = 1$ is exactly (B.1) in δ-coordinates, $\delta_\iota \mid \sigma_\iota^2 \sim \mathrm{N}(m_\iota, \sigma_\iota^2\Omega)$, which is what makes the MCMC twin of the conjugate model an end-to-end oracle. (iii) $\mathrm{E}[B' \mid A] = A^{-1}\mathrm{E}[\Gamma \mid A] = M'$ holds for every A if and only if $\mathrm{E}[\Gamma \mid A] = AM'$, so D3 centring is the only linear centring that keeps the reduced-form prior mean at M conditionally on A. The equivalence is tested in two ways. The structural normal–inverse-gamma log marginal likelihood equals the Phase 1 NIW value to 10^{-10} in five cases (constant only; SOC and SUR with own-lag means $(1, 0.5, 0, 0.9)$; trend and SUR; no deterministic terms with `soc_scope="all"`; $n = 1$) and under perturbed hyperparameters. Kolmogorov–Smirnov tests on functionals of Σ and on coefficient marginals, from 20,000 prior draws, give a minimum p-value above 10^{-3}.

B.2 Mode I moment matching and the implied reduced-form prior

Mode I keeps the coefficients independent of λ_ι. With $\bar{\sigma}_\iota^2 = \psi_\iota/\iota$ it sets $a_{\iota j} \sim \mathrm{N}(0, \kappa_a \bar{\sigma}_\iota^2/\psi_j)$ and $\delta_\iota \sim \mathrm{N}(m_\iota, V_\iota)$, where V_ι has entries $\kappa^2/(\iota\, l^\alpha)$ on the own lag l, $\kappa^2\kappa_c^2\psi_\iota/(\iota\, l^\alpha \psi_m)$ on lag l of variable $m \neq \iota$ and $\kappa_d\psi_\iota/\iota$ on the deterministic terms; under Constant volatility $\lambda_\iota \sim \mathrm{IG}(1+\iota/2, \psi_\iota/2)$, whose mean is $\bar{\sigma}_\iota^2$. At $\kappa_c = 1$, $V_\iota = \bar{\sigma}_\iota^2\Omega$ with Ω the conjugate prior's Ω at $d = n+2$. Write $u_t = A^{-1}e_t$ for the reduced-form residual and β_j for column j of B.

1. *Mean.* From $\Gamma = \Delta + (A - I)M'$, where Δ stacks the δ_ι,
$$B' = A^{-1}\Gamma = M' + A^{-1}(\Delta - M'), \qquad \mathrm{E}[B \mid A] = M \text{ for every } A. \tag{B.2}$$

2. *Covariance prior.* The rows a_ι have mean zero and are independent of $u_{<\iota}$, so $\mathrm{E}[\Sigma_{\iota\iota}] = \mathrm{E}[\lambda_\iota] + \sum_{j<\iota}\mathrm{E}[a_{\iota j}^2]\,\mathrm{E}[\Sigma_{jj}]$. At $\kappa_a = 1$ this is $\psi_\iota/\iota + \sum_{j<\iota}(\psi_\iota/(\iota\psi_j))\,\psi_j = \psi_\iota$ by induction, and the off-diagonal terms vanish by sign symmetry, so $\mathrm{E}[\Sigma] = \operatorname{diag}\psi$. The "naive" choice $a_{\iota j} \sim \mathrm{N}(0, \psi_\iota/\psi_j)$ with $\mathrm{E}[\lambda_\iota] = \psi_\iota$ instead gives $\mathrm{E}[\Sigma_{\iota\iota}] = 2^{\iota-1}\psi_\iota$, the growth pointed out by Chan et al. (2024).

3. *Marginal variances.* From (B.2), $\mathrm{Var}(\beta_j \mid A) = c_j(A)\,\Omega$ with $c_j(A) = \sum_\iota (A^{-1})_{j\iota}^2 \bar{\sigma}_\iota^2$ at $\kappa_c = 1$. Since $(A^{-1})_{j\iota} = \delta_{j\iota} - \sum_{l<j} a_{jl}(A^{-1})_{l\iota}$ with independent zero-mean rows, $c_j = \mathrm{E}[c_j(A)]$ satisfies $c_j = \bar{\sigma}_j^2 + \sum_{l<j}(\bar{\sigma}_j^2/\psi_l)\,c_l$, the recursion of item 2, so $c_j = \psi_j$. Because $\mathrm{E}[B \mid A]$ does not depend on A, $\mathrm{Var}(B_{(l,m),j}) = \kappa^2\psi_j/(l^\alpha\psi_m)$, the NIW marginal variance.

So, at $\kappa_c = \kappa_a = 1$, Mode I reproduces three moments of the NIW prior: the reduced-form mean (for every A), the marginal coefficient variances and $\mathrm{E}[\Sigma]$. It is not the NIW distribution.

Cross-lag leak. With $\kappa_c < 1$, $\kappa_a = 1$, $m_a = 0$ and $d = n+2$, the same calculation with the own-lag entry separated out gives, for equation j and variable m (1-based),

$$\frac{\mathrm{Var}(B_{(l,m),j})}{\kappa^2\psi_j/(l^\alpha\psi_m)} = \kappa_c^2 + (1-\kappa_c^2)\,s_{jm}, \qquad s_{jm} = \frac{\mathrm{E}[(A^{-1})_{jm}^2]\,\bar{\sigma}_m^2}{\psi_j} = \begin{cases} 1/j & m = j, \\ 1/(m(m+1)) & m < j, \\ 0 & m > j. \end{cases} \tag{B.3}$$

Here s_{jm} is the prior share of structural shock m in $\mathrm{Var}(u_j)$, so $\sum_m s_{jm} = 1$. The closed form follows from the share recursion $r_{j,m} = j^{-1} \sum_{l=m}^{j-1} r_{l,m}$ with $r_{m,m} = 1/m$, whose solution is $1/(m(m+1))$ for every $j > m$; it holds in Mode II as well, because $(A^{-1})_{j\iota}$ does not involve row ι of A. For example, at $\kappa_c = 0.5$ every later equation's coefficients on the lags of variable 1 get relative variance 0.625 instead of the intended 0.25. The implied reduced-form moments are reported by `fit.summary()`.

B.3 Tests of the prior equivalences

Besides the tests stated with Appendix B.1 (the Mode II prior and its closed-form log marginal likelihood) and Appendix B.2 (the Mode I moments), two committed tests check the equivalences of Section 3.2: the analytic log marginal likelihood satisfies the Chib identity, and the Σ implied by `iw_equivalent` passes Kolmogorov–Smirnov tests against $\mathrm{IW}(\mathrm{diag}\,\psi, n+2)$. The Mode II identity makes the analytic engine an exact posterior oracle for the Mode II `Constant` compositions of the MCMC engine; the Mode I and `iw_equivalent` tests check priors only (Section 8).

B.4 The exact mixture correction and the single ASIS accept/reject

Consider one equation with cleaned residuals $\tilde{e}_t$ (transitory scales removed), log-volatility path h and SV parameters $\theta = (\mu, \phi, \sigma)$. The sampler works with $y_t^* = \log(\tilde{e}_t^2 + c)$ and an auxiliary model $y_t^* = h_t + r_t$, where r_t has the mixture density $g(r) = \sum_j p_j \,\mathrm{N}(r; m_j, v_j)$ (Kim et al., 1998; Omori et al., 2007). Define the augmented target

$$\pi(\theta, h, s) = p(\theta, h \mid \tilde{e}) \prod_t \frac{p_{s_t} \,\mathrm{N}(y_t^* - h_t; m_{s_t}, v_{s_t})}{g(y_t^* - h_t)}, \tag{B.4}$$

whose marginal in (θ, h) is the exact posterior, because each factor of the product is a probability mass function in s_t. Drawing $s \mid h, \theta$ from that product is therefore an exact Gibbs step. Let $q(h \mid s, \theta) \propto p(h \mid \theta) \prod_t \mathrm{N}(y_t^* - h_t; m_{s_t}, v_{s_t})$ be the Gaussian conditional of the auxiliary model, which the precision sampler draws. Under (B.4), the conditional of h given (s, θ) is $q(h \mid s, \theta) \exp\{w(h)\}$ up to a constant, with

$$w(h) = \sum_t m_t \Big[\log \mathrm{N}(\tilde{e}_t; 0, e^{h_t}) - \log g(y_t^* - h_t) \Big], \qquad \log \mathrm{N}(\tilde{e}_t; 0, e^{h_t}) = -\tfrac{1}{2}\big(\log 2\pi + h_t + \tilde{e}_t^2 e^{-h_t} \big), \tag{B.5}$$

where m_t masks padded rows. An independence Metropolis–Hastings step with proposal q therefore accepts h' with probability $\min\{1, \exp(w(h') - w(h))\}$. This is Algorithm 3 of Del Negro and Primiceri (2015); its acceptance probability has the form of eq. (11) of Stroud et al. (2003). Because the same g appears in the indicator law and in w, the invariant law is exact for any mixture table and any offset $c \geq 0$; the table and the offset change only the acceptance rate and the mixing. This is why mutants that corrupt the table or the indicator law are not bugs in exact mode (Appendix D.8).

The non-centred step. The ancillarity–sufficiency interweaving step (Yu and Meng, 2011; Kastner and Frühwirth-Schnatter, 2014) holds $\tilde{h} = (h - \mu)/\sigma$ fixed and redraws (μ, σ) from the s-conditional Gaussian regression $y_t^* - m_{s_t} = \mu + \sigma \tilde{h}_t + \sqrt{v_{s_t}}\, z_t$ with a Gaussian prior on (μ, σ). That draw is an exact Gibbs step for the auxiliary augmented target $\tilde{\pi}$. The exact target is $\pi = \tilde{\pi} \cdot \exp\{w(h)\} \cdot \mathrm{const}$, and the reparametrisation Jacobian σ^{-T} appears in both and cancels, so using the $\tilde{\pi}$-conditional as a proposal under π gives the acceptance ratio $\exp\{w(\mu' + \sigma'\tilde{h}) - w(\mu + \sigma\tilde{h})\}$. If the non-centred step is itself a Metropolis–Hastings step under $\tilde{\pi}$, with ratio α_{aux}, a single accept/reject with

$$\log \alpha = \log \alpha_{\mathrm{aux}} + w(h_{\mathrm{new}}) - w(h_{\mathrm{old}}) \tag{B.6}$$

is used; two sequential tests, as in delayed acceptance (Christen and Fox, 2005), would also be valid but define a different kernel. The value $w(h_{\text{old}})$ must be that of the state after the h step of the same sweep; a stale value is a bug. The centred (γ, ϕ) and σ^2 steps need no correction: they do not move h, and the indicator law in (B.4) does not involve (μ, ϕ, σ). On acceptance, a negative σ' is folded by $(\sigma', \tilde{h}) \mapsto (-\sigma', -\tilde{h})$. The fold leaves $h = \mu + \sigma\tilde{h}$ unchanged and is a measure-preserving involution of the non-centred posterior, whose prior is symmetric under the joint sign change (Frühwirth-Schnatter and Wagner, 2010); folding σ alone is not. The combination (B.6) was derived in the repository's design notes and checked there on a toy posterior by quadrature; it is gated by the identity tests above, by the toy quadrature twin (A6), by Geweke tests in exact mode, and by mutant M7 with a coarse mixture table. It is the standard Metropolis–Hastings ratio under π for the auxiliary step's proposal. Published samplers correct the h draw (Del Negro and Primiceri, 2015), or correct it and update the parameters under the exact model in separate steps (Hosszejni and Kastner, 2019), and `stochvol` offers an optional, off-by-default correction for the mixture approximation (Hosszejni and Kastner, 2021). We do not claim it as new.

B.5 The path-integrated log predictive density, and a bound on its delta-method MCSE

Derivation. Fix a forecast draw s: reduced-form lag matrices $A_1, \dots, A_p$, deterministic terms, and a drawn path of effective volatilities, which determines the conditional Cholesky factors $L_k = \texttt{cond_chol}_{s,k}$ for $k = 1, \dots, h$. The simulated path satisfies $y_{T+k} = \text{cond_mean}_k + u_k$ with $u_k = L_k \varepsilon_k$, where cond_mean_k is the VAR mean evaluated on the simulated path up to $T + k - 1$. Let $\Phi_0 = I$ and $\Phi_j = \sum_{l=1}^{\min(j,p)} A_l \Phi_{j-l}$. By linearity, $y_{T+h} = \tilde{m}_h + \sum_{k=1}^{h} \Phi_{h-k} u_k$, where $\tilde{m}_h$ is the zero-shock path, and $\text{cond_mean}_h = y_{T+h} - u_h$. Hence

$$\begin{aligned} \tilde{m}_h &= \text{cond_mean}_h - \sum_{k<h} \Phi_{h-k} u_k, \\ y_{T+h} \mid \theta_s, \lambda_s \sim \mathrm{N}(\tilde{m}_h, M_h M_h'), &\qquad M_h = [\Phi_{h-1} L_1 \mid \Phi_{h-2} L_2 \mid \cdots \mid \Phi_0 L_h], \end{aligned} \tag{B.7}$$

because the u_k are independent $\mathrm{N}(0, L_k L_k')$ given the volatility path, and the deterministic terms cancel. For a subset $\mathcal{I}$ of variables, the lower factor of $(M_h M_h')_{\mathcal{I}\mathcal{I}}$ is R' from the QR decomposition $M_h[\mathcal{I}, :]' = QR$ with $\operatorname{diag}(R) > 0$, so $M_h M_h'$ is never formed and no additional random numbers are drawn. The estimator is $\operatorname{logsumexp}_s \log \mathrm{N}(y; \tilde{m}_{s,h}, M_{s,h} M_{s,h}') - \log S$ (equation (15) in the notation of Section 6.4). At $h = 1$, $M_1 = L_1$ and $\tilde{m}_1 = \text{cond_mean}_1$, so it coincides with the path-conditioned estimator, and the library runs the conditioned code there. It is exact given each draw's parameters and volatility path, and it is still a Monte Carlo average over draws. Given each draw's volatility path, the conditioning is the one Hiraki et al. (2026, Appendix D) use for predictive moments; with constant volatility the estimator is that of Warne et al. (2017), specialised to a VAR. In a synthetic persistent VAR at $h = 8$ with an outcome 6 standard deviations out, the test's docstring records a path-conditioned estimate of -35.2 (MCSE 1.0) against the exact -22.3; the test asserts a gap of more than 5 nats and an MCSE above 0.5, and that the integrated estimator is exact.

Further checks and scope. The value -19.1 quoted in Section 6.4 for the replication's 2008Q2 cell is free of the path-conditioning error only; it is itself a Monte Carlo estimate over the same 1000 draws. The commit message of b9c66c0 records an agreement of 1.8×10^{-14} with the companion-form oracle (test tolerance 10^{-10}). The estimator is equivariant to rescaling variables by $10^{\pm 6}$, and on real MCMC fits each draw's density matches moments obtained by `jax.jacfwd` of the forecast core to 10^{-10}. The estimator is opt-in; the SPEC default is unchanged. It needs each

draw's lag matrices, which MCMC forecasts of linear or zero means carry (`lag_coefs`, Table C.3); analytic, benchmark and ordering-pooled forecasts raise at $h > 1$. Combining the estimator with `rb_transitory` raises at $h > 1$ because it would require integrating the transitory scales of $T + 1, \ldots, T + h - 1$ jointly.

A bound on the delta-method MCSE. The library reports the delta-method MCSE $\mathrm{sd}(w)/(\sqrt{S}\,\bar{w})$ with $w_s = \exp(\log p_s - \max_r \log p_r)$ and the sample standard deviation (`ddof=1`); for MCMC draws it multiplies this by $\sqrt{S/\mathrm{ESS}}$, with ESS the effective sample size of the weights over the (chain, draw) array.

Proposition B.5. *Let $w_1, \ldots, w_S \geq 0$, not all zero, with mean $\bar{w}$, sample variance $s_w^2 = (S-1)^{-1}\sum_s (w_s - \bar{w})^2$ and Kish effective sample size $\mathrm{ESS}_K = (\sum_s w_s)^2 / \sum_s w_s^2$ (Kish, 1965). Then*

$$\frac{s_w^2}{S\,\bar{w}^2} = \frac{S/\mathrm{ESS}_K - 1}{S-1} \leq 1, \tag{B.8}$$

with equality if and only if exactly one w_s is non-zero. With the population variance $\sigma_w^2 = S^{-1}\sum_s (w_s - \bar{w})^2$ in place of s_w^2, the bound is $\sigma_w/(\sqrt{S}\bar{w}) \leq \sqrt{(S-1)/S}$.

Proof. $(S-1)s_w^2 = \sum_s w_s^2 - S\bar{w}^2 = S^2\bar{w}^2/\mathrm{ESS}_K - S\bar{w}^2$; dividing by $S(S-1)\bar{w}^2$ gives the identity. For non-negative weights $\sum_s w_s^2 \leq (\sum_s w_s)^2$, that is $\mathrm{ESS}_K \geq 1$, with equality if and only if $\sum_{s \neq r} w_s w_r = 0$, that is, if at most one weight is non-zero. The population case follows from $\sigma_w^2 = (S-1)s_w^2/S$. ∎

Four consequences follow. (i) The iid delta-method MCSE of a Rao–Blackwellised log score cannot exceed 1 nat, however far off the estimate is; it approaches 1 as the weight concentrates on one draw, and reaches it in floating point when the other weights underflow. A reported MCSE near 1 at a tail origin is therefore the ceiling of the formula, not an estimate of the error. (ii) The iid MCSE is a function of ESS_K alone; for large S it is approximately $(1/\mathrm{ESS}_K - 1/S)^{1/2}$, so the descriptive RB-degenerate threshold of 0.9 used in the replication corresponds, before the MCMC adjustment, to $\mathrm{ESS}_K \approx 1/0.81 \approx 1.2$. (iii) The factor $\sqrt{S/\mathrm{ESS}}$ applied to MCMC draws is not covered by the proposition and can raise the reported value above 1. (iv) An MCSE is a model of the error, not a certificate: at tail origins the weight ESS should be inspected directly. The bound, the identity, the `ddof=1` convention and consequence (iii) are unit-tested against the library's `log_pred_density`.

B.6 Dummy rows: the closed-form λ marginal and an exact rejection envelope

The SOC/SUR rows (y_d, F_d) of equation i (m rows) are pseudo-observations that update a base prior $\theta \sim \mathrm{N}(\mu_0, V_0)$ of the equation's coefficients; write $r = y_d - F_d\mu_0$.

Mode II. Here $V_0 = \lambda\tilde{V}_0$ and the rows have noise variance λ/w, so $d \mid \lambda \sim \mathrm{N}(F_d\mu_0, \lambda\tilde{S})$ with $\tilde{S} = F_d\tilde{V}_0F_d' + I/w$. With the base prior $\lambda \sim \mathrm{IG}(c, C)$,

$$\lambda \mid d \sim \mathrm{IG}\big(c + m/2,\; C + \tfrac{1}{2}r'\tilde{S}^{-1}r\big), \tag{B.9}$$

and $p(d)$ has a closed form. The Geweke reference therefore samples the prior with the rows exactly: draw $\lambda \mid d$, then condition θ on the rows by Matheron's rule, $\theta = \theta_0 + V_0F_d'(F_dV_0F_d' + RI)^{-1}(y_d - F_d\theta_0 - \varepsilon)$ with $\theta_0 \sim \mathrm{N}(\mu_0, V_0)$ and $\varepsilon \sim \mathrm{N}(0, RI)$, which is exact Gaussian conditioning (Journel and Huijbregts, 1978; Chilès and Delfiner, 2012; Wilson et al., 2020). The reference is written in this covariance form ($m \times m$ systems), while the kernels use the precision form ($q \times q$ Gram updates), so the two implementations differ in algebra, not only in authorship.

iw_equivalent. Only the a-block is λ-scaled and the rows have a fixed weight, so $d \mid \lambda \sim \mathrm{N}(r; 0, S(\lambda))$ with $S(\lambda) = S_0 + \lambda UU'$, $S_0 = F_\delta V_\delta F_\delta' + I/w$ and $U = F_a\tilde{V}_a^{1/2}$. This is not of

inverse-gamma form. Let $q(\lambda) = r'S(\lambda)^{-1}r$ and $q_\infty = \min_u (r-Uu)'S_0^{-1}(r-Uu)$, the generalised least-squares residual.

Proposition B.6. *For every $\lambda > 0$,*

$$g(\lambda) = \Big(\frac{|S_0|}{|S(\lambda)|}\Big)^{1/2} \exp\big\{-\tfrac{1}{2}\big(q(\lambda) - q_\infty\big)\big\} \le 1, \tag{B.10}$$

so drawing $\lambda \sim \mathrm{IG}(c, C)$ and accepting with probability $g(\lambda)$ yields exact draws from $p(\lambda \mid d) \propto \mathrm{IG}(\lambda; c, C)\,\mathrm{N}(r; 0, S(\lambda))$.

Proof. First, $|S(\lambda)| = |S_0|\,|I + \lambda U'S_0^{-1}U| \ge |S_0|$ because $U'S_0^{-1}U \succeq 0$. Second, for $\lambda > 0$ the function $f(u) = (r-Uu)'S_0^{-1}(r-Uu) + u'u/\lambda$ is minimised at $u^* = (U'S_0^{-1}U + I/\lambda)^{-1}U'S_0^{-1}r$, with minimum $r'S_0^{-1}r - r'S_0^{-1}U(U'S_0^{-1}U + I/\lambda)^{-1}U'S_0^{-1}r = r'(S_0 + \lambda UU')^{-1}r = q(\lambda)$ by the Woodbury identity. Dropping the non-negative term $u'u/\lambda$ gives $q(\lambda) \ge q_\infty$. Hence $\mathrm{N}(r; 0, S(\lambda)) \le (2\pi)^{-m/2}|S_0|^{-1/2}e^{-q_\infty/2}$, and $g(\lambda)$ is the ratio of the two sides. The usual rejection argument completes the proof. ∎

The same argument shows that $q(\lambda)$ is non-increasing and tends to q_∞ as $\lambda \to \infty$, which explains the name. Proposals are drawn in batches; the first accepted proposal of a batch is the sequential rejection draw, so the batched sampler is exact. The repository notes record an acceptance rate of 0.6–1.0 on the test rows. The log prior with the rows is exact up to the intractable evidence, which is enough for every density-ratio identity and every Geweke and SBC statistic. The reference is checked against the closed forms to 10^{-10} and by Kolmogorov–Smirnov tests of the λ draws against a quadrature CDF; Geweke row 205 gives a G-small maximum $|z|$ of 2.24, and the repository notes record G-full 2.76 and an SBC minimum p of 0.018 for the same row. In the sampler, Mode II rows are weighted $1/\lambda_i$ like data rows, so the conditional of λ_i gains $m/2$ in its shape and the rows' λ-free squared residuals in its scale.

B.7 The successive-conditional chain of a scalar conjugate model

Take $\theta \sim \mathrm{N}(0, \tau^2)$ and $y \mid \theta \sim \mathrm{N}(\theta, s^2)$, where s^2 stands for the data variance ($s^2 = s_0^2/T$ for T observations), and let $\zeta = \tau^2/(\tau^2 + s^2)$. One successive-conditional sweep of a Geweke test draws $y_{m+1} \sim \mathrm{N}(\theta_m, s^2)$ and then $\theta_{m+1} \mid y_{m+1} \sim \mathrm{N}(\zeta y_{m+1}, \zeta s^2)$, so

$$\theta_{m+1} = \zeta(\theta_m + e_m) + \sqrt{\zeta s^2}\, z_m, \qquad e_m \sim \mathrm{N}(0, s^2),\ z_m \sim \mathrm{N}(0,1). \tag{B.11}$$

This is an AR(1) with coefficient ζ and stationary variance $\zeta^2(\tau^2 + s^2) + \zeta s^2 = \tau^2$, as it must be. With $r = \mathrm{Var}_{\mathrm{prior}} / \mathrm{Var}_{\mathrm{post}} = \tau^2/(\zeta s^2) = 1/(1-\zeta)$,

$$\zeta = 1 - 1/r, \qquad \mathrm{IAT}(\theta) = \frac{1+\zeta}{1-\zeta} = 2r - 1, \qquad \mathrm{RNE} \approx \frac{1}{2r-1}. \tag{B.12}$$

If the posterior sampler is itself autocorrelated with coefficient ρ, the chain is AR(1) with coefficient $1 - (1-\zeta)(1-\rho)$. The design rule $r \le 5$ (IAT ≤ 9) for every scalar parameter follows, and it is targeted with small T (20 for G-small, 50 for G-full) and proper priors that are tight relative to T observations. This is a derivation for the scalar conjugate model; that the rule holds for ϕ, σ, p and ν in the registered compositions is not verified. The minimum relative numerical efficiencies measured at G-small range from 0.008 to 0.129 across compositions; in the four compositions for which the lowest-efficiency functions were extracted (`structural_constant`, `reduced_constant`, `structural_sv` and the univariate `zero_sv`) they occur on intercepts, on the data function $\mathrm{mean}_t \tanh y_t$ and, in `zero_sv`, on h levels, not on ϕ or σ, so they are not direct evidence either way; under the parallel-chain design low efficiency affects power, not size.

B.8 Why the CCM-2019 triangular step is exact under a flat prior

Write the reduced form in triangular form, $\tilde{y}_t = Ay_t$, and take two equations. Equation 2 reads $\tilde{y}_{2,t} = a_{21}x_t'\pi^{(1)} + x_t'\pi^{(2)} + \lambda_{2,t}^{1/2}\varepsilon_{2,t}$. The map $(\pi^{(1)}, \pi^{(2)}) \mapsto (\pi^{(1)}, \gamma_2 = a_{21}\pi^{(1)} + \pi^{(2)})$ has unit Jacobian, and because all equations share the regressors x_t and A does not vary over time, equation 2 depends on $(\pi^{(1)}, \pi^{(2)})$ only through γ_2, whatever the volatility weights. Under a flat prior on $(\pi^{(1)}, \pi^{(2)})$ the posterior of $\pi^{(1)}$ therefore depends on equation 1 alone, and drawing $\pi^{(1)}$ from equation 1, as the triangular algorithm of Carriero et al. (2019) does, is exact. Under a proper prior $\pi^{(2)} \sim \mathrm{N}(\mu_2, V_2)$, independent of $\pi^{(1)}$, the term the algorithm omits is

$$p(\tilde{y}_2 \mid y_1, \pi^{(1)}) = \mathrm{N}\big(\tilde{y}_2;\ X(a_{21}\pi^{(1)} + \mu_2),\ XV_2X' + \Lambda_2\big), \tag{B.13}$$

which depends on $\pi^{(1)}$ whenever $a_{21} \neq 0$. In general the triangular step draws $\pi^{(j)}$ from equation j of the triangular system alone, and its conditional is correct only for the last equation. The error is thus a prior effect: it is large with tight Minnesota priors and short samples and fades as the data dominate. Averaged over simulated datasets, per-dataset shifts of the posterior mean in both directions can cancel in the ranks of the parameters themselves, while a data-dependent quantity such as the log-likelihood keeps its power: an incorrect posterior passes SBC for a given quantity only when its rank deviations over different parts of the data space cancel, and a quantity that combines it with an aspect of the data can make them add up instead (Modrák et al., 2025, Sections 3.2–3.3, Theorem 5). This is the mechanism behind lesson 3 of the mutation suite: the block SBC of the π_0 draw catches M1 through the log-likelihood rank ($p = 2.8 \times 10^{-11}$, control minimum $p = 0.10$), whereas one-block invariance of the whole scan did not in a recorded run that has not been repeated ($q = 0.29$). The derivation is recorded in the design notes, not in a committed test. Bognanni (2022) showed that the triangular algorithm does not, in general, sample from the intended posterior, and the corrected algorithm is that of Carriero et al. (2022). With a time-varying A_t the shift $a_{21,t}x_t'\pi^{(1)}$ cannot be absorbed into one γ_2, and the flat-prior exactness fails.

B.9 The excess-pivot recursion and the QR whitened draw

The recursion. Let $Q = \mathrm{Lap}_w + \mathrm{diag}(\delta)$ be the scaled precision of the random-walk log-volatility path, with $\delta_t \geq 0$, edge weights $w_t \geq 0$ ($w_{-1} = w_T = 0$), $Q_{tt} = \delta_t + w_{t-1} + w_t$ and $Q_{t,t-1} = -w_{t-1}$. In the SV application $\delta_0 = \sigma^2/V_0$, $\delta_t = \sigma^2/v_{s_t}$ at observed periods (0 at masked ones; notation of Section 5.5) and the live edges have unit weight.

Proposition B.9. *The Cholesky pivots p_t (squared diagonal entries of the lower factor) of Q are $p_t = q_t + w_t$, where*

$$q_0 = \delta_0, \qquad q_t = \delta_t + \frac{w_{t-1}\, q_{t-1}}{w_{t-1} + q_{t-1}} \quad (q_t = \delta_t \text{ if } w_{t-1} = 0), \tag{B.14}$$

the subdiagonal of the factor is $-w_{t-1}/\sqrt{p_{t-1}}$, and $\log|Q| = \sum_t \log(q_t + w_t) = \sum_t[\log w_t + \mathrm{log1p}(q_t/w_t)]$, with $\log q_t$ for the terms with $w_t = 0$.

Proof. For a tridiagonal matrix, $p_0 = Q_{00}$ and $p_t = Q_{tt} - Q_{t,t-1}^2/p_{t-1}$. Substituting $p_t = q_t + w_t$ gives $q_t = \delta_t + w_{t-1} - w_{t-1}^2/(q_{t-1} + w_{t-1}) = \delta_t + w_{t-1}q_{t-1}/(w_{t-1} + q_{t-1})$. ∎

Every operation in (B.14) combines non-negative numbers by addition, multiplication and division, so nothing cancels. The assembled matrix, by contrast, stores $Q_{tt} = \sigma^2/v_{s_t} + 2$: when σ^2/v_{s_t} falls below half an ulp of 2, rounding returns the singular path Laplacian, and the level pivot, of order $\sigma^2(1/V_0 + \sum_t 1/v_{s_t})$, is lost. The recursion is an instance of subtraction-free Gaussian elimination for diagonally dominant M-matrices parametrised by off-diagonal entries and row

sums (Grassmann et al., 1985; O'Cinneide, 1996; Alfa et al., 2002; Ye, 2008): with off-diagonal entry $-w_{t-1}$, pivot-row sum q_{t-1} and pivot p_{t-1}, the Schur-complement row-sum update of Demmel et al. (2008, Section 2.3.4) is (B.14), with δ_t the row sum of row t. We apply it within the precision-sampler setting of Rue (2001); we claim the application, not the algorithm, and we do not prove an error bound here. In exact arithmetic the pivots are those of the assembled factorisation, so the sampler is the same function of the injected noise.

The QR whitened draw. For the contemporaneous row a_i with prior precision diag(prec), prior mean $\bar{a}$, regressors Z and weights w_t, let $\mathcal{A} = [\mathrm{diag}(\sqrt{\mathrm{prec}});\ \sqrt{w} \odot Z]$ and $\rho = [\sqrt{\mathrm{prec}} \odot \bar{a};\ \sqrt{w} \odot (\tilde{y}_i - g_i)]$, so that $\mathcal{A}'\mathcal{A} = K$ and $\mathcal{A}'\rho = b$ are the conditional precision and the canonical mean. With the Householder decomposition $\mathcal{A} = \mathcal{Q}R$, $\mathrm{diag}(R) > 0$, R is the upper Cholesky factor of K, and $\mathcal{Q}'\rho = (R')^{-1}b$, so

$$a = R^{-1}(\mathcal{Q}'\rho + z) = K^{-1}b + R^{-1}z. \tag{B.15}$$

The Jacobi-scaled precision draw computes $\theta = s \odot (L_s')^{-1}(L_s^{-1}(s \odot b) + z)$ with $s = \mathrm{diag}(K)^{-1/2}$ and $L_s L_s' = sKs$; since $s^{-1}L_s$ is the lower Cholesky factor of K, this is also $K^{-1}b + R^{-1}z$. The two draws are therefore the same function of z in exact arithmetic, but the QR route works with $\mathrm{cond}(\mathcal{A}) = \mathrm{cond}(K)^{1/2}$ and never squares it (Higham, 2002; Björck, 1996).

C Algorithms

The pseudocode below follows the committed code of the MCMC engine (file and symbol named in each caption) and is written for one chain and, where stated, one equation; equations and chains are batched by `vmap`. All arithmetic is float64. Every kernel is written as `core(noise, ...)` applied to `draw_noise(key, ...)`, so the noise z or u in each algorithm is drawn from the key named in the text and could be injected by a test. A step reports success through a flag `ok` and the smallest pivot it met; a failure is recorded as a status code and never raises inside compiled code (Algorithm C.11). Mask $m_t \in \{0, 1\}$ marks observed rows of the padded sample. Figure C.1 shows the plan as the package prints it, and Table C.3 the predictive contract that the forecast core (Algorithm C.12) fills.

C.1 Construction-time resolution

The constructor of `BVAR` resolves the `"auto"` defaults (a Minnesota form left at `"auto"` becomes `"conjugate"` with a `Constant` prior left at `"auto"` or set to `"iw"`, and `"structural"` otherwise; `Constant("auto")` then becomes `"iw"` or `"ig"`), applies the nine rules of Table C.1, and, for MCMC compositions, rejects what the engine does not support: Lenza–Primiceri scaling and the Metropolis–Hastings hyperparameter hierarchy (both planned for a later phase), and empirical Bayes over ψ with cross-lag tightness $\kappa_c \neq 1$ in structural form.

C.2 The Gibbs sweep and the coefficient steps

Algorithm C.1 is one sweep; Algorithms C.2 to C.4 are the structural coefficient and contemporaneous draws, and Algorithm C.5 the reduced-form step. In the δ-coordinates of Section 3.2, equation i is the regression

$$y_i = (XM_{<i} - Y_{<i})\, a_i + X\delta_i + e_i, \tag{C.1}$$

whose contemporaneous regressors are minus the prior-mean residuals of the earlier variables; Algorithm C.2 draws all equations with the shared design F of its caption.

#	Prior form	Volatility	Engine and mode
1	conjugate	`Constant("iw")`	analytic (conjugate NIW)
2	conjugate	any other	`ValueError`
3	structural	`Constant("ig")`	MCMC, Mode I
4	structural	`Constant("iw")`	MCMC, Mode II (NIW twin), labelled `scaling=conjugate`
5	structural	`Constant("iw_equivalent")`	MCMC, Mode I δ prior, (a, λ) prior of Chan (2022)
6	structural	`SV(...)`	MCMC, Mode I
7	reduced	`Constant("ig")`, `SV(...)`	MCMC, corrected triangular (CCCM) step
8	reduced	`Constant("iw_equivalent")`	MCMC, CCCM, Chan's (a, λ) prior; order-invariant
9	reduced	`Constant("iw")`	`ValueError`

Table C.1: Engine resolution: rules that map a resolved (prior form, volatility prior) to an engine and mode; the first match applies. `BVAR()`, `BVAR(vol=Constant("ig"))` and `BVAR(vol=SV())` reach rules 1, 3 and 6.

Algorithm C.1 One Gibbs sweep of one chain (`engines/mcmc/sweep.py`, `_sweep_fn`). The order of the steps is the `Plan` built by `build_plan`.

Input: chain key k_c, absolute sweep index it (uint32), state, data, prior, cache

for each global pre-step g (reduced form: the CCCM step, Algorithm C.5) **do**
    update g's block with key `step_key`$(k_c, it, g.\text{block})$; record its status
end for
$\tilde{Y} \leftarrow$ pre-residual target (Y, or $Y - X\Pi$ in reduced form), masked
for all equations i in parallel **do**
    for each step of `plan.equation`, in order **do**
        rebuild the equation context from the *current* state; weights $m_t \exp(-\log \lambda_{i,t}^{\text{eff}})$
        key $\leftarrow$ `eq_key(step_key`$(k_c, it, \text{block}), i)$
        `equation`: draw $(\delta_i, a_i) \mid \lambda^{\text{eff}}$ (Algorithms C.2 and C.3); or `contemp`: draw $a_i \mid g_i$ (Algorithm C.4)
        `residual`: $e_i \leftarrow \tilde{y}_i - (\text{cols} \odot \text{cmask})\, a_i - g_i$ (zero on padded rows)
        `vol`: update the volatility block atomically (Algorithm C.6)
    end for
end for
keep the first non-zero code $1 + 1000 \cdot \text{block} + \text{eq}$ per chain and the running minimum pivot

Algorithm C.2 Structural (δ_i, a_i) draw with D3 centring, weighted path (`components/mean/linear.py`; `linalg.weighted_gram`, `precision_draw`). $F = [X \mid XM_c - Y_{:,1:n-1}]$ is shared by all equations; c_θ marks the active slots of equation i.

per fit: packed products $FF_{t,c} = F_{t,r_c} F_{t,s_c}$, $r_c \le s_c$, column-major ($T_{\text{pad}} \times q(q+1)/2$)
$\text{prec}_\theta \leftarrow [\text{pm.prec} \cdot e^{-\log \lambda_i \, 1\{\text{Mode II}\}} \mid \text{cm.prec}]$; $\text{mean}_\theta \leftarrow [\text{pm.mean} \mid \text{cm.mean}]$
$w_t \leftarrow m_t \exp(-\log \lambda_{i,t}^{\text{eff}})$ ▷ masked before the exponential
$G \leftarrow (w'FF)$ unpacked to $q \times q$; $f \leftarrow F'(w \odot y_i)$ ▷ one GEMM under `vmap`
$K \leftarrow (c_\theta c_\theta') \odot G + \text{diag}(\text{prec}_\theta)$; $b \leftarrow \text{prec}_\theta \odot \text{mean}_\theta + c_\theta \odot f$
$z \sim \text{N}(0, I_q)$ from `sub_key`(key, `Sub.Z`)
$s \leftarrow \text{diag}(K)^{-1/2}$; $L \leftarrow \text{chol}(sKs)$ ▷ Jacobi scaling, an exact change of variables
$\theta \leftarrow s \odot (L')^{-1}(L^{-1}(s \odot b) + z)$ ▷ $\sim \text{N}(K^{-1}b, K^{-1})$
ok $\leftarrow \text{diag}\, L$ finite and > 0, θ finite; min_pivot $\leftarrow \min \text{diag}(L)^2$
$\theta \leftarrow \theta \odot c_\theta$; $\delta_i \leftarrow \theta_{1:k}$; $a_i \leftarrow \theta_{k+1:q}$; $\gamma_i = \delta_i + M_c a_i$
coupling $\leftarrow 0$ (Mode I), or $(n_{\text{active}}, \frac{1}{2} \sum \tilde{P}_\theta (\theta - \text{mean}_\theta)^2)$ over the λ-scaled slots (Mode II, iw_equivalent)

Structural SV(outliers, t): BVAR(Minnesota(structural, eb), SV(outliers,t), p=2)

```
>>> print(fit.plan.describe())
phase     step        block            writes
equation  equation    EQ_MEAN(0)       eq.mean, eq.struct.a
                                       (δ_i, a_i) | λ_eff,i joint Gaussian (fused contemporaneous draw)
equation  residual    EQ_MEAN(0)       —
                                       e_i = ỹ_i + Σ_{j<i} a_ij ỹ_j − g_i from the new draws
equation  vol         EQ_VOL(2)        eq.vol
                                       VolBlock.sample_eq: o → p → ν → τ, then s → h → θ_SV (atomic)
post      (none)


read sets (Step.reads of the same plan):
  equation    reads eq.coef, eq.mean, eq.struct, eq.vol, glob
  residual    reads eq.mean, eq.struct, glob.mean
  vol         reads eq.vol, glob.vol
```

Reduced-form SV: BVAR(Minnesota(reduced, eb), SV, p=2)

```
>>> print(fit.plan.describe())
phase     step        block            writes
pre       cccm        G_CCCM(16)       glob.mean.Pi
equation  contemp     EQ_CONTEMP(1)    eq.struct.a
                                       a_i | g_i, λ_eff,i: regress ỹ_i − g_i on −ỹ_{<i}
equation  residual    EQ_MEAN(0)       —
                                       e_i = ỹ_i + Σ_{j<i} a_ij ỹ_j − g_i from the new draws
equation  vol         EQ_VOL(2)        eq.vol
                                       VolBlock.sample_eq: o → p → ν → τ, then s → h → θ_SV (atomic)
post      (none)


read sets (Step.reads of the same plan):
  cccm        reads eq.struct.a, eq.vol, glob.mean.Pi
  contemp     reads eq.coef, eq.mean, eq.struct, eq.vol, glob
  residual    reads eq.mean, eq.struct, glob.mean
  vol         reads eq.vol, glob.vol
```

Figure C.1: The Gibbs plan as data: the output of `fit.plan.describe()` (each step's block and write set) for a structural SV(outliers, t) model and a reduced-form SV model, fitted to the simulated data of Listing 2, followed by each step's declared read set (`Step.reads`). The mixture indicators are local to the `vol` step, so no step runs between the indicators and h.

Algorithm C.3 Graded eigen fast path for Mode I × Constant (`linalg.eigen_prepare`, `eigen_draw`). G_{const} is the time-constant Gram and $\text{FtY}_i = F'y_i$.

---

per fit, per equation:

1: $d \leftarrow \text{prec}_\theta$; $\tilde{G} \leftarrow d^{-1/2}(c_\theta c_\theta' \odot G_{\text{const}})\, d^{-1/2}$

2: graded decomposition $\tilde{G} = Q\,\text{diag}(g)Q'$: unit-diagonal rescaling, Cholesky (eigendecomposition fallback), Householder QR of the rescaled factor sorted by decreasing scale, QR-iteration SVD of R'

3: $u_1 \leftarrow Q'(d^{1/2} \odot \text{mean}_\theta)$; $u_2 \leftarrow Q'(d^{-1/2} \odot c_\theta \odot \text{FtY}_i)$

per sweep, $\lambda = \lambda_i$:

4: $v \leftarrow \lambda/(\lambda + g)$ ▷ g clipped at 0

5: $\theta \leftarrow d^{-1/2} \odot Q\,(v \odot u_1 + u_2/(\lambda + g) + \sqrt{v} \odot z)$ ▷ exact: $K = d^{1/2}(I + \tilde{G}/\lambda)d^{1/2}$

6: ok $\leftarrow$ g finite, $g \geq -10^{-10} \max|g|$, θ finite

---

The graded decomposition is needed because $\kappa_d = 10^7$ makes one diagonal entry of $\tilde{G}$ about 10^9 times the others; its accuracy is observed against 40–45-digit solves, not proven. With dummy rows the same per-fit decomposition is applied after factoring the dense prior precision, $K = L_P(I + L_P^{-1}G(L_P')^{-1}/\lambda)L_P'$.

The scan is the textbook SUR Gibbs conditional with weight $A'\Lambda_t^{-1}A$, so it is a block Gauss–Seidel sweep on the joint precision of $\text{vec}\,\Pi$ (Fox and Parker, 2017); A is never inverted. The noise row z_j comes from `sub_key(eq_key(step_key, j), Sub.Z)`. Mutant M1 replaces the scan by the triangular step of Carriero et al. (2019).

Algorithm C.4 Contemporaneous row a_i by QR, for non-fusing means and the reduced form (`components/structure/recursive.py`, `contemp_core`; `linalg.lstsq_draw`). The target is $\tilde{y}_i - g_i = -\tilde{y}_{<i}\, a_i + e_i$.

1: $Z \leftarrow \text{cols} \odot \text{cmask}$ with cols $= -\tilde{Y}_{:,1:n-1}$; $\sqrt{w_t} \leftarrow \sqrt{m_t \exp(-\log \lambda_{i,t}^{\text{eff}})}$
2: $\mathcal{A} \leftarrow [\text{diag}(\sqrt{\text{prec}});\ \sqrt{w} \odot Z]$; $\rho \leftarrow [\sqrt{\text{prec}} \odot \text{mean};\ \sqrt{w} \odot (\tilde{y}_i - g_i)]$ ▷ $\mathcal{A}'\mathcal{A} = K_a$, $\mathcal{A}'\rho = b_a$
3: Householder QR $\mathcal{A} = \mathcal{Q}R$, signs fixed so that $\text{diag}(R) > 0$ ▷ R is the upper Cholesky factor of K_a
4: $a \leftarrow R^{-1}(\mathcal{Q}'\rho + z)$ ▷ $= K_a^{-1} b_a + R^{-1} z$, Appendix B.9
5: min_pivot $\leftarrow \min_j (R_{jj}/\|\mathcal{A}_{:,j}\|)^2$; ok $\leftarrow \text{diag}(R) > 0$ and a finite
6: $a \leftarrow a \odot$ cmask; coupling as in Algorithm C.2 if the row is λ-scaled

Algorithm C.5 One corrected triangular (CCCM) scan for the reduced-form coefficients (`components/mean/linear_reduced.py`, `cccm_core`), following Carriero et al. (2022).

1: $A \leftarrow I + \text{strict-lower}(a)$; $\omega_{i,t} \leftarrow m_t \exp(-\log \lambda_{i,t}^{\text{eff}})$
2: $W \leftarrow \omega'(A \circ A)$ ▷ $W_{tj} = \sum_i a_{ij}^2/\lambda_{it} = (\Sigma_t^{-1})_{jj}$
3: **for all** j in parallel **do** ▷ K_j does not depend on Π
4: $\quad K_j \leftarrow X' \,\text{diag}(W_{:,j}) X + \text{diag}(\text{prec}_{0,j})$; $(L_j, s_j) \leftarrow$ Jacobi-scaled Cholesky of K_j
5: **end for**
6: $E \leftarrow (Y - X\Pi)A'$ ▷ recomputed from scratch every sweep
7: **for** $j = 1, \ldots, n$ (sequential) **do**
8: $\quad f \leftarrow X\pi_j$; $G \leftarrow (E \odot \omega')A_{:,j}$ ▷ $(\Sigma_t^{-1} u_t)_j$
9: $\quad b \leftarrow \text{prec}_{0,j} \odot \text{mean}_{0,j} + X'(G + W_{:,j} \odot f)$
10: $\quad \pi_j^{\text{new}} \leftarrow s_j \odot (L_j')^{-1}(L_j^{-1}(s_j \odot b) + z_j)$
11: $\quad E \leftarrow E - X(\pi_j^{\text{new}} - \pi_j)A_{:,j}'$; $\Pi_{:,j} \leftarrow \pi_j^{\text{new}}$
12: **end for**
13: ok $\leftarrow$ all factorisations ok and Π finite

C.3 Static numerical paths

In the structural coefficient draw the slots a_{ij}, $j \geq i$, are inactive (prior mean 0, precision 1, zero columns) and are zeroed after the draw, which leaves the active draw exact (Algorithm C.2). The weighted Gram $F^\top \Lambda_i^{-1} F$, $\Lambda_i = \text{diag}(\lambda_{i,\cdot}^{\text{eff}})$, comes by default from a packed product of the columns of F stored once per fit, so under `vmap` the Grams of all chains and equations are one matrix product (Algorithm C.2). Under constant volatility without a transitory scale the weights do not vary over time, so the data part of the precision is G/λ_i with G fixed per fit: $K(\lambda_i) = \text{diag}(d) + G/\lambda_i$ in Mode I, and $K = \tilde{K}/\lambda_i$ with a λ-free $\tilde{K}$ in Mode II, where the prior precision also scales with $1/\lambda_i$. A deterministic rule then picks an exact fast path: `eigen` for Mode I (one graded eigendecomposition per fit, then $O(q^2)$ per draw; Algorithm C.3), `fixed_factor` for Mode II and `cholesky_const` for `iw_equivalent`. In the reduced-form step (Algorithm C.5) the n precisions K_j are factored in parallel before the sequential scan by default; a deterministic shape rule turns this precompute off above a memory bound.

C.4 The volatility block

Algorithm C.6 is the atomic volatility update of one equation, with the transitory steps of Table C.2, the SV core of Algorithm C.7 and its random-walk path draw, Algorithm C.8.

Nothing runs between the indicator draw inside the SV core and the h draw, so the step-order rule of Del Negro and Primiceri (2015) holds by construction; the indicators are local to the core and never part of the state. The Constant core draws $\lambda_i \sim \text{IG}(c_i + T_{\text{eff}}/2 + n_{\text{active}}/2,\, C_i + \frac{1}{2}\sum_t m_t \tilde{e}_t^2 + \text{quad})$, where $(n_{\text{active}}, \text{quad})$ is the coupling returned by the coefficient step (zero in Mode I).

The collapsed steps are partially collapsed Gibbs blocks (van Dyk and Park, 2008) whose

Algorithm C.6 The volatility block of one equation, transitory then persistent, atomic (`components/vol/robust.py`, `VolBlockImpl.sample_eq`).

1: $\log e_t^2 \leftarrow \log e_t^2$ on observed rows, 0 on padded rows ▷ mask before the log (erratum E23)
2: transitory update $o \to p \to \nu \to \tau$ given the current persistent variance (Table C.2)
3: $\log \tilde{e}_t^2 \leftarrow \log e_t^2 - (2 \log o_t + \log \tau_t)$ on observed rows
4: persistent core on $\log \tilde{e}^2$: Constant (inverse-gamma draw with the coupling channel) or SV (Algorithm C.7)
5: ok $\leftarrow$ core ok and every transitory leaf finite

Table C.2: Transitory steps (`components/vol/transitory.py`); all conditionals are exact. $\log z_t^2 = \log e_t^2 - \log \lambda_t^{\text{pers}}$, r_t^2 the corresponding squared standardised residual; $c(\nu) = \nu - 2$ in the standardised parametrisation (the default) and $c(\nu) = \nu$ under `t_scale="unit"`.

Kind	Steps, in order	Conditionals
outlier	$o \mid e, h, p$; $p \mid o$	logits $\log w_k - \log k - z_t^2/(2k^2)$, $w_1 = 1-p$, $w_k = p/(K-1)$, $K = 20$; $p \sim \text{Beta}(a_p + N_1, b_p + T_{\text{eff}} - N_1)$
t	$\nu \mid e, h$ (τ integrated out); $\tau \mid e, h, \nu$	grid logits $\log \pi_g + T_{\text{eff}} A_g - \frac{\nu_g+1}{2} \sum_t m_t \operatorname{log1p}(r_t^2/c_g)$ on $\nu \in \{3, \ldots, 40\}$; $\tau_t \sim \text{IG}\big(\frac{\nu+1}{2}, \frac{c(\nu)+r_t^2}{2}\big)$
outlier $+ t$	$o \mid e, h, \nu, p$ (τ int.); $p \mid o$; $\nu \mid e, h, o$ (τ int.); $\tau \mid e, h, o, \nu$	collapsed pmf $\log w_k - \log k - \frac{\nu+1}{2} \operatorname{log1p}\big(z^2/(c\,k^2)\big)$

trimmed draws are never conditioned on before they are redrawn. The τ draws use a vectorised Marsaglia–Tsang gamma sampler (Marsaglia and Tsang, 2000) with two pre-drawn candidates per element and a loop only for elements that rejected both; the first accepted candidate of an iid sequence is the rejection sampler, so the draw is exact.

Algorithm C.7 SV update of one equation, AR(1) log-volatility, with the exact correction and ASIS (`components/vol/sv.py`, `SVCore`). Steps (c), (a), (b) and (b**) follow the design document; $w(\cdot)$ is (B.5).

1: $y_t^* \leftarrow \text{logaddexp}(\log \tilde{e}_t^2, \log c_i)$ on observed rows ▷ $c_i = 10^{-12} \psi_i$
2: (c) $s_t \leftarrow$ inverse-CDF categorical draw from $\ell_{tj} = \log p_j - \frac{1}{2} \log v_j - (y_t^* - h_t - m_j)^2/(2v_j)$; keep $\log g(y_t^* - h_t)$
3: (a) assemble the non-centred AWOL tridiagonal system $\tilde{Q} = \sigma^2 Q$, $\tilde{b}$ given s; $\tilde{x}' \leftarrow$ tridiagonal precision draw (random walk: Algorithm C.8); $h' \leftarrow \mu + \sigma \tilde{x}'$
4: **if** exact **then** accept h' iff $\log u_h < w(h') - w(h)$ ▷ Del Negro and Primiceri (2015), Alg. 3
5: **end if**
6: (b) centred (γ, ϕ) by independence MH, rejecting $|\phi'| \geq 1$; σ^2 by MH under $B_\sigma \chi_1^2$ or an exact inverse-gamma draw
7: (b**) $\tilde{h} \leftarrow (h - \mu)/\sigma$; ϕ by MH; (μ', σ') from the exact 2×2 weighted regression of $y^* - m_s$ on $(1, \tilde{h})$
8: **if** exact **then** one accept/reject with $\log \alpha = \log \alpha_{\text{aux}} + w(\mu' + \sigma' \tilde{h}) - w(h)$, $w(h)$ from after step (a)
9: **end if**
10: **if** accepted and $\sigma' < 0$ **then** $(\sigma', \tilde{h}) \leftarrow (-\sigma', -\tilde{h})$ ▷ fold; h unchanged
11: **end if**
12: refresh padded states; ok $\leftarrow$ pivot > 0, $C_T > 0$, $\sigma > 0$, $|\phi| < 1$, new state finite

The cold start draws h once from the AWOL system after the initial values, because with $h \equiv \mu$ and a rejected first exact step the σ^2 proposal is the point 0, which is then absorbing; this was found on the pilot prototype, and a regression check tests the cold start. The AWOL precision draw follows Chan and Jeliazkov (2009) and McCausland et al. (2011). For a random-walk log variance the centred step draws σ^2 only, the non-centred regression is on (h_0, σ), and the path is drawn by Algorithm C.8.

Algorithm C.8 Random-walk SV path as a path Laplacian (`linalg.laplacian_precision_sample`): $Q = \mathrm{Lap}_w + \mathrm{diag}(\delta)$, $Q_{tt} = \delta_t + w_{t-1} + w_t$, $Q_{t,t-1} = -w_{t-1}$ (Appendix B.9).

Input: $\delta_{0:T} \geq 0$; edge weights $w_{0:T-1} \in \{0, 1\}$, $w_T = 0$; b; $z \sim \mathrm{N}(0, I)$

1: $q_0 \leftarrow \delta_0$; $q_t \leftarrow \delta_t + w_{t-1} q_{t-1}/(w_{t-1} + q_{t-1})$ ($q_t = \delta_t$ after a zero edge) ▷ excess over the next edge
2: $p_t \leftarrow q_t + w_t$; $\ell_t \leftarrow \sqrt{p_t}$; $c_t \leftarrow -w_{t-1}/\ell_{t-1}$ ▷ all terms ≥ 0
3: forward: $v_t \leftarrow (b_t - c_t v_{t-1})/\ell_t$; backward: $x_t \leftarrow (v_t + z_t - c_{t+1} x_{t+1})/\ell_t$
4: $\log|Q| \leftarrow \sum_t [\log w_t + \mathrm{log1p}(q_t/w_t)]$ ($\log q_t$ where no next edge); min_pivot $\leftarrow \min_t p_t$
5: in the SV step (`sv.py`): $h \leftarrow \sigma x$ ▷ random walk: no separate level

C.5 Runner, failures and replay

Algorithms C.9 and C.10 are the compiled bodies of one chain and the host driver, and Algorithm C.11 the failure path with its deterministic replay.

Algorithm C.9 Burn-in and sampling bodies of one chain (`engines/mcmc/runner.py`, `_bodies`). Counters are uint32 throughout (erratum E7).

BURN$(k_c, \mathrm{state}, it_0, n)$:
1: **return** `fori_loop` over $r = 0, \ldots, n-1$ of SWEEP$(k_c, it_0 + r, \cdot)$ ▷ traced trip count: no recompile across burn sizes
SAMPLE$(k_c, \mathrm{state}, it_0, \mathrm{thin}, \mathrm{stop})$:
2: `scan` over $j = 0, \ldots, \mathrm{chunk} - 1$ with carry (state, snap):
3: base $\leftarrow it_0 + j \cdot \mathrm{thin}$; run thin sweeps SWEEP$(k_c, \mathrm{base} + r, \cdot)$
4: snap $\leftarrow$ state if $j = \mathrm{stop}$ ▷ exact state at the last kept draw
5: record monitors and the parameter draw; optionally run the forecast core on selected draws (Algorithm C.12)
6: **return** (state, snap, records)

Algorithm C.10 Host driver (`runner.run_chains`).

1: with float64 and one OpenBLAS thread: check the static configuration and burn $+ \lceil \mathrm{draws/chunk} \rceil \cdot \mathrm{chunk} \cdot \mathrm{thin} < 2^{32}$
2: chain keys $\leftarrow$ `fold_in` chain over (seed, `Stream.MCMC`, data salt, chain); init keys likewise from `Stream.INIT`
3: cache $\leftarrow$ PREPARE(data, prior); state $\leftarrow$ INIT, or a remapped warm state
4: plan storage from the abstract record shapes; raise `StorageBudgetError` before sampling if over budget
5: **while** $it <$ burn **do**
6: $n \leftarrow \min(\mathrm{burn} - it, \mathrm{chunk} \cdot \mathrm{thin})$; copy the start state to the host if replay is on
7: state $\leftarrow$ BURN$(\cdot, it, n)$; CHECK (Algorithm C.11); $it \leftarrow it + n$
8: **end while**
9: zero the accumulators
10: **for** each sampling chunk c **do**
11: $it_0 \leftarrow \mathrm{burn} + c \cdot \mathrm{chunk} \cdot \mathrm{thin}$; host copy if replay is on; SAMPLE; CHECK; append the kept records
12: **end for**
13: final state $\leftarrow$ snapshot after exactly burn $+$ draws $\cdot$ thin sweeps; acceptance rates and latent moments from the accumulators

Replay costs one host copy of the start state per executable call and can be disabled (`Runtime(replay=False)`); no chain is ever dropped. The replay of an equation failure after the production CCCM step originally raised a `TracerArrayConversionError` on host arrays; it now converts the state to device arrays first.

C.6 The forecast core and the predictive contract

Algorithm C.12 simulates one forecast path per parameter draw, and Table C.3 lists the predictive contract that it fills.

Algorithm C.11 Failures as values, and deterministic replay (`runner._Run`, `failure_detail`; `state.encode_error`).

inside compiled code, every block:
ok $\Leftrightarrow$ pivots finite and > 0, draws finite (and block-specific checks); a NaN pivot counts as 0
the sweep keeps the first (code $= 1 + 1000 \cdot$ block + eq, *it*) per chain; nothing raises, no jitter is added
on the host, after every executable call (CHECK):
if some chain has a non-zero code **then**
    pick the chain with the smallest failing *it*; (block, eq) $\leftarrow$ decode(code)
    replay from the host copy of the start state to sweep *it* ▷ deterministic: keys depend on *it* only
    re-run the steps of sweep *it* before the failing one, then evaluate the failing block's exposed conditional: dimension, non-finite entries, non-positive diagonal, minimum eigenvalue, weight and prior-precision ranges; for a global step, locate the failing column
    raise `MCMCNumericalError`(block, equation, iteration, detail, chain)
else if a state invariant is violated without a code **then**
    bisect the replay for the first offending sweep
end if

Algorithm C.12 Forecast core for one parameter draw (`engines/mcmc/forecast.py`, `forecast_core`).

Input: draw, last p observations, shocks $\varepsilon_{1:H}$ and volatility, mean and global keys
$(\log \lambda_i^{\text{pers}}, \log \lambda_i^{\text{eff}})_{T+1:T+H} \leftarrow$ `draw_ahead_eq`: SV $h_{T+j} = \mu + \phi(h_{T+j-1} - \mu) + \sigma\eta_j$; one uniform per period for *o*; $\log \tau = \log(c(\nu)/2) - \text{loggamma}(\nu/2)$
for $h = 1, \ldots, H$, carrying the lag window **do**
    $x \leftarrow$ regressors from the window at $T + h$
    $m \leftarrow A^{-1}\Gamma x$ (structural) or $\Pi' x$ (reduced) ▷ unit-triangular solve
    $L \leftarrow A^{-1} \operatorname{diag}(\exp(\frac{1}{2} \log \lambda^{\text{eff}}_{\cdot,T+h}))$ ▷ conditional Cholesky factor at no extra cost
    $y \leftarrow m + L\varepsilon_h$; shift y into the window
end for
return paths, cond_mean $= m$, cond_chol $= L$, scales

Given the draw, the path up to $T + h - 1$ and the drawn λ_{T+h}, $y_{T+h} \sim \mathrm{N}(m, LL')$ exactly. Forecast keys derive from (forecast seed, chain, kept-draw index), so in-loop and post-hoc forecasts agree, to 10^{-12} in the tests.

The contract tests run for every registered analytic model and for 13 MCMC compositions: structural $\times$ {SV, SV(outliers), SV(t), SV(outliers, t), SV(rw), Constant(ig), Constant(iw), Constant(iw_equivalent)} and reduced $\times$ {SV, SV(outliers), SV(rw), Constant(ig), Constant(iw_equivalent)}; reduced $\times$ SV(rw), the primary arm of the replication, was added in 7f410dc with validation row 18 (Appendix D.5). Reduced $\times$ SV(t), reduced $\times$ SV(outliers, t) and the replication's variant reduced $\times$ SV(rw, t) are not in this list.

C.7 Forecast draws, impulse responses and variance decompositions

The analytic engine draws (B, Σ) from the NIW posterior and uses $L = \text{chol}(\Sigma)$, scaled by the future Lenza–Primiceri factor when that treatment is active (Lenza and Primiceri, 2022). The MCMC forecast draws are evenly spaced stored draws of each chain (Algorithm C.12). Besides the effective variances $\lambda^{\text{eff}}_{s,T+h}$ of (12), which include the future outlier and Student-t scales, the forecast core returns the persistent variances $\lambda^{\text{pers}}_{s,T+h}$, which exclude them. Impulse responses use $\Phi_0 = I$, $\Phi_j = \sum_{l=1}^{\min(j,p)} A_l \Phi_{j-l}$ with A_l the reduced-form lag matrices; unit-shock responses $\Phi_j A^{-1}$ are date-free, while one-standard-deviation responses depend on the date and use the persistent variance by default; the forecast error variance decomposition is computed from the responses.

Table C.3: The predictive contract: fields of `ForecastResult`, which wraps an `xarray.Dataset`. The accelerator variables are carried by MCMC forecasts only, are ignored by `check_structure` and by every consumer, and are never contract fields.

Field	Dimensions	Meaning
`paths`	(chain, draw, horizon, variable)	joint simulated paths on the transformed scale; each slice is one path from one parameter draw
`paths_level`	same	pathwise inverse of the tcodes to raw levels (Table A.2)
`cond_mean`	same	$m_{s,h}$: mean of y_{T+h} given the draw, the simulated path to $T+h-1$ and λ_{T+h}
`cond_chol`	(…, variable, variable_)	$L_{s,h}$: lower-triangular factor of the conditional covariance
attrs (all engines)	—	`origin`, `freq`, `model`, `engine`, `spec`, `spec_hash`, `tcodes`, `seed`, `converged`, `warnings`, `rai_bvar_version`; analytic adds `mh_seed`
attrs (MCMC)	—	`ordering`, `n_chains`, `mixture`, `exact`, `transitory`, `outlier_K`, `t_scale`, `fseed` and the fit's reproducibility metadata (Appendix G)
Accelerator variables (MCMC only; not contract fields)		
`cond_A`, `cond_scale`	(chain, draw, variable, variable_), (chain, draw, horizon, variable)	A and the effective scales, so that cond_chol $= A^{-1}\,\mathrm{diag}$(cond_scale); $\mathrm{O}(n^2)$ factored densities
`cond_scale_persistent`; `log_p`, `log1m_p`, `nu`	(chain, draw, horizon, variable); (chain, draw, variable)	inputs of the opt-in transitory Rao–Blackwellisation (`rb_transitory=True`)
`lag_coefs`	(chain, draw, lag, variable, variable_)	A_l of each draw, used by the path-integrated estimator at $h>1$ (D-34)

Invariants. L^{-1}(paths−cond_mean) is iid $\mathrm{N}(0, I)$; draws are joint across horizons and variables; stacking (chain, draw) gives the shape (draws, H, n) in chain-major order. Analytic forecasts carry no `ordering` attribute; their ordering is the `variable` coordinate.

C.8 Scores, tests and the ordering check

Scores. All scores are computed per origin, horizon and model on the transformed scale: the joint and per-variable log score with the chosen estimator (an integrated run also stores the conditioned values from the same forecast); the fair CRPS in its probability-weighted-moment form, computed with scoringrules (Zanetta and Allen, 2024); quantile-weighted CRPS with centre and tail weights (Gneiting and Ranjan, 2011) and optional threshold-weighted CRPS; energy (Gneiting and Raftery, 2007) and variogram ($p = 0.5$; Scheuerer and Hamill, 2015) scores on a subsample of at most 1000 draws; squared and absolute errors; and PIT values with 50% and 90% coverage. An opt-in option (`rb_transitory=True`) also integrates the outlier and t scales of $T+h$ out of the log score (13) analytically.

The Diebold–Mariano statistic. For loss differences $d_t = \ell_{M,t} - \ell_{B,t}$ over P common origins (for the log score, $\ell = -\log \hat{p}$), the Diebold–Mariano statistic (Diebold and Mariano, 1995) uses a rectangular-kernel long-run variance with $h-1$ autocovariances $\hat{\gamma}_k$ and the small-sample correction of Harvey et al. (1997),

$$\mathrm{DM}_{\mathrm{HLN}} = \frac{\bar{d}}{\sqrt{\hat{\omega}/P}} \sqrt{\frac{P+1-2h+h(h-1)/P}{P}}, \qquad \hat{\omega} = \hat{\gamma}_0 + 2\sum_{k=1}^{h-1} \hat{\gamma}_k, \tag{C.2}$$

compared with Student-t_{P-1} quantiles; Bartlett (Newey–West) weights (Newey and West, 1987)

are used only if $\hat{\omega}$ is not positive. The test is DM with the HLN correction applied to log-score losses; Amisano and Giacomini (2007) give the analogous weighted likelihood ratio test for density forecasts.

The model confidence set and the volatility grid. The model confidence set uses the $T_{\max}$ statistic on the joint log-score losses with a circular block bootstrap (block length $\max(1, \text{round}(P^{1/3}))$, 2000 replications); $\mathcal{M}_{25\%}$ in (16) is the 75% set ($\alpha = 0.25$). With two models the statistic is $|Z^*|$ of one loss difference, so the route is a two-sided test at level 0.25, and the block length is the same at every horizon (Section 6.6). The comparison table prints two-sided DM p-values while the verdict uses one-sided ones (0.029 against 0.014 in Listing G.2). `verdict(axis="vol")` pairs each model with the arm that has the baseline's value on that axis and agrees on all others, and `grid()` harmonises the arms so that this pairing is meaningful: in a volatility grid the Constant baseline becomes the MCMC Mode I model `Constant("ig")`, not the analytic NIW model, and Mode II is labelled a prior change.

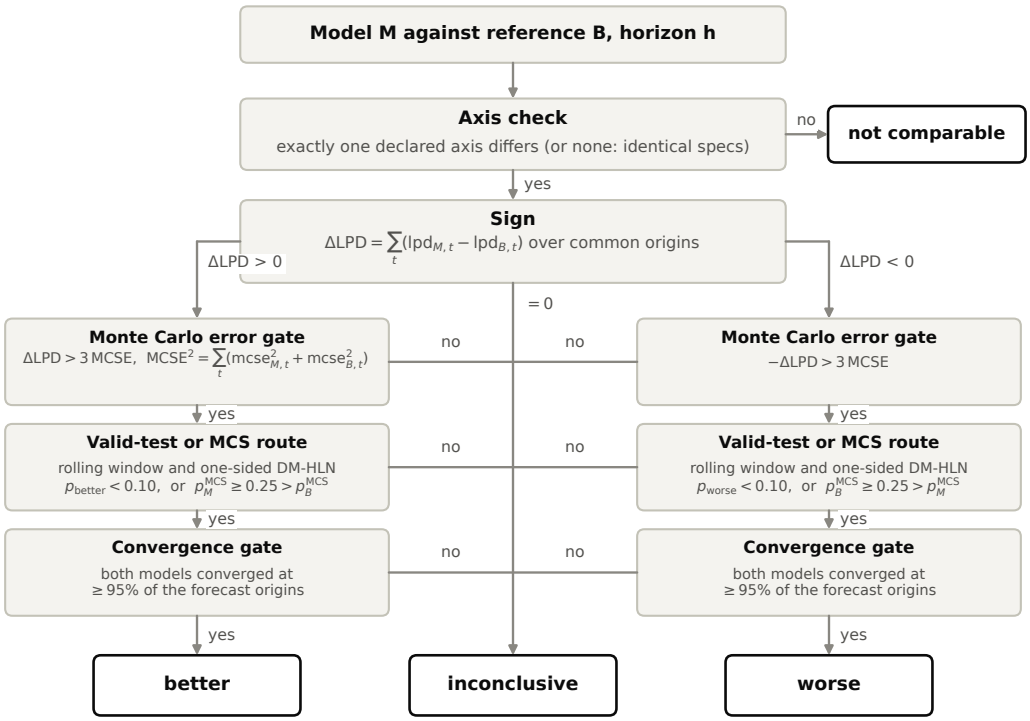


Figure C.2: The verdict rule (16) of Section 6.6, with the convergence gate, as a decision flow.

The ordering check. A metric is flagged as material only when it exceeds its threshold, 0.05 nats per variable or 0.10 for the joint score, besides the signal-to-noise condition of Section 6.7. Besides the analytic NIW engine, whose forecasts are exactly order-invariant, the reduced-form `Constant("iw_equivalent")` model is order-invariant because its implied Σ prior is inverse Wishart (Chan, 2022). In prior mode `rb.ordering_sensitivity()` simulates the prior under each ordering without a fit; in forecast or backtest mode it refits, recomputing every data-dependent quantity from the permuted window. Optionally, `rb.ordering_sensitivity()` also returns the equal-weight mixture over orderings, which is exactly order-invariant when all orderings are used. MCMC fits of compositions that are not order-invariant warn from $n = 4$ variables on (R8).

D Validation detail

This appendix gives the configurations, statistics and run records behind Section 8. Its tables and figures are generated from the release validation record: the weekly tiers run at commit 7f880e1, the code of release 0.2.0, on an Apple M5 Max on 24 October 2026, with the JSON records that the Geweke, SBC and mutation tests write when `RAI_BVAR_VALIDATION_RECORD` points to a directory and, for the tiers without such a hook, pytest's JUnit XML (Appendix G). The per-PR tier on the release commit has only pass or fail outcomes, from continuous integration on Linux. The one x86-64 run of the weekly tiers, in continuous integration at commit 779957f (before D-35), passed the slow tests of `tests/analytic` and `tests/contract` and all 43 slow Geweke tests, then reached its 330-minute limit about half-way, before the mutation, parity and SBC tests, with one failure: the replay test of F1 (Section 9.1), which replays one recorded chain and asserts its trajectory; the log does not show which assertion failed. Where a number below was reproduced on an earlier commit, the commit and the machine are stated.

D.1 Tier map and cadence

Table D.1 lists the seven tiers. Tier U, G-small and the kernel-level Tier B run on every pull request, on Linux for Python 3.12–3.14; the continuous-integration job named "nightly" runs weekly, so composition-level Tier B, G-full, SBC, the large analytic twins, long parity runs and the mutation suite run weekly. Every `RuntimeWarning` is an error, as is any silent float32 truncation in the MCMC tests.

D.2 Tier U: tolerances and infrastructure checks

One tolerance lesson generalises: two exact samplers of the same Gaussian agree only to $O(\epsilon_{\text{mach}}\,\text{cond}(K))$, with $\text{cond}(K)$ the condition number of the Jacobi-scaled precision, which reaches 10^6 for Minnesota priors on level data. Path-against-path comparisons therefore use the envelope $2 \times 10^{-12} + 5\epsilon_{\text{mach}}\,\text{cond}(K)(1 + t)$ for means (with t the largest posterior t-ratio) and $2 \times 10^{-12} + 5\epsilon_{\text{mach}}\,\text{cond}(K)$ for covariances. Infrastructure checks cover float64 on every array, a JAX-free package import, typed keys with JAX's key-reuse checker, the invariances of Section 4.6 and the failure path.

D.3 Geweke tests at the per-PR size

The G-small configuration runs $C = 1000$ independent successive-conditional chains of $M_c = 100$ sweeps, each started at a prior draw with no burn-in, against $M_1 = 2 \times 10^5$ iid marginal-conditional draws, with family level $\alpha = 10^{-3}$, t_{C-1} critical values and the sequential wrapper of Gandy and Scott (2021) with $k = 3$ stages and multiplier $\Delta = 4$. The test data have $T_{\text{eff}} = 20$, never a multiple of the 32-row bucket, $n = 2$ in structural and $n = 3$ in reduced form (with the design table's overrides: $n = 3$ for row 205 and $n = 1$ for the univariate row 17), $p = 1$ and an intercept. Table D.2 gives the result for each of the 16 per-PR compositions: the 15 registered at 62d15f6 and row 18, added in 7f410dc. On commit 62d15f6 (Apple M5 Max) every one of the 15 passed: the largest final-stage $|z|$ was 3.03 against critical values of 4.10–4.50; row 201 went to a second, fresh and larger stage after a first-stage Bonferroni q of 0.0012 ($|z| = 4.21$ on a log-scale data autocorrelation) and passed there with $q = 1.0$; and the minimum relative numerical efficiency per composition was 0.008–0.129 (where it falls is stated in Appendix B.7). Row 18 passed at stage 1 with a largest $|z|$ of 1.98 against 4.42, in one run recorded with 7f410dc (Appendix D.5). On the release commit all 16 pass in continuous integration on Linux, which records no statistics; the

Table D.1: Validation tiers: content, decision rule, cadence as designed and as run in continuous integration (the job named "nightly" runs weekly), and parametrised test IDs on the release commit, counted under the tier's pytest marker where one exists. Tiers U, B and A have no tier-wide marker (for B see note a); the G count includes the kernel-level Geweke tests.

Tier	Content	Decision rule	Cadence (designed / run)	IDs
U	density-ratio and Metropolis–Hastings identities; dense oracles with injected noise; closed forms; infrastructure	relative error below a fixed tolerance (10^{-8} for identities)	PR / PR	–
B	block-conditional SBC ($N = 200$, $L = 99$); one-block invariance ($N = 1000$, $L = 20$)	Bonferroni at $\alpha = 10^{-3}$; sequential wrapper	PR / kernel level PR, composition level weekly	46[a]
G	Geweke: G-small ($C = 1000$, $M_c = 100$, $M_1 = 2 \cdot 10^5$) and G-full ($C = 4000$, $M_c = 500$, $M_1 = 10^6$)	$\min_g p_g \leq \alpha/G$, $\alpha = 10^{-3}$ (small) or 10^{-2} (full); sequential wrapper	small PR, full nightly / small PR, full weekly	100
S	SBC, $N = 1000$, $L = 99$, two-pass thinning, ESS gates	γ test, Bonferroni at $\alpha = 10^{-2}$; no failed replication; ESS gates	nightly / weekly	23
A	analytic twins and exact oracles A1–A7	$\lvert z\rvert \leq \max(4, z_{\text{Bonf}})$ (A2–A5); log ML to 10^{-10} (A1)	PR: A1, A2, A5 ($n = 3$), A6, a small A7; nightly: A3, A4, the rest / the nightly part weekly	–
P	parity with independent implementations	quantile $\lvert z\rvert \leq \max(4, z_{1-0.01/(2m)})$; log ML rtol 10^{-8}	short PR, long nightly / short PR, long weekly	15[b]
M	18 injected mutants (Table 2)	caught iff the tier fails on the mutant and passes on the control	weekly / weekly	37[c]

[a]Composition-level tests in `tests/sbc/test_block_tier.py`. [b]Marker `parity_long`; `tests/parity` has 364 IDs in all. [c]Mutant × tier checks; the registry test runs on every PR.

release validation record covers the weekly tiers only and has no G-small row. Batch means within chains (Flegal and Jones, 2010) give a non-gating cross-check of the chains' long-run variance. The sequential design bounds the probability of a false rejection by $\sum_i \gamma^{i-1}\beta_i = \alpha$, with $\beta_1 = \alpha/k$, $\gamma = \beta_1^{1/k}$ and $\beta_{i+1} = \beta_i/\gamma$; a stage fails if $q_i \leq \beta_i$, passes if $q_i > \gamma + \beta_i$, and otherwise hands the decision to a new, independent experiment, enlarged once by Δ. At the per-PR settings $\gamma = 0.0693$ and $\beta = (3.33 \times 10^{-4}, 4.81 \times 10^{-3}, 0.0693)$, so a correct sampler is rerun at four times the size with probability at most about 7%.

D.4 Geweke test functions and the finite-variance rule

The test functions of every composition are drawn from fixed families: every scalar parameter; latent states at fixed dates ($h_{i,0}$, $h_{i,1}$, $h_{i,\lceil T/2\rceil}$, $h_{i,T}$) and the path means $\text{mean}_t\, h_{i,t}$ and $\text{mean}_t\, |h_{i,t}|$; squares where their variance is finite; cross-products within blocks ($a_{ij}\gamma_{i,(\text{lag}\,1,j)}$, $\mu\phi$, $\phi\sigma$, $p\,N_1$, $\nu \cdot \text{mean}\log\tau$); cross-equation products of reduced-form coefficients; the exact log-likelihood given the latent states and the log prior density $\log p(\theta)$; the standardised residual sums $\sum_t e_{it}^2/\lambda_{it}$; and data functions. The number of functions per per-PR composition ranges from 22 (univariate SV) to 134 (reduced form with SV and outliers). Geweke (2004) states his tests for test functions with $\text{var}\, g(\theta, y) < \infty$, the condition under which their asymptotic normality is derived. Under the SV

Table D.2: G-small Geweke results for the 16 per-PR compositions: number of test functions G, t_{C-1} critical value, maximum $|z|$ and its argument, Gandy–Scott stage and Bonferroni q, minimum relative numerical efficiency, and time, from runs at earlier commits (note under the table); all 16 pass on the release commit.

Row	Composition	G	t_{C-1}	max $\|z\|$ (argmax)	Stage: q	min RNE	Time
1	`structural_constant`	43	4.25	2.77 (`data.acov1_y[1]`)	1: 0.249	0.031	2.9 s
2	`structural_sv`	64	4.34	2.59 (`eq1.core.sigma^2`)	1: 0.623	0.034	3.6 s
3	`structural_sv_outliers`	80	4.39	2.51 (`eq1.struct.a0^2`)	1: 0.991	0.022	6.6 s
4	`structural_sv_t`	78	4.39	2.12 (`eq1.trans.nu_le_5`)	1: 1.0	0.025	10.8 s
5	`structural_mode2_constant`	38	4.22	2.13 (`eq1.mean.delta0^2`)	1: 1.0	0.033	2.1 s
7	`reduced_constant`	80	4.39	3.03 (`glob.mean.Pi[3,0]`)	1: 0.198	0.022	3.2 s
8	`reduced_sv`	110	4.46	2.74 (`glob.mean.Pi[3,1]^2`)	1: 0.697	0.021	5.1 s
9	`reduced_sv_outliers`	134	4.50	2.98 (`eq0.core.phi^2`)	1: 0.394	0.024	8.9 s
10	`reduced_sv_t`	131	4.50	2.61 (`data.corr_y[0,1]`)	1: 1.0	0.020	15.8 s
17	`zero_sv`	22	4.10	2.34 (`data.mean_tanh_y[0]`)	1: 0.427	0.008	2.5 s
18*	`reduced_sv_rw`	92	4.42	1.98 (—)	1: —	—	—
201	`structural_constant_dummies`	45	4.26 → 4.25	4.21 (`data.acf1_log_y2[1]`) → 1.94 (`eq0.mean.delta1`)	1: 0.0012 → 2: 1.0	0.103	7.5 s
202	`structural_mode2_constant_dummies`	40	4.23	2.26 (`eq0.core.log_lam`)	1: 0.953	0.129	2.3 s
203	`reduced_constant_dummies`	83	4.40	2.61 (`glob.mean.Pi[1,2]^2`)	1: 0.753	0.055	2.9 s
204	`structural_sv_dummies`	66	4.35	2.65 (`eq0.mean.delta1^2`)	1: 0.543	0.020	3.9 s
205	`structural_iw_equivalent_dummies`	72	4.37	2.24 (`eq2.mean.delta1^2`)	1: 1.0	0.099	15.5 s

16 per-PR compositions of `tests/geweke/test_geweke_compositions.py` (`PR_ROWS`). Exact mode, $C = 1000$, $M_c = 100$, $M_1 = 200{,}000$, $\alpha = 10^{-3}$, seed 20261022, Gandy–Scott $k = 3$, $\Delta = 4$; a stage after the first is a fresh run at four times C and M_1. G and the stage-1 critical value $t_{C-1}(1 - \alpha/2G)$ are recomputed from the committed code and equal the recorded ones. Time: pytest call duration.

Statistics: rows other than 18 are the G-small run of dossier 04 §6.6 on commit 62d15f6 (Apple M5 Max; reproduced twice, deterministic for fixed code and seed on that machine); *row 18 (`reduced_sv_rw`, in the PR tier since 7f410dc) is recorded only in DESIGN §9.5 and the commit message of 7f410dc (Apple M5 Max under load; stage 1, no per-stage record). The release validation record (7f880e1, Appendix D) covers the slow tiers and has no G-small row; on the release commit the 16 per-PR G-small tests passed in the fast suite of CI run 40061724239 (ubuntu-latest, Python 3.12–3.14), which records pass or fail only.

test prior $\mathrm{E}[y^2] = \mathrm{E}[e^h]$ is infinite, because σ^2 has a χ^2 tail and ϕ has prior mass near 1; under $\mathrm{IG}(3, 2)$ the variance of λ^2 is infinite; and with Student-t errors $\mathrm{E}[y^{2k}]$ is finite only for $\nu > 2k$. The rules adopted are therefore: under SV only log-scale or bounded data functions ($\mathrm{mean}_t \log y_t^2$, sample correlations, first autocorrelations, $\mathrm{mean}_t \tanh y_t$); $\log \lambda$ in place of λ^2; and a Constant test prior $\mathrm{IG}(5, 4)$. Each reference function carries a finite-variance flag. The test priors are proper and tight, $M \neq 0$ so that D3 centring is exercised, $a_{ij} \sim \mathrm{N}(0, 0.7^2)$, the outlier probability is $\mathrm{Beta}(2, 18)$ so that outliers occur at $T = 20$, $\nu \in \{5, \ldots, 12\}$, and the in-window recomputation of hyperparameters is off, so that no hyperparameter depends on y. The size of the test needs only that the sweep leaves the posterior invariant: a chain started at a prior draw is stationary from its first sweep under the null, and its mean is an unbiased estimate of $\mathrm{E}_p[g]$; ergodicity affects only power. This answers the concern of Talts et al. (2018, Section 3) about the convergence of Geweke's single successive-conditional chain. At $G = 100$ and $\alpha = 10^{-3}$ the normal critical value would give an actual size of 1.64×10^{-3} at $C = 200$ and 1.11×10^{-3} at $C = 1000$, which is why the t_{C-1} quantile is used.

D.5 G-full, the approximate rows, Tier B and SBC

G-full uses $C = 4000$, $M_c = 500$, $M_1 = 10^6$ and $\alpha = 10^{-2}$ at $n = 3$, $p = 2$ and $T_{\text{eff}} = 50$ (rows 14–16: $n = 2$; row 17: $n = 1$); the twelve `exact=False` rows (the auxiliary-mixture sampler) are recorded but do not gate, because that sampler targets the mixture approximation rather than the exact posterior; only a non-finite test function, which is a bug whatever the approximation, fails them. Composition-level Tier B consists of block-conditional SBC ($N = 200$, $L = 99$) of every exact-draw block, with ranks of the block's coordinates and of the exact log-likelihood, and one-block invariance ($N = 1000$ joint draws, $L = 20$ applications of the live blocks, two-sample Kolmogorov–Smirnov tests) for Metropolis–Hastings, ASIS and Gibbs-scan blocks; its 46 tests run

weekly, while each kernel suite runs its own Tier B per PR. SBC uses $N = 1000$ replications, $L = 99$ kept draws, burn-in 300, a pilot of 1000 sweeps, $t_{\max} = 200$ (400 for row 18) and $\alpha = 10^{-2}$ for the 23 compositions (rows 1–18 and 201–205), inside the same sequential wrapper. The thinning cap was raised from the design's 50 to 200 because, in runs recorded in the test module, a cap of 50 made the ESS rule fail on slowly mixing datasets of a t row and an outlier-t row while the γ test was clean. Row 18 has a cap of 400 by the same rule (below); the cap bounds the work, not the gate, which is stated in ESS. Each replication draws $(\tilde{\theta}, \tilde{y})$ from the joint model, applies only the data filter $\text{finite}(\tilde{y}) \wedge \max|\tilde{y}| < 10^3$, runs a pilot chain from an independent prior draw, sets its thinning to $t_i = \min(\lceil 2\,\text{IAT}_i \rceil, t_{\max})$ with the Geyer initial-monotone IAT (Geyer, 1992) of the quantities and of their tail indicators, and then keeps L draws from a fresh chain; ranks are uniform conditionally on $\tilde{y}$, so data-dependent thinning preserves uniformity. Uniformity is tested with the statistic of Säilynoja et al. (2022): for N ranks in $\{0, \dots, L\}$, $z_i = i/(L+1)$ and R_i the number of ranks below i,

$$\gamma_{\text{S}} = 2 \min_{1 \le i \le L} \min\big\{ F_{\text{Bin}}(R_i; N, z_i),\ 1 - F_{\text{Bin}}(R_i - 1; N, z_i) \big\}, \tag{D.1}$$

whose exact null distribution follows from the forward recursion of Säilynoja et al. (2022) over the binomial increments of R_i; our implementation accumulates the mass that leaves the envelope at each step, so that far-tail p-values keep relative precision. The quantities include the parameters, h at fixed dates and its path means, the total and per-equation log-likelihood, $\log p(\theta)$ and the predictive unknowns h_{T+1} and y_{T+1} through the production forecast core; data-dependent quantities are needed because a "posterior" equal to the prior passes SBC on every data-free quantity (Modrák et al., 2025, Section 3.4, Theorem 7).

The ESS erratum. The design asked for a per-replication thinned ESS of at least $0.8L$ in 99% of replications. With $L = 99$ kept draws that cannot be certified for a single replication: even with iid draws the Geyer ESS of the kept draws reaches $0.8L$ only in about three replications out of four, according to a simulation stated in the documentation of the SBC module. The per-replication ESS is therefore predicted from the pilot IAT with an AR(1) form, $\rho_i = (\text{IAT}_i - 1)/(\text{IAT}_i + 1)$ and $\text{ESS}_i = L(1 - \rho_i^{t_i})/(1 + \rho_i^{t_i})$, which is at least $0.96L$ whenever t_i is not capped, and a second gate requires the replication-pooled ESS of the kept draws, computed from autocovariances of ECDF indicators pooled over replications (a level is used only when Kish's effective number of replications is large enough), to be at least $0.8L$ for every quantity. Table D.3 collects the G-full, `exact=False`, composition-level B and SBC records of the release commit: every composition passes G-full and SBC, row 2 at the third G-full stage and row 15 at the second SBC stage, every composition-level B test passes, and no `exact=False` row has a non-finite test function. Row 205 repeats the values recorded in the repository notes (G-full 2.76, SBC minimum p 0.018) and row 18 its G-full 1.90.

Row 18. Row 18 (`reduced_sv_rw`, reduced-form CCCM × random-walk SV) is the replication's primary SV arm built through the production resolve path: its static configuration equals the arm's in every component field, and it differs where every row differs by construction, in the shapes ($n = 3$ at both sizes, so that the middle column of the CCCM step is exercised; $p = 1$ and 2, $T_{\text{eff}} = 20$ and 50, against $n = 4$, $p = 4$ and $T_{\text{eff}} = 77$ to 182 in the replication), in the inert ν grid and in the test priors (random-walk $h_0 \sim \text{N}(0, 1)$ against the library default $\text{N}(\log(\psi_i/\iota), 4)$; the tight test Minnesota prior). Its references compose the reduced-form linear reference and the random-walk SV reference, so the row needed no new reference code. In one run each recorded with 7f410dc (Apple M5 Max under load), its largest $|z|$ is 1.98 in G-small (critical value 4.42) and 1.90 in G-full (3.94), all at stage 1, and the recorded `exact=False` variant has a G-full largest $|z|$ of 2.49; the release record gives the same G-full values (Table D.3). In one-off runs on the row, with no committed test, its G-small rule fails, against the control's 1.98, for three mutants of the suite, M1 (the CCM-2019

Table D.3: G-full, the recorded `exact=False` rows, composition-level Tier B and full SBC per composition on the release commit: maximum $|z|$ or minimum p and stage, Tier B tests passed, the thinning cap and the largest thinning used, the number of replications failing the per-replication ESS rule, and the pooled ESS.

		G-full					SBC				
Row	Composition	G	t_{C-1}	max $\|z\|$, stage	`exact=False`	Tier B	min p, stage	$t_{\max}$	max thin	ESS fail	pooled ESS
1	`structural_constant`	96	3.88	2.47, 1	—	1/1	0.0357, 1	200	6	0	99
2	`structural_sv`	126	3.95	2.90, 3	2.81	1/1	0.00803, 1	200	107	0	99
3	`structural_sv_outliers`	150	3.99	2.35, 1	2.47	1/1	0.0339, 1	200	200	1	99
4	`structural_sv_t`	147	3.99	2.41, 1	2.51	1/1	0.00399, 1	200	192	0	99
5	`structural_mode2_constant`	87	3.86	2.71, 1	—	1/1	0.0323, 1	200	5	0	99
6	`structural_iw_equivalent`	87	3.86	2.62, 1	—	1/1	0.0193, 1	200	5	0	99
7	`reduced_constant`	107	3.91	2.86, 1	—	3/3	0.0315, 1	200	84	0	98
8	`reduced_sv`	137	3.97	2.67, 1	2.64	3/3	0.0346, 1	200	120	0	98
9	`reduced_sv_outliers`	161	4.01	3.30, 1	3.11	3/3	0.0158, 1	200	200	1	99
10	`reduced_sv_t`	158	4.00	2.93, 1	2.60	3/3	0.00321, 1	200	200	0	99
11	`reduced_iw_equivalent`	98	3.89	2.19, 1	—	3/3	0.0266, 1	200	60	0	99
12	`structural_sv_outliers_t`	168	4.02	2.59, 1	2.04	1/1	0.0184, 1	200	200	0	99
13	`reduced_sv_outliers_t`	179	4.03	3.04, 1	2.92	3/3	0.0294, 1	200	200	3	98
14	`structural_sv_rw`	60	3.77	2.01, 1	3.02	1/1	0.0656, 1	200	157	0	99
15	`structural_sv_invgamma`	72	3.81	2.03, 1	2.16	1/1	0.1, 2	200	137	0	99
16	`structural_sv_fixed_b0`	74	3.82	2.09, 1	1.64	1/1	0.0187, 1	200	93	0	99
17	`zero_sv`	22	3.51	2.40, 1	—	—	0.0997, 1	200	102	0	98
18	`reduced_sv_rw`	119	3.94	1.90, 1	2.49	3/3	0.0192, 1	400	400	0	98
201	`structural_constant_dummies`	99	3.89	2.23, 1	—	1/1	0.0125, 1	200	5	0	99
202	`structural_mode2_constant_dummies`	90	3.87	2.75, 1	—	1/1	0.0505, 1	200	5	0	99
203	`reduced_constant_dummies`	110	3.92	2.13, 1	—	3/3	0.0137, 1	200	37	0	99
204	`structural_sv_dummies`	129	3.96	2.70, 1	—	1/1	0.00591, 1	200	105	0	99
205	`structural_iw_equivalent_dummies`	90	3.87	2.76, 1	—	1/1	0.0178, 1	200	5	0	99

23 compositions of `src/rai_bvar/validation/joint.py`. G-full: $C = 4000$, $M_c = 500$, $M_1 = 10^6$, $\alpha = 10^{-2}$; G and the stage-1 critical value are recomputed from the committed code and equal the recorded ones; max $|z|$ is that of the last stage. `exact=False`: the inexact variant (12 SV rows), recorded, not gating; — where the row has none. Tier B: composition-level tests of `tests/sbc/test_block_tier.py` for the row, passed of run (— where the row has none); its 8 invariance tests of the volatility blocks, parametrised by block, all passed. SBC: $N = 1000$, $L = 99$, pilot 1000, $\alpha = 10^{-2}$; $t_{\max}$ from `T_MAX` and `SBC_OVERRIDES` (equal to the recorded one); max thin: the largest thinning used; ESS fail: replications failing the per-replication ESS rule; pooled ESS: minimum over quantities of the replication-pooled ESS. Release validation record at commit 7f880e1 (clean tree), Apple M5 Max, 2026-10-24: `record/geweke.jsonl`, `record/sbc.jsonl` and `junit_tests_sbc.xml` in `benchmarks/reports/validation/7f880e1_apple-m5-max/`.

triangular step, 17.8), M15 (53.4) and M2 (461), and for two errors that are not registered mutants, a dropped h_0 prior in the random-walk AWOL proposal (22.6) and a dropped last random-walk edge (30.0). At the default test prior a prior variance V_0 used as a precision, or a dropped prior mean m_0, in the random-walk AWOL step is numerically invisible, while the replication runs at $V_0 = 4$ with data-centred m_0; `test_g_small_rw_h0_prior` therefore runs G-small at $h_0 \sim \mathrm{N}(-1, 4)$ for row 18 on every PR and for row 14 weekly, and a V_0-as-precision mutant fails there (largest $|z|$ 13.8 for row 18 and 11.3 for row 14, against 2.33 and 1.39 for production; critical values 4.42 and 4.29). SBC at the shared cap $t_{\max} = 200$ had a clean γ test (minimum p 0.019 over 59 quantities), but 10 of 997 replications had pilot IATs of 187–319, one more than the 99% ESS rule allows; the row's cap was then raised to 400, at which the rule covers every pilot IAT observed and the row passes at stage 1, as it does on the release commit (minimum p 0.019). That pass rests on the ESS rule's blind spot, a pilot that underestimates the IAT: with a pilot of 4000 sweeps instead of the design's 1000, the same run fails the per-replication rule (981 of 997; pilot IAT 99th percentile 530, maximum 1438 against 319), while the γ test and the pooled-ESS gate (minimum 98.7) stay clean. A reduced-form coefficient, an intercept in 19 of 35 cases, is the slowest quantity in every replication with a pilot IAT above 91, and extending the 20 slowest chains to 16,000 sweeps leaves the IAT estimate growing in 10 of them (up to 4376). About 1–2% of prior-drawn $T = 50$ datasets therefore mix with IATs in the hundreds to thousands of sweeps under this composition, a mixing tail that the γ test does not see as a calibration error; the design's pilot of 1000 is kept, and the

limitation is stated in Section 12. Row 18 took 5.7 min of SBC at $t_{\max} = 200$ and 11.4 min at 400. The contract tests pass for the composition (10 of 10, on the release commit too), which is also fitted through the public `BVAR.fit` path.

D.6 Analytic twins and exact oracles

Table D.4 lists the twins A1–A7 with their oracle, statistic, tolerance, negative control and cadence. Two methodological details matter for reading it. First, the twins compare quantiles in CDF form: each exact quantile x_q is tested by $z = (\hat{F}_{\mathrm{MCMC}}(x_q) - q)/\sqrt{\mathrm{MCSE}^2(\hat{F}) + q(1-q)/N_{\mathrm{ref}}}$, with the indicator's own ESS. To first order this equals the quantile-MCSE test, but it avoids the density estimate of the order-statistic MCSE, whose noise made the usual statistic heavy-tailed at $n = 20$; the decision rule is $|z| \leq \max(4, z_{\mathrm{Bonf}})$ at family level 10^{-2}. Second, the twins carry negative controls whose rejection is part of the test: for example, A2 rejects an NIW posterior with one extra degree of freedom (a change of about 1.5% in Σ), and A5 rejects an inverse-gamma shape off by $\frac{1}{2}$ (about 1.6% in $\mathrm{E}[\lambda_i]$). The quadrature oracle of A5 checks itself on every call (doubling the rule and widening the interval changes nothing above 10^{-10}), and with a λ-scaled prior on all coordinates it must reproduce the closed form of Chan (2022). The twins are also the tier that caught a missing term the dense oracles of the coefficient step did not: in Mode II with dummy rows, the λ step must count the rows (Appendix B.6).

D.7 Parity with independent implementations

Parity runs both implementations on the same committed datasets with one parameter set from which both sides' settings are derived, and compares the 5, 25, 50, 75 and 95% quantiles with $z = (\hat{q}_{\mathrm{ours}} - \hat{q}_{\mathrm{ref}})/\sqrt{\mathrm{mcse}^2_{\mathrm{ours}} + \mathrm{mcse}^2_{\mathrm{ref}}}$, passing if $|z| \leq \max(4, z_{1-0.01/(2m)})$ over the m comparisons; the MCSE is the larger of the quantile MCSE of Vehtari et al. (2021) and a batch-quantile MCSE. Table D.5 summarises the rows used as evidence in this paper; the parametrisation notes in `tests/parity/` record every convention that had to be matched.

- *Analytic engine.* R BVAR 1.0.5 (Kuschnig and Vashold, 2021) and the Bank of England bvar 0.3.5 (Labonne et al., 2026). The tests assert log marginal likelihoods at fixed hyperparameters to a relative tolerance of 10^{-8}; the recorded agreement is below 10^{-11} for BVAR in the exact cases and at most 1.5×10^{-11} for the Bank of England package. Posterior moments are tested to a relative tolerance of 10^{-8} (the largest recorded deviation is 6×10^{-9}), empirical-Bayes modes to 10^{-3}, and Metropolis–Hastings and forecast quantiles within Monte Carlo error.

- *Univariate SV.* stochvol 3.2.1 (Kastner, 2016; Hosszejni and Kastner, 2021) against `ZeroMean` $\times$ SV with $n = 1$, on the main and default-prior rows of four datasets: maximum $|z|$ 1.63–2.59 at the per-PR size and 1.64–3.57 at the size of the weekly tier, against a bound of 4.00. The design document names stochvol 3.2.9, but the committed fixtures were generated with version 3.2.1.

- *Standardised Student-t SV.* The exit criterion for this sampler is an exact posterior obtained by self-normalised importance sampling on a four-observation problem, against which the maximum $|z|$ is 1.83. A small problem with an exact posterior is an oracle that does not depend on any other implementation.

- *Reduced form.* bayesianVARs (Gruber et al., 2026) could not be built in the test environment (no Fortran compiler), so no finding about it is stated. The exit criterion was met against a

Table D.4: Analytic twins and exact oracles A1–A7: oracle, statistic and tolerance, negative control and its measured power, cadence, and result on the release commit.

	Oracle	Statistic, tolerance	Negative control	Cadence	Tests	Result
A1	Structural NIG log ML (Chan, 2022) = Phase 1 analytic log ML (Mode II, $\kappa_c = \kappa_a = 1$), with and without SOC/SUR; EB λ at `cross=1`; MCMC Mode II × `Constant("iw")` with dummy rows against the Phase 1 NIW posterior	log ML abs. 10^{-10}; EB modes abs. 10^{-6}; dummy-row extension: CDF-form quantiles, $\lvert z\rvert \le \max(4, z_{\text{Bonf}})$	the same draws against the posterior without the rows must fail	PR, weekly	7	passed
A2	MCMC Mode II × `Constant("iw")` against the Phase 1 NIW posterior (B, Σ, A, Λ), $n = 3$, $p = 2$, $T = 60$	CDF-form quantiles (5–95%), $\lvert z\rvert \le \max(4, z_{\text{Bonf}})$ at family level 10^{-2}	NIW posterior with one extra degree of freedom (about 1.5% on Σ) must fail	PR, weekly	3	passed
A3	as A2 at $n = 20$, $p = 4$, $T = 300$ (the realistic-size oracle)	as A2; about 10^4 quantiles at family level 10^{-2}	—	weekly	1	passed
A4	Mode II with $\kappa_c = 0.5$ (asymmetric conjugate prior, not an NIW; Chan, 2022) against the closed form: $t_{2\hat{\nu}}$, IG($\hat{\nu}, \hat{S}$) and iid draws	as A2	—	weekly	1	passed
A5	Mode I × Constant: per-equation 1-D quadrature of $p(\lambda_i \mid y)$, exact E[λ_i], E[θ_i], Var(θ_i); the quadrature checks itself on every call	$\lvert z\rvert \le \max(4, z_{\text{Bonf}})$ with the MCSE of the mean	inverse-gamma shape off by $\frac{1}{2}$ (about 1.6% in E[λ_i]) must fail	PR, weekly	6	passed
A6	toy SV posteriors by 2-D quadrature or SIR: exact-mode invariance of the h step, the ASIS correction, the step order, T13	moments at the exact values, $\lvert z\rvert < 4.5$	wrong kernels (`exact=False` with a one-component table, uncorrected ASIS (M7), Primiceri's Algorithm 1 order) must reach $\max \lvert z\rvert > 8$	PR	7	passed
A7	reduced × Constant: exact system-wide joint draw of vec Π (NumPy test code, $O((nk)^3)$) against the production CTA Gibbs, $n \in \{2, 3, 4\}$, also inside the production sweep	MCSE z, Bonferroni (engine-level: $\alpha = 5 \times 10^{-4}$ per comparison)	the CCM-2019 Π step and an unweighted a step must fail	PR, weekly	7	passed

Tests: the number of test functions (each may be parametrised) in `tests/analytic/test_mcmc_twin.py`, `tests/analytic/test_mode1_quadrature.py`, `tests/analytic/test_structural_nig.py`, `tests/mcmc/blocks/test_cccm.py`, `tests/mcmc/blocks/test_sv_toy_quadrature.py` at commit 4ac7969; cadence from their `slow` markers (PR: every pull request; weekly: the CI job named "nightly"). CDF form: each exact quantile x_q is tested by $z = (\hat{F}_{\text{MCMC}}(x_q) - q)/\sqrt{\text{MCSE}^2 + q(1-q)/N_{\text{ref}}}$ with the indicator's own ESS. A negative control passes when the comparison rejects the perturbed oracle. Result: the weekly test functions passed in the release validation record at commit 7f880e1 (clean tree; Apple M5 Max, 2026-10-24; `junit_tests_analytic.xml` and `junit_tests_mcmc_blocks.xml` in `benchmarks/reports/validation/7f880e1_apple-m5-max/`), the PR test functions in the fast suite of CI run 40061724239 on the same commit (ubuntu-latest, Python 3.12–3.14).

substitute: an exact system-wide Gibbs sampler written in R in rai-bvar's own conventions (vec $\Pi \mid A, \Lambda$ jointly, A rows by regression, SV by one sweep of stochvol's sampler at offset 0, or $\lambda_i \mid e$ inverse gamma), which gives maximum $|z|$ 2.76, 3.45 and 3.56 against bounds of 4.05–4.51.

Parity does not establish everything: there is no external VAR-level oracle for structural Mode I × SV, and exact mode has no external oracle beyond the reweighted stochvol rows.

D.8 The mutant catalogue

A mutant is injected only through a production seam: a named kernel internal overridden through `compose(static, overrides=...)`, the volatility slot, the sampler's prior arrays, the observables simulator, or the configuration. A tier catches a mutant when its decision rule fails on the mutant and passes on the unmutated control with common seeds; for a U check, the mutant's error must exceed the tolerance and the control's must not, at the same joint draw. `test_registry` requires every mutant to have at least one confirmed catching tier, and a mutant caught by none blocks the release. The U checks use a tolerance of 10^{-8} (M8 and M11: 10^{-12}; M12: 10^{-4}; M15: 10^{-10}); the B checks are the block SBC of π_0 or one-block invariance of the volatility block; G is `geweke_sequential` at G-small; S is a short SBC ($N = 200$, four times that after stage 1, $L = 99$, burn-in 100, pilot 300, $t_{\max} = 30$). The mutant and control statistics are in Table 2; below, each entry gives the seam, the error as a mathematical statement, its source where it has one, and the catching tiers.

M1 (`cccm_core`; reduced Constant.) π_j is drawn from equation j alone, the triangular step of Carriero et al. (2019), instead of the corrected full conditional (Carriero et al., 2022). Caught by U, B (through the log-likelihood rank) and G; one-block invariance of the whole scan does not catch it (recorded; Appendix B.8).

M2 (volatility slot, `exact=False`; structural SV.) Stale indicators: s is carried across sweeps and drawn after h, the order of Algorithm 1 of Primiceri (2005) corrected by Del Negro and Primiceri (2015). Caught by G.

M3 (`JointSpec(fresh_latents=True)`; structural SV.) The observables simulator draws fresh latent states instead of using the current ones, the observables-simulator error in the constructed example of Geweke (2004). Caught by G.

M4 (`tau_conditional`; structural SV(t).) $\tau_t \mid \cdot$ has shape $\nu/2$ instead of $(\nu + 1)/2$. Caught by U and G.

M5 (sampler prior arrays; structural SV.) B_σ is doubled in the sampler only, so the sampler's prior differs from the model's. Caught by G.

M6 (`nc_fold`; univariate SV.) The non-centred step takes $|\sigma|$ without flipping $\tilde{h}$, which is not measure-preserving (Appendix B.4). Caught by U and G.

M7 (`nc_accept`, with the one-component table COARSE1 in mutant and control; univariate SV.) The non-centred move is accepted without the w-correction in exact mode. With OCSN-10 the missing correction is too small to see at test sizes. Caught by U and G.

M8 (`structural_gamma`; structural Constant.) D3 centring is dropped, $\gamma_i = \delta_i$. Caught by U, G and S.

M9 (`outlier_logits`; structural SV(outliers).) The outlier pmf omits the factor $1/k$. Caught by U and G.

M10 (`p_conditional`; structural SV(outliers).) $p \mid o \sim \text{Beta}(a_p + N_1, b_p + T)$ instead of $b_p + T - N_1$. Caught by U and G.

Table D.5: Parity summary: oracle and version, rows, statistic, bound, and maximum $|z|$ or relative tolerance.

Oracle	Quantity	Tolerance	Recorded deviation
R BVAR 1.0.5	log marginal likelihood at fixed hyperparameters	rel. 10^{-8}	$< 10^{-11}$
	NIW posterior parameters (relative to the largest entry)	rel. 10^{-8}	up to 6×10^{-9}
	empirical-Bayes modes of the hyperparameters	rel. 10^{-3}	within tolerance
	Metropolis–Hastings and forecast quantiles	4 combined MCSE	within tolerance
bvar 0.3.5 (Bank of England)	log marginal likelihood at fixed hyperparameters	rel. 10^{-8}	$\leq 1.5 \times 10^{-11}$
	posterior mean $\bar{B}$ on level data (column-scaled)	10^{-8}	2.9×10^{-9}
	means of 4000 iid posterior draws	5 MCSE	within tolerance
	hyperparameter modes	rel. 10^{-3}	within tolerance
	forecast quantiles	Monte Carlo tolerance	within tolerance

			max $\lvert z \rvert$	
Oracle and fixture	m	Bound	per-PR size	weekly size
stochvol 3.2.1, `ZeroMean` × SV, $n = 1$				
`stochvol_i_main`	40	4.00	1.86	3.57
`stochvol_i_defaults`	40	4.00	1.66	2.12
`stochvol_ii_main`	40	4.00	1.63	2.53
`stochvol_ii_defaults`	40	4.00	1.95	2.75
`stochvol_iii_main`	40	4.00	2.34	2.92
`stochvol_iii_defaults`	40	4.00	2.59	2.59
`stochvol_iv_main`	40	4.00	1.96	1.76
`stochvol_iv_defaults`	40	4.00	1.79	1.64
`stochvol_i_exact` (`exact=True`)	40	4.00	—	2.14
`stochvol_iv_exact` (`exact=True`)	40	4.00	—	2.56
exact posterior by self-normalised importance sampling (Student-t SV)				
four observations, `T_pad=320`	45	4.00	1.83	—
exact system-wide Gibbs sampler in R (reduced form; replaces bayesianVARs)				
`bvars_swa_n3p2_const`	195	4.05	2.76 (4 × 3,000)	2.75
`bvars_swa_n3p2_sv`	300	4.15	—	3.45
`bvars_swa_n7p4_sv`	1,540	4.51	—	3.56

Upper panel: `tests/parity/test_r_bvar.py` and `test_boe_bvar.py`; tolerances and recorded deviations from `tests/parity/BVAR_PARAMETRIZATION.md` §12 and `BOE_BVAR_NOTES.md` §14. Lower panel: `tests/parity/test_stochvol.py` and `test_bayesianvars.py`, results table of `SV_PARAMETRIZATION.md` §6; $z = (\hat{q}_{\text{ours}} - \hat{q}_{\text{ref}})/\sqrt{\text{mcse}^2_{\text{ours}} + \text{mcse}^2_{\text{ref}}}$ over the 5, 25, 50, 75 and 95% quantiles, bound $\max(4, z_{1-0.01/(2m)})$; per-PR size 4×6000 kept draws (4×3000 where stated), weekly size $4 \times 40{,}000$ (stochvol) or $4 \times 20{,}000$ (VAR); — not run at that size. Recorded in the notes, main repository at 4ac7969, whose machine label ("M1 Pro") is not verified here. The notes' Student-t stochvol fixture and fold fractions are not shown: they concern another package's sampler and were not re-tested on its current release. On the release commit 7f880e1 the weekly parity tests of `tests/parity` gave 13 passed, 1 skipped (no bayesianVARs fixture) and 1 expected failure (the Student-t stochvol row not shown here) in the release validation record (Apple M5 Max, `junit_tests_parity.xml`); in the fast suite of CI run 40061724239 the per-PR parity tests passed, the per-PR Student-t stochvol row again an expected failure.

M11 (`indicator_logits`; auxiliary sampler.) The indicator pmf omits $v_j^{-1/2}$. Caught by U, B and G under `exact=False`; in exact mode it is not a bug, and the G check passes there (recorded).

M12 (`mixture_table`; auxiliary sampler.) The effective component means are shifted by -1.2704 twice. Caught by U, B and G under `exact=False`; not a bug in exact mode (recorded).

M13 (`constant_conditional`; structural Constant.) The inverse-gamma shape uses T_{pad} instead of T_{eff}. Caught by U and G.

M14 (`contemp_conditional`; reduced Constant.) The A-row regression uses $+u_{<j}$ instead of $-u_{<j}$. Caught by U and G.

M15 (`nc_step`; univariate SV.) The ASIS map omits the rescaling of $\tilde{h}$. Caught by U, B and G.

M16 (configuration, `exact=False`; univariate SV.) Offset $c = 0.1$ at $\lambda \approx 0.01$ (prior level centred at $\log 0.01$) against a control offset of 10^{-12}. In exact mode a large offset is not a bug, so the mutant is defined for the auxiliary sampler only. Caught by G.

M17 (`linear_conditional`; structural Constant with dummies.) Mode I SOC/SUR rows are weighted by the current $1/\lambda_i$ instead of the fixed $1/\bar{\sigma}_i^2$. Caught by U and G; its G control runs on the dense eigen path (control statistic 1.94; 2.44 on the `cholesky_const` path).

M18 (`cccm_core`; reduced Constant with dummies.) The SOC/SUR rows are dropped from the CCCM step. Caught by U and G.

Coverage is U 14 of 18, G 18 of 18, B 4 (every mutant with a registered B check) and S 1, which is 37 mutant $\times$ tier checks. The four mutants without a U check, M2, M3, M5 and M16, carry `u_check=None` in the registry: they are errors of step order, of the simulator, of a sampler–model prior mismatch and of a model approximation, and no deterministic identity is registered for them. Whether an identity could detect them is not tested; for M5, a density-ratio identity of the σ^2 step against the model prior is a plausible detector. The design expected U catches for nine mutants; the suite confirmed fourteen, and G for all eighteen. In a review of the dummy-row work by a separate agent instance on x86-64 Linux (Appendix J), the tests also caught four further dummy-row errors that the reviewing agent injected ad hoc (Mode II rows weighted $1/\bar{\sigma}^2$, rows counted twice in the λ step, CCCM rows weighted by $(\Sigma^{-1})_{jj}$, a fixed-factor weight $2w$); these are not registered mutants.

Figure D.1 shows, for each mutant and tier, the mutant's statistic relative to the tier's decision threshold, and Figure D.2 places the G statistics of the mutants against the per-composition controls (G-full rows of the release validation record and the mutation suite's G-small controls). Recorded runs at earlier commits on x86-64 Linux, made in cloud agent sessions (M1–M16, then M17–M18; Appendix J), also caught every mutant; statistics agree across machines to two or three significant figures, not bitwise (M2: 702 / 2.48 in the committed evidence, labelled M1 Pro; 705 / 2.48 on the M5 Max; 705 / 2.47 on x86-64).

D.9 Runtime convergence diagnostics

Under stochastic volatility the primary quantities of the convergence flag of Section 6.6 include the last log-volatilities $h_{i,T}$. The convergence flag is computed by in-repository NumPy code: rank normalisation with normal scores $(r - 0.375)/(S + 0.25)$, split chains, R-hat as the maximum of the bulk and folded versions (Vehtari et al., 2021), ESS with Geyer's initial positive and monotone sequence truncation (Geyer, 1992), and tail ESS, mean ESS and the MCSE of means and quantiles. The diagnostics are tested against reference values from ArviZ 0.23.4 and arviz-stats 1.3.3 (Kumar et al., 2019), which agree with each other to the last digit on every case and are hard-coded in the tests (ArviZ is not a dependency), to a relative tolerance of 10^{-10}; against stationary Gaussian

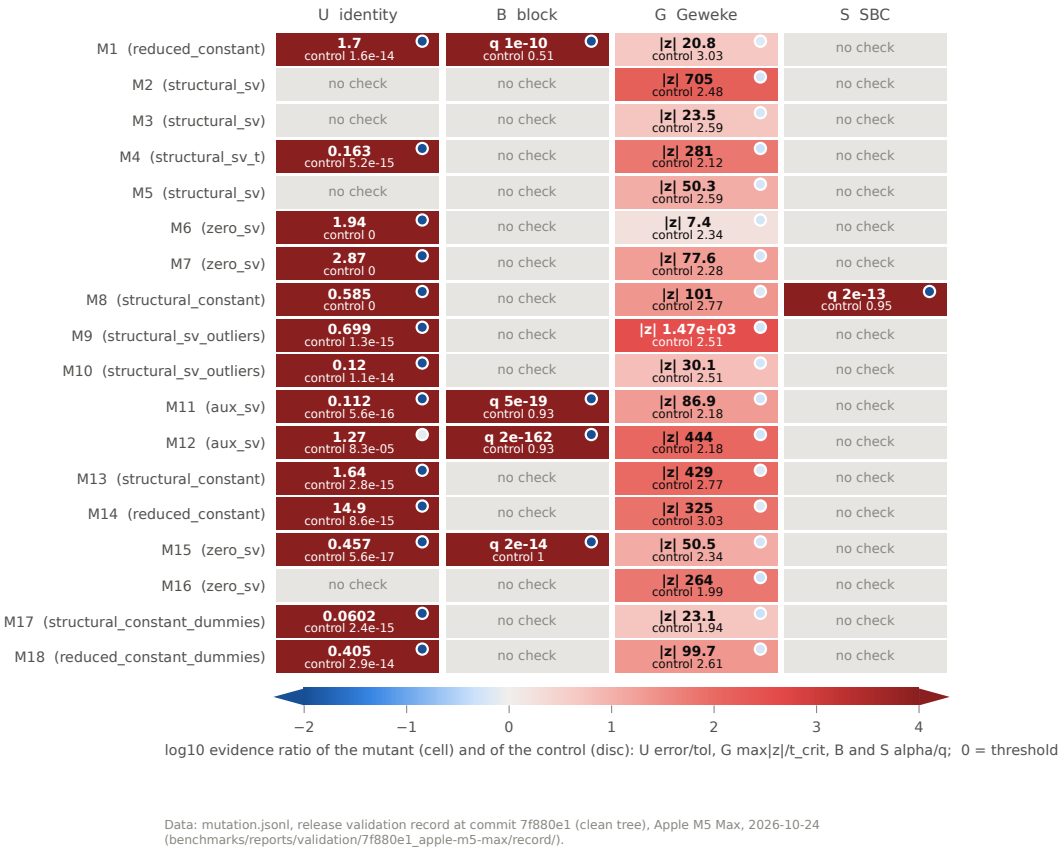
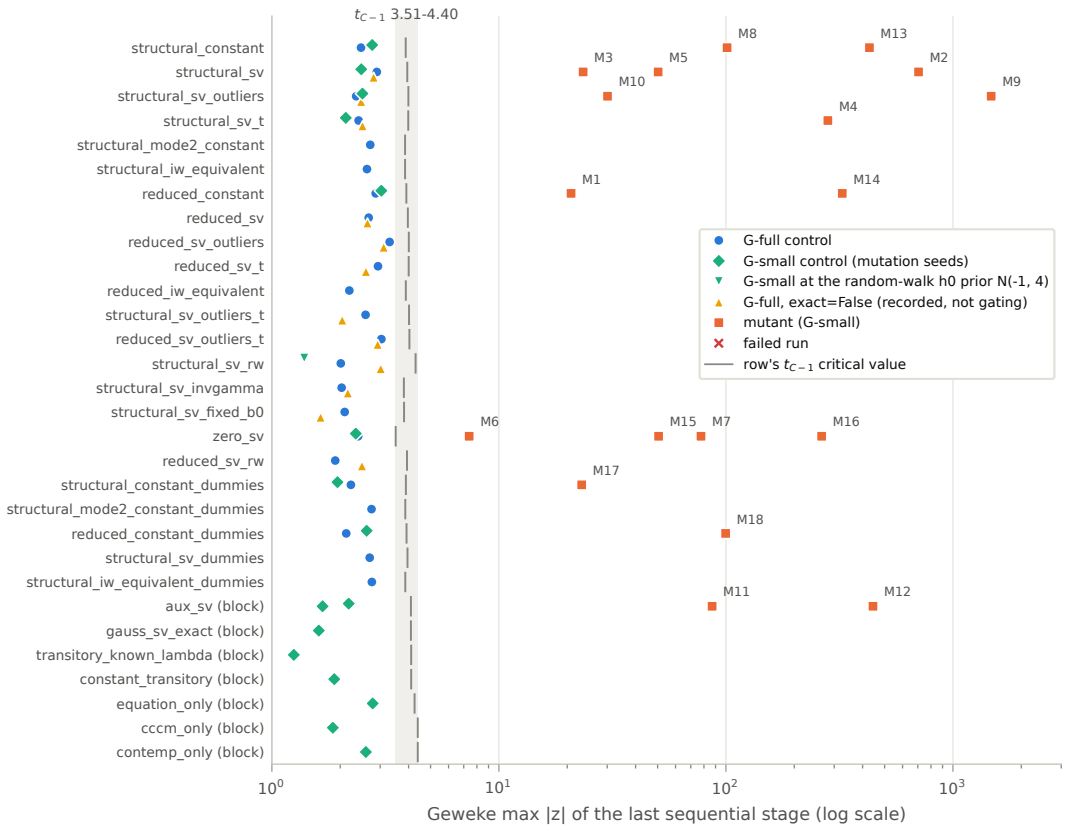


Figure D.1: Mutation suite: mutants M1–M18 × tiers U, B, G, S. Cell colour is $\log_{10}$ of the mutant's statistic divided by the tier's decision threshold; a marker shows the control; grey means that no check is registered. From the release validation record (commit 7f880e1, Apple M5 Max).

AR(1) chains with closed-form IATs, including the indicator IAT; and by statistical bounds calibrated over 30 seed sets. The gate is decided by the primary monitor set only: at $n = 20$ it has 41 quantities for Constant, 101 for SV, 61 for random-walk SV and 101 for SV with outliers and t, against secondary sets of 211, 251, 251 and 291, which are reported but never gate. A single chain is never converged, because split R-hat cannot see disagreement between chains.

D.10 Suite size and runtimes

On the release commit the suite collects 5,256 parametrised test IDs: 4,771 in the per-PR fast suite (`not slow and not network and not bench`), 482 in the CI job named "nightly", which runs weekly, and 3 benchmarks (4,964 at 62d15f6). The harness code in `src/rai_bvar/validation/` has 4,233 lines on the release commit (4,230 at c823d20, before row 18). In continuous integration on the release commit (Linux, Python 3.12, 3.13 and 3.14) the fast suite gave 4,761 passed, 9 skipped and 1 expected failure in each job, with no failures, in 49–72 min at 7f880e1 and, with the same counts, in 80–85 min at 4ac7969 itself; the skips are six live ArviZ comparisons (ArviZ not installed) and three phase-gating checks. The weekly tiers of the release validation record took about three hours on the Apple M5 Max: 83 min for `tests/geweke` (G-full 18 s to 7.8 min per composition), 82 min for `tests/sbc` (SBC 16 s to 11.2 min per composition), and 2–6 min each for the mutation suite, parity, the analytic twins and the block tier, run one directory at a time. Earlier, at commit 62d15f6, on the same kind of machine with a cold compilation cache and other



Data: geweke.jsonl, mutation.jsonl, release validation record at commit 7f880e1 (clean tree), Apple M5 Max, 2026-10-24 (benchmarks/reports/validation/7f880e1_apple-m5-max/record/).

Figure D.2: Geweke maximum $|z|$ per composition for the unmutated samplers (G-full rows and the recorded `exact=False` rows of the validation record, and the mutation suite's G-small controls at the mutation seeds) against the t_{C-1} critical band, with the mutants' G statistics overlaid on a logarithmic scale; failed runs would be drawn as crosses (there are none). From the release validation record (commit 7f880e1, Apple M5 Max), which also gives the block configurations and row 14 at the random-walk h_0 prior $\mathrm{N}(-1, 4)$ but no per-PR G-small row (Table D.2).

jobs running (load average 3.5–18), G-small for all per-PR compositions took 94.6 s and 99.7 s in two runs, and the mutation suite 181.9 s and 177.4 s, with identical statistics.

E Numerical robustness and performance detail

E.1 The three float64 failures

Table E.1 summarises the three failures of Section 9. None was a user report: the first two occurred at states that are legitimate under the model and were found by validation or replication runs, and the third was a bug in the failure-reporting path that the second exposed. Status codes decode as $1003 = (\texttt{EQ_CONTEMP}, \text{equation } 2)$ and $2001 = (\texttt{EQ_VOL}, \text{equation } 0)$.

The recorded diagnostics of F1 at the real state (the QR draw matches the exact conditional to 1.6×10^{-8} standard deviations) and of the old path of F2 at the test state (minimum pivot 2.22×10^{-16} against an exact 5.308×10^{-15}; log-determinant -36.04 against -32.8695; $h_0 = -73.90$ against -3.024) come from commit messages and scratch computations, not from committed tests.

Table E.1: Float64 failures F1–F3: state, mechanism, how each was found, the fix and its evidence. The lower panel, generated by `tab_rw_pivots.py`, recomputes the minimum pivot, the log-determinant and h_0 of the random-walk AWOL step at $\sigma = 5 \times 10^{-9}$ for the assembled factorisation, the excess-pivot recursion and a 60-digit reference.

	State	Mechanism	Found by	Fix
F1	G-full `reduced_sv`, chain 3214 of 4000, sweep 194; equation 0 at $h \approx 31$–37, $\lvert y\rvert \approx 2 \times 10^8$	equation 2's a-row precision has exact eigenvalues 14.8 and 1.9×10^{17}; forming $Z'WZ$ loses the small one, pivot 0 (code 1003); the conditional itself is benign	Geweke tier (weekly), with a heavy-tailed test prior	Householder QR of the whitened rows (Algorithm C.4); same noise map
F2	first full CR attempt (8a622c5), SV(rw), reduced form, origin 1987Q2, ordering GDP, TBILL, INFL, UR, burn-in sweep 318, $\sigma = 4.98 \times 10^{-9}$	$\sigma^2/v_{s_t} \approx 10^{-17}$ is below the ulp of the Laplacian diagonal (2); the assembled matrix is the singular path Laplacian; level pivot 0 (code 2001)	the replication's 24-ordering stage	excess-pivot recursion (Algorithm C.8, Appendix B.9)
F3	the replay of F2	the replay re-ran the production CCCM step on host arrays: `TracerArrayConversionError`; the first crash's diagnostics were lost	the replay itself	convert the replayed state to device arrays (93f2bcd), with a test on the production program

Factorisation of $\tilde{Q}$	Min. pivot	Rel. error	$\log\lvert\tilde{Q}\rvert$	$h_0 = \sigma x_0$	$\max_t \lvert h_t - h_t^{\mathrm{ref}}\rvert$	Failure reported
Assembled tridiagonal (before `f60d736`)	2.220×10^{-16}	0.96	-36.0437	-73.903	71	no
Excess-pivot recursion (since `f60d736`)	5.308×10^{-15}	1.3×10^{-15}	-32.8695	-3.024	4.3×10^{-14}	no
60-digit reference	5.308×10^{-15}	—	-32.8695	-3.024	—	—

Recomputed from committed code at the synthetic state of `tests/mcmc/blocks/test_sv_core.py::test_rw_awol_at_a_tiny_sigma` ($\sigma = 5 \times 10^{-9}$, $T_{\mathrm{eff}} = 88$, $T_{\mathrm{pad}} = 96$, 89 live states, $V_0 = 1.5$, $m_0 = -0.3$, noise seed 41), not at the production state of F2. The smallest pivot is that of the level direction (the last live state). "Failure reported": the engine's status would flag a pivot that is not positive or a non-finite draw. At this state the assembled factorisation returns a positive pivot and finite draws, so no failure is reported, while h_0 is off by 70.9.

Tests of the two fixes. The QR draw of F1 is tested three ways: it agrees with the Cholesky precision draw within the path-versus-path envelope of Appendix D.2 at 20 random states of each of the five reduced-form compositions; at a synthetic state with $\mathrm{cond}(K_a) > 10^{15}$ it matches the mean of a 60-digit exact conditional to $50\epsilon_{\mathrm{mach}}\,\mathrm{cond}(K_a)^{1/2}$ and reproduces its noise map (only the QR side is run there); and a slow test replays chain 3214 of the failing G-full run, where the old draw returns status 1003 and the production sweep is finite. The tiny-σ tests of F2 use the synthetic problem `problem(4, T_eff=88, T_pad=96, rw=True, sigma=5e-9)` of `test_sv_core.py`: 88 periods, two of them masked, padded to 96. Against its 60-digit reference the excess-pivot recursion gives the smallest pivot to a relative 10^{-12}, $\log\lvert\tilde{Q}\rvert$ to 10^{-13} and $h = \sigma x$ to 10^{-12}, and the whole random-walk update at that σ returns `ok` with every quantity finite, for five noise draws. The random-walk `aux_log_likelihood` had the same flaw (17534.8 instead of -1650.8 at $\sigma = 5 \times 10^{-9}$); the fix is tested against the dense Gaussian likelihood under the closed-form random-walk covariance $V_0 + \sigma^2 \min(s,t)$. Each fix was checked for neutrality elsewhere, on one machine. After the F1 fix every structural program was bitwise unchanged (only non-fusing means use the step), and the reduced-form states agreed with the old draw to 3.7×10^{-12} (relative) after 3 Geweke sweeps and 4.1×10^{-10} after 60-sweep fits; after the F2 fix 639 of 656 arrays were bitwise identical (every non-random-walk composition and fit), and random-walk states agreed to 3.3×10^{-11} after

3 sweeps and 6.7×10^{-10} after 60-sweep fits. The rejected alternative for F1, narrowing the test prior's h tail, would have hidden a real numerical weakness and dropped high-persistence coverage. The other formed-Gram draws, the structural coefficient step and the CCCM Π step, still accept any positive pivot, without a relative floor; whether reachable states expose them is not known. No test asserts the old-path values; they are recomputed by the script of Figure E.1. On Linux, in continuous integration, the tiny-σ tests and the fast F1 tests pass, and the replay test failed in the one weekly run (Appendix D). Figure E.1 shows the level pivot and the error in the level of h as σ decreases, for both factorisations.

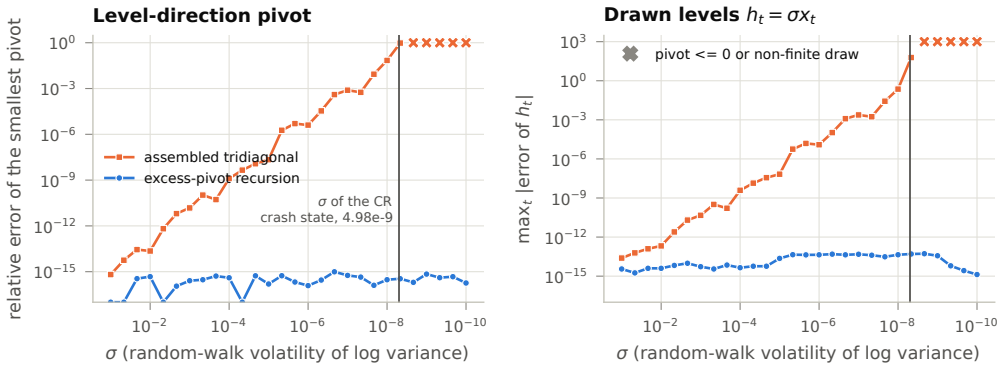


Figure E.1: The random-walk SV path system at small σ: level-direction pivot and error in the level of h against σ, for the assembled factorisation and the excess-pivot recursion, measured against a 60-digit reference, at the synthetic state of `test_rw_awol_at_a_tiny_sigma` (88 periods), not at the production fit of F2.

The old path at small σ. Before f60d736 the assembled path could return a finite but wrong draw without reporting a failure, because the step accepts any finite positive pivot. Figure E.1 shows where: on its grid of 28 values of σ from 10^{-1} to 10^{-10}, the assembled factorisation returns a zero pivot, and so a reported failure, only for the five values $\sigma \leq 2.2 \times 10^{-9}$; at $\sigma = 10^{-8}$ and 4.6×10^{-9} it returns finite draws whose level pivot is off by 7% and 95% and whose h is off by up to 0.23 and 61. The excess-pivot recursion stays within a relative 10^{-15} of the reference pivot, and within 5.3×10^{-14} of the reference h, at every σ of the grid. These are recomputations at the synthetic state of `test_rw_awol_at_a_tiny_sigma`, not at the production fit of F2; of the replication's runs, only the quick run predates the fix (Appendix F.5).

E.2 Performance

Three sources of performance numbers exist in the repository, and only the third measures the production engine: the stand-in scratch scripts of the design stage (a simplified SV block, data closed over as constants), the prototypes of the pilot (the real SV block, on a loaded machine), and the BENCH-P2 runs of the engine itself through `BVAR.fit` (the reference baseline and the continuous-integration report below). The paper quotes only the third and never blends them. BENCH-P2 fits structural Linear × SV with fixed hyperparameters to a simulated VAR(4) with SV ($n = 20$, $T = 300$; $k = 81$, $q = 100$, $T_{\text{pad}} = 320$), with 4 chains of 1000 burn-in and 10,000 kept draws on one CPU device, and times the whole fit, including compilation, storage and diagnostics, in fresh processes; wall and compilation times are medians over three cold or three warm runs, and the per-sweep time and peak memory are medians over all six. The per-sweep time is the minimum over three blocks of 1000 sweeps of the compiled burn-in executable, for all four chains

together (the runner's own rate over the sampling sweeps is 4.07 ms); the peak resident set is reported in MiB, and ESS per second is taken over the warm-run wall time. Table E.2 gives the result recorded at commit deb3929 on an Apple M5 Max that was not idle (one-minute load 2–6, desktop applications open): 52.3 s cold and 49.6 s warm, compilation 3.46 s cold and 1.00 s from the persistent cache, 3.90 ms per sweep, and a peak resident set of 3,043 MiB. `converged` was false in all six runs; according to the commit message, 4 of the 101 primary quantities missed R-hat 1.01, while every group's minimum bulk ESS exceeded 400. The slowest-mixing quantities were σ_i and ϕ_i (minimum ESS per second 10.2 and 11.2), against about 400 for the own first-lag coefficients. Table E.3 gives the kernel microbenchmarks at 80 equation–chain lanes, with XLA's flop and byte counts and the design budget lines: the packed Gram halves the flops of the batched oracle and is about twice as fast, and the packed Gram, the precision draw and the SV core together take 3.44 ms of the 3.87 ms sweep. These figures are from one non-idle machine at one commit and are not portable. The x86-64 column of Table E.2 is the report of the continuous-integration bench job on a shared GitHub runner with two logical CPUs (AMD EPYC 7763), at commit 779957f, before D-35: 228.9 s cold and 198.4 s warm, 16.8 ms per sweep (17.2 ms at the runner's rate over the sampling sweeps), with `converged` false in all six runs. Such a runner is not the protocol's x86 machine, an idle laptop with four to eight cores, so the design reports this run and does not gate on the 120 s target. No BENCH-P2 run exists on an idle machine, on the release commit, on an idle x86 laptop or on four devices.

Not measured on the production engine. The 4-device stretch target (only pilot projections exist), the reduced-form CCCM sampler, SV(t) and SV with outliers and t, Linear × Constant without dummy rows, and an idle x86 machine. An earlier x86 baseline in the history, measured under heavy load on four cores and labelled provisional, is superseded by the continuous-integration report and not quoted.

One OpenBLAS thread while chains run. On CPU, jaxlib lowers Cholesky factorisations and triangular solves to LAPACK and BLAS calls into SciPy's library, and XLA already runs those calls on its own threads; the engine therefore restricts OpenBLAS to one thread while chains run. The function is a no-op where no OpenBLAS with a thread API is loaded, as on macOS Accelerate, so the reference baseline did not exercise it. The claim that results do not depend on the thread count for factors smaller than 128×128 is recorded in the module docstring, not tested; the committed tests check only the set and restore semantics.

Dense eigen path with dummy rows. For Mode I × Constant with SOC/SUR rows at $n = 20$, $p = 4$, $T = 300$, the dense eigen path took 0.46 ms per sweep against 1.53 ms for the per-sweep Cholesky, on an Apple M5 Max under a concurrent load of 3–5; the two paths agree within the path-versus-path envelope of Section 8.

E.3 Platform caveats

On macOS with NumPy and SciPy on Accelerate, `slogdet` and `det` emitted runtime warnings on two well-conditioned symmetric positive definite tridiagonal matrices of a dense oracle, which the suite's warning filter turned into failures; the oracle now takes the log-determinant from the Cholesky factor, and the code under test is unchanged. A Phase 1 empirical-Bayes test intermittently crashes the interpreter on the same platform; the root cause is unknown and it is not tied to the MCMC code. Linux CI is the reference platform; neither observation is attributed to a particular library.

Table E.2: BENCH-P2 on the production engine, on the reference Apple M5 Max (not idle) and on a 2-vCPU x86-64 continuous-integration runner (reported, not gated): wall time cold and warm, compilation time, time per sweep, peak RSS, ESS and ESS per second by quantity group, the convergence flag, and the machines, load and commits.

Quantity	Apple M5 Max	x86-64 runner, 2 vCPU
Wall time, cold cache (median of 3)	52.3 s	228.9 s
Wall time, warm cache (median of 3)	49.6 s	198.4 s
Compilation, cold / from the persistent cache	3.46 s / 1.00 s	33.99 s / 3.81 s
Run time, wall minus compilation (median of 6)	48.7 s	194.8 s
After the runner: assembly, diagnostics (median of 6)	4.02 s	5.99 s
Per sweep: min. of 3×1000 compiled burn-in sweeps (median of 6)	3.90 ms	16.79 ms
Per sweep: runner, over the 10,000 kept sweeps (median of 6)	4.07 ms	17.19 ms
Peak resident set (median of 6)	3,043 MiB	2,054 MiB
`converged` (R-hat < 1.01, bulk ESS > 400, primary set)	false in 6 of 6	false in 6 of 6
120 s target (cold / warm)	met / met	missed / missed (not gated)

	Apple M5 Max			x86-64 runner	
Quantity group	Min. bulk ESS	Min. ESS/s	Median ESS/s	Min. bulk ESS	Min. ESS/s
log-likelihood	1,562	31.5	31.5	1,838	9.3
$h_{i,T}$	1,870	37.7	134.6	1,931	9.7
μ_i	3,835	77.4	165.9	4,102	20.7
σ_i	504	10.2	13.7	494	2.5
ϕ_i	554	11.2	17.3	618	3.1
own first-lag coefficients	19,997	403.4	494.5	19,671	99.1
primary monitor set (all)	504	10.2	115.2	494	2.5
cross-lag coefficient `B[y1.L1, y0]`	8,968	180.9	—	9,150	46.1

Recorded, not re-measured. Apple M5 Max: `tests/bench/baseline.json` as committed in `d490730`, measured at commit `deb3929` on 2026-10-22; 18 cores (6 performance), macOS-26.6.2-arm64-arm-64bit, Python 3.12.13, NumPy 2.5.4, jax/jaxlib 0.12.0.1; one-minute load average 2.4–6.1 before the six runs (not idle). x86-64: `benchmarks/reports/bench_p2_x86_github_779957f.json`, the report of the CI bench job on a GitHub runner, measured at commit `779957f` (before D-35) on 2026-10-24; AMD EPYC 7763 64-Core Processor, 2 logical CPUs, Linux-6.17.0-1022-azure-x86_64-with-glibc2.39, Python 3.12.3, NumPy 2.5.4, jax/jaxlib 0.12.0.1; one-minute load average 1.0–1.7. A shared 2-vCPU runner is not the protocol's x86 machine, so the 120 s target is neither required nor asserted there (reported, not gated). $n = 20$, $p = 4$, $T = 300$; 4 chains $\times$ 10,000 kept draws after 1,000 burn-in, thin 1, 1 CPU device. ESS per second is bulk ESS over the end-to-end wall time of the warm runs; the ESS of the cross-lag coefficient is of the stored draws.

F Clark–Ravazzolo supplement

This appendix documents the replication of Section 11: the provenance of the acceptance run, its pre-registration, the criteria and deviations exactly as coded, the run history, the data and models, and further tables and figures. Every number of the acceptance run is filled automatically from `benchmarks/reports/clark_ravazzolo_acceptance.json`; numbers of the first full run (report commit 551f181, code 4bb978a, path-conditioned estimator, 4×5000 draws) are history only and are marked as such. Line numbers refer to `benchmarks/clark_ravazzolo.py` at commit c823d20.

F.1 Provenance of the acceptance run

The acceptance configuration and its pre-registration were committed together in c823d20, before the run was launched. According to the author's run record, the run was started afterwards from a detached worktree at that commit with a clean working tree, with the com-

Table E.3: Kernel microbenchmarks at 80 lanes: packed and batched Gram, precision draw, SV indicators, AWOL draw, exact correction, SV core and sweep; XLA flops and bytes accessed; and the design budget lines.

		ms					Budget
Kernel	Production code	min	median	GFLOP	MiB acc.	Temp MiB	ms
`gram_packed`	`linalg.weighted_gram(mode="packed")`: `(w @ FF)[pos]` (D-25)	0.467	0.488	0.259	24.8	3.1	1.1
`gram_batched`	`weighted_gram(mode="batched")` (the oracle; for the D-25 ratio)	1.050	1.168	0.517	58.1	25.6	---
`precision_draw`	`linalg.precision_draw`: Jacobi-scaled Cholesky + two solves (D-14)	1.167	1.193	0.005	37.7	12.3	1.9 (with GEMMs)
`sv_indicators`	`vol.sv.draw_indicators` (inverse-CDF categorical, D-4)	0.270	0.295	0.006	14.0	3.9	1.5 (with AWOL)
`sv_awol`	`awol_system` + `linalg.tridiag_precision_sample`	0.141	0.149	0.001	7.8	1.4	(with indicators)
`sv_exact`	the exact correction of the `h` move: one extra `log g` + two `w`	0.363	0.384	0.002	10.6	2.7	0.4 (pilot 1.1)
`sv_core`	`SVCore.sample`: the whole SV update (indicators, AWOL, exact `h`, C and NC steps)	1.802	1.888	0.026	60.8	5.9	about 3.6
`sweep`	the runner's compiled `burn` executable, per sweep (all 80 lanes)	3.867	3.877	0.307†	139.1†	16.7	5.6

Recorded (not re-measured): `tests/bench/baseline.json` `kernels`, committed in `d490730`, measured at commit `deb3929` on 2026-10-22, Apple M5 Max, cpu backend, 1 device, one-minute load average 5.8 (not idle). Each kernel is the production function vmapped over the 80 lanes and AOT-compiled; times are the minimum and median over 50 timed calls (5 calls of 200 sweeps for `sweep`, given per sweep). GFLOP and MiB accessed are XLA's `cost_analysis()`, temporary MiB its `memory_analysis()`, per call of the compiled executable († the 200-sweep executable, as reported, not divided by the number of sweeps). Budget: the DESIGN §10.3 line in `tests/bench/bench_kernels.py`. The packed Gram, the precision draw and the SV core take 3.44 ms of the 3.87 ms sweep.

mand `benchmarks/clark_ravazzolo.py benchmarks/reports/clark_ravazzolo_acceptance --config acceptance --n-jobs 10` and a fresh score cache named after the commit. The run log, committed with the report in 65dd212, opens with the start time 2026-10-23 16:10:56 UTC, eleven seconds after the commit time of c823d20, and with the commit c823d20 in its configuration line; it ends at 19:46:21 UTC. These two timestamps are local: a commit time is set by the author's machine, and the log was written by the run itself. An independent record exists as well: GitHub's repository activity log, which the server writes when it receives a push, records the creation of the branch `phase-2-cr-acceptance` at commit c823d20 at 16:10:46 UTC, ten seconds before the run's first log line; the first arm's scores were written 88 s into the run. The pull request that later merged the branch was opened at 19:58:06 UTC, after the run had ended. The report's `generated` field (2026-10-23 16:31 UTC) is set when the report is first assembled, after the five reduced-form main-stage arms and their Gaussian scores (1193 s into the run, by the log) and before the structural pair and the ordering stages; it is not the end of the run. The script labels a report as the acceptance run only when `acceptance_gaps` (l. 1022) returns nothing: the configuration must be the registered one (estimator, MCMC budget, forecast draws, origin and ordering steps, noise replications, backtest seed 0), every stage must have run (the structural pair and all 24 orderings), and the commit must be known with no uncommitted changes to tracked files. The acceptance report records `config.name` = `acceptance`, `config.lpd_method` =

`integrated`, `config.draws` = 8000, `config.mcmc` = 4 chains × 20,000 kept draws, burn-in 1000, `config.seed` = 0, `versions.git_commit` = `c823d20` and `versions.git_dirty` = `false`, and whether the JSON has no `pending` key (absent). The values that `acceptance_gaps` requires are `"acceptance"`, `"integrated"`, 8000, 4 chains × 20,000 kept draws with burn-in 1000, seed 0, a known commit and `git_dirty` = `false`, with no `pending` key. The function does not compare the commit with c823d20; that the run is at c823d20, the commit that adds the pre-registration, is a requirement of the pre-registration itself and is checked from `versions.git_commit`. Two sentences of the repository's ledgers that still said the acceptance configuration "keeps 1000 forecast draws" were corrected after the launch, in d58c9ef, which changes documentation only; the script at c823d20 uses 8000 (l. 194), as pre-registered.

F.2 The pre-registration

The pre-registration is reproduced below as committed at c823d20, in the file `benchmarks/reports/clark_ravazzolo_preregistration.md`. Only the typography has changed (Markdown to LaTeX); the words, numbers and structure are those of the file. It was written in an agent session, and the reviews it mentions were agent reviews (Appendix J).

Clark–Ravazzolo replication: pre-registration of the acceptance run

Written 2026-10-23, before any score of the acceptance run was computed. The acceptance run is `benchmarks/clark_ravazzolo.py benchmarks/reports/clark_ravazzolo_acceptance --config acceptance` with a fresh cache (`--cache ~/.cache/rai-bvar-cr-<short commit>`; the command is in the script's docstring), on a clean working tree at the commit that adds this file. The report header records that commit and whether tracked files differ from it; the report is labelled the acceptance run only when they do not, every stage ran, and the config, estimator and backtest seed are the ones below. The previous full run (`--config full`, report `benchmarks/reports/clark_ravazzolo.md` committed in `551f181`) is the reference this document departs from.

Why there is a second run

The full run met criteria 1, 2, 3 and 5 and missed 4 and 6 (N08 §C5, as coded in `benchmarks/clark_ravazzolo.py`). The two misses had diagnosable causes. Neither was a sign that SV(rw) loses to Constant.

- **Criterion 6** failed only on `compare()`'s convergence gate. The SV(rw) verdict was "better" in 24/24 orderings with the gate off and 0/24 with it on, because SV(rw) converged at 57–82% of origins against the 95% gate.
- **Criterion 4** failed on one cell: full-sample UR, ΔLPS(h=8) = +1.954 > ΔLPS(h=1) = +0.159. A handful of 2007–08 origins drive it. There the path-conditioned Rao–Blackwellised estimator is degenerate: one forecast draw carries all the weight, and the score is biased low by up to tens of nats, more for the thinner-tailed Constant arm.

These changes are motivated by that failure analysis, so the run is not blind to the previous results. What is pre-registered is that the settings below were fixed before any score of the new run existed, and that the criteria are evaluated on them as coded, whatever the outcome.

What changes (and only this)

1. **MCMC budget for every MCMC arm, in every stage:** 4 chains × 20,000 kept draws, burn-in 1000, thin 1 (previously 4 × 5000, burn 1000). With the default `store_max = 2500` the parameters of one in $\lceil 20{,}000/2500 \rceil = 8$ kept draws are stored (10,000 in all), and the forecast draws are taken from those; R-hat and ESS use every kept draw.
   - Evidence: fit-level convergence diagnostics only, with no forecasts or scores. The data are the same and the fit seeds are the CR main-stage ones.
   - SV(rw), identity ordering: 72/106 origins converge at 4 × 5000; 79/106 with burn 5000; 92/106 with 8 chains; **106/106 at 4 × 20,000** (max R-hat 1.0061, min bulk ESS 1763).

- The two worst orderings of the previous run (INFL > UR > GDP > TBILL and INFL > UR > TBILL > GDP): 71/106 and 63/106 at 4 × 5000 with the main-stage fit seeds; **106/106 and 106/106 at 4 × 20,000** (max R-hat 1.0049 and 1.0046, min ESS 1689 and 1717). The previous run's ordering stage, which fits each ordering with its own backtest seed, had 63/106 and 60/106 there (and 69/106 for the identity ordering against 72/106 in the main stage), so this evidence is at the main-stage seeds, not at the ordering stage's.
- All 520 + 424 diagnostic fits finished without error. Every base-budget failure is on a σ quantity and is marginal (R-hat < 1.03), consistent with R-hat sampling noise at bulk ESS ∼600, not with a stuck sampler.

2. **Primary log predictive density: the path-integrated Rao–Blackwellised estimator** (`lpd_method="integrated"`; DESIGN, the decision row added with it).
   - For h > 1 it conditions each draw on its parameters and its drawn latent scale path only, and integrates the intermediate y's out analytically.
   - At h = 1 it is identical to the previous estimator.
   - The previous (path-conditioned) estimator is reported as a secondary number in criterion 4's detail, and per cell in the JSON (`dlps_conditioned`, `dlpd_conditioned`); it never enters a rule.
3. **Forecast draws per origin: 8000** (previously 1000), in every stage.
   - Reason: the integrated LPD is exact given each draw but is still an average over draws, and at crisis origins the weights concentrate on a few of them. In the review of the estimator (2008Q2, UR h = 8, at the budget of item 1; precision only, no criterion evaluated), Constant's 1000-draw value sat on average about 0.3–0.6 nats below the 80,000-draw value, with an sd of 0.75–1.06 nats across thinning offsets. SV(rw)'s varied with sd ≈ 1.4 nats across forecast seeds, more than its MCSE, and its expected bias was as large as Constant's, so the error favours neither arm.
   - With 8000 draws the thinning error dropped to an sd of 0.09–0.19 nats with bias ≈ 0.
   - It does not remove the MCMC component: the 80,000-draw values still differ by about 1.3 nats across fit seeds at that origin.
   - The cost is small next to the fits.

What does not change

- **Data:** FRED-QD/FRED-MD vintage 2026-09 with the same SHA-256 checks, the four series and their transformations, and the estimation start 1965Q1.
- **Models:** Constant(iw_equivalent), SV(rw), SV, SV(rw,t10), AR-SV(0.9), and the structural pair. The mean prior is `Minnesota(form='reduced', hierarchy='fixed', lambda_=0.2, alpha=2, cross=1, mean=0)` with p = 4.
- **SV priors:** the library defaults. CR's own RW-SV σ^2 prior IG(1/2, 0.0175) is **not** adopted. It stays a documented deviation, and adopting it now would be a modelling change decided after seeing results.
- **Origins and horizons:** origins 1985Q1–2011Q2 (106) with the 1985Q1–2007Q4 pre-crisis subset (92); expanding window; horizons 1, 2, 4, 8.
- **Seeds:** backtest seed 0, per-origin fit seeds (DESIGN §11.6), cold starts; the ordering stage with all 24 orderings and the same noise replications.
- **Rules:** the criteria 1–6 rules and thresholds exactly as coded. That includes the 95% convergence gate of `compare()` (R-hat < 1.01 and bulk ESS > 400 on every primary quantity) and the RB-degenerate flag (joint `lpd_mcse` ≥ 0.9; descriptive only).

Stated in advance

- **Criterion 4 may still fail on UR.** The WP23 review's estimate with corrected tail origins was UR h = 8 ≈ +0.23 against +0.157 at h = 1, which breaks the coded rule ΔLPS(h=8) < ΔLPS(h=1). If it fails, it is reported as a difference from CR, attributed with evidence where possible (final vintage instead of real-time data, Chan's iw_equivalent baseline, library-default SV priors). It is not tuned.

- **Criterion 6** is expected to pass if the convergence gate is met. Its h = 1 scores are estimator-independent.
- **The integrated LPD still carries Monte Carlo error at tail origins** (item 3): about 1.3 nats per arm at a crisis origin from the MCMC itself. That amounts to a few hundredths of a nat in a 106-origin mean, but it is not zero. The RB-degenerate flag stays descriptive.
- **No further run with different settings** is made after this one's scores are seen, except to fix a bug. A bug fix is documented, and the affected stages are rerun in full with these settings and a fresh cache (the score-cache keys do not change with the code).

The fit-level diagnostic that the first item cites is not part of release 0.2.0, so its numbers are those of the pre-registration (Table F.9); a script and its 520 + 424 fit records are kept with the paper's sources (branch `paper-draft`, `docs/paper/figures/data/convergence_diagnostic/`).

F.3 The criteria as coded

Table F.1 gives each criterion in the words of the replication plan, with its outcome in the first full run and in the acceptance run.

Table F.1: The six acceptance criteria: wording of the replication plan, and outcome of the rule as coded (quoted below) in the first full run (history; report at 551f181, path-conditioned LPD, 4×5000 draws) and in the acceptance run (path-integrated LPD, $4 \times 20{,}000$ draws).

#	Wording (plan)	First run	Acceptance
1	SV beats Constant in average LPD at $h = 1, 2$ for all four variables in both samples; the joint $h = 1$ gain is clearly positive	met	met
2	CRPS ratio < 1 at $h \leq 2$ for all four variables	met	met
3	RMSE ratios $\leq$ about 1 at short horizons	met	met
4	Full sample: the $h = 8$ gain shrinks or turns negative for UR and the bill; pre-crisis it stays positive for GDP and inflation	not met	not met
5	Neither SV-t nor AR-SV systematically beats RW or AR SV	met	met
6	`compare()` says "better" (DM one-sided at 10% for at least GDP and the bill at $h = 1$) in all 24 orderings, or the report states which fail	not met	met

The six criteria of the replication plan are qualitative; the script turns them into the operational rules below, which are the project's own choices (for example, "about 1" became ≤ 1.02 and "systematically" became at least 3 of 4 joint horizons or 8 of 16 cells). The rules are quoted from the `rule` strings of `criteria` (l. 666), with typography changed only.

1. "ΔLPS(SV(rw) − Constant) > 0 at h = 1, 2 for all four variables in the full and pre-crisis samples, and the joint ΔLPD at h = 1 > 0 with one-sided DM/HLN p < 0.10 in both samples"
2. "CRPS ratio SV(rw)/Constant < 1 at h = 1, 2 for all four variables (full sample)"
3. "RMSE ratio SV(rw)/Constant ≤ 1.02 ('about 1') at h = 1, 2 for all four variables (full sample); CR's gains for GDP and the bill are 5–10%"
4. "full sample: ΔLPS(h=8) < ΔLPS(h=1) for UR and TBILL; pre-crisis: ΔLPS(h=8) > 0 for GDP and INFL"

5. "neither SV(rw,t10) nor AR-SV(0.9) is better than both SV(rw) and SV (one-sided DM/HLN $p < 0.10$, positive ΔLPD) at ≥ 3 of 4 joint horizons or ≥ 8 of 16 marginal cells (full sample)"

6. "in every one of the 24 orderings: compare() verdict SV(rw) vs Constant at h = 1 is 'better', and the one-sided DM/HLN p-value of the marginal log score is < 0.10 for GDP and TBILL at h = 1"

Two features of the coded rules are worth stating. Criterion 4's rule compares the $h = 8$ gain with the $h = 1$ gain only, whereas the qualitative wording is "shrinks or turns negative". Criterion 6 is met only if the verdict is "better" in every ordering both with the convergence gate on, as `compare()` returns it, and with the gate off, and if both p-values are below 0.10 in every ordering. In the acceptance run the outcome, as coded, is: criteria 1, 2, 3, 5 and 6 met; 4 not met. The script prints the numbers behind it as follows (under the integrated estimator, criterion 4's detail also reports secondary numbers, which are not part of the rule): 16 of 16 cells positive; joint h=1: full +0.819 nats/quarter (p=0.000), pre +0.791 nats/quarter (p=0.000); 8 of 8 cells < 1; pre-crisis: all 8 < 1; GDP/h1 0.908, GDP/h2 0.901, INFL/h1 0.899, INFL/h2 0.848, UR/h1 0.969, UR/h2 0.946, TBILL/h1 0.935, TBILL/h2 0.945; full UR: h1 +0.160 → h8 +0.273; full TBILL: h1 +0.377 → h8 -0.195; pre-crisis GDP h8 +0.105; pre-crisis INFL h8 +0.307. Secondary (not part of the rule): full UR h8: median +0.140 (largest origins: 2007Q4 +7.6, 2008Q1 +6.6, 2008Q2 +6.5), Gaussian-approx h1 +0.122 → h8 -0.526; full TBILL h8: median -0.059 (largest origins: 2000Q4 -2.3, 2000Q2 -2.2, 2000Q3 -2.1), Gaussian-approx h1 +0.315 → h8 -0.230; pre-crisis GDP h8 Gaussian-approx -0.001; pre-crisis INFL h8 Gaussian-approx +0.354; path-conditioned (SPEC §7) estimator ΔLPS: full UR: h1 +0.160 → h8 +1.309; full TBILL: h1 +0.377 → h8 -0.193; pre-crisis GDP h8 +0.105; pre-crisis INFL h8 +0.307; SV(rw,t10): significantly better than both SV(rw) and SV at 0/4 joint horizons and 0/16 marginal cells; AR-SV(0.9): significantly better than both SV(rw) and SV at 0/4 joint horizons and 0/16 marginal cells; verdict 'better' (convergence gate on, as compare() returns it) in 24/24 orderings; with the gate off in 24/24; DM GDP and TBILL < 0.10 in 24/24.

F.4 Deviations from the target

The report prints the following list, quoted from `DEVIATIONS` (l. 1380), followed, for the integrated estimator, by `LPD_INTEGRATED_DEVIATION` (l. 1372). References to sections are to the repository's replication plan (N08) and design documents.

1. "Not a deviation from CR, but from the library default: every own-lag prior mean is 0, as in CR (WP App. 6.1), including the UR and TBILL levels; the library warns that this contradicts their level transformation codes (it would centre them on 1)."

2. "Pseudo real time on the final FRED-QD 2026-09 vintage instead of CR's RTDSM real-time vintages and second-release actuals (D4; §C2). Score levels differ; rankings are expected to be robust (§C6)."

3. "Estimation starts in 1965Q1, not 1955Q1 (FRED-QD starts in 1959; §C2)."

4. "Inflation uses the chain-type GDPCTPI (FRED-QD has no GDPDEF; §C6)."

5. "Baseline: Constant('iw_equivalent') (Chan 2022) stands in for CR's normal-diffuse VAR (§C3); the SV arms use the library's default SV priors except the two CR variants (no prior was tuned). CR's RW-SV σ^2 prior IG(1/2, 0.0175) and its 36-observation training prior for h_0 are not used (the latter would violate R9, §C6)."

6. "AR-SV variant: CR's $a_1 \sim N(0.9, 0.05^2)$ is mapped to $(\phi+1)/2 \sim$ Beta(71.25, 3.75) (same mean and sd on ϕ); the SV level μ keeps the library's in-window data-centred prior instead of CR's $a_0 \sim N(0, 0.4^2)$."

7. "LPD: the primary score is the Rao–Blackwellised predictive density; CR's Gaussian approximation is reported alongside (§C4, §C6)."

8. "Significance: one-sided DM/HLN p-values are descriptive under an expanding window; compare() therefore decides 'better' through the Model Confidence Set (SPEC §9.5)."

9. "compare() turns a verdict 'inconclusive' when fewer than 95% of origins converged (R-hat/ESS targets of DESIGN §8.2); verdicts are reported with that gate on (as returned) and off."

   `LPD_INTEGRATED_DEVIATION`: "LPD estimator: the primary score is the library's path-integrated Rao–Blackwellised estimator (`lpd_method='integrated'`, DESIGN §7.3, D-34), not the path-conditioned SPEC §7 default; at h = 1 the two are identical, so every h = 1 cell and criterion 6 (h = 1 only) are estimator-independent. The path-conditioned ΔLPS is reported for the criterion-4 cells only, as a secondary number (never in a rule)."

The Gaussian-approximation log score of item 7 (Adolfson et al., 2007) equals, up to an additive constant, $-\frac{1}{2}$ times the Dawid–Sebastiani score, which is proper but not strictly proper (Dawid and Sebastiani, 1999; Gneiting and Raftery, 2007); it is a parity metric only.

Four further differences from the target are not in the script's list: (c) Clark and Ravazzolo's VAR-SV puts $\mathrm{N}(0, 1000^2 I)$ on the contemporaneous matrix A, whereas the library's SV arms use the Mode I prior of a_{ij} and `Constant("iw_equivalent")` the λ_i-scaled one, so the baseline and the SV arms also differ in this prior (Section 6.6); (d) their σ^2 prior $\mathrm{IG}(1/2, 0.0175)$ has zero density at 0 and no mean, whereas the library's $0.1\chi^2_1$ for the random walk has mean 0.1 and an unbounded density at 0, a difference in shape, not only in scale; (e) FRED-QD would allow an estimation start in 1960Q2, closer to their 1955Q1 than the 1965Q1 used; (f) our samples are defined by origin at every horizon, so pre-crisis targets at $h = 8$ run to 2009Q4; Clark and Ravazzolo's forecasts are also indexed by origin (1985Q1–2011Q2), but with data through $t-1$ their h-step target is $t+h-1$, one quarter earlier than ours for the same origin, and since their last vintage (2011Q4) ends in 2011Q3, their samples at $h = 4$ and 8 cannot use every origin to 2011Q2 (nor at $h = 2$ with second-release actuals for growth and inflation); the working paper states neither how they trim those origins nor whether its 2007Q4 cut-off applies to origins or targets (Clark and Ravazzolo, 2012, Section 2 and the notes to Tables 1–3), and the article was not accessible. Their data are in the journal's data archive (Clark and Ravazzolo, 2015b) (current-vintage unemployment and bill rate, and real-time vintages of GDP and its price index), so the three candidate causes of Section 11 could be tested; this run, as pre-registered, does not.

F.5 Run history and the first attempt's crash

Table F.2 gives each run with its commit, configuration, machine and load, wall time and outcome. In order: a quick run with the library's then-default MCMC settings (4×2000), on a shared four-core x86-64 virtual machine, main stage only, labelled as not the acceptance run; the first full attempt on 8a622c5 (4×5000), which aborted in the SV(rw) ordering stage on F2; the fixes 93f2bcd (failure replay) and f60d736 (excess-pivot recursion); the second full run on 4bb978a, which contains both fixes and the QR a-row draw 3d3f51b, report committed in 551f181; the follow-ups in 4fc0162 of a review of f60d736 by a separate agent instance (including the random-walk auxiliary likelihood), committed after the second run had started and therefore not part of it; the path-integrated estimator in b9c66c0; the pre-registration and acceptance configuration in c823d20; and the acceptance run. The second full run took 3950 s (1.10 h) from a fresh cache with six worker processes, while weekly-tier jobs ran alongside. It differs from the aborted attempt in 41 files, not only by the fix: the dummy-row work, the QR a-row draw, the replay fix and the random-walk fix all entered between the two, so the second run is not described as differing "only by the fix". The

acceptance run's wall time is 12,905 s (3 h 35 min) (stage times in its committed log); its software versions and platform are rai_bvar 0.0.1; jax 0.12.0.1; platform Darwin arm64, python 3.12.13; commit `c823d20`; tracked files unchanged.

The crash of the first attempt unfolded as follows. After the main stage and the Constant orderings had finished, an SV(rw) fit (reduced form, origin 1987Q2, ordering GDP, TBILL, INFL, UR) raised `MCMCNumericalError` in `EQ_VOL`, equation 0, chain 0, at burn-in sweep 318, with code 2001 and pivot 0. The deterministic replay that should have attached diagnostics itself raised (F3), so the state was reconstructed by a separate replay, which showed $\sigma = 4.98 \times 10^{-9}$ for GDP. The fix f60d736 replaced the assembled factorisation by the excess-pivot recursion, and the second full run, on 4bb978a, completed all 24 orderings without a crash. A review of f60d736 by a separate agent instance (Appendix J) found no defect; its follow-ups (4fc0162) came after that run had started and left the 656-array invariance set bitwise unchanged, so the run's report stands. The committed quick report predates f60d736, so its random-walk SV numbers may have been affected by the old path; they serve only as context, including as one run of the yardstick of reading rule B (Appendix F.7).

F.6 Data and models

Tables F.3 and F.4 give the data and the model arms as coded.

The FRED-QD series are those of McCracken and Ng (2021) and the monthly series those of McCracken and Ng (2016); the data are downloaded on demand and not bundled. CR's estimation used real-time vintages (Croushore and Stark, 2001); the replication runs in pseudo real time on the final vintage, with the start date of Clark (2011). The benchmark FRED-QD codes of these series are 5, 6, 2 and 2 (McCracken and Ng, 2021, Tables A1A, A1F, A1C and A1H), and the 2026-09 file carries the same codes; the script follows Clark and Ravazzolo's definitions instead, which agree with the benchmark for `GDPC1` (100 times code 5) but keep inflation in log first differences and UR and TBILL in levels.

CR's target is Clark and Ravazzolo (2015a); the numbers compared are those of the working paper's Tables 1–3 (Clark and Ravazzolo, 2012), as hard-coded in the script (`CR_FULL`, l. 147; `CR_PRE_DLPS`, l. 167). The article's tables were not accessible to us; the working paper's are used throughout. CR's VAR(4) has a Minnesota prior without cross-variable shrinkage (tightness $\theta = 0.2$, $1/l^2$ lag decay, zero prior means) in the normal-diffuse form of Kadiyala and Karlsson (1997), normal on the coefficients and diffuse on the error covariance; their VAR-SV has random-walk log-volatilities, sampled by the single-move step of Jacquier et al. (1994), with a σ^2 prior IG$(1/2, 0.0175)$ that the replication does not adopt. The variants we use are their AR-SV with a nearly fixed autoregressive coefficient and their SV with Student-t errors with fixed degrees of freedom d (Jacquier et al., 2004). The working paper uses a unit lower-triangular A in its time-varying-volatility VARs (its Section 3.2) but does not state the variable order, which is why all 24 orderings are run; the article's text was not accessible.

F.7 Further results of the acceptance run

Table F.6 gives the pre-crisis comparison of SV(rw) and Constant at $h = 1$ and $h = 8$, Table F.7 every SV arm against Constant with the convergence shares, the verdicts with the gate on and off, MCS membership and the variant comparisons of criterion 5, and Table F.8 the Constant arm's levels against CR's benchmark VAR together with the compute record. The reading rules below apply to every number here. They say which numbers may move between the two runs. They are the paper's, not part of the pre-registration, and were first committed (33dd9f8) after the run had

Table F.2: Run history of the replication: commit, configuration, machine and load, wall time and outcome of each run, from the quick run to the acceptance run.

Run	Code	Configuration	Machine, load, start	Wall time	Outcome
Quick	not recorded	`quick`: 4 × 2,000, burn-in 1,000; 1,000 forecast draws	shared 4-core x86-64 VM (load average about 10)	main stage 1.08 h; ordering stage stopped	criteria 1, 2, 3, 5 met; 4 not met; 6 not evaluated; report `1ae5765`
Full #1	`8a622c5`	`full`: 4 × 5,000, burn-in 1,000; 1,000 forecast draws; 6 workers	Apple M5 Max, weekly tier alongside; started 02:59:57Z	aborted; reported 04:10Z	crash F2 in the ordering stage (`MCMCNumericalError`, `EQ_VOL`, code 2001); no report
Full #2	`4bb978a`	`full`: 4 × 5,000, burn-in 1,000; 1,000 forecast draws; 6 workers	Apple M5 Max, load about 25 (weekly tier alongside); started about 04:12Z	3,950 s (1.10 h); main stage 649 s; Constant orderings 982 s; SV(rw) orderings 2308 s	criteria 1, 2, 3, 5 met; 4, 6 not met; report `551f181`
Acceptance	`c823d20`	`acceptance`: 4 chains × 20,000 kept draws, burn-in 1000; 8000 forecast draws; `integrated`; 10 workers	Darwin arm64, python 3.12.13	12,905 s (3 h 35 min)	criteria 1, 2, 3, 5 and 6 met; 4 not met; report `65dd212`

			In the code of		
Commit	Committed (UTC)	Subject	full #1	full #2	acceptance
`1ae5765`	10-23 01:25Z	WP23: CR acceptance command, --n-jobs, and the quick report (not the acceptance run)	yes	yes	yes
`3d3f51b`	10-23 02:48Z	Recursive a-row draw by QR of the whitened rows (fixes test_g_full[reduced_sv])	no	yes	yes
`93f2bcd`	10-23 03:37Z	Failure replay converts host arrays; review follow-ups	no	yes	yes
`f60d736`	10-23 04:09Z	RW AWOL draw factors the path Laplacian without cancellation (fixes the CR crash)	no	yes	yes
`4bb978a`	10-23 04:09Z	Merge phase-2-wp25: RW AWOL Laplacian factorisation (queue item 7 crash fix)	no	yes	yes
`4fc0162`	10-23 04:36Z	RW Laplacian follow-ups from the independent review	no	no	yes
`551f181`	10-23 05:16Z	Clark–Ravazzolo full replication report (queue item 7)	no	no	yes
`b9c66c0`	10-23 16:10Z	Path-integrated Rao–Blackwellised log predictive density (D-34, opt-in)	no	no	yes
`c823d20`	10-23 16:10Z	Clark–Ravazzolo acceptance config and its pre-registration	no	no	yes
`d58c9ef`	10-23 16:46Z	Docs: the CR acceptance config uses 8000 forecast draws (as pre-registered)	no	no	no
`7f410dc`	10-23 18:44Z	Validation row 18: reduced-form x SV(rw), the CR primary arm	no	no	no
`65dd212`	10-23 19:47Z	Clark–Ravazzolo acceptance run report (pre-registered, c823d20)	no	no	no

Quick run: report banner and header (1ae5765; its code commit was not recorded); its assembling invocation read cached origins. Full runs: config and outcome from the reports, start times, machine, load and stage times from the comments of issue #3 (comments of 03:01Z, 04:10Z and 05:17Z on 2026-10-23). Lower panel: commit times are committer times converted to UTC; "In the code of" is `git merge-base --is-ancestor` against 8a622c5, 4bb978a and c823d20. 4fc0162 came after full run #2 had started and is not part of it.

Table F.3: Data of the replication. Rates are non-annualised quarterly rates; the estimation sample starts in 1965Q1 (one raw quarter earlier is kept for the difference). Clark and Ravazzolo define growth and inflation as $100\,\Delta\log$ (their Section 2), although the notes to their Tables 1–3 call all variables annualised percentage points; score differences and CRPS and RMSE ratios do not depend on this scale, but the benchmark levels in their tables do.

Variable	CR definition	FRED-QD series	Transformation in the script
GDP	real GDP growth, $100\,\Delta\log$	`GDPC1`	$100\log$ level, then tcode 2
INFL	inflation in the GDP price index, $100\,\Delta\log$	`GDPCTPI`	$100\log$ level, then tcode 2 (chain-type index; FRED-QD has no deflator series)
UR	unemployment rate, quarterly average of monthly values	`UNRATE`	level, tcode 1
TBILL	3-month Treasury bill rate, quarterly average	`TB3MS`	level, tcode 1

Sample. Vintage 2026-09 of FRED-QD and FRED-MD, SHA-256 checked (Appendix G); estimation from 1965Q1; expanding window; origins 1985Q1–2011Q2 (106) and the pre-crisis subset 1985Q1–2007Q4 (92); horizons 1, 2, 4, 8; $p = 4$. Both samples are defined by origin at every horizon, so the targets at horizon h run h quarters past the last origin (at $h = 8$: to 2009Q4 pre-crisis, to 2013Q2 in the full sample).
Run-time checks. (i) FRED-QD `UNRATE` and `TB3MS` equal the within-quarter means of the FRED-MD monthly series of the same vintage (tolerance 0.01 over more than 200 quarters; first full run: maximum absolute difference 3.3×10^{-5} over 270 quarters; acceptance run: `UNRATE`: maximum absolute difference 3.3×10^{-5} over 270 quarters; `TB3MS`: maximum absolute difference 3.3×10^{-5} over 270 quarters). (ii) No look-ahead: at origins 1985Q1, 2007Q4 and 2011Q2 the resolved prior, hyperparameters and padded data are bitwise identical when the vintage is cut at the origin, checked for the primary arm only (bitwise identical at 1985Q1, 2007Q4 and 2011Q2). (iii) The checksums of the vintage files.

Table F.4: Model arms. Every arm uses the mean block `Linear(prior=Minnesota(form="reduced", hierarchy="fixed", lambda_=0.2, alpha=2.0, cross=1.0, mean=0.0))` with $p = 4$, sampled by the corrected triangular algorithm (Carriero et al., 2022), except the structural pair, which uses `form="structural"` (Mode I). SV priors are the library defaults unless stated.

Arm	Volatility specification	CR counterpart	Role
Constant	`Constant("iw_equivalent")`, the order-invariant prior of Chan (2022)	VAR with a normal-diffuse prior	baseline of every relative measure; negative control of the ordering stage
SV(rw)	`SV(rw=True)`: $h_0 \sim \mathrm{N}(\log(\psi_\iota/\iota), 4)$, $\sigma^2 \sim 0.1\chi^2_1$, in-window	VAR-SV (random walk; $\sigma^2 \sim \mathrm{IG}(1/2, 0.0175)$, $A \sim \mathrm{N}(0, 1000^2 I)$)	primary arm of criteria 1–4 and 6
SV	`SV()`: $\mu \sim \mathrm{N}(\log(\psi_\iota/\iota), 10)$, $(\phi + 1)/2 \sim \mathrm{Beta}(20, 1.5)$, $\sigma^2 \sim \chi^2_1$	—	second reference of criterion 5
SV(rw,t10)	`SV(rw=True, tails="t", nu_prior=DiscreteUniform(10, 10), t_scale="unit")`	VAR-SVt with $d = 10$	criterion 5 variant; ν fixed, so its grid-bound warning fires by design
AR-SV(0.9)	`SV(phi_prior=Beta(71.25, 3.75))`	VAR-ARSV with $a_1 \sim \mathrm{N}(0.9, 0.05^2)$ (same mean and sd on ϕ)	criterion 5 variant
Constant / SV(rw) [structural]	`Constant("ig")` / `SV(rw=True)` in structural Mode I	—	robustness pair

written its main-stage report and before the run's report was committed (65dd212), so their timing relative to the numbers cannot be checked.

A. *Estimator:* every RB ΔLPS and ΔLPD at $h \geq 2$, the DM p-values at $h \geq 2$, the RB/Gaussian sign disagreements, the criterion-4 cells, the variant comparisons and the ordering spread at $h \geq 2$.

B. *Monte Carlo noise only:* all $h = 1$ LPD numbers, CRPS and RMSE ratios, Gaussian scores and $h = 1$ DM p-values (the two estimators coincide at $h = 1$). Between the quick run (4×2000) and the first full run, the full-sample $h = 1$ ΔLPS of SV(rw) moved by at most 0.003 per marginal cell and 0.002 joint, and over all arms and both samples by at most 0.0044 per marginal cell and 0.0122 joint; this is the Monte Carlo yardstick of rule B.

C. *Budget:* converged shares, gate-on verdicts, criterion 6's gate-on count, the MC standard errors of the totals and the run time.

D. *Unchanged by design:* data checks, models, priors, origins, rules and fit seeds.

Reading rule B is the check on the $h = 1$ numbers: they should differ from the first run's by Monte Carlo noise only. They do, by this yardstick: no $h = 1$ ΔLPS or joint ΔLPD of any arm in either sample moved by more than it. The comparison is descriptive: the yardstick is the largest move over all arms between a single pair of runs, one of which predates the random-walk fix (Appendix F.5), and SV(rw)'s own joint move (0.0106) is close to it. For scale, the order-invariant `Constant` arm's joint $h = 1$ score spans 0.008 nats over the 24 seeded orderings; that is one arm's Monte Carlo noise, a lower bound for that of a difference between two arms, and does not by itself make an SV-arm move "noise". Under the expanding window, any "better" verdict can only come through the model confidence set (Hansen et al., 2011). In the first full run the pre-crisis GDP $h = 8$ cell, which criterion 4 needs to be positive, was $+0.108$ under the path-conditioned estimator and -0.004 under the Gaussian approximation; its acceptance values are $+0.105$ (path-integrated), $+0.105$ (path-conditioned) and -0.001 (Gaussian). The fit-level diagnostic covered only SV(rw), and only three of the 24 orderings at the main-stage seeds, so before the run the gate-on verdicts of SV, SV(rw,t10) and AR-SV(0.9) could have stayed inconclusive; in the event every reduced-form arm converged at 100% of origins in both samples, and no verdict depends on the gate. The converged shares of every arm, in both samples, are in the last column of Table F.7. Across arms and samples the RB and Gaussian ΔLPS differ in sign in 28 marginal and 7 joint cells. Across the 24 orderings the joint $h = 1$ gain of SV(rw) over `Constant` runs from $+0.800$ to $+0.877$ nats.

The inflation gains are larger than Clark and Ravazzolo's: the full-sample ΔLPS is $+0.233$ at $h = 1$ and $+0.293$ at $h = 8$, against their $+0.075$ and $+0.138$, and the $h = 8$ RMSE and CRPS ratios are 0.725 and 0.720, against their 0.883 and 0.887. The `Constant` arm's own inflation forecasts are more accurate than their benchmark VAR's (RMSE 0.188 against 0.269 at $h = 1$; Table F.8), so the two sets of levels are not on the same data. The final-vintage chain-type price index and the down-weighting of volatile periods by SV are possible explanations, untested conjectures; magnitudes are not comparable across these designs, which is why the replication targets the ranking only.

Table F.5 collects the cells of criterion 4 with the descriptive numbers that Section 11.5 reports next to them; none of them enters the rule.

F.8 Convergence against the MCMC budget

Table F.9 gives the fit-level convergence diagnostic that set the acceptance budget: SV(rw) fits at the main-stage fit seeds, with no forecasts or scores, at 4×5000, with burn-in 5000, with 8

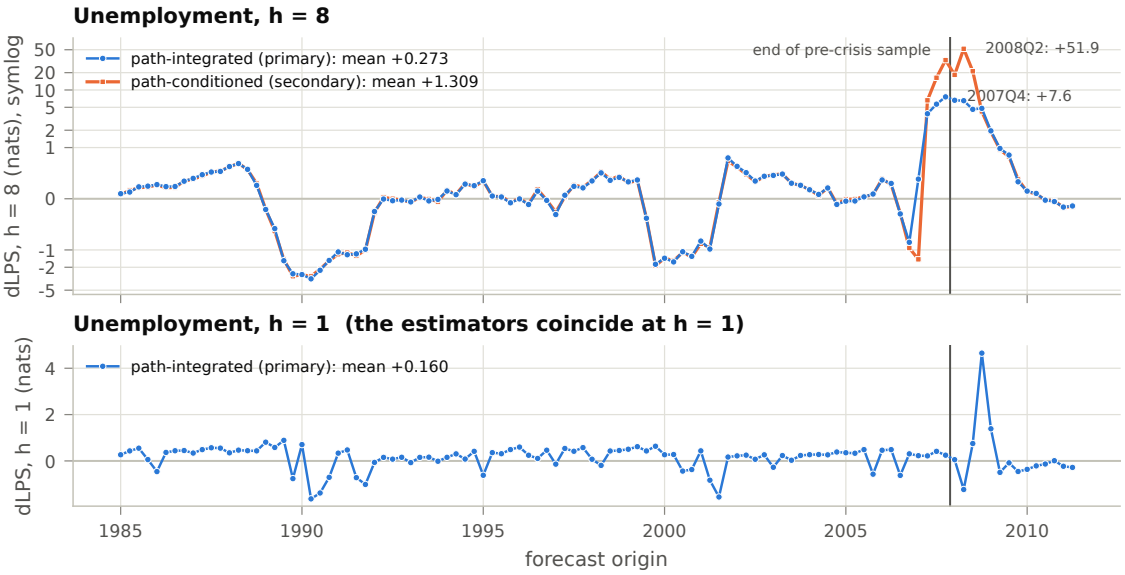


Figure F.1: Per-origin ΔLPS of SV(rw) against Constant for unemployment at $h = 8$ (and $h = 1$ for contrast), origins 1985Q1–2011Q2: path-conditioned against path-integrated, from the same forecast draws of the acceptance run. The $h = 8$ panel has a symmetric-log axis (linear within ± 1 nat), and the largest value of each series is labelled with its origin. Drawn from a per-origin export of the run's score cache, kept with the paper's sources (its check reproduces the report's values exactly).

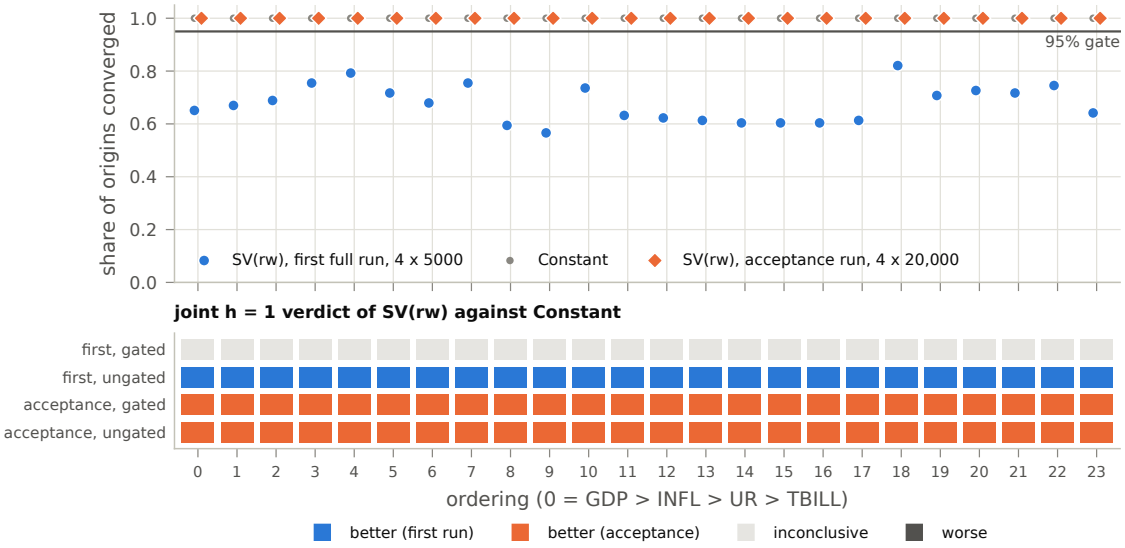


Figure F.2: Share of converged origins per ordering for SV(rw) and Constant: first full run (4×5000) against the acceptance run ($4 \times 20{,}000$), with the 95% gate; gated and ungated $h = 1$ verdicts are marked.

Table F.5: Criterion 4's cells, SV(rw) against `Constant`, acceptance run (descriptive; not part of the rule). Mean: the primary path-integrated RB ΔLPS that the rule uses; Gauss.: Clark and Ravazzolo's Gaussian-approximation score; Cond.: path-conditioned RB estimator on the same draws; Median: per-origin median of the primary difference; Top-3 share and Mean w/o top 3: share of the summed difference from the three origins largest in absolute value, and the mean without them, recomputed from the report's `n`, `dlps` and `dlps_top` (typed, not `\textbackslash acc` values).

Sample, variable	h	Mean	Gauss.	Cond.	Median	Top-3 share	Mean w/o top 3
Full, UR	1	+0.160	+0.122	+0.160	+0.264	—	+0.151
	8	+0.273	−0.526	+1.309	+0.140	72%	+0.079
Full, TBILL	1	+0.377	+0.315	+0.377	+0.453	—	+0.449
	8	−0.195	−0.230	−0.193	−0.059	32%	−0.136
Pre-crisis, GDP	8	+0.105	−0.001	+0.105	+0.222	—	+0.152
Pre-crisis, INFL	8	+0.307	+0.354	+0.307	+0.269	12%	+0.280
Pre-crisis, UR	8	+0.026	−0.683	+0.427	+0.096	>100%	−0.166

Rule: full-sample $h = 8$ below $h = 1$ for UR and TBILL; pre-crisis $h = 8$ positive for GDP and INFL. The pre-crisis UR row is not part of the rule. A share is printed only where the three values have the sign of the sum; above 100%, the other origins sum to the opposite sign.

Table F.6: Pre-crisis sample (1985Q1–2007Q4), SV(rw) against Constant at $h = 1$ and $h = 8$: ΔLPS under the integrated (primary) and the path-conditioned estimator, the Gaussian-approximation ΔLPS and the number of RB-degenerate origins, with CR's working-paper values.

		ΔLPS, SV(rw) − Constant				CR
Var.	h	Integrated	Conditioned	Gaussian	Degen.	ΔLPS
GDP	1	+0.115	+0.115	+0.121	0	+0.222
	8	+0.105	+0.105	−0.001	0	+0.214
INFL	1	+0.260	+0.260	+0.233	0	+0.077
	8	+0.307	+0.307	+0.354	0	+0.152
UR	1	+0.148	+0.148	+0.126	0	+0.187
	8	+0.026	+0.427	−0.683	0	−0.065
TBILL	1	+0.423	+0.423	+0.370	0	+0.316
	8	−0.205	−0.204	−0.249	0	−0.261
joint	1	+0.791	+0.791	+0.671	0	—
	8	+0.330	+1.341	−0.527	0	—

Origins 1985Q1–2007Q4 of the expanding-window backtest. Integrated: path-integrated RB log score (primary); Conditioned: path-conditioned RB estimator on the same draws (secondary, in no rule); Gaussian: Gaussian approximation; Degen.: RB-degenerate origins (joint MCSE ≥ 0.9 for either arm). CR: Clark and Ravazzolo's VAR-SV against VAR, working paper, Gaussian-approximation log score (`CR_PRE_DLPS` of `benchmarks/clark_ravazzolo.py`).

chains and at $4 \times 20{,}000$, for the identity ordering and the two worst orderings of the first run, together with the main-stage against ordering-stage seed comparison. Its numbers are those of the pre-registration until the diagnostic is committed. Figure F.2 compares the converged shares per ordering of the first full run and the acceptance run; in the acceptance run, SV(rw)'s smallest converged share across the 24 orderings is 100%. The seed comparison is itself evidence of R-hat noise at the threshold: the same models in the same ordering converged at 72 (main-stage seeds) and 69 (ordering-stage seeds) of 106 origins, with joint $h = 1$ ΔLPD of +0.809 and +0.838.

Table F.7: All SV arms against Constant, full and pre-crisis samples: joint ΔLPD by horizon, converged share, the `compare()` verdict with the gate on and off, and MCS membership; and the variant comparisons of criterion 5 (full sample only).

		Joint ΔLPD against Constant				
Sample	Arm	$h = 1$	$h = 2$	$h = 4$	$h = 8$	Converged
Full	SV(rw)	+0.819	+0.743	+0.673	+0.604	100%
	SV	+0.830	+0.835	+0.845	+0.760	100%
	SV(rw,t10)	+0.813	+0.735	+0.623	+0.552	100%
	AR-SV(0.9)	+0.837	+0.838	+0.846	+0.785	100%
	Constant	—	—	—	—	100%
Pre-crisis	SV(rw)	+0.791	+0.593	+0.319	+0.330	100%
	SV	+0.776	+0.651	+0.471	+0.537	100%
	SV(rw,t10)	+0.788	+0.601	+0.333	+0.317	100%
	AR-SV(0.9)	+0.781	+0.654	+0.479	+0.555	100%
	Constant	—	—	—	—	100%

		Full sample				Pre-crisis			
		Verdict		In MCS		Verdict		In MCS	
Arm	h	gate on	off	model	ref.	gate on	off	model	ref.
SV(rw)	1	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	2	"inconclusive"	"inconclusive"	`false`	`false`	"better"	"better"	`true`	`false`
	4	"inconclusive"	"inconclusive"	`false`	`false`	"inconclusive"	"inconclusive"	`false`	`false`
	8	"inconclusive"	"inconclusive"	`false`	`false`	"inconclusive"	"inconclusive"	`false`	`false`
SV	1	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	2	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	4	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	8	"inconclusive"	"inconclusive"	`false`	`false`	"better"	"better"	`true`	`false`
SV(rw,t10)	1	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	2	"inconclusive"	"inconclusive"	`false`	`false`	"better"	"better"	`true`	`false`
	4	"inconclusive"	"inconclusive"	`false`	`false`	"inconclusive"	"inconclusive"	`false`	`false`
	8	"inconclusive"	"inconclusive"	`false`	`false`	"inconclusive"	"inconclusive"	`false`	`false`
AR-SV(0.9)	1	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	2	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	4	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`
	8	"better"	"better"	`true`	`false`	"better"	"better"	`true`	`false`

Variants (criterion 5), full sample, joint ΔLPD						
Variant	against	$h = 1$	$h = 2$	$h = 4$	$h = 8$	DM p, $h = 1$
SV(rw,t10)	SV(rw)	−0.006	−0.007	−0.050	−0.052	0.646
SV(rw,t10)	SV	−0.017	−0.100	−0.222	−0.208	0.692
AR-SV(0.9)	SV(rw)	+0.018	+0.096	+0.172	+0.181	0.309
AR-SV(0.9)	SV	+0.007	+0.003	0.000	+0.025	0.089

Joint ΔLPD: path-integrated RB estimator, mean over origins, model minus reference. Converged: share of origins at which the fit met the convergence rule (R-hat < 1.01, bulk ESS > 400 on every primary quantity). Verdict: `compare()` on the volatility axis, with the 95% convergence gate on and off. In MCS: whether the model and the reference are in the model confidence set. Under the expanding window the DM test is descriptive only, so a "better" verdict can come only through the model confidence set (model in, reference out); each verdict's route is stated in the report's `reasons`. The variant comparisons exist for the full sample only.

Table F.8: Constant-arm levels against CR's benchmark VAR at $h = 1$ (average log score and RMSE), and the compute record of the acceptance run: machine, number of worker processes, stage times and wall time. The two log-score rows use different estimators and data vintages, so this is a sanity check, not a parity test.

	Average log score, $h = 1$		RMSE, $h = 1$	
Variable	Constant arm	CR benchmark VAR	Constant arm	CR benchmark VAR
GDP	−1.116	−1.118	0.728	0.675
INFL	0.138	−0.235	0.188	0.269
UR	−0.024	−0.034	0.246	0.240
TBILL	−0.906	−0.832	0.458	0.454

Compute record of the acceptance run	
Commit, and whether tracked files differed from it	`c823d20`; modified: `false`
Platform	Darwin arm64, python 3.12.13
rai-bvar, jax	0.0.1, 0.12.0.1
Configuration and LPD estimator	`acceptance`, `integrated`
MCMC budget per fit	4 chains × 20,000 kept draws, burn-in 1000
Forecast draws per origin	8000
Worker processes	10
Stage times	main stage 1,484 s; ordering stage, Constant 2,294 s and SV(rw) 9,114 s; done at 12,905 s (run log, 65dd212)
Wall time	12,905 s (3 h 35 min)

Constant arm: the reference arm's levels in the SV(rw) comparison, full sample 1985Q1–2011Q2, path-integrated RB log score (at $h = 1$ identical to the path-conditioned one). CR benchmark VAR: Clark and Ravazzolo's working paper as recorded in `docs/phase2/08_common_sv_and_ordering.md` §C5 (Gaussian-approximation log score, real-time data). The two log-score columns differ in estimator and data vintage: a sanity check of levels, not a parity test.

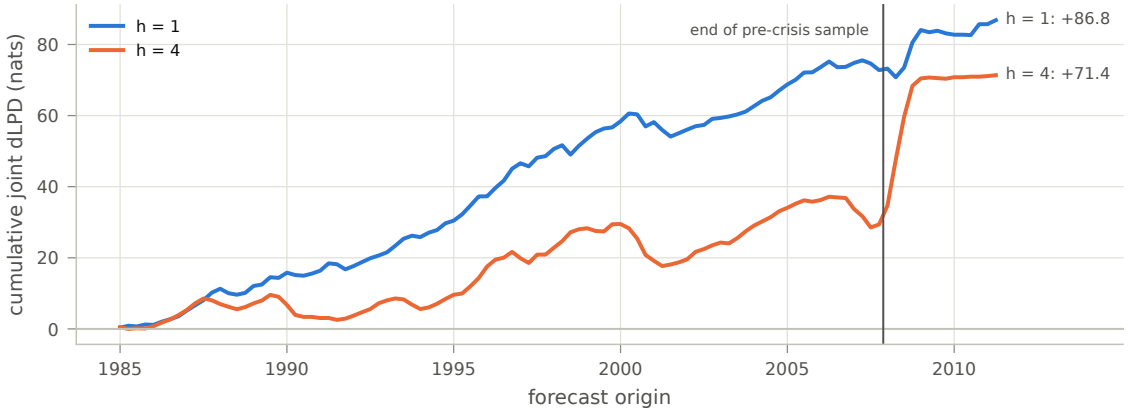


Figure F.3: Cumulative joint ΔLPD of SV(rw) against Constant over origins at $h = 1$ and $h = 4$, with the end of the pre-crisis sample marked. Drawn from the same uncommitted per-origin export as Figure F.1.

F.9 The first full run (history)

Table F.10 reproduces the first full run's tables for the record: the headline and pre-crisis comparisons, all arms, the variants, the structural pair, the RB-degenerate origins and the cells in which the RB and Gaussian ΔLPS differ in sign. They were produced by the path-conditioned estimator with 4×5000 draws and 1000 forecast draws, and they are history, not results; Section 11 summarises what they showed and why the acceptance run differs. In the failing criterion-4 cell (full sample, unemployment, $h = 8$) the median per-origin difference was +0.099, and the Gaussian approximation gave −0.499. Across all SV arms and both samples, the RB and Gaussian ΔLPS differed in sign in 28 marginal and 8 joint cells, all at $h \geq 2$. In the report's example warning for each SV arm (main stage), a fit missed on one σ quantity out of 13 primary quantities (random-

Table F.9: Convergence against the MCMC budget for SV(rw), fit-level only: 4×5000, burn-in 5000, 8 chains and $4 \times 20{,}000$; identity ordering and the two worst orderings; converged origins, maximum R-hat and minimum bulk ESS; and the main-stage against ordering-stage seed comparison.

Ordering	Budget	Converged origins	Max. R-hat	Min. bulk ESS
GDP > INFL > UR > TBILL (identity)	4×5000, burn-in 1000 (first full run)	72/106	1.0237	394
	4×5000, burn-in 5000	79/106	1.0281	327
	8×5000, burn-in 1000	92/106	1.0163	855
	$4 \times 20{,}000$, burn-in 1000 (acceptance)	106/106	1.0061	1763
INFL > UR > GDP > TBILL	4×5000, burn-in 1000 (first full run)	71/106	1.0196	401
	$4 \times 20{,}000$, burn-in 1000 (acceptance)	106/106	1.0049	1689
INFL > UR > TBILL > GDP	4×5000, burn-in 1000 (first full run)	63/106	1.0262	372
	$4 \times 20{,}000$, burn-in 1000 (acceptance)	106/106	1.0046	1717

Same ordering, 4×5000*: main-stage fit seeds against the first full run's ordering stage*

Ordering	Main-stage seeds	Ordering-stage seeds
GDP > INFL > UR > TBILL	72/106	69/106
INFL > UR > GDP > TBILL	71/106	63/106
INFL > UR > TBILL > GDP	63/106	60/106
joint $h = 1$ ΔLPD, identity	+0.809	+0.838

Upper panel, the pre-registration's diagnostic: SV(rw) fits only (no forecasts or scores) at the main-stage fit seeds over the 106 origins 1985Q1–2011Q2, from the fit records `docs/paper/figures/data/convergence_diagnostic/*.jsonl`, which are kept with the paper's sources, not with release 0.2.0; the generator checks that they reproduce item 1 of the pre-registration (c823d20) to the digits stated there; all 520 + 424 fits finished without error. Converged: R-hat < 1.01 and bulk ESS > 400 on every primary quantity. Lower panel: first full run (report 551f181, history); the ordering stage fits each ordering with its own backtest seed.

walk arms) or 21 (AR arms), with $\hat{R}$ 1.012–1.020 and bulk ESS 537–743. With the gate off, all four SV arms were "better" than `Constant` at $h = 1$ in the full sample through the MCS route (total ΔLPD +85.7 to +87.5 nats, MCSEs 1.00–1.06). In the first run the joint $h = 1$ range across orderings was 0.083 nats against a gain of 0.809, and the per-variable gains were 2.2–7.8 times their ranges. The order-invariant Constant arm was flagged "material" by the ordering diagnostic at two $h > 1$ cells (joint $h = 2$ and UR $h = 8$), with a noise floor resting on one replicate; this is reported as a limitation of the diagnostic, and the acceptance run's flag is `false`. Differences from CR's levels, such as inflation gains of 1.6–3.2 times CR's (our RB score against their Gaussian one), are not attributed to any single cause; final-vintage chain-type prices and the iw_equivalent baseline are hypotheses that we did not test.

Table F.10: The first full run's tables, for the record: history, not results (the note names the run).

		Full sample, 1985Q1–2011Q2 (106 origins)							Pre-crisis, 1985Q1–2007Q4 (92)			
Var.	h	ΔLPS	Gauss.	Sign	Degen.	DM p	CRPS	RMSE	ΔLPS	Gauss.	Sign	Degen.
GDP	1	+0.126	+0.107		0	0.003	0.897	0.909	+0.115	+0.124		0
	2	+0.124	+0.054		1	0.002	0.893	0.899	+0.107	+0.122		0
	4	+0.073	−0.063	yes	7	0.141	0.928	0.945	+0.073	−0.076	yes	2
	8	+0.111	+0.010		12	0.006	0.938	0.977	+0.108	−0.004	yes	8
INFL	1	+0.231	+0.213		0	< 0.001	0.844	0.898	+0.259	+0.240		0
	2	+0.267	+0.265		1	< 0.001	0.808	0.850	+0.296	+0.291		0
	4	+0.249	+0.250		7	< 0.001	0.769	0.791	+0.272	+0.269		2
	8	+0.282	+0.319		12	< 0.001	0.724	0.728	+0.295	+0.345		8
UR	1	+0.159	+0.125		0	0.008	0.936	0.970	+0.147	+0.132		0
	2	+0.266	+0.043		1	0.090	0.925	0.948	+0.116	+0.055		0
	4	+0.969	−0.256	yes	7	0.153	0.929	0.937	+0.018	−0.137	yes	2
	8	+1.954	−0.499	yes	12	0.178	0.978	0.963	+1.032	−0.654	yes	8
TBILL	1	+0.378	+0.322		0	< 0.001	0.799	0.937	+0.425	+0.378		0
	2	+0.153	+0.094		1	0.073	0.899	0.950	+0.176	+0.114		0
	4	−0.081	−0.143		7	0.713	0.988	0.971	−0.083	−0.150		2
	8	−0.207	−0.234		12	0.864	1.022	0.988	−0.220	−0.253		8
joint	1	+0.809	+0.665		0	< 0.001			+0.790	+0.683		0
	2	+0.858	+0.452		1	0.017			+0.578	+0.408		0
	4	+2.254	−0.032	yes	7	0.146			+0.061	−0.157	yes	2
	8	+5.013	−0.320	yes	12	0.162			+1.883	−0.514	yes	8

	Full, joint ΔLPD				Pre-crisis, joint ΔLPD				Converged		Sign cells	
Arm	$h = 1$	2	4	8	1	2	4	8	full	pre	marg.	joint
SV(rw)	+0.809	+0.858	+2.254	+5.013	+0.790	+0.578	+0.061	+1.883	68%	67%	7	4
SV	+0.822	+0.987	+2.607	+5.146	+0.776	+0.643	+0.384	+2.047	37%	37%	7	0
SV(rw,t10)	+0.811	+0.866	+2.023	+4.636	+0.788	+0.584	+0.102	+1.659	57%	55%	7	4
AR-SV(0.9)	+0.825	+0.957	+2.508	+5.326	+0.782	+0.642	+0.370	+2.191	62%	60%	7	0
Constant				(reference)					100%	100%		

Variants (criterion 5), full sample, joint

Variant	against	$h = 1$	2	4	8	DM p, $h = 1$
SV(rw,t10)	SV(rw)	+0.002	+0.008	−0.231	−0.377	0.446
SV(rw,t10)	SV	−0.011	−0.120	−0.584	−0.510	0.625
AR-SV(0.9)	SV(rw)	+0.017	+0.099	+0.254	+0.313	0.319
AR-SV(0.9)	SV	+0.004	−0.030	−0.099	+0.180	0.281

Structural pair, SV(rw) against Constant, full sample

Variable	$h = 1$	2	4	8	DM p, $h = 1$
GDP	+0.103	+0.107	+0.059	+0.109	0.007
INFL	+0.232	+0.260	+0.260	+0.282	< 0.001
UR	+0.128	+0.197	+0.873	+1.754	0.025
TBILL	+0.315	+0.060	−0.178	−0.239	< 0.001
joint	+0.751	+0.589	+1.368	+2.913	< 0.001

History: report `benchmarks/reports/clark_ravazzolo.json` committed in 551f181, code 4bb978a, config `full` (4 chains × 5000 kept draws, burn-in 1000, 1000 forecast draws per origin). ΔLPS: path-conditioned RB estimator (the primary estimator of that run); Gauss.: Gaussian approximation; Sign: RB and Gaussian ΔLPS differ in sign (28 marginal and 8 joint cells over all arms and both samples, none at $h = 1$); Degen.: RB-degenerate origins (joint MCSE ≥ 0.9); DM p: one-sided DM/HLN p-value for "model better", descriptive under the expanding window. Converged: share of origins at which the fit met the convergence rule.

G Reproducibility record

Code and data availability. rai-bvar is developed at `https://github.com/yigitokar/rai-bvar` under Apache-2.0. This paper describes release 0.2.0, commit 4ac7969 on `main` (the full hash is in the install line below). Its code is that of commit 7f880e1, to which it changes only the version string (in `pyproject.toml` and in the lock file, which uv also rewrote in its newer format without changing any locked dependency version) and adds the committed reports below. It needs Python 3.12 or later and installs with

```
pip install git+https://github.com/yigitokar/rai-bvar@v0.2.0
```

(the tag `v0.2.0` points to commit 4ac7969 and carries a GitHub release; there is no PyPI release and no archived DOI). The repository holds, under `benchmarks/reports/`, the reports of the Clark–Ravazzolo runs (the quick run, the first full run, and the acceptance run with its log; Section 11 and Appendix F) and the pre-registration of the acceptance run (`clark_ravazzolo_preregistration.md`, committed with its configuration in c823d20, Appendix F.2); the release validation record of Appendix D, in `benchmarks/reports/validation/7f880e1_apple-m5-max/`; and the x86-64 BENCH-P2 report of Table E.2. The scripts that produced them are in the same commit (`benchmarks/clark_ravazzolo.py`, `benchmarks/phase2_perf.py` and the test suite), and the paper's sources, figure and table scripts are on the branch `paper-draft` (`docs/paper`).

Validation records. With `RAI_BVAR_VALIDATION_RECORD` set to a directory, three test modules append one JSON line per run to it: `geweke.jsonl` from `tests/geweke`, for every Geweke tier (G-small, the block configurations, G-full, the recorded `exact=False` rows and the random-walk h_0 variant, distinguished by a `tier` field), `sbc.jsonl` from `tests/sbc` and `mutation.jsonl` from `tests/mutation`. The release validation record ran the weekly tiers (`-m "(slow or network) and not bench"`) at 7f880e1 on a clean tree, on an Apple M5 Max, from 04:17 to 07:18 UTC on 24 October 2026: its `record/` directory holds those three files (43 Geweke, 23 SBC and 37 mutation records), and pytest's JUnit XML of each test directory gives the pass or fail outcome of the tiers without a record hook (composition-level Tier B, parity, the analytic twins, the contract tests); `meta.json` names the commit, machine, versions and load, and `README.md` the results. `tests/unit` was left out of that run, because its slow empirical-Bayes tests of Phase 1 hit an intermittent heap corruption in macOS Accelerate within one process (issue #3); they pass in isolation and feed no table. The per-PR tier passed on the release commit in continuous integration (Appendix D.10).

Data. The FRED vintages are downloaded on demand from the St. Louis Fed, verified against the SHA-256 checksums of the package's registry, and cached; they are not bundled.

Dataset, vintage	SHA-256 of the published file
FRED-MD 2026-09	`d6e239038afbdbf4667c2a63c13c942817bc196fedab4355cd7e0d353ecdd65d`
FRED-QD 2026-09	`a144b1547ddbad5c68fc934c90c0c0f9837a5139571ba8e64afcc2637aaa3feb`

Regeneration. The `Makefile` in `docs/paper` regenerates the figures and the generated table bodies from committed code and data, `make tables` the 14 generated tables and `make figures` the figures (20 s and 39 s on the Apple M5 Max), and `make pdf` builds the paper; each script names its data source in its header, and Table G.2 gives the command for each float. The remaining tables, among them 11 of the floats that the plan lists in Table G.2, are typed into the sources, with the source of each entry in its evidence tags. The acceptance tables and figures need `benchmarks/reports/clark_ravazzolo_acceptance.json`, and Figures F.1 and F.3 the per-origin export of the acceptance run, which the paper branch holds with its export script.

What reproduces how. Table G.1 states each guarantee with its scope. Three levels must be kept apart. Bitwise equality holds for the listed variations within one platform and one version set, and

part of it is established only with the runner's test doubles ("toys"), not with the production kernels. Equality to rounding holds where XLA vectorises the reductions of the production SV and CCCM kernels differently for different batch sizes: chains are then equal only until a last-bit difference flips an accept/reject decision, and in distribution afterwards. Nothing is guaranteed across platforms (OpenBLAS against Accelerate), JAX versions or PRNG implementations. In practice, on one machine and version set, re-runs reproduced the printed output of the analytic listing of Section 7 line for line, that of the MCMC listing except its timing lines (and, on the release commit, its version line), and the fixed-seed validation statistics to every printed digit; across machines the mutation statistics agreed to two or three significant figures, and MCMC results can be compared only within Monte Carlo error.

Every MCMC fit records the information needed to reproduce it: the jax and jaxlib versions, the PRNG implementation and whether threefry is partitionable, the platform, device count and chain method, the seed and the data salt, the resolved Gram mode, constant-volatility path and T_{pad}, the mixture, offset and exact flag, the ordering, the specification hash and the package version; the sweep range and the burn-in, draws and thinning are stored with the draws. Keys are counter-based (Salmon et al., 2011), derived by `fold_in` over seed, stream, data salt, chain, absolute sweep, block, equation and sub-draw, with uint32 counters and no key in any loop carry, so draws cannot depend on chunking, burn-in pieces or thinning alignment; with partitionable threefry, the words drawn directly from a parent key equal the key data of its `fold_in` children, which JAX's key-reuse checker cannot see, and the engine avoids this by giving every consumer its own sub-key. A test checks this for the volatility core's forecast draws, where a draw from the block key would reuse the words of the transitory sub-keys.

G.1 Backtest seeding, caching and parallel runs

Instead of a start date, an end date and a `step`, `backtest()` also accepts a given list of origins. Each origin of a backtest has its own forecast generator and, for MCMC models, its own fit seed, both derived from the backtest seed and the origin; a data salt keeps chains independent across windows and orderings. Scores are cached per origin under a key of spec, sample, outcomes, origin and settings, with tags for the MCMC seeding and a non-default estimator, so runs resume and adding a model fits only that model. A pool of spawned workers reproduces the serial results to 10^{-12}. Warm starts are opt-in and require strictly increasing origins.

G.2 The walk-through scripts and their output

Listings G.1 and G.3 are the scripts of Section 7 and Listings G.2 and G.4 their standard output, verbatim; nothing was written to stderr by `ex_a.py`. They were run on release 0.2.0 (commit 4ac7969, clean working tree) with `docs/paper/code/run_examples.py`, which records the commit, the platform and the package versions (Table G.2); the MCMC numbers are platform-specific.

Listing G.1: `ex_a.py` in full (from line 6; lines 1–5 are comments naming the commit of the run).

```
6  import numpy as np
7  import pandas as pd
8
9  import rai_bvar as rb
10
11 # A tiny quarterly data set: a stationary VAR(1) in (GDP growth, inflation, rate),
12 # stored as raw levels (two log-levels, one rate) with FRED-style transformation codes.
13 rng = np.random.default_rng(0)
14 A = np.array([[0.4, 0.0, -0.001], [0.1, 0.6, 0.0], [15.0, 25.0, 0.85]])
15 mu = np.array([0.0075, 0.006, 4.0])
16 sd = np.array([0.007, 0.003, 0.25])
```

Table G.1: Reproducibility matrix: for a fixed specification, data, seed and device count, each variation, the guarantee, the test that establishes it, and its scope. "Toys" are the runner's test doubles in `tests/mcmc/runner/toys.py`, which implement the block protocols but are not the production kernels.

Variation	Guarantee	Scope	Tests
chunk size and burn-in piece size	bitwise	toys; production follows from the key layout, not tested	a
thinning with aligned sweeps	bitwise	toys	b
fused against unfused plan	bitwise	toys (3 configurations)	c
`vmap` over chains against a sequential loop	bitwise	toys	d
chain-count prefix, `chains="map"`	bitwise	toys and production `structural_sv`	e
chain-count prefix, `chains="vmap"`	toys: bitwise on macOS, to rounding on x86 Linux; production: rtol 10^{-9}	toys; production SV and CCCM kernels, short runs	f
`vmap` against `shard_map` over 4 devices	toys: rtol 10^{-12}; production: rtol 10^{-9}	toys; production `structural_sv`, short burn-in	g
leading fit axis against looped fits	rtol 10^{-10}	toys	h
in-loop against post-hoc forecasts	10^{-12}	production	i
memory-mapped against in-memory store	equal	toys	j
OpenBLAS thread count (factors $< 128 \times 128$)	bitwise	recorded (module docstring)	—
adding the dummy-row code, with no rows	bitwise (537 arrays)	recorded, one machine; structural test	k
platform, JAX version, PRNG implementation, partitionable threefry, spec fields	none	by design	—

Tests in `tests/mcmc/`: (a) `runner/test_chunking.py::test_chunk_and_burn_piece_size_invariance`; (b) `runner/test_chunking.py::test_thinning_with_aligned_sweeps`; (c) `runner/test_sweep.py::test_fused_equals_unfused_bitwise`; (d) `runner/test_sweep.py::test_vmap_over_chains_equals_sequential_loop`; (e) `runner/test_chunking.py::test_chain_prefix_invariance_with_mapped_chains`, `::test_mapped_chains_production_prefix`; (f) `runner/test_chunking.py::test_chain_prefix_invariance`, `::test_production_chain_prefix_with_vmapped_chains_to_rounding`, `::test_production_chain_prefix_with_vmapped_reduced_form_to_rounding`; (g) `runner/test_devices_bitwise.py::test_shard_map_matches_vmap_in_subprocess`; (h) `runner/test_runner.py::test_fit_axis_equals_looped_fits`; (i) `forecast/test_forecast_core.py::test_in_loop_forecast_equals_post_hoc_sv`, `::test_in_loop_forecast_equals_post_hoc_constant`, `::test_in_loop_forecast_equals_post_hoc_with_thinned_storage`; (j) `runner/test_storage.py::test_memmap_store_equals_memory_store`; (k) `blocks/test_dummies.py::test_dummy_path_is_closed_without_dummies`.

```
corr = np.array([[1.0, 0.2, 0.3], [0.2, 1.0, 0.2], [0.3, 0.2, 1.0]])
L = np.linalg.cholesky(corr * np.outer(sd, sd))
y, Y = mu.copy(), []
for _ in range(200 + 240):
    y = (np.eye(3) - A) @ mu + A @ y + L @ rng.standard_normal(3)
    Y.append(y)
Y = np.array(Y[200:])
raw = pd.DataFrame(
    {"gdp": 100 * np.exp(np.cumsum(Y[:, 0])), "prices": 50 * np.exp(np.cumsum(Y[:, 1])),
     "rate": Y[:, 2]},
    index=pd.period_range("1960Q1", periods=240, freq="Q"),
)
data = rb.VARData.from_frame(raw, tcodes={"gdp": 5, "prices": 5, "rate": 1})
```

Table G.2: Environment and regeneration record: the release validation record and the walk-through listings (commit, machine, versions of Python, NumPy, SciPy, jax, jaxlib, pandas, xarray and scoringrules), the BLAS backend, the benchmark machines, the release, and the command that regenerates each table and figure.

Item	Release validation record	Walk-through listings
Commit	`7f880e1`, clean	`4ac7969` (branch `main`), clean
Started (UTC)	2026-10-24 04:17	2026-10-24 08:18
Machine, system	Apple M5 Max, 18 cores, macOS-26.6.2-arm64	macOS-26.6.2-arm64-arm-64bit
Python; NumPy, SciPy	3.12.13; 2.5.4, 1.18.1	3.12.13; 2.5.4, 1.18.1
jax, jaxlib	0.12.0.1, —	0.12.0.1, 0.12.0.1
pandas, xarray, scoringrules	—	3.0.6, 2026.9.0, 0.11.0
rai-bvar version (`pyproject.toml`)	0.0.1	0.2.0
BLAS / LAPACK	accelerate (BLAS), accelerate (LAPACK) (environment of this generator)	
BENCH-P2 machines	Apple M5 Max, 18 cores (reference baseline); AMD EPYC 7763 64-Core Processor, 2 vCPU (CI runner)	
Release	0.2.0, `4ac7969` on `main`: the code of `7f880e1`, with the version string set	

Float	§	Generator	Float	§	Generator
Fig1	4.1	`figures/make_fig_architecture.py`	FigD1	D	`figures/make_fig_mutation_heatmap.py`
Tab1	3.5	typed in `sections/03_framework.tex`	FigD2	D	`figures/make_fig_geweke_dots.py`
Tab2	4.2	typed in `sections/appendix.tex`	TabE1	E	`figures/tab_rw_pivots.py`
Fig3	6.8	`figures/make_fig_verdict_flow.py`	FigE1	E	`figures/make_fig_laplacian_pivots.py`
Fig4	6.4	`figures/rb_tail_cell.py`	TabE2	E	`figures/tab_bench.py`
Lst1	7.2	`code/run_examples.py`	TabE3	E	`figures/tab_kernels.py`
Lst2	7.3	`code/run_examples.py`	TabF1	F	`figures/tab_runs.py`
Tab5	8.3	typed in `sections/appendix.tex`	TabF2	F	typed in `sections/appendix.tex`
Tab7	8.11	typed in `sections/08_validation.tex`	TabF3	F	typed in `sections/appendix.tex`
Tab12	11.6	typed in `sections/appendix.tex`	TabF4	F	`figures/tab_cr_precrisis.py`
Tab13	11.9	typed in `sections/11_replication.tex`	TabF5	F	`figures/tab_cr_arms.py`
Fig6	11.9	`figures/make_fig_cr_orderings.py --acc $ACC`	TabF6	F	`figures/tab_cr_budget.py`
TabA1	A	typed in `sections/appendix.tex`	TabF7	F	`figures/tab_cr_levels_compute.py`
FigC1	C	`figures/make_fig_plan.py`	TabF8	F	`figures/tab_first_run.py`
TabC1	C	typed in `sections/appendix.tex`	FigF1	F	`figures/make_fig_cr_per_origin.py` (after `make per-origin`)
TabD1	D	`figures/tab_geweke.py`	FigF2	F	`figures/make_fig_cr_convergence.py --acc $ACC`
TabD2	D	`figures/tab_parity.py`	FigF3	F	`figures/make_fig_cr_cumulative.py` (after `make per-origin`)
TabD3	D	`figures/tab_geweke_full_sbc.py`	TabG1	G	typed in `sections/appendix.tex`
TabD4	D	`figures/tab_twins.py`	TabG2	G	`figures/tab_environment.py`

Upper panel: the release validation record, `meta.json` in `benchmarks/reports/validation/7f880e1_apple-m5-max/`, and the environment record of the walk-through listings, `docs/paper/code/out/env.json` (written by `code/run_examples.py`); — not recorded. Lower panel: every float planned in OUTLINE §14, with its generator (paths from `docs/paper`); run each with the main repository's Python (`make figures` in `docs/paper` runs the figure generators; `$ACC` is the acceptance report, `make per-origin` its per-origin export). 27 of the 38 planned floats have a generator.

```
print(data)
print("prior means of own first lag:", data.prior_means())

# Analytic engine: conjugate Minnesota with SOC/SUR dummies and GLP empirical Bayes.
bvar = rb.BVAR(mean=rb.Linear(prior=rb.Minnesota(soc=1.0, sur=1.0)), lags=2)
print(bvar.engine, "|", bvar.name)
fit = bvar.fit(data)
print(fit.summary())
fc = fit.forecast(horizon=4, draws=2000, seed=0)
print(fc)
print(fc.quantiles([0.05, 0.5, 0.95]).sel(horizon=1).round(4).to_pandas())
print("paths_level h=1 median:", fc.median(level=True).sel(horizon=1).round(3).values)

# Pseudo out-of-sample comparison on rolling windows (valid DM/AG p-values).
res = rb.compare(
    {"RW": rb.RandomWalk(), "OLS-VAR(2)": rb.OLSVAR(lags=2), "BVAR(2)": bvar},
    data,
    origins=rb.Origins(start="2010Q1", end="2019Q3"),
    window=rb.Rolling(size=120),
    horizons=[1, 4],
    baseline="RW",
```

```
)
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)
pd.set_option("display.max_colwidth", 200)
cols = ["origins", "dlpd", "dlpd_mcse", "crps_rel", "dm_pvalue", "tests_valid", "in_mcs_25", "converged"]
print(res.table()[cols].round(3))
v = res.verdict()
print(v[["model", "reference", "axis", "horizon", "verdict"]])
for r in v.itertuples():
    print(f"{r.model} vs {r.reference} h={r.horizon}: {r.reasons}")
```

Listing G.2: Output of Listing G.1 (stdout, verbatim).

```
VARData(variables=['gdp', 'prices', 'rate'], T=239, 1960Q2..2019Q4, freq='Q-DEC', vintage=None)
prior means of own first lag: {'gdp': 0.0, 'prices': 0.0, 'rate': 1.0}
analytic | BVAR(Minnesota(conjugate, eb, SOC+SUR), p=2)
            mode
lambda  0.268149
soc     1.119513
sur     1.176805
ForecastResult(origin=2019Q4, horizon=4, variables=['gdp', 'prices', 'rate'], samples=2000)
variable     gdp  prices    rate
quantile
0.05     -0.0053  0.0027  4.2629
0.50      0.0053  0.0073  4.6754
0.95      0.0158  0.0120  5.1097
paths_level h=1 median: [349.266 182.507   4.675]
                   origins    dlpd  dlpd_mcse  crps_rel  dm_pvalue  tests_valid  in_mcs_25  converged
model      horizon
RW         1             39   0.000      0.000     1.000        NaN         True      False        1.0
OLS-VAR(2) 1             39  14.447      0.102     0.907      0.029         True       True        1.0
BVAR(2)    1             39  15.368      0.094     0.893      0.020         True       True        1.0
RW         4             36   0.000      0.000     1.000        NaN         True      False        1.0
OLS-VAR(2) 4             36  33.191      0.884     0.860      0.001         True       True        1.0
BVAR(2)    4             36  35.051      0.946     0.842      0.002         True       True        1.0
        model reference                                                 axis  horizon         verdict
0  OLS-VAR(2)        RW                                                model        1          better
1     BVAR(2)        RW  deterministic,lags,mean,model,outliers,prior,vol        1  not comparable
2  OLS-VAR(2)        RW                                                model        4          better
3     BVAR(2)        RW  deterministic,lags,mean,model,outliers,prior,vol        4  not comparable
OLS-VAR(2) vs RW h=1: ΔLPD=+14.45 (MC s.e. 0.10); DM p=0.014 (valid: rolling window); MCS25: model in, reference out
BVAR(2) vs RW h=1: differs along ['deterministic', 'lags', 'mean', 'model', 'outliers', 'prior', 'vol']; a verdict needs
    ↪ exactly one axis (SPEC R2)
OLS-VAR(2) vs RW h=4: ΔLPD=+33.19 (MC s.e. 0.88); DM p=0.000 (valid: rolling window); MCS25: model in, reference out
BVAR(2) vs RW h=4: differs along ['deterministic', 'lags', 'mean', 'model', 'outliers', 'prior', 'vol']; a verdict needs
    ↪ exactly one axis (SPEC R2)
```

The log difference (code 5) consumes the first period: 239 of the 240 raw rows remain. Line 35 prints the resolved engine, and lines 40–41 summarise the forecast on the transformed scale and, through the pathwise inverse, on the level scale. Empirical Bayes sets the tightness (printed as `lambda`; κ in Section 3) and the SOC and SUR weights. The comparison runs 39 rolling-window origins at $h = 1$ and 36 at $h = 4$. A comparison that credits the prior would pair the BVAR with an arm that differs from it only in the prior.

Listing G.3: `ex_b.py` in full (from line 8; lines 1–7 are comments).

```
import platform

import numpy as np
import pandas as pd

import rai_bvar as rb

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)
```

```python
pd.set_option("display.max_colwidth", 200)

# Same simulated data as ex_a.py
rng = np.random.default_rng(0)
A = np.array([[0.4, 0.0, -0.001], [0.1, 0.6, 0.0], [15.0, 25.0, 0.85]])
mu = np.array([0.0075, 0.006, 4.0])
sd = np.array([0.007, 0.003, 0.25])
corr = np.array([[1.0, 0.2, 0.3], [0.2, 1.0, 0.2], [0.3, 0.2, 1.0]])
L = np.linalg.cholesky(corr * np.outer(sd, sd))
y, Y = mu.copy(), []
for _ in range(200 + 240):
    y = (np.eye(3) - A) @ mu + A @ y + L @ rng.standard_normal(3)
    Y.append(y)
Y = np.array(Y[200:])
raw = pd.DataFrame(
    {"gdp": 100 * np.exp(np.cumsum(Y[:, 0])), "prices": 50 * np.exp(np.cumsum(Y[:, 1])),
     "rate": Y[:, 2]},
    index=pd.period_range("1960Q1", periods=240, freq="Q"),
)
data = rb.VARData.from_frame(raw, tcodes={"gdp": 5, "prices": 5, "rate": 1})
print(platform.platform(), "| rai-bvar", rb.__version__)

# MCMC engine: same Linear mean, stochastic volatility; form="auto" resolves to "structural".
mc = rb.MCMCSettings(chains=4, draws=1000, burn=1000)
sv = rb.BVAR(vol=rb.SV(), lags=2, mcmc=mc)
print(sv.engine, "|", sv.name, "|", sv.mean.prior.form, "|", sv.axes)
fit = sv.fit(data.upto("2019Q3"))
s = fit.summary()
print("prior mode:", s.attrs["prior_mode"], "| hyper:", s.attrs["hyper"], "| converged:", fit.converged)
print(s[["mean", "sd", "rhat", "ess_bulk"]].round(3))
fc = fit.forecast(horizon=4, draws=2000, seed=0)
print(fc, "| engine attr:", fc.dataset.attrs["engine"])
y_obs = data.values[-1]  # 2019Q4, transformed scale
print("joint LPD h=1 and MCSE:", fc.log_pred_density(y_obs, horizon=1, return_mcse=True))
print("marginal LPD (rate):", fc.log_pred_density(y_obs[2:], horizon=1, variables=["rate"]))
irf = fit.irf(horizon=8, draws=500, seed=0)
print(irf)
print(irf.median().sel(horizon=[0, 4, 8], shock="rate").round(4).to_pandas())

# Volatility-axis comparison: both arms structural Mode I, differing only in vol (R2).
models = {
    "Linear x Constant": rb.BVAR(vol=rb.Constant("ig"), lags=2, mcmc=mc),
    "Linear x SV": sv,
}
for k, m in models.items():
    print(k, m.axes)
res = rb.compare(
    models, data,
    origins=rb.Origins(start="2017Q1", end="2019Q3"),
    window=rb.Rolling(size=120),
    horizons=[1],
    draws=1000,
    baseline="Linear x Constant",
)
print(res.table()[["origins", "dlpd", "dlpd_mcse", "dm_pvalue", "tests_valid", "in_mcs_25", "converged"]].round(3))
v = res.verdict(axis="vol")
print(v[["model", "reference", "axis", "horizon", "verdict"]])
for r in v.itertuples():
    print(r.reasons)
```

Listing G.4: Output of Listing G.3 (stdout, verbatim; MCMC numbers are platform-specific).

```
macOS-26.6.2-arm64-arm-64bit | rai-bvar 0.2.0
mcmc | BVAR(Minnesota(structural, eb), SV, p=2, mcmc=45b567) | structural | {'mean': 'Linear', 'prior': 'Minnesota(
    ↪ structural, eb)', 'vol': 'SV', 'lags': '2', 'deterministic': 'c', 'outliers': 'none'}
prior mode: Mode I | hyper: {'hierarchy': 'eb', 'lambda': 0.24549046712778935, 'alpha': 2.0, 'cross': 1.0, 'contemp': 1.0,
    ↪ 'var_const': 10000000.0, 'eb_method': 'conjugate_twin', 'eb_converged': True, 'offset_rule': 'relative', 'c_rel': 1
```

```
    ↪ e-12} | converged: False
                  mean     sd   rhat  ess_bulk
quantity
loglik          1928.304  7.948  1.031   179.391
B_own1[gdp]        0.461  0.069  1.000  3010.071
B_own1[prices]     0.558  0.065  0.999  2735.739
B_own1[rate]       0.948  0.054  1.001  3537.035
mu[gdp]          -10.108  0.129  1.003   665.300
mu[prices]       -11.834  0.160  1.001   757.856
mu[rate]          -2.892  0.155  1.002   692.202
phi[gdp]           0.806  0.118  1.035   131.873
phi[prices]        0.831  0.111  1.034   125.015
phi[rate]          0.861  0.110  1.017   197.937
sigma[gdp]         0.156  0.112  1.036   134.415
sigma[prices]      0.200  0.113  1.042   109.911
sigma[rate]        0.088  0.062  1.008   362.438
h_T[gdp]         -10.226  0.356  1.009   756.713
h_T[prices]      -11.766  0.361  1.010  1264.266
h_T[rate]         -2.831  0.246  1.002   887.965
ForecastResult(origin=2019Q3, horizon=4, variables=['gdp', 'prices', 'rate'], samples=2000) | engine attr: mcmc
joint LPD h=1 and MCSE: (8.575356603927622, 0.007996504228586188)
marginal LPD (rate): -0.4000308979305567
IRFResult(horizon=8, variables=['gdp', 'prices', 'rate'], shock_size='sd', samples=500)
response     gdp  prices    rate
horizon
0         0.0000     0.0  0.2397
4        -0.0002     0.0  0.1230
8        -0.0001    -0.0  0.0456
Linear x Constant {'mean': 'Linear', 'prior': 'Minnesota(structural, eb)', 'vol': 'Constant', 'lags': '2', 'deterministic':
    ↪  'c', 'outliers': 'none'}
Linear x SV {'mean': 'Linear', 'prior': 'Minnesota(structural, eb)', 'vol': 'SV', 'lags': '2', 'deterministic': 'c', '
    ↪ outliers': 'none'}
                         origins   dlpd  dlpd_mcse  dm_pvalue  tests_valid  in_mcs_25  converged
model            horizon
Linear x Constant 1             11  0.000      0.000        NaN         True       True        1.0
Linear x SV       1             11 -0.107      0.102      0.842         True       True        0.0
         model          reference axis  horizon       verdict
0  Linear x SV  Linear x Constant  vol        1  inconclusiveΔ
LPD=-0.11 (MC s.e. 0.10, not precise enough); DM p=0.421 (valid: rolling window); MCS25: model in, reference in; converged
    ↪ at only 0% of origins
```

Unlike Listing 1, the default mean of Listing G.3 has no dummy observations, and its Minnesota hyperparameters are the empirical-Bayes plug-in of the conjugate twin (`eb_method: conjugate_twin` in line 3 of the output). The summary reports $\hat{R}$ and bulk ESS for every primary quantity. On the homoskedastic DGP of the walk-through the SV parameters mix slowly: ϕ_i and σ_i have bulk ESS between 110 and 362 from 4000 kept draws and $\hat{R}$ up to 1.042, so the fit is flagged as not converged; the library's warning (on stderr, not shown) reports that 7 of the 16 primary quantities fail, which by the table are the six SV parameters ϕ_i, σ_i and the log-likelihood ($\hat{R}$ 1.031, bulk ESS 179). In the final comparison the axis labels of the two arms differ only in `vol`. With the interest rate ordered last, a one-s.d. rate shock has no impact effect on output growth or inflation in the recursive identification (lines 52–54 of Listing G.3). A longer chain is the first remedy for the unconverged SV fit of the walk-through. The `engine` attribute of the forecast records which engine produced it, and the MCMC forecast also carries the ordering, sampler metadata and accelerator variables (Table C.3).

H Licensing and source-reading practice

rai-bvar is released under the Apache License 2.0. The package's practice with respect to GPL-licensed reference software is stated in these words:

No code was translated from GPL sources. Algorithms were implemented from the published papers. GPL packages were run as numerical oracles, and in a few documented cases (named in the repository notes) their source was read to pin down an oracle's behaviour, defaults or constants, never to write implementation code.

The sessions that read source and those that wrote code were AI agent sessions; what the practice cannot exclude for code written by language models is stated in Appendix J. Table H.1 lists the repository notes in which GPL or unlicensed source was read, with the versions and the purpose each note records, and the notes in which none was read. The versions in the table are those of the source that was read; the parity fixtures were produced with stochvol 3.2.1 and R BVAR 1.0.5 (Appendix D.7). Not every package whose source was read is an oracle of this release: the notes also record reading the sources of bayesianVARs, bvarsv, bvartools, factorstochvol and bsvars to confirm published constants and observable behaviour. The source of the MIT-licensed Bank of England package was read to document its conventions as an oracle.

Table H.1: Repository notes and the reading of third-party source they record.

Note	What was read, and for what
Notes in which source was read	
`docs/phase2/02_sv_sampler.md`	stochvol (GitHub sources and CRAN manuals), and the bayesianVARs 0.1.8 and bvarsv 1.1 sources, in the note itself; stochvol 3.2.9, bayesianVARs 0.1.8 and bvarsv 1.1 in its cross-check; read only for published defaults, constants and behaviour; no code transcribed
`docs/phase2/01_omori_mixture.md`	six GPL packages opened only to confirm the published mixture constants and observable behaviour (which table, which offset, whether a shift is applied)
`docs/phase2/03_sv_outliers_t.md`	greps of the stochvol C++ and R sources (the ν proposal, the update order, the t scaling of the simulator), and greps of the replication code of Carriero et al. (2024), which carries no licence (prior constants, outlier grid, mixture table, the ν log-likelihood constant)
`docs/phase2/07_jax_patterns.md`	in the cross-check, one stochvol header searched only for the 10 published mixture constants
`tests/parity/BVAR_PARAMETRIZATION.md`	R BVAR 1.0.5 sources and the JSS article, read to establish the behaviour of the oracle; "not a description of code to be ported"
Notes in which no GPL source was read	
`tests/parity/SV_PARAMETRIZATION.md` (packages only run; behaviour from manuals and experiments); `docs/phase2/04_cccm_corrected.md`; `docs/phase2/05_structural_equation_sampler.md`; `docs/phase2/06_validation_geweke_sbc.md`; `docs/phase2/08_common_sv_and_ordering.md`.	

The committed parity fixtures that the R oracles produced (in `tests/parity/fixtures/`) are outputs of GPL programs run as oracles; the project treats them as data, not code. The Bank of England bvar package (Labonne et al., 2026) is MIT-licensed. FRED-MD and FRED-QD (McCracken and Ng, 2016, 2021) are downloaded on demand and never bundled, because some series are third-party copyrighted; the named variable sets of the loader, and the committed fixtures, use only U.S. federal-agency series.

I Design-review errata relevant to numerics and reproducibility

The Phase 2 design was reviewed before implementation by two judge agents (Appendix J), and its errata table maps each of their 28 findings (E1–E28) to its resolution. Four of those that concern numerics and reproducibility are listed below; each was found in the proposals and resolved in the design before the code it concerned was written.

E7 The counter $it_0 + j \cdot \text{thin} + r$ promotes to int64 under 64-bit mode. All counters are uint32 with explicit casts, `step_key` raises on any other counter type, and the host checks that the sweep count stays below 2^{32} (Algorithm C.10).

E9 An earlier proposal validated the samplers with the blocks' own prior samplers and log priors. The design adopted an independent reference registry with separate owners (separate agent instances, Appendix J), which became the two-implementation rule of Section 8.

E14 Two "runtime" settings, the Gram mode and padding to a maximum length, were not bitwise-neutral. The Gram mode is now resolved by a fixed shape rule and stored in the specification, there is no padding to a maximum, and `Runtime` holds only settings that have an invariance test, bitwise except for the device count, which is to rounding (Table G.1).

E23 $2 \log |e_t|$ is $-\infty$ on padded rows and produced NaN. The residual is masked before the logarithm (Algorithm C.6).

The review process is described in Appendix J; the generic lessons for other implementations are in Section 9.

J Development process and AI-use statement

Statement. rai-bvar and this paper were written with AI coding agents: Claude (Anthropic), used through Claude Code, in local sessions and in cloud sessions on a four-core x86-64 Linux virtual machine. Every commit except the nine pull-request merges was made in an agent session (below). The agents wrote the algorithm notes and the Phase 2 design, implemented and reviewed the Phase 2 work packages (the benchmark and replication scripts among them) and drafted this paper with its figure and table scripts, this appendix included (below); the specification (`SPEC.md`) and the code of Phases 0 and 1 were committed from agent sessions as well. They also ran the tests, benchmarks, replication runs and reviews described below. The author takes responsibility for the content of the paper and of the software, including any error that the agents made and the checks did not catch. An AI system is not an author of this paper. The author set the product idea and the requirements; the specification (its v1 is not in the repository) and all of the code, that of Phases 0 and 1 included, were written by the agents under his direction.

The commit record. At `main` 4ac7969 (release 0.2.0) and `paper-draft` fd8a3f7 the repository has 77 commits, made between 21 October 2026 (21:54 UTC) and 24 October (07:26 UTC). Sixty-eight carry a `Co-Authored-By` trailer that names the agent: both commits of Phase 0, both of Phase 1, 57 of the 65 commits of Phase 2 (with its close-out and the release) and the seven commits of the paper branch. The other nine are the merge commits of the nine pull requests, made on GitHub. Every commit has the author's name in its author field. On the 16 commits made in cloud sessions the author address is an Anthropic no-reply address and the committer is the agent (12 of them also link their session); on the other 52 commits outside GitHub, merges between branches among them, author and committer are the author's own name and address, as on commits from local sessions. On those 52 only the trailer marks a commit made in an agent session, and no trailer shows how much of a commit, if anything, the author wrote.

Who did what.

Algorithm notes. The eight notes `docs/phase2/01`–`08`, from which the samplers were implemented, were written in agent sessions from the papers and package documentation and, in

the notes that Appendix H lists, from package source read for constants and behaviour. Each records an independent cross-check that re-read the sources and re-ran numerical checks with scripts of its own; four of the notes (02–05) say that a second agent made it, and the others do not say who did.

Design. Three competing design proposals (correctness-, performance- and extensibility-first) were written by agents and ranked by two judge agents; the highest-ranked became the base of `docs/phase2/DESIGN.md`, and every finding of the judges was mapped to a resolution (errata E1–E28, Appendix I).

Implementation. The design splits Phase 2 into 25 work packages, each owning its files. Agents of a multi-agent workflow implemented the packages, several at a time on the same machine, coordinated by an orchestrating session that also recorded the design amendments; the references of the validation harness came from packages whose agents wrote no kernel code (Section 8.1).

Reviews. Phase 1: three adversarial review rounds by 63, 49 and 10 agents found and fixed 96 defects before the Phase 1 commit. Phase 2: in the first Phase 2 pull request (WP1–WP22) each stage was reviewed before the next began; the backtest and replication script (WP23) and the benchmark work (WP24) were reviewed together, with the fixes made in the cloud, the dummy-row work (WP25) in the cloud on x86-64 Linux, the fix f60d736 and the estimator D-34 by further agents (D-34 by a "four-lens" review that found two defects), and the close-out by a verifier agent that checked 131 changed statements against the code and corrected 13.

Runs. Agents ran, or started through continuous integration, the tests, benchmarks and replication runs reported here. The x86-64 runs of the mutation suite at earlier commits, an earlier BENCH-P2 under load and the quick replication run were made in cloud sessions; the x86-64 BENCH-P2 report of Table E.2 and the per-PR tier on the release commit come from continuous integration, and the Apple runs, the release validation record among them, from local sessions. On the night of 22 to 23 October the cloud session wrote a work queue for a local session at the author's request ("The user is asleep: don't wait for their input"). The local session ran the reference benchmark, the weekly tier and the full replication runs, diagnosed the crash F2 and wrote its fix (Section 9.2), took over the orchestration and opened one combined pull request. The queue told the local session not to open a pull request or merge (the orchestrator was to combine the work into one pull request), not to install system software, and not to change priors, settings or criteria to make anything pass; the orchestrator left the merge to the author. The pre-registration of the acceptance run (c823d20) was written in an agent session before the run was launched. The session fixed its settings under the author's standing instruction of 2026-10-23 to complete the replication and its evaluation.

This paper. Agents drafted the paper in stages: six evidence dossiers on the repository, each fact-checked against it by a separate agent; three competing outlines, of which two judge agents chose one, with parts of the other two grafted onto it (`docs/paper/OUTLINE.md` §0); nine section writers, each followed by a fact-checker; an assembly pass; a cut to the page budget, checked claim by claim against the earlier draft; and a review round in which three agents wrote referee reports from the standpoints of computational statistics, macro-econometrics and research software (67 comments; the triage is in `docs/paper/REVIEW_RESPONSE.md`, the reports themselves are not committed). An agent compiled the bibliography and checked each entry against an authoritative record: Crossref for entries with a DOI, otherwise the

arXiv, CRAN, PyPI or GitHub record, a publisher's or library record, or for two books a review that states the details (`docs/paper/dossiers/99_bibliography_report.md`; each entry's source is named in `refs.bib`). By the drafting rule, every number carries an evidence tag in a comment of the LaTeX source that names the test, file, commit or run it rests on, and the acceptance-run numbers were filled by a script from the report and checked against it (Appendix F.1). This appendix was drafted by an agent from the git history, the pull requests and issue #3.

What "independent" means in this paper. Where this paper calls a review, re-run, cross-check, fact-check or reference implementation independent, or says that work packages had separate owners (the design judges, the notes' cross-checks, the work-package owners of the two-implementation rule, the reviews of WP23/WP24, WP25, f60d736 and D-34, the close-out verifier, the x86-64 re-runs, the dossiers' fact-checks, the outline judges and the referee reports), the parties were separate instances of the same AI coding agents (sessions, or agents started by a session's workflow), each with its own context, and not independent people. The record says so in words for the Phase 1 review rounds, four of the notes' cross-checks and the work-package implementers; elsewhere it names a judge, reviewer, verifier or fact-checker without saying who that was, but every such record was written in an agent session and none names a person. That secures some things and not others. It secures separate code paths: a test enforces that no reference imports a kernel module, and the references use other algebra than the kernels (Section 8.1); the dummy-row reference, though, was written by the kernels' owners. The mechanical checks (identities, Geweke and SBC tests, analytic twins, the mutation suite) reach the same verdict whoever runs them and can be re-run from the committed tests, and a run on a second platform is a genuine re-execution. It does not give independent human judgement. Instances of the same agents can share blind spots and misreadings, and all of them worked from the same documents (`SPEC.md`, `DESIGN.md`, the notes), so an error in a shared document, or one misreading of a paper, can enter a kernel and its reference alike; the Geweke and SBC tests in addition share our prior and simulator (Section 8.1). Fewer checks have a reference external to the agents: the third-party oracles of Tier P (stochvol, R BVAR, the Bank of England bvar), library routines used as oracles in some tests (SciPy's distributions, for example), and published constants and formulas; even for these the agents wrote the comparison code and chose what to compare. No reviewer other than the agents is on record for the code or for this paper (no pull request has a GitHub review). No person took part in the reviews, judgements and fact-checks listed above: every one was an agent instance.

Licensing. The source-reading practice of Appendix H records what the sessions read and wrote. The agents are language models whose training data are not known to the author and may include GPL-licensed code, so that practice cannot exclude that their output resembles such code; no check for such resemblance is recorded.

Authorship and responsibility. Yigit Okar is the sole author. On record, he directed the agents (the overnight queue, for example, was written at his request), and the design leaves some decisions to him as the project's owner: DESIGN Q14, the default convergence policy for SV fits, was marked "the owner decides" and open at ae5f8d6. Q14 was resolved as an agent recommended, at the author's instruction of 2026-10-24 (D-35); DESIGN §12.2 records that the recommendations for the earlier open questions were "accepted as stated" on 2026-10-22, without saying by whom. Merging was reserved for him ("merging is the user's decision", issue #3), and all nine pull requests were merged from his GitHub account. Pull requests #1 and #4–#10 were merged by an agent

session at the author's explicit instruction in that session (#8–#10 under his instruction to finish the paper and prepare its arXiv package); who merged #2 is not on record beyond the account. He is responsible for the paper and for the software.

References

Oriol Abril-Pla, Virgile Andreani, Colin Carroll, Larry Dong, Christopher J. Fonnesbeck, Maxim Kochurov, Ravin Kumar, Junpeng Lao, Christian C. Luhmann, Osvaldo A. Martin, Michael Osthege, Ricardo Vieira, Thomas Wiecki, and Robert Zinkov. PyMC: A modern, and comprehensive probabilistic programming framework in Python. *PeerJ Computer Science*, 9:e1516, 2023. doi: 10.7717/peerj-cs.1516.

Malin Adolfson, Jesper Lindé, and Mattias Villani. Forecasting performance of an open economy DSGE model. *Econometric Reviews*, 26(2–4):289–328, 2007. doi: 10.1080/07474930701220543.

Attahiru Sule Alfa, Jungong Xue, and Qiang Ye. Accurate computation of the smallest eigenvalue of a diagonally dominant M-matrix. *Mathematics of Computation*, 71(237):217–236, 2002. doi: 10.1090/S0025-5718-01-01325-4.

Gianni Amisano and Raffaella Giacomini. Comparing density forecasts via weighted likelihood ratio tests. *Journal of Business & Economic Statistics*, 25(2):177–190, 2007. doi: 10.1198/073500106000000332.

Jonas E. Arias, Juan F. Rubio-Ramírez, and Minchul Shin. Macroeconomic forecasting and variable ordering in multivariate stochastic volatility models. *Journal of Econometrics*, 235(2):1054–1086, 2023. doi: 10.1016/j.jeconom.2022.04.013.

Marta Bańbura, Domenico Giannone, and Lucrezia Reichlin. Large Bayesian vector auto regressions. *Journal of Applied Econometrics*, 25(1):71–92, 2010. doi: 10.1002/jae.1137.

Dominik Bertsche and Robin Braun. Identification of structural vector autoregressions by stochastic volatility. *Journal of Business & Economic Statistics*, 40(1):328–341, 2022. doi: 10.1080/07350015.2020.1813588.

Åke Björck. *Numerical Methods for Least Squares Problems*. Society for Industrial and Applied Mathematics, 1996. doi: 10.1137/1.9781611971484.

Maximilian Boeck, Martin Feldkircher, and Florian Huber. BGVAR: Bayesian global vector autoregressions with shrinkage priors in R. *Journal of Statistical Software*, 104(9):1–28, 2022. doi: 10.18637/jss.v104.i09.

Mark Bognanni. Comment on "large Bayesian vector autoregressions with stochastic volatility and non-conjugate priors". *Journal of Econometrics*, 227(2):498–505, 2022. doi: 10.1016/j.jeconom.2021.10.008.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. *JAX: Composable Transformations of Python+NumPy Programs*, 2018. URL `https://github.com/jax-ml/jax`. Python package version 0.12.0.1.

Paul-Christian Bürkner, Jonah Gabry, and Aki Vehtari. Approximate leave-future-out cross-validation for Bayesian time series models. *Journal of Statistical Computation and Simulation*, 90(14):2499–2523, 2020. doi: 10.1080/00949655.2020.1783262.

Andrea Carriero, Todd E. Clark, and Massimiliano Marcellino. Bayesian VARs: Specification choices and forecast accuracy. *Journal of Applied Econometrics*, 30(1):46–73, 2015. doi: 10.1002/jae.2315.

Andrea Carriero, Todd E. Clark, and Massimiliano Marcellino. Common drifting volatility in large Bayesian VARs. *Journal of Business & Economic Statistics*, 34(3):375–390, 2016. doi: 10.1080/07350015.2015.1040116.

Andrea Carriero, Todd E. Clark, and Massimiliano Marcellino. Large Bayesian vector autoregressions with stochastic volatility and non-conjugate priors. *Journal of Econometrics*, 212(1):137–154, 2019. doi: 10.1016/j.jeconom.2019.04.024.

Andrea Carriero, Joshua Chan, Todd E. Clark, and Massimiliano Marcellino. Corrigendum to "large Bayesian vector autoregressions with stochastic volatility and non-conjugate priors" [J. Econometrics 212 (1) (2019) 137–154]. *Journal of Econometrics*, 227(2):506–512, 2022. doi: 10.1016/j.jeconom.2021.11.010.

Andrea Carriero, Todd E. Clark, Massimiliano Marcellino, and Elmar Mertens. Addressing COVID-19 outliers in BVARs with stochastic volatility. *Review of Economics and Statistics*, 106(5):1403–1417, 2024. doi: 10.1162/rest_a_01213.

Joshua C. C. Chan. Asymmetric conjugate priors for large Bayesian VARs. *Quantitative Economics*, 13(3):1145–1169, 2022. doi: 10.3982/QE1381.

Joshua C. C. Chan. BVARs and stochastic volatility. In Michael P. Clements and Ana Beatriz Galvão, editors, *Handbook of Research Methods and Applications in Macroeconomic Forecasting*, chapter 3, pages 43–67. Edward Elgar Publishing, 2024. doi: 10.4337/9781035310050.00007. Preprint: arXiv:2310.14438.

Joshua C. C. Chan. *bvar-toolkit: MATLAB Library for Large Bayesian VARs*, 2026a. URL `https://github.com/joshuaccchan/bvar-toolkit`.

Joshua C. C. Chan. *bvar-toolkit-python: Python Version of bvar-toolkit*, 2026b. URL `https://github.com/joshuaccchan/bvar-toolkit-python`.

Joshua C. C. Chan and Ivan Jeliazkov. Efficient simulation and integrated likelihood estimation in state space models. *International Journal of Mathematical Modelling and Numerical Optimisation*, 1(1/2):101–120, 2009. doi: 10.1504/IJMMNO.2009.030090.

Joshua C. C. Chan, Gary Koop, and Xuewen Yu. Large order-invariant Bayesian VARs with stochastic volatility. *Journal of Business & Economic Statistics*, 42(2):825–837, 2024. doi: 10.1080/07350015.2023.2252039.

Jean-Paul Chilès and Pierre Delfiner. *Geostatistics: Modeling Spatial Uncertainty*. Wiley Series in Probability and Statistics. John Wiley & Sons, 2nd edition, 2012. doi: 10.1002/9781118136188.

J. Andrés Christen and Colin Fox. Markov chain Monte Carlo using an approximation. *Journal of Computational and Graphical Statistics*, 14(4):795–810, 2005. doi: 10.1198/106186005X76983.

Todd E. Clark. Real-time density forecasts from Bayesian vector autoregressions with stochastic volatility. *Journal of Business & Economic Statistics*, 29(3):327–341, 2011. doi: 10.1198/jbes.2010.09248.

Todd E. Clark and Michael W. McCracken. Tests of equal forecast accuracy and encompassing for nested models. *Journal of Econometrics*, 105(1):85–110, 2001. doi: 10.1016/S0304-4076(01)00071-9.

Todd E. Clark and Francesco Ravazzolo. The macroeconomic forecasting performance of autoregressive models with alternative specifications of time-varying volatility. Working Paper 12-18, Federal Reserve Bank of Cleveland, 2012. Also circulated as Norges Bank Working Paper 9/2012.

Todd E. Clark and Francesco Ravazzolo. Macroeconomic forecasting performance under alternative specifications of time-varying volatility. *Journal of Applied Econometrics*, 30(4):551–575, 2015a. doi: 10.1002/jae.2379.

Todd E. Clark and Francesco Ravazzolo. Macroeconomic forecasting performance under alternative specifications of time-varying volatility (replication data). Journal of Applied Econometrics Data Archive, 2015b. URL `http://qed.econ.queensu.ca/jae/2015-v30.4/clark-ravazzolo/`. Files readme.cr.txt, cr-data-csv.zip, cr-data-xls.zip; accessed 2026-10-23.

Todd E. Clark, Florian Huber, Gary Koop, Massimiliano Marcellino, and Michael Pfarrhofer. Tail forecasting with multivariate Bayesian additive regression trees. *International Economic Review*, 64(3):979–1022, 2023. doi: 10.1111/iere.12619.

Timothy Cogley and Thomas J. Sargent. Drifts and volatilities: Monetary policies and outcomes in the post WWII US. *Review of Economic Dynamics*, 8(2):262–302, 2005. doi: 10.1016/j.red.2004.10.009.

Samantha R. Cook, Andrew Gelman, and Donald B. Rubin. Validation of software for Bayesian models using posterior quantiles. *Journal of Computational and Graphical Statistics*, 15(3):675–692, 2006. doi: 10.1198/106186006X136976.

Dean Croushore and Tom Stark. A real-time data set for macroeconomists. *Journal of Econometrics*, 105(1):111–130, 2001. doi: 10.1016/S0304-4076(01)00072-0.

A. Philip Dawid and Paola Sebastiani. Coherent dispersion criteria for optimal experimental design. *The Annals of Statistics*, 27(1):65–81, 1999. doi: 10.1214/aos/1018031101.

Marco Del Negro and Giorgio E. Primiceri. Time varying structural vector autoregressions and monetary policy: A corrigendum. *Review of Economic Studies*, 82(4):1342–1345, 2015. doi: 10.1093/restud/rdv024.

Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.

James Demmel, Ioana Dumitriu, Olga Holtz, and Plamen Koev. Accurate and efficient expression evaluation and linear algebra. *Acta Numerica*, 17:87–145, 2008. doi: 10.1017/S0962492906350015.

Francis X. Diebold and Roberto S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995. doi: 10.1080/07350015.1995.10524599.

Alistair Dieppe, Romain Legrand, and Björn van Roye. The BEAR toolbox. Working Paper Series 1934, European Central Bank, 2016. URL `https://www.ecb.europa.eu/pub/pdf/scpwps/ecbwp1934.en.pdf`. Also available from SSRN, DOI 10.2139/ssrn.2811020.

Thomas Doan, Robert Litterman, and Christopher Sims. Forecasting and conditional projection using realistic prior distributions. *Econometric Reviews*, 3(1):1–100, 1984. doi: 10.1080/07474938408800053.

Saikat Dutta, Owolabi Legunsen, Zixin Huang, and Sasa Misailovic. Testing probabilistic programming systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)*, pages 574–586. ACM, 2018. doi: 10.1145/3236024.3236057.

James M. Flegal and Galin L. Jones. Batch means and spectral variance estimators in Markov chain Monte Carlo. *The Annals of Statistics*, 38(2):1034–1070, 2010. doi: 10.1214/09-AOS735.

Colin Fox and Albert Parker. Accelerated Gibbs sampling of normal distributions using matrix splittings and polynomials. *Bernoulli*, 23(4B):3711–3743, 2017. doi: 10.3150/16-BEJ863.

Roy Frostig, Matthew James Johnson, and Chris Leary. Compiling machine learning programs via high-level tracing. In *SysML Conference 2018*, 2018. URL `https://mlsys.org/Conferences/doc/2018/146.pdf`.

Sylvia Frühwirth-Schnatter and Helga Wagner. Stochastic model specification search for Gaussian and partial non-Gaussian state space models. *Journal of Econometrics*, 154(1):85–100, 2010. doi: 10.1016/j.jeconom.2009.07.003.

Axel Gandy and James Scott. Unit testing for MCMC and other Monte Carlo methods, 2021. URL `https://arxiv.org/abs/2001.06465`. arXiv:2001.06465v2.

Andrew Gelman. Correction to Cook, Gelman, and Rubin (2006). *Journal of Computational and Graphical Statistics*, 26(4):940, 2017. doi: 10.1080/10618600.2017.1377082.

John Geweke. Getting it right: Joint distribution tests of posterior simulators. *Journal of the American Statistical Association*, 99(467):799–804, 2004. doi: 10.1198/016214504000001132.

John Geweke and Gianni Amisano. Comparing and evaluating Bayesian predictive distributions of asset returns. *International Journal of Forecasting*, 26(2):216–230, 2010. doi: 10.1016/j.ijforecast.2009.10.007.

Charles J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992. doi: 10.1214/ss/1177011137.

Raffaella Giacomini and Halbert White. Tests of conditional predictive ability. *Econometrica*, 74(6):1545–1578, 2006. doi: 10.1111/j.1468-0262.2006.00718.x.

Domenico Giannone, Michele Lenza, and Giorgio E. Primiceri. Prior selection for vector autoregressions. *Review of Economics and Statistics*, 97(2):436–451, 2015. doi: 10.1162/REST_a_00483.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Tilmann Gneiting and Roopesh Ranjan. Comparing density forecasts using threshold- and quantile-weighted scoring rules. *Journal of Business & Economic Statistics*, 29(3):411–422, 2011. doi: 10.1198/jbes.2010.08110.

Winfried K. Grassmann, Michael I. Taksar, and Daniel P. Heyman. Regenerative analysis and steady state distributions for Markov chains. *Operations Research*, 33(5):1107–1116, 1985. doi: 10.1287/opre.33.5.1107.

Roger B. Grosse and David K. Duvenaud. Testing MCMC code, 2014. URL `https://arxiv.org/abs/1412.5218`. arXiv:1412.5218. Presented at the 2014 NIPS Workshop on Software Engineering for Machine Learning.

Luis Gruber and Gregor Kastner. Forecasting macroeconomic data with Bayesian VARs: Sparse or dense? It depends! *International Journal of Forecasting*, 41(4):1589–1619, 2025. doi: 10.1016/j.ijforecast.2025.02.001.

Luis Gruber, Stefan Haan, and Gregor Kastner. *bayesianVARs: MCMC Estimation of Bayesian Vectorautoregressions*, 2026. URL `https://CRAN.R-project.org/package=bayesianVARs`. R package version 0.1.8.

A. K. Gupta and D. K. Nagar. *Matrix Variate Distributions*, volume 104 of *Monographs and Surveys in Pure and Applied Mathematics*. Chapman & Hall/CRC, Boca Raton, 2000. ISBN 1-58488-046-5.

Peter R. Hansen, Asger Lunde, and James M. Nason. The model confidence set. *Econometrica*, 79 (2):453–497, 2011. doi: 10.3982/ECTA5771.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

David Harvey, Stephen Leybourne, and Paul Newbold. Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2):281–291, 1997. doi: 10.1016/S0169-2070(96)00719-4.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2nd edition, 2002. doi: 10.1137/1.9780898718027.

Daichi Hiraki, Siddhartha Chib, and Yasuhiro Omori. Dynamic factor stochastic volatility-in-mean VAR for large macroeconomic panels, 2026. URL `https://arxiv.org/abs/2604.04529`. arXiv:2604.04529v1.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL `https://www.jstor.org/stable/4615733`.

Daniel Hook and Diane Kelly. Mutation sensitivity testing. *Computing in Science & Engineering*, 11(6):40–47, 2009. doi: 10.1109/MCSE.2009.200.

Darjus Hosszejni and Gregor Kastner. Approaches toward the Bayesian estimation of the stochastic volatility model with leverage. In Raffaele Argiento, Daniele Durante, and Sara Wade, editors, *Bayesian Statistics and New Generations: Selected Contributions from BAYSM 2018*, volume 296 of *Springer Proceedings in Mathematics & Statistics*, pages 75–83, Cham, 2019. Springer. doi: 10.1007/978-3-030-30611-3_8.

Darjus Hosszejni and Gregor Kastner. Modeling univariate and multivariate stochastic volatility in R with stochvol and factorstochvol. *Journal of Statistical Software*, 100(12):1–34, 2021. doi: 10.18637/jss.v100.i12.

Stephan Hoyer and Joe Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 5(1):10, 2017. doi: 10.5334/jors.148.

Florian Huber and Luca Rossini. Inference in Bayesian additive vector autoregressive tree models. *The Annals of Applied Statistics*, 16(1):104–123, 2022. doi: 10.1214/21-AOAS1488.

Eric Jacquier, Nicholas G. Polson, and Peter E. Rossi. Bayesian analysis of stochastic volatility models. *Journal of Business & Economic Statistics*, 12(4):371–389, 1994. doi: 10.1080/07350015.1994.10524553.

Eric Jacquier, Nicholas G. Polson, and Peter E. Rossi. Bayesian analysis of stochastic volatility models with fat-tails and correlated errors. *Journal of Econometrics*, 122(1):185–212, 2004. doi: 10.1016/j.jeconom.2003.09.001.

Oleg Jarma, Maja Dall, Raúl Pardo, Mahsa Varshosaz, and Andrzej Wąsowski. Software testing meets Bayesian workflow: A literature-driven artifact study of probabilistic programs, 2026. URL `https://arxiv.org/abs/2610.16594`. arXiv:2610.16594v1.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

Alexander Jordan, Fabian Krüger, and Sebastian Lerch. Evaluating probabilistic forecasts with scoringRules. *Journal of Statistical Software*, 90(12):1–37, 2019. doi: 10.18637/jss.v090.i12.

André G. Journel and Charles J. Huijbregts. *Mining Geostatistics*. Academic Press, London, 1978.

K. Rao Kadiyala and Sune Karlsson. Numerical methods for estimation and inference in Bayesian VAR-models. *Journal of Applied Econometrics*, 12(2):99–132, 1997. doi: 10.1002/(SICI)1099-1255(199703)12:2<99::AID-JAE429>3.0.CO;2-A.

Gregor Kastner. Dealing with stochastic volatility in time series using the R package stochvol. *Journal of Statistical Software*, 69(5):1–30, 2016. doi: 10.18637/jss.v069.i05.

Gregor Kastner. Sparse Bayesian time-varying covariance estimation in many dimensions. *Journal of Econometrics*, 210(1):98–115, 2019. doi: 10.1016/j.jeconom.2018.11.007.

Gregor Kastner. *factorstochvol: Bayesian Estimation of (Sparse) Latent Factor Stochastic Volatility Models*, 2026. URL `https://CRAN.R-project.org/package=factorstochvol`. R package version 1.1.2.

Gregor Kastner and Sylvia Frühwirth-Schnatter. Ancillarity-sufficiency interweaving strategy (ASIS) for boosting MCMC estimation of stochastic volatility models. *Computational Statistics & Data Analysis*, 76:408–423, 2014. doi: 10.1016/j.csda.2013.01.002.

Sangjoon Kim, Neil Shephard, and Siddhartha Chib. Stochastic volatility: Likelihood inference and comparison with ARCH models. *Review of Economic Studies*, 65(3):361–393, 1998. doi: 10.1111/1467-937X.00050.

Young Geun Kim and Changryong Baek. *bvhar: Bayesian Vector Heterogeneous Autoregressive Modeling*, 2026. URL `https://CRAN.R-project.org/package=bvhar`. R package version 2.4.1.

Leslie Kish. *Survey Sampling*. John Wiley & Sons, New York, 1965.

Fabian Krueger. *bvarsv: Bayesian Analysis of a Vector Autoregressive Model with Stochastic Volatility and Time-Varying Parameters*, 2015. URL `https://CRAN.R-project.org/package=bvarsv`. R package version 1.1.

Fabian Krüger, Sebastian Lerch, Thordis Thorarinsdottir, and Tilmann Gneiting. Predictive inference based on Markov chain Monte Carlo output. *International Statistical Review*, 89(2):274–301, 2021. doi: 10.1111/insr.12405.

Ravin Kumar, Colin Carroll, Ari Hartikainen, and Osvaldo Martin. ArviZ a unified library for exploratory analysis of Bayesian models in Python. *Journal of Open Source Software*, 4(33):1143, 2019. doi: 10.21105/joss.01143.

Nikolas Kuschnig and Lukas Vashold. BVAR: Bayesian vector autoregressions with hierarchical prior selection in R. *Journal of Statistical Software*, 100(14):1–27, 2021. doi: 10.18637/jss.v100.i14.

Nikolas Kuschnig and Lukas Vashold. *BVAR: Hierarchical Prior Selection in R*, 2024. URL `https://CRAN.R-project.org/package=BVAR`. R package version 1.0.5.

Paul Labonne, Andrea Renzetti, and Joseph Oyegoke. *bvar: Bayesian Vector Autoregressions for Forecasting*. Bank of England, 2026. URL `https://github.com/bank-of-england/bvar`. Python package version 0.3.5.

Michele Lenza and Giorgio E. Primiceri. How to estimate a vector autoregression after March 2020. *Journal of Applied Econometrics*, 37(4):688–699, 2022. doi: 10.1002/jae.2895.

Robert B. Litterman. Forecasting with Bayesian vector autoregressions – five years of experience. *Journal of Business & Economic Statistics*, 4(1):25–38, 1986. doi: 10.1080/07350015.1986.10509491.

Rui Liu, Andrés Ramirez Hassan, and Tomasz Woźniak. *bvars: Bayesian Forecasting with Large Vector Autoregressions*, 2026. URL `https://CRAN.R-project.org/package=bvars`. R package version 1.1.

George Marsaglia and Wai Wan Tsang. A simple method for generating gamma variables. *ACM Transactions on Mathematical Software*, 26(3):363–372, 2000. doi: 10.1145/358407.358414.

William J. McCausland, Shirley Miller, and Denis Pelletier. Simulation smoothing for state–space models: A computational efficiency analysis. *Computational Statistics & Data Analysis*, 55(1):199–212, 2011. doi: 10.1016/j.csda.2010.07.009.

Michael W. McCracken and Serena Ng. FRED-MD: A monthly database for macroeconomic research. *Journal of Business & Economic Statistics*, 34(4):574–589, 2016. doi: 10.1080/07350015.2015.1086655.

Michael W. McCracken and Serena Ng. FRED-QD: A quarterly database for macroeconomic research. *Federal Reserve Bank of St. Louis Review*, 103(1):1–44, 2021. doi: 10.20955/r.103.1-44.

Martin Modrák, Angie H. Moon, Shinyoung Kim, Paul-Christian Bürkner, Niko Huurre, Kateřina Faltejsková, Andrew Gelman, and Aki Vehtari. Simulation-based calibration checking for Bayesian computation: The choice of test quantities shapes sensitivity. *Bayesian Analysis*, 20 (2):461–488, 2025. doi: 10.1214/23-BA1404.

Franz X. Mohr. *bvartools: Bayesian Inference of Vector Autoregressive and Error Correction Models*, 2026. URL `https://CRAN.R-project.org/package=bvartools`. R package version 0.3.0.

Whitney K. Newey and Kenneth D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987. doi: 10.2307/1913610.

Colm Art O'Cinneide. Relative-error bounds for the LU decomposition via the GTH algorithm. *Numerische Mathematik*, 73(4):507–519, 1996. doi: 10.1007/s002110050203.

Yasuhiro Omori, Siddhartha Chib, Neil Shephard, and Jouchi Nakajima. Stochastic volatility with leverage: Fast and efficient likelihood inference. *Journal of Econometrics*, 140(2):425–449, 2007. doi: 10.1016/j.jeconom.2006.07.008.

Du Phan, Neeraj Pradhan, and Martin Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro, 2019. URL `https://arxiv.org/abs/1912.11554`. arXiv:1912.11554. NeurIPS 2019 Program Transformations for Machine Learning Workshop.

Thomas Pinder. *impulso: Bayesian Vector Autoregression (VAR) in Python*, 2026. URL `https://github.com/QuantClimate/Impulso`. Python package version 0.1.3.

Giorgio E. Primiceri. Time varying structural vector autoregressions and monetary policy. *Review of Economic Studies*, 72(3):821–852, 2005. doi: 10.1111/j.1467-937X.2005.00353.x.

PyMC Developers. *pymc-extras*, 2026. URL `https://github.com/pymc-devs/pymc-extras`. Python package version 0.15.1.

Håvard Rue. Fast sampling of Gaussian Markov random fields. *Journal of the Royal Statistical Society B*, 63(2):325–338, 2001. doi: 10.1111/1467-9868.00288.

Teemu Säilynoja, Paul-Christian Bürkner, and Aki Vehtari. Graphical test for discrete uniformity and its applications in goodness-of-fit evaluation and multiple sample comparison. *Statistics and Computing*, 32(2):32, 2022. doi: 10.1007/s11222-022-10090-6.

Teemu Säilynoja, Marvin Schmitt, Paul-Christian Bürkner, and Aki Vehtari. Posterior SBC: Simulation-based calibration checking conditional on data, 2025. URL `https://arxiv.org/abs/2502.03279`. arXiv:2502.03279.

John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, pages 1–12. ACM, 2011. doi: 10.1145/2063384.2063405.

Michael Scheuerer and Thomas M. Hamill. Variogram-based proper scoring rules for probabilistic forecasts of multivariate quantities. *Monthly Weather Review*, 143(4):1321–1334, 2015. doi: 10.1175/MWR-D-14-00269.1.

Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010. doi: 10.25080/Majora-92bf1922-011.

Charles Shaw. *srvar-toolkit: Shadow-Rate VAR Toolkit for Bayesian Macroeconomic Forecasting*, 2026. URL `https://pypi.org/project/srvar-toolkit/`. Python package version 0.4.0.

Christopher A. Sims and Tao Zha. Bayesian methods for dynamic multivariate models. *International Economic Review*, 39(4):949–968, 1998. doi: 10.2307/2527347.

Christopher A. Sims and Tao Zha. Were there regime switches in U.S. monetary policy? *American Economic Review*, 96(1):54–81, 2006. doi: 10.1257/000282806776157678.

James H. Stock and Mark W. Watson. Core inflation and trend inflation. *Review of Economics and Statistics*, 98(4):770–784, 2016. doi: 10.1162/REST_a_00608.

Jonathan R. Stroud, Peter Müller, and Nicholas G. Polson. Nonlinear state-space models with state-dependent variances. *Journal of the American Statistical Association*, 98(462):377–386, 2003. doi: 10.1198/016214503000161.

Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration, 2018. URL `https://arxiv.org/abs/1804.06788`. arXiv:1804.06788.

David A. van Dyk and Taeyoung Park. Partially collapsed Gibbs samplers: Theory and methods. *Journal of the American Statistical Association*, 103(482):790–796, 2008. doi: 10.1198/016214508000000409.

Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

Xiaolei Wang and Tomasz Woźniak. *bsvarSIGNs: Bayesian SVARs with Sign, Zero, and Narrative Restrictions*, 2026. URL `https://CRAN.R-project.org/package=bsvarSIGNs`. R package version 3.0.

Anders Warne, Günter Coenen, and Kai Christoffel. Marginalized predictive likelihood comparisons of linear Gaussian state-space models with applications to DSGE, DSGE-VAR, and VAR models. *Journal of Applied Econometrics*, 32(1):103–119, 2017. doi: 10.1002/jae.2514.

James T. Wilson, Viacheslav Borovitskiy, Alexander Terenin, Peter Mostowsky, and Marc Peter Deisenroth. Efficiently sampling functions from Gaussian process posteriors. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*,

volume 119 of *Proceedings of Machine Learning Research*, pages 10292–10302, 2020. URL https://proceedings.mlr.press/v119/wilson20a.html.

Tomasz Woźniak. *bsvars: Bayesian Estimation of Structural Vector Autoregressive Models*, 2026. URL https://CRAN.R-project.org/package=bsvars. R package version 4.0.

Qiang Ye. Computing singular values of diagonally dominant matrices to high relative accuracy. *Mathematics of Computation*, 77(264):2195–2230, 2008. doi: 10.1090/S0025-5718-08-02112-1.

Yaming Yu and Xiao-Li Meng. To center or not to center: That is not the question – an ancillarity-sufficiency interweaving strategy (ASIS) for boosting MCMC efficiency. *Journal of Computational and Graphical Statistics*, 20(3):531–570, 2011. doi: 10.1198/jcgs.2011.203main.

Francesco Zanetta and Sam Allen. *scoringrules: A Python Library for Probabilistic Forecast Evaluation*, 2024. URL https://github.com/frazane/scoringrules. Python package version 0.11.0.